

東華大學
DONGHUA UNIVERSITY

ICPC / CCPC
Standard Code Library

Gospel_rock



Coach

老黄畅谈

Contestants

ZnZrYb

xixii_

47523

Build Time: October 12, 2026 by ZnZrYb

Contents

1 梦的开始 . . . **7**
- 1.1 ASCII 表 . . . 7
- 1.2 dxdy 4 . . . 7
- 1.3 一维数组二维化 . . . 7
- 1.4 初始化模板 . . . 7
- 1.5 CMake 配置 . . . 8
  - 1.5.1 Sanitizer 小技巧 . . . 8
  - 1.5.2 Cmake Release 配置小技巧 . . . 8
  - 1.5.3 小心 vector 大小等相关调试信息信息显示错误 . . . 8
- 1.6 本地手测时怎么发 EOF . . . 8
- 1.7 CLion 常用功能与快捷键（默认 keymap） . . . 9
- 1.8 CLion 语言引擎：赛场机上先切 Nova . . . 9
- 1.9 Windows：禁用 `Shift` 切换中英文输入 . . . 9
  - 1.9.1 微软拼音（系统自带，Win10 逐步截图） . . . 10
  - 1.9.2 搜狗输入法 . . . 12

2 基础数学 . . . **13**
- 2.1 快速幂 . . . 13
- 2.2 矩阵快速幂 . . . 13
  - 2.2.1 矩阵快速幂加速递推 . . . 14
- 2.3 自动取模类 . . . 15
- 2.4 异或线性基（linear basis） . . . 16
- 2.5 带权 bitset 线性基（F2 线性方程组求解） . . . 17
- 2.6 区间异或线性基（prefix linear basis） . . . 18
- 2.7 n 进制异或线性基 . . . 19
- 2.8 MEX (Minimum Excluded Value) . . . 20
- 2.9 中位数 (Median) . . . 20
  - 2.9.1 中位数的双向判定不等式 . . . 20
  - 2.9.2 ± 1 编码 + 前缀和：$O(1)$ 区间中位数判定 . . . 20
  - 2.9.3 多段奇长度划分公共中位数 = 全局中位数 . . . 20
  - 2.9.4 L_1 最优化是中位数（对比 L_2 平方和最优化是平均数） . . . 21
  - 2.9.5 流式 / 增量中位数：对顶堆 . . . 21
  - 2.9.6 区间第 k 大（中位数是 $k = \lceil \text{len}/2 \rceil$ 的特例） . . . 21
- 2.10 常见结论（conclusion） . . . 21
  - 2.10.1 约数个数定理 . . . 21
  - 2.10.2 Legendre 公式（$n!$ 的质因子指数） . . . 21
  - 2.10.3 二次方程根与二阶线性递推 . . . 22
  - 2.10.4 逆序对奇偶性与置换环 . . . 22

3 数论 . . . **23**
- 3.1 线性筛 . . . 23
- 3.2 Miller-Rabin 与 Pollard's Rho . . . 23
- 3.3 欧拉定理，扩展欧拉定理，欧拉函数的求解（线性筛，试除法） . . . 24
  - 3.3.1 欧拉定理与费马小定理的证明 . . . 25
- 3.4 gcd（迭代与递归） . . . 25
- 3.5 exgcd 与逆元 . . . 26
- 3.6 在线快速逆元（$O(1)$ 查询） . . . 26
- 3.7 中国剩余定理（CRT） . . . 27
  - 3.7.1 CRT 双射形式（一维循环 $\leftrightarrow$ 二维网格） . . . 27
  - 3.7.2 扩展中国剩余定理（exCRT） . . . 27
  - 3.7.3 CRT 的典型应用 . . . 28
- 3.8 裴蜀定理（Bézout's identity） . . . 28
  - 3.8.1 裴蜀定理的典型应用 . . . 28
- 3.9 莫比乌斯反演（Möbius inversion） . . . 29
  - 3.9.1 莫比乌斯函数 μ . . . 29
  - 3.9.2 反演公式 . . . 29
  - 3.9.3 关键引理：$\sum_{d|n} \mu(d) = [n = 1]$ . . . 29

3.9.4 反演公式（因数形式）的证明 . . . 30

4 计数与组合 . . . 32
4.1 Catalan 数（合法括号串数量） . . . 32
4.2 求组合数（递推法） . . . 32
4.3 求组合数（逆元） . . . 32
4.4 插板法（stars and bars）求非负整数解的数量 . . . 33
4.5 Lucas 定理 . . . 33
4.6 组合相关公式以及技巧 . . . 33
4.6.1 离散变量概率变期望 . . . 33
4.6.2 朱世杰恒等式（Hockey-stick identity） . . . 34
4.7 二项式反演 . . . 34
4.8 范德蒙德卷积 . . . 34
4.8.1 例：中位数为 X（2026 CCPC 网络赛 M） . . . 34
4.9 斯特林数与球盒模型 . . . 35
4.10 整数分拆 . . . 35
4.11 伯努利数与自然数幂和 . . . 35
4.12 常用组合恒等式 . . . 36
4.13 Hook-length 公式（树版本）：递增标号计数 . . . 36
4.14 Erdős–Szekeres 定理（单调子序列） . . . 37

5 博弈论 . . . 38
5.1 Nim 博弈 . . . 38
5.1.1 三角形博弈（带约束的 Nim） . . . 38
5.1.2 Moore's Nim-K（每回合可同时动 K 堆） . . . 39
5.1.3 反 Nim（Anti-Nim / Misère Nim） . . . 39
5.1.4 阶梯 Nim . . . 39
5.1.5 Lasker's Nim（可拆分的 Multi-SG） . . . 39
5.2 巴什博弈（单堆有上限取） . . . 39
5.3 SG 函数的暴力打表与 Anti / Every 变体 . . . 39
5.3.1 反 SG（Anti-SG） . . . 40
5.3.2 Every-SG（每回合走" 所有还能动的石子"） . . . 40
5.3.3 实战范例：CCPC 2025 网络赛 F「连线博弈」——XOR 哈希切子游戏 . . . 40
5.4 威佐夫博弈（双堆联动） . . . 41
5.5 斐波那契博弈（前手两倍约束） . . . 41
5.6 图上删边游戏 . . . 41
5.6.1 树上删边（Colon Principle） . . . 41
5.6.2 无向图删边（Fusion Principle） . . . 41

6 进阶数学 . . . 42
6.1 FFT 多项式乘法（非递归） . . . 42
6.1.1 高精度乘法（基于 FFT） . . . 42
6.1.2 FFT 的使用方法与常见应用 . . . 42
6.2 NTT 卷积（ac-library 精简移植） . . . 44
6.3 多项式全家桶（Poly） . . . 47
6.4 任意模数卷积（三模数 NTT + Garner） . . . 51
6.5 分治 FFT . . . 51
6.6 线性递推：Berlekamp-Massey 与 Bostan-Mori . . . 52
6.7 拉格朗日插值 . . . 53
6.8 FWT 与子集卷积 . . . 54
6.9 拉格朗日反演 . . . 55
6.10 生成函数常用结论 . . . 56
6.11 线性规划（两阶段单纯形 Simplex） . . . 57

7 位运算相关恒等式 . . . 61
7.1 位运算辅助函数（GNU 与 C++20） . . . 61
7.1.1 GNU 常用函数：popcount、clz、ctz、`__lg` . . . 61
7.1.2 C++20：`<bit>` 的标准接口 . . . 61
7.1.3 竞赛中直接套用的写法 . . . 62

7.2 加法与位运算转换 . . . 62
7.3 或、与、异或的关系 . . . 62
7.4 减法与位运算 . . . 62
7.5 & 与 | 的分配律 . . . 62
7.6 lowbit 的推导 . . . 63
7.7 **去除最低位的 1** . . . 63
7.8 连续异或前缀和 . . . 63
7.9 对相邻元素异或的操作转化为前缀异或交换 . . . 63
7.10 集合内两两最小异或和 . . . 64

8 基础算法 . . . **65**
8.1 二维前缀和 . . . 65
8.2 二维差分 . . . 65
8.3 双指针模板 . . . 65
8.4 整数二分 . . . 66
8.4.1 partition_point 实现二分答案（C++20） . . . 66
8.5 三分（实数） . . . 66
8.6 三分整数 . . . 67
8.7 倍增法 . . . 67
8.8 LIS 最长上升子序列及其还原 . . . 67
8.8.1 Dilworth 定理（最小链覆盖 / 偏序划分） . . . 68
8.9 **高精度大整数类** . . . 69

9 数据结构 . . . **74**
9.1 单调栈 . . . 74
9.2 离散化 . . . 74
9.3 ST 表（Sparse Table） . . . 75
9.4 树状数组（BIT / Fenwick Tree） . . . 76
9.4.1 单点加，区间求和 . . . 76
9.4.2 区间加，单点查询 . . . 77
9.4.3 区间加，区间求和 . . . 77
9.4.4 单点改，区间最值查询（BIT 实现 RMQ） . . . 78
9.5 珂朵莉树（ODT / Chtholly Tree） . . . 79
9.6 带权可撤销并查集（判二分图） . . . 80
9.7 线段树分治 . . . 81
9.8 **ACL `segtree`（单点修改）** . . . 82
9.8.1 模板本体 . . . 82
9.8.2 示例：单点赋值 + 区间最大值 . . . 84
9.8.3 示例：单点修改 + 最大子段和 . . . 84
9.8.4 示例：第 k 个 1 与左右边界 . . . 84
9.8.5 示例：含负数时找首个前缀和 $\geq x$ . . . 85
9.9 **ACL `lazy_segtree`（区间修改）** . . . 86
9.9.1 模板本体 . . . 86
9.9.2 示例：区间加 + 区间和 . . . 87
9.9.3 示例：仿射修改 + 一至三次幂和 . . . 88
9.9.4 示例：赋值/加 + 和、最值与位置 . . . 88
9.9.5 示例：最小代价位置的权值和 . . . 88
9.9.6 示例：权值线段树求最小众数 . . . 89
9.10 动态开点线段树（区间加，区间求和） . . . 90
9.11 `rope`（O(1) 拷贝的可持久化序列） . . . 91
9.12 FHQ-treap（无旋 Treap） . . . 91
9.13 隐式 FHQ-treap（可区间维护的动态序列） . . . 93
9.14 主席树（可持久化权值线段树） . . . 98
9.15 分块（单点修改） . . . 99
9.16 回滚莫队 . . . 100

10 图论 . . . **101**

10.1 倍增法求 LCA . . . 101
10.2 虚树（Virtual Tree） . . . 101
10.3 **并查集（DSU）与 DSU on next 技巧** . . . 103
10.3.1 带权并查集（class 封装，维护到根距离与连通块大小） . . . 104
10.3.2 带权并查集（群作用泛型版）+ 不动点钉死 `pinned_dsu` . . . 105
10.4 Borůvka 最小生成树 . . . 107
10.5 树上背包 . . . 108
10.6 Floyd 全源最短路 . . . 108
10.7 Dijkstra（堆优化） . . . 109
10.8 二分图判定（染色法） . . . 109
10.9 匈牙利算法（二分图最大匹配） . . . 109
10.9.1 Hall 婚配定理（完美匹配判定） . . . 110
10.10差分约束 . . . 110
10.11Tarjan 缩点为边双连通分量（EBCC） . . . 110
10.12Tarjan 求强连通分量（SCC） . . . 111
10.13点分治 . . . 111
10.14重链剖分（ACL `lazy_segtree`） . . . 112
10.15欧拉回路与欧拉路径 . . . 114
10.16Color Coding（随机染色 + 状压 DP） . . . 114
10.16.1 转移骨架 . . . 115
10.16.2 剪枝技巧 . . . 115
10.16.3 复杂度 . . . 115
10.16.4 一句话总结 . . . 115
10.17合法括号序列 ↔ 有根有序树 . . . 115
10.18 **网络流（Dinic）：最大流 / 最小割容量及割边集合** . . . 116
10.19最小费用最大流（SPFA 势能 + Dijkstra 增广） . . . 118
10.20图论常见结论 . . . 120
10.20.1 树直径端点的” 远点” 性质 . . . 120
10.20.2 相邻点单源最短路差不超过 1 . . . 120
10.20.3 广义凯莱公式（森林连通块版） . . . 120
10.20.4 两条树上路径是否相交（LCA 判定） . . . 120

11 字符串算法 . . . **122**
11.1 Manacher 算法 . . . 122
11.2 Z 函数（扩展 KMP） . . . 122
11.3 KMP（前缀函数 pi） . . . 123
11.4 **字符串哈希（mod $2^{61}-1$ 版本）** . . . 123
11.5 子序列自动机 . . . 123
11.6 后缀自动机（SAM） . . . 124
11.7 Trie 树 . . . 125
11.8 01 Trie（XOR Trie） . . . 126
11.9 字符串分词与输出技术（大模拟读入） . . . 127

12 动态规划（dp） . . . **128**
12.1 二进制子集枚举技巧 . . . 128
12.1.1 枚举某个集合的所有子集 . . . 128
12.1.2 枚举固定大小的子集 . . . 128
12.1.3 枚举补集元素做前向转移（附「交判子集」） . . . 128
12.2 SOS DP（Sum / Max over Subsets，高维前缀和） . . . 129
12.2.1 典型应用：按位无交最大值 . . . 129
12.3 子集格最短路（超集和 + 状压 DP 求最优排列） . . . 129
12.4 线段树维护 dp 求最大独立集 . . . 131

13 计算几何 . . . **133**
13.1 点线基础模板 . . . 133
13.2 点在线上的投影点 . . . 134
13.3 对称点 . . . 134
13.4 点线位置判断（CCW） . . . 135

13.4.1 逆时针弧角 ccw_angle（标量 / 向量两版） . . . 135
13.5 判断两直线是否垂直 / 平行 . . . 135
13.6 判断两线段是否相交 . . . 135
13.7 两条直线交点 . . . 136
13.8 两线段间最小距离 . . . 136
13.9 多边形面积 . . . 137
13.10圆 Circle：测度五件套 . . . 137
13.11两圆位置关系 circle_relation . . . 138
13.12三角形内接圆 / 外接圆 . . . 138
13.13直线与圆的关系 / 交点 circle_line_relation / getCrossPointsCL . . . 139
13.14线段与圆的关系 / 交点 circle_segment_relation / getCrossPointsCS . . . 139
13.15两圆的关系 / 交点 circle_circle_relation / getCrossPointsCC . . . 140
13.16过一点圆的切点 getTangentPoints . . . 141
13.17两圆公切线 getCommonTangentPoints . . . 141
13.18两圆交集面积 two_circle_intersect_area . . . 142
13.19 **简单多边形与圆的交集面积 polygon_circle_intersect_area** . . . 142
13.20多边形凸性检验 . . . 143
13.20.1 反例 1：内角为反向凹角（外角为锐角） . . . 143
13.20.2 反例 2：自交多边形（蝴蝶结） . . . 143
13.20.3 反例 3：内角恰为 180° . . . 143
13.21点与多边形关系 . . . 144
13.22极角排序 . . . 145
13.23Minkowski 闵可夫斯基和 . . . 145
13.24 **半平面交（half_plane_cut）** . . . 146
13.25凸包（Andrew 算法） . . . 147
13.25.1 对偶点技巧：直线上半包络 ⇔ 对偶点上凸壳 . . . 148
13.26凸多边形直径（旋转卡壳） . . . 148
13.27最远点对（旋转卡壳，返回原始下标） . . . 149
13.28最小矩形覆盖（旋转卡壳，4 指针） . . . 149
13.29平面最近点对 . . . 150
13.30两凸多边形最近距离（旋转卡壳） . . . 151
13.31随机凸多边形生成 . . . 152
13.32最小圆覆盖 . . . 152
13.33常用公式 . . . 153
13.33.1 正多边形面积公式 . . . 153
13.33.2 三角恒等式 . . . 153
13.34结论 . . . 154
13.34.1 Pick 定理 . . . 154

14 STL 及其使用 . . . 156
14.1 **gp_hash_table（pb_ds 哈希表，替 unordered_map）** . . . 156
14.2 **pb_ds 红黑树（tree）** . . . 156
14.3 string::find 的起始位置参数与 string_view . . . 157
14.4 **mt19937_64 套 splitmix64 抗 BM 攻击** . . . 157
14.5 unordered_map 自定义 hash & 结构体做 key . . . 158
14.6 views::keys / views::values 遍历 map 的键 / 值（C++20 ranges） . . . 159
14.7 sort + unique 按 key 去重并保留指定值 . . . 159
14.8 lower_bound / upper_bound 自定义比较器参数顺序相反 . . . 159
14.9 __int128 读入输出（重载运算符版） . . . 160
14.10next_permutation 枚举全排列 . . . 160
14.11 **FMT_STRUCT：让结构体能被 format / println 打印** . . . 160

15 常见问题与错误 . . . 162
15.1 -O2 下的 FMA 乘加融合踩崩浮点 ceil . . . 162
15.2 __float128 使用要点 . . . 162
15.3 DP 不可达状态必须用 INF/-INF 显式标识 . . . 163

15.4 大网格 DFS 遇到障碍要跳过，不要继续标记递归 . . . 163

16 debug 技巧 . . . **164**
16.1 除了打印之外，善用 `assert` . . . 164
16.1.1 增量 debug . . . 164
16.2 `std::string` 条件断点的陷阱 . . . 164

17 环境测试 . . . **165**
17.1 试机赛与纪律 . . . 165
17.2 编译器版本与扩展头 . . . 165
17.3 `__int128`, `__float128`, `long double` 实测 . . . 165
17.4 评测机限制与编译选项 . . . 165
17.5 `#pragma` 开优化 / SIMD 实测 . . . 166
17.6 性能基准（map 跑分） . . . 166

18 常用 Python 命令 . . . **167**
18.1 启动交互解释器 . . . 167
18.2 当计算器：科学计数法速估 . . . 167

19 常用技巧（卡常，对拍，打表，调试等） . . . **168**
19.1 **快读快写** . . . 168
19.1.1 读入，输出科学计数法小数 . . . 168
19.2 开大栈空间 . . . 169
19.3 C++ 中 `struct` 自定义比较器与 `std::array` 性能对比 . . . 169
19.4 计时 . . . 169
19.5 对拍 . . . 170
19.5.1 **Special Judge（答案不唯一）题目怎么对拍** . . . 170
19.6 打表 . . . 171
19.6.1 打表中需要输出的东西（答案） . . . 171
19.6.2 打表中需要输出的东西（前缀答案） . . . 172
19.7 凸包数据生成 . . . 172
19.8 编译 shell . . . 173

1 梦的开始

1.1 ASCII 表

!	33	-	45	9	57	E	69	Q	81	]	93	i	105	u	117
"	34	.	46	:	58	F	70	R	82	^	94	j	106	v	118
#	35	/	47	;	59	G	71	S	83	_	95	k	107	w	119
$	36	0	48	<	60	H	72	T	84	`	96	l	108	x	120
%	37	1	49	=	61	I	73	U	85	a	97	m	109	y	121
&	38	2	50	>	62	J	74	V	86	b	98	n	110	z	122
'	39	3	51	?	63	K	75	W	87	c	99	o	111	{	123
(	40	4	52	@	64	L	76	X	88	d	100	p	112	|	124
)	41	5	53	A	65	M	77	Y	89	e	101	q	113	}	125
*	42	6	54	B	66	N	78	Z	90	f	102	r	114	~	126
+	43	7	55	C	67	O	79	[	91	g	103	s	115		
,	44	8	56	D	68	P	80	\	92	h	104	t	116		

1.2 dxdy 4

```
/* -> (Right, Down, Left, Up)
  |  |
   <-
*/
int dx[4] = {1, 0, -1, 0};
int dy[4] = {0, 1, 0, -1};
```

1.3 一维数组二维化

```
// 如果 n * m <= 1e6, 那么应该开 3e6
↪ (如果一行为M+2)
static constexpr ll MAXN = (ll)3e6 + 10
struct GRID {
  ll a[MAXN] = {};

  ll *operator[](int i) {
    return a + i * (M + 2);
  }
} grid;
```

1.4 初始化模板

```
// teamname: Gospel_rock
/**
 * Problem: ${PROBLEM_NAME}
 * Contest: ${PROBLEM_GROUP_NAME}
 * Judge: ${ONLINE_JUDGE_NAME}
 * URL: ${PROBLEM_URL}
 * Created: ${YEAR}-${MONTH}-${DAY}
 ↪  ${HOUR}:${MINUTE}:${SECOND}
 * Author: Gospel_rock
 * My blog: https://znzryb.com/
 *
 * Powered by AutoCp
 ↪  https://github.com/Pushpavel/AutoCp
 */

#include <bits/stdc++.h>
#define all(vec) vec.begin(), vec.end()
#define lson(o) (o << 1)
#define rson(o) (o << 1 | 1)
#define SZ(a) ((long long)a.size())
#define fsp(x) fixed << setprecision(x)
#define pct(x) __builtin_popcountll(x)
#define debug(x) println(stderr, "{}:{}", #x, x)
#define cend cerr << "\n----------------------
↪  ----------\n"
#define cEnd cerr << "\n**********************
↪  **********\n"

using namespace std;

using ll = long long;
using ull = unsigned long long;
using DB = long double;
using i128 = __int128;
using CD = complex<double>;

static constexpr ll MAXN = (ll)1e6 + 10, INF =
↪  (1ll << 61) - 1;
static constexpr ll mod = 998244353;  //
↪  (ll)1e9+7;
static constexpr double eps = 1e-8;
// 不用 M_PI: 后者是 POSIX 扩展, glibc <cmath>
↪  严格模式会 undef
const long double PI = acosl(-1.0);

ll lT, testcase;

/*
 *
 */
```

```
ll N;

void Solve() {
  cin >> N;
}

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  cout.tie(nullptr);
#ifdef LOCAL
  cout.setf(ios::unitbuf); // 无缓冲流，
  ↪ 方便我们调试
#endif

  cin >> lT;
  for (testcase = 1; testcase <= lT; ++testcase)
    Solve();
  return 0;
}

/*

*/
```

这个 debug(x) 走 C++23 的 println(stderr, ...), **需要 -std=c++23**(<print> 已被 bits/stdc++.h 带上)。只要 x 能被 std::format 格式化就能打：基本类型、string、vector / set / map 等容器、pair / tuple 以及它们的任意嵌套，例如 debug(a) 输出 a:[1, 2, 3], debug(mp) 输出 mp:{1: "one", 2: "two"}; x 也可以是表达式（debug(a[1] + n) 输出 a[1] + n:7)。自定义结构体先用 STL 章的 FMT_STRUCT 宏注册一下即可。不做 LOCAL/非 LOCAL 切换——OJ 上 stderr 通常不影响判分，介意的话提交前把 debug(...) 行删了或包成 #ifdef LOCAL。

1.5 CMake 配置

1.5.1 Sanitizer 小技巧

有些越界或未定义行为问题，仅开 address 不一定能完整暴露。实践中建议同时开启 address + undefined，并保留帧指针，方便定位调用栈。

```
cmake_minimum_required(VERSION 3.31)
project(Edu_CF_186)

set(CMAKE_CXX_STANDARD 20)

add_executable(Edu_CF_186 main.cpp)

add_compile_definitions(LOCAL)
add_compile_options(-fsanitize=address,undefine
↪ d -fno-omit-frame-pointer)
add_link_options(-fsanitize=address,undefined)

# 这个不要随便启用，启用后可能 debug 示数不可信
# add_compile_definitions(_GLIBCXX_DEBUG)

add_executable(A_New_Year_String
↪ "src/A_New_Year_String.cpp")
add_executable(B_New_Year_Cake
↪ "src/B_New_Year_Cake.cpp")
add_executable(odt_callback_bench "speed
↪ test/odt_callback_bench.cpp")
target_compile_options(odt_callback_bench
↪ PRIVATE "SHELL:${CMAKE_CXX_FLAGS_RELEASE}")
```

1.5.2 Cmake Release 配置小技巧

这个写法的主要目的有两个：一是只给特定目标启用 Release 进行性能测试，二是避免在 GUI 里把全局配置切到 Release 后忘记切回 Debug，进而出现调试信息异常、断点行为不符合预期等问题。

```
add_executable(odt_callback_bench "speed
↪ test/odt_callback_bench.cpp")
target_compile_options(odt_callback_bench
↪ PRIVATE "SHELL:${CMAKE_CXX_FLAGS_RELEASE}")
```

1.5.3 小心 vector 大小等相关调试信息信息显示错误

```
#ifdef LOCAL
#define _GLIBCXX_DEBUG
#endif
```

不要在这个 LLDB 调试器下使用这个东西，可能会导致显示错误，因为类型变为了 std::__debug::vector，调试器不支持。

1.6 本地手测时怎么发 EOF

本地跑 while (cin >> x) / while (scanf("%d",&x) != EOF) 这类按 EOF 终止的 CP 代码，手动在终端输完样例后要发 EOF 让 stdin 关流，否则程序一直阻塞读不到末尾。**EOF 的发送键在不同系统/终端里不一样 **:

- **macOS / Linux 真终端**: Ctrl-D（在空输入行上按)。
- **Windows cmd / PowerShell**: Ctrl-Z 后按 Enter。
- **WSL / Git Bash on Windows**: Ctrl-D（同 Unix)。
- **CLion / VSCode Run 控制台**: 看 Clion KeyMap SendEOF，mac/Linux ⇒ Cmd-D Ctrl-D, Windows ⇒ Ctrl-Z+Enter。

1.7 CLion 常用功能与快捷键（默认 keymap）

赛场 / 平时最常用的几个 CLion 操作与默认快捷键如下。赛场机一律 Windows / Linux，认第二列即可；第三列是 macOS 默认 keymap，平时在自己机器上对照用。**Run context configuration** 指「运行光标当前所在的那个目标」——多目标 CMake 工程里免去手动切下拉框，直接跑当前打开的这道题；**Reformat Code** 是一键格式化当前文件 / 选区；**Rename** 是语义级重命名，改变量 / 函数名时所有引用一起改，比全文替换安全（不会误伤字符串和同名的别的符号）。快捷键随 keymap 版本略有出入，忘了就用 **Find Action**（`Ctrl+Shift+A`）搜功能名，弹出项右侧直接显示当前绑定。

Move Line 按物理行搬、不管语法；**Move Statement** 按语句 / 代码块整体对调，且拒绝挪到破坏语法的位置。复制行只有 **Duplicate Line**（只能向下复制一份），没有 VSCode 那种「上下移动时自动复制」。

功能	Windows / Linux	macOS
Run context configuration（跑光标所在目标）	`Ctrl+Shift+F10`	`Ctrl+Shift+R`
Run（当前选中的配置）	`Shift+F10`	`Ctrl+R`
Debug（当前选中的配置）	`Shift+F9`	`Ctrl+D`
Build Project（只编译不跑）	`Ctrl+F9`	`Cmd+F9`
Reformat Code（格式化当前文件 / 选区）	`Ctrl+Alt+L`	`Opt+Cmd+L`
Rename（重命名符号 + 所有引用）	`Shift+F6`	`Shift+F6`
注释 / 取消注释行	`Ctrl+/`	`Cmd+/`
Duplicate Line（复制当前行 / 选区到下方）	`Ctrl+D`	`Cmd+D`
删除整行	`Ctrl+Y`	`Cmd+Backspace`
Move Line（纯按行上 / 下移）	`Alt+Shift+Up/Down`	`Opt+Shift+Up/Down`
Move Statement（按语句块上 / 下移）	`Ctrl+Shift+Up/Down`	`Cmd+Shift+Up/Down`
Find Action（搜任意功能 / 快捷键）	`Ctrl+Shift+A`	`Cmd+Shift+A`

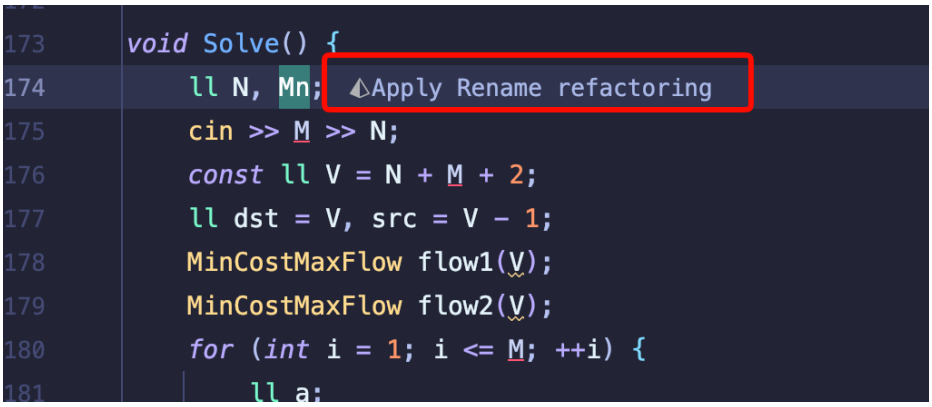
1.8 CLion 语言引擎：赛场机上先切 Nova

CLion 有两套语言引擎：**Classic**（补全 / 高亮走 clangd）和 **Nova**（换成 ReSharper C++ 引擎，核心功能不走 clangd）。**赛场机开机先切 Nova**——最直接的好处是类模板补全带尖括号：敲 `vec` 回车，Nova 给 `vector<>` 且光标落在尖括号中间，Classic 只给 `vector`。竞赛里 `vector` / `map` 满屏都是，省下的按键很可观。

Classic 这边**没有开关能打开**：clangd 只对函数模板补尖括号，类模板一律不补（C++17 有了 CTAD，`vector v{1, 2};` 本身合法，判断不出要不要那对尖括号）。

切换：右侧工具栏齿轮 → **Switch to Nova Engine** → **Enable and Restart**；或 **Settings** → **Advanced Settings** → 勾 **Use the ReSharper C++ language engine (CLion Nova)**。要重启 + 重新解析项目，所以**热身赛就切好**；设置每台机独立，换机器要重切。CLion 2025.3 起 Nova 已是默认，新版本不用管这节。

切到 Nova 后还多一个改名捷径：直接把变量名敲成新的，行内浮出 **Apply Rename refactoring**（下图），点它或按 `Alt+Enter` 回车，所有引用一起改掉，省掉先按 `Shift+F6` 开对话框这一步。Classic 引擎没有这个提示，行内提示样式则要 CLion 2025.2 以上。

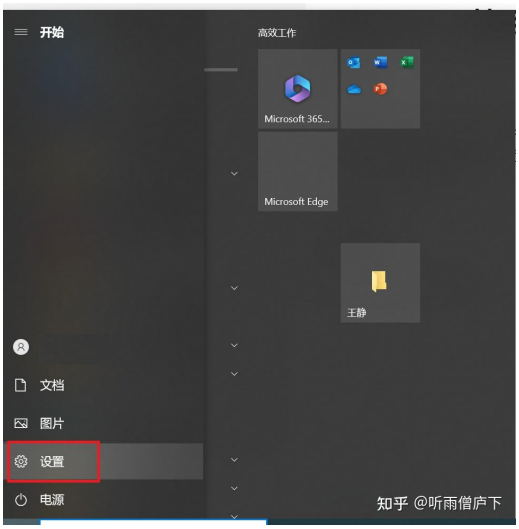


1.9 Windows：禁用 `Shift` 切换中英文输入

Windows 赛场机上敲代码，误碰 `Shift`（打大写、打符号松手稍慢）就会被输入法切到中文模式，接着敲出一串全角字符——开赛前先把这个切换键关掉。

1.9.1 微软拼音（系统自带，Win10 逐步截图）

1. 开始菜单 → **设置**：



2. 进入**时间和语言**：



3. 左侧栏切到**语言**：



4.「首选语言」列表里点**中文 (简体，中国)**：



5. 点展开出来的**选项**按钮：



6. 键盘列表里点**微软拼音** → 进入**按键**：



7.「中/英文模式切换」里**把 `Shift` 的勾去掉**（保留 `Ctrl + 空格键`，这就是之后手动切中英文的键）：



改完后 `Shift` 只做普通修饰键；切中英文用 `Ctrl + Space`，切输入法 / 语言用 `Win + Space`。Win11 路径名稍有不同：设置 → 时间和语言 → **语言和区域** → 中文右侧「…」→ 语言选项 → 微软拼音 → 键盘选项 → 按键，最后一步相同。

1.9.2 搜狗输入法

1. 任务栏搜狗图标打开工具面板 → **更多设置 (P)**：



2. 属性设置 → **按键** → 「中英切换」选**无**（不想彻底禁可选 `Ctrl`）：



QQ / 百度输入法路径类似，都在「按键设置」里。

更省事的做法（比赛全程不需要中文输入时）：直接在上面第 4 步的语言列表里**删掉中文输入法**或把默认语言切到 ENG，Shift 就完全没有切换语义了。快速定位入口：开始菜单搜「输入法」→「编辑语言和键盘选项」。

2 基础数学

2.1 快速幂

```
// 在 k<=0 的时候返回 1
ll binpow(ll a,ll k,const ll &p=mod){
  ll res=1;
  a%=p;
  while(k>0){
    if((k&1)==1) res=a*res%p;
    a=a*a%p;
    k>>=1;
  }
  return res;
}
```

2.2 矩阵快速幂

注意，我们这个东西依赖于后面的这个**自动取模类**，所以代码还是比较长的。

```
/*
* 2025/11/5
↪ https://www.luogu.com.cn/record/245403611
↪ 矩阵快速幂模版题
* 2025/11/5
↪ https://www.luogu.com.cn/record/245411884
↪ P1939 矩阵加速（数列）
* 2026/3/4 去除了过时的宏定义和没有意义的压行
*/
template<class T>
struct Matrix {
  int n, m;
  vector<vector<T> > a; // 1-based

  Matrix() : n(0), m(0) {
  }

  Matrix(int n_, int m_, const T &val = T()) {
    assign(n_, m_, val);
  }

  // 用 1-based 的 vv 构造
  Matrix(const vector<vector<T> > &vv) {
  ↪ load(vv); }

  // vv.size() == n+1, vv[1].size() == m+1
  void load(const vector<vector<T> > &vv) {
    n = (int) vv.size() - 1;
    m = n ? (int) vv[1].size() - 1 : 0;
    a = vv;
  }

  void assign(int n_, int m_, const T &val =
  ↪ T()) {
    n = n_;
    m = m_;
    a.assign(n + 1, vector<T>(m + 1, val));
  }

  // 重载
  vector<T> &operator[](int i) { return a[i]; }
  const vector<T> &operator[](int i) const {
  ↪ return a[i]; }

  // += 负责真正干活
  Matrix &operator+=(const Matrix &rhs) {
    assert(n == rhs.n && m == rhs.m);
    for (int i = 1; i <= n; ++i) {
      for (int j = 1; j <= m; ++j) {
        a[i][j] += rhs[i][j];
      }
    }

    return *this;
  }

  // + 基于 +=
  Matrix operator+(const Matrix &rhs) const {
```

```
    Matrix res(*this);
    res += rhs;
    return res;
  }

  Matrix &operator-=(const Matrix &rhs) {
    assert(n == rhs.n && m == rhs.m);
    for (int i = 1; i <= n; ++i) {
      for (int j = 1; j <= m; ++j) {
        a[i][j] -= rhs[i][j];
      }
    }

    return *this;
  }

  Matrix operator-(const Matrix &rhs) const {
    Matrix res(*this);
    res -= rhs;
    return res;
  }

  Matrix &operator*=(const Matrix &rhs) {
    *this = (*this) * rhs;
    return *this;
  }

  Matrix operator*(const Matrix &rhs) const {
    assert(m == rhs.n);
    Matrix res(n, rhs.m, T());
    for (int i = 1; i <= n; ++i) {
      for (int k = 1; k <= m; ++k) {
        if (a[i][k] != T()) {
          for (int j = 1; j <= rhs.m; ++j) {
            res[i][j] += a[i][k] * rhs[k][j];
          }
        }
      }
    }
    return res;
  }

  static Matrix identity(int n) {
    Matrix I(n, n, T());
    for (int i = 1; i <= n; ++i) I[i][i] = T(1);
    return I;
  }

  friend ostream &operator<<(ostream &os, const
  ↪  Matrix &ma) {
    for (int i = 1; i <= ma.n; ++i) {
      for (int j = 1; j <= ma.m; ++j) {
        os << ma[i][j] << " ";
      }
      os << "\n";
    }
    return os;
  }
};

// 矩阵快速幂：必须是方阵
template<class T>
Matrix<T> mat_pow(Matrix<T> base, long long
↪  exp) {
  assert(base.n == base.m);
  Matrix<T> res = Matrix<T>::identity(base.n);
  while (exp > 0) {
    if (exp & 1) res = res * base;
    base = base * base;
    exp >>= 1;
  }
  return res;
}
```

2.2.1 矩阵快速幂加速递推

已知一个数列 a，它满足：

$$a_x = \begin{cases} 1 & x \in \{1,2,3\} \\ a_{x-1} + a_{x-3} & x \geq 4 \end{cases}$$

求 a 数列的第 n 项对 10^9+7 取余的值。

对于递推式 $a_x = a_{x-1} + a_{x-3}$，因为递推涉及到了前 3 项的状态，我们可以构造一个大小为 3×1 的状态列向量 $\begin{bmatrix} a_x \\ a_{x-1} \\ a_{x-2} \end{bmatrix}$。

根据递推关系，我们可以将其展开为矩阵乘法的形式，寻找转移矩阵 T：

$$\begin{aligned} a_x &= 1 \cdot a_{x-1} + 0 \cdot a_{x-2} + 1 \cdot a_{x-3} \\ a_{x-1} &= 1 \cdot a_{x-1} + 0 \cdot a_{x-2} + 0 \cdot a_{x-3} \\ a_{x-2} &= 0 \cdot a_{x-1} + 1 \cdot a_{x-2} + 0 \cdot a_{x-3} \end{aligned}$$

通过提取系数，可以得出转移矩阵 T：

$$\begin{bmatrix} a_x \\ a_{x-1} \\ a_{x-2} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} a_{x-1} \\ a_{x-2} \\ a_{x-3} \end{bmatrix}$$

(这里就是普通的矩阵乘法，按照矩阵乘法规则展开，上式和下式是等价的)

初始状态列向量为 $V = \begin{bmatrix} a_3 \\ a_2 \\ a_1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$。

将转移矩阵 T 自乘 $n-1$ 次后，再乘以初始向量 V，即可得到最终状态：

$$T^{n-1}V = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} a_3 \\ a_2 \\ a_1 \end{bmatrix} = \begin{bmatrix} a_{n+2} \\ a_{n+1} \\ a_n \end{bmatrix}$$

由此可见，经过 $n-1$ 次状态转移后，底部的元素恰好从 a_1 递推变为了 a_n。因此结果矩阵的第 3 行第 1 列即为所求的 a_n（在代码中由于加入了空的 `{}` 占位使下标从 1 开始，对应为 `mat[3][1]`）。

```
void Solve() {
  ll n;
  cin >> n;
  vector<vector<modi> > A{
        {},      // 递推矩阵
```

```
        {{}, 1, 0, 1},// fn = fn-1 + 0*fn-2 +
        ↪  fn-3
        {{}, 1, 0, 0},
        {{}, 0, 1, 0}
      },
      B = {       // 原向量 v
      ↪ (也可以认为是一个矩阵)
        {},
        {{}, 1}, // a3
        {{}, 1}, // a2
        {{}, 1}  // a1
      };
  Matrix mat(A),matb(B);
  mat = mat_pow(mat, n-1);
  mat*=matb; // 递推矩阵 T* 原向量 v
  // a1 递推 n-1 次，是这个 an
  cout << mat[3][1] << "\n";
}
```

对于递推式 $S_n = 22S_{n-1} - S_{n-2}$，我们可以将其转化为矩阵乘法的形式来加速计算。

首先构造状态列向量 $\begin{bmatrix} S_n \\ S_{n-1} \end{bmatrix}$，根据递推关系可以得出转移矩阵 T：

$$\begin{bmatrix} S_n \\ S_{n-1} \end{bmatrix} = \begin{bmatrix} 22 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} S_{n-1} \\ S_{n-2} \end{bmatrix}$$

初始状态列向量为 $V = \begin{bmatrix} S_2 \\ S_1 \end{bmatrix} = \begin{bmatrix} 482 \\ 22 \end{bmatrix}$。

将转移矩阵 T 自乘 $n-1$ 次后，再乘以初始向量 V，即可得到最终状态：

$$T^{n-1}V = \begin{bmatrix} 22 & -1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} S_2 \\ S_1 \end{bmatrix} = \begin{bmatrix} S_{n+1} \\ S_n \end{bmatrix}$$

由此可见，计算结果矩阵的第 2 行第 1 列即为所求的 S_n。代码中因为使用了一个空的 {} 进行占位（即下标从 1 开始），所以对应的取值为 T[2][1]。

```
vector<vector<modi> >
// 构造转移矩阵 T
// [ S_n   ] = [ 22 -1 ] * [ S_{n-1} ]
// [ S_{n-1} ]   [  1  0 ]   [ S_{n-2} ]
// 注意：用 {} 占位使得二维 vector 的下标从 1 开始
tmp={{},
  {{},22,-1},
  {{},1,0},
  };
Matrix T(tmp);

// 将转移矩阵自乘 n-1 次
T=mat_pow(T,n-1);

// 构造初始状态列向量 V
// V = [ S_2 ] = [ 482 ]
//     [ S_1 ]   [  22 ]
vector<vector<modi> > tmp2=
  {{},
  {{},482},
  {{},22},
  };
Matrix V(tmp2);

// 将转移矩阵和初始向量相乘：T^{n-1} * V
T*=V;

// 提取结果 S_n
// 经过 n-1 次转移后，结果列向量变为 [ S_{n+1} ]
//                                  [ S_n     ]
// 故所求结果在第 2 行第 1 列（1-indexed）
modi res=T[2][1];
```

2.3 自动取模类

```
// T: 底层存储类型 (uint32_t / uint64_t)
// MOD: 模数
// Wide: 乘法时用的更宽类型
template <class T, T MOD, class Wide = ull>
struct modint {
  T v;
  modint(ll x = 0) {
    ll t = x % (ll)MOD;
    if (t < 0) t += MOD;
    v = (T)t;
  }

  T val() const { return v; }

  modint& operator+=(const modint &o) {
    T x = v + o.v;
    if (x >= MOD) x -= MOD;
    v = x;
    return *this;
  }

  modint& operator-=(const modint &o) {
    T x = v >= o.v ? v - o.v : (T)(v + MOD -
    ↪  o.v);
    v = x;
    return *this;
  }

  modint& operator*=(const modint &o) {
    Wide t = (Wide)v * (Wide)o.v;
    v = (T)(t % MOD);
    return *this;
  }

  // 幂
  static modint powmod(modint a, ll e) {
    modint r = 1;
    while (e) {
      if (e & 1) r *= a;
      a *= a;
      e >>= 1;
    }
    return r;
  }

  // 逆元（假定 MOD 是质数）
  static modint inv(modint a) {
    return powmod(a, (ll)MOD - 2);
  }
```

```
  modint& operator/=(const modint &o) {
    return *this *= inv(o);
  }

  // 一元
  modint operator+() const { return *this; }
  modint operator-() const { return modint(0) -
  ↪  *this; }

  // 二元
  friend modint operator+(modint a, const
  ↪  modint &b) { return a += b; }
  friend modint operator-(modint a, const
  ↪  modint &b) { return a -= b; }
  friend modint operator*(modint a, const
  ↪  modint &b) { return a *= b; }
  friend modint operator/(modint a, const
  ↪  modint &b) { return a /= b; }

  friend bool operator==(const modint &a, const
  ↪  modint &b) { return a.v == b.v; }
  friend bool operator!=(const modint &a, const
  ↪  modint &b) { return a.v != b.v; }

  friend ostream& operator<<(ostream &os, const
  ↪  modint &x) {
    return os << x.v;
  }
  friend istream& operator>>(istream &is,
  ↪  modint &x) {
    ll t;
    if (!(is >> t)) return is;
    x = modint(t);
    return is;
  }
};
using modi=modint<uint32_t,mod>;
```

2.4 异或线性基（linear basis）

异或线性基用于解决在一个整数集合中，任意选取元素进行异或操作所能产生的数值组合问题。其核心原理基于线性代数中的向量空间，将整数的二进制表示看作位于 $\mathbb{F}_2$ 域（即模 2 加法，等价于异或）上的向量。

模板中各核心操作的理论基础如下：

- **插入（insert）**：本质是向极大线性无关组中添加新向量的过程。从高位到低位遍历，若当前数字的第 i 位为 1，且该位已存在基底 d_i，则将数字异或 d_i 以消去该位；若不存在基底，则将当前数字作为 d_i 存入，插入成功。若最终数字变为 0，说明它可由已有基底线性表出。
- **查询最大值（query_max）**：基于贪心思想。从高位到低位遍历，对于第 i 位的基底 d_i，因为它是集合中唯一最高位为 i 的基底，若当前答案异或 d_i 后能变大，则必定选择异或。因为无论后面低位如何选择，都无法弥补当前第 i 位错失的 1。
- **查询最小值（query_min）**：如果在插入时出现过能被线性表出的数字（即能异或出 0），那么最小值就是 0；否则，由于基底中的元素互相无法表出对方，线性基数组中最小的非零基底就是能异或出的最小值。
- **重构与求第 k 小（rebuild & query_kth）**：rebuild 的本质是对线性基矩阵进行高斯消元，化为“简化阶梯型矩阵”。经过重构，对于任意基底 p_j，其所有的二进制位 1 在其它基底的对应位上一定都是 0。这使得基底之间变成了完全独立的“正交”状态，它们能表出的所有异或和严格按基底的选择与否呈现字典序递增。因此，将排名 k 转化为二进制表示，若第 j 位为 1，则异或上第 j 小的非零基底，即可拼凑出第 k 小的异或和。

```
/*
 * 2026-04-06: AC
 ↪  https://vjudge.net/solution/69022982
 * 主要是测试了 rebuild 和 query-kth
 * 2026-04-06: AC
 ↪  https://www.luogu.com.cn/record/272234814
 * 主要测试了 merge 功能和这个 query_max 功能
 * 2026-04-07: AC https://acm.hdu.edu.cn/contes」
 ↪  t/view-code?cid=1202&rid=3312&from=rank
 * 测试了 query_min_with
 */
struct LinearBasis {
  static const int MAXL = 60;
  vector<long long> d;
  vector<long long> p; // 正交化后的基底，用于求第
  ↪  k 小
  int cnt; // 线性基内元素的数量（秩）
  bool zero; // 原数组是否能异或出 0

  LinearBasis() {
    d.assign(MAXL + 1, 0);
    cnt = 0;
    zero = false;
  }

  // 向极大线性无关组中添加新向量
  bool insert(long long val) {
    // 从高到低遍历二进制位
    for (int i = MAXL; i >= 0; i--) {
      // 若当前数字的第 i 位为 1
      if (val & (1LL << i)) {
        // 若不存在该位的基底，则作为新基底存入，
        ↪  插入成功
        if (!d[i]) {
          d[i] = val;
          cnt++;
          return true;
        }
        // 若已存在该位基底，则异或消去该位的 1
        val ^= d[i];
      }
    }
    // 若最终数字变为 0，说明它可由已有基底线性表出
    zero = true;
    return false;
  }

  // 查询能异或出的最大值
```

```
  long long query_max() {
    long long res = 0;
    // 基于最高位独占特性的贪心，从高位到低位遍历
    for (int i = MAXL; i >= 0; i--) {
      // 若当前答案异或基底 d[i] 后能变大，
      ↪ 则必然选择异或
      if ((res ^ d[i]) > res) {
        res ^= d[i];
      }
    }
    return res;
  }

  // 查询能异或出的最小值
  long long query_min() {
    // 若在插入时出现过能被线性表出的数字
    ↪ （即能异或出 0）
    if (zero) return 0;
    // 否则，线性基数组中最小的非零基底就是能异或」
    ↪ 出的最小值
    for (int i = 0; i <= MAXL; i++) {
      if (d[i]) return d[i];
    }
    return 0;
  }

  // 重构线性基，通过类似高斯消元化为简化阶梯型矩阵
  void rebuild() {
    // 使所有基底互不干扰，达到完全正交状态
    for (int i = MAXL; i >= 0; i--) {
      for (int j = i - 1; j >= 0; j--) {
        // 如果 d[i] 的第 j 位为 1，则用 d[j]
        ↪ 消去这一位
        if (d[i] & (1LL << j)) {
          d[i] ^= d[j];
        }
      }
    }
    p.clear();
    // 将正交化后的非零基底提取出来，
    ↪ 方便后续按字典序查询
    for (int i = 0; i <= MAXL; i++) {
      if (d[i]) p.push_back(d[i]);
    }
  }

  // 查询去重后的第 k 小值（要求先调用 rebuild）
  long long query_kth(long long k) {
    // 如果原集合能异或出 0，并且 0 算第一小，
    ↪ 先扣除 0 的排名
    if (zero) k--;
    if (k == 0) return 0;
    // 若查询排名超过了能异或出的总个数（2^cnt - 1
    ↪ 个非零值）
    if (k >= (1LL << cnt)) return -1;

    long long res = 0;
    // 将排名 k 转化为二进制表示
    for (int i = 0; i < (int) p.size(); i++) {
      // 若第 i 位为 1，则异或上正交化后的第 i
      ↪ 小的非零基底
      if (k & (1LL << i)) res ^= p[i];
    }
    return res;
  }

  // 合并另一个线性基
  void merge(const LinearBasis &other) {
    for (int i = 0; i <= MAXL; i++) {
      // 将另一个线性基的非零基底全部插入当前线性」
      ↪ 基
      if (other.d[i]) insert(other.d[i]);
    }
    zero |= other.zero;
  }

  // 给定初值 init，从线性基里选任意子集 XOR，使
  ↪ init ^ subset_xor 最小
  // 典型场景：1→n 最短 XOR 路径（HDU「月之都」）—
  ↪ —先求任意一条路径的异或和
  // init = (1 ^ n) ^ 额外边的 x^y^w，再用环
  ↪ （linearly 入基的 x^y^w）去最小化
  long long query_min_with(long long init) {
    long long res = init;
    // 从高位到低位贪心：d[i] 的最高位就是第 i 位，
    ↪ 不会污染更高位；
    // 若 res 的第 i 位为 1，异或 d[i] 能把它消掉，
    ↪ res 严格变小；反之不改
    for (int i = MAXL; i >= 0; i--) {
      if ((res ^ d[i]) < res)
        res ^= d[i];
    }
    return res;
  }
};
```

2.5 带权 bitset 线性基（F2 线性方程组求解）

给出 K 个已知向量 $\text{mask}_j \in \mathbb{F}_2^N$ 及各自挂着的权值 S_j，询问目标向量能否被它们线性表出：能表出就返回对应权值的异或，表不出返回 -1。本质是对增广矩阵 $[\,\text{mask} \mid S\,]$ 做高斯消元——左半边每做一次行异或，右半边同步做一次；N 远大于 64 时整行用 `bitset` 异或。来源：HDU 2026 多校 5「链上 Nim」。

- `d[i]` 非空时最高位恰为第 i 位；`query` 把目标消成全 0 即表出成功，还剩非零位说明不在张成的子空间里，返回 -1。
- **落位后必须立刻 `break`**（代码中高亮那行）：`d[i]` 存的是 mask 的副本，不 `break` 会带着同一个 mask 继续往低位走、在更低的空位上再落位一次，秩虚增后 `query` 会把本来表不出的 mask 判成可表出。
- 单次 `insert` / `query` 为 $O(N^2/w)$，整体 $O((K+Q)\,N^2/w)$，空间 $O(N^2/w)$。

行	mask（位 $5 \to 0$）						val
d_5	1	0	1	0	1	0	7
d_3	0	0	1	0	0	1	4

d_4, d_2, d_1, d_0 仍为空（none()）

图注：$N=6$，依次插入 $(\{5,3,1\},7)$、$(\{3,0\},4)$、$(\{5,1,0\},3)$。前两个各自落位成主元行（蓝底 = 主元位，也就是该行最高位）；第三个先异或 d_5 变成 $\{3,0\}$、权值 $3\oplus 7=4$，再异或 d_3 归零，说明它能被前两行表出，直接丢弃。查询 $\{5,1,0\}$ 消到全 0，返回 $7\oplus 4=3$；查询 $\{5,0\}$ 消到最后剩第 1 位而 d_1 为空，返回 -1。

```
/*
 * 2026-08-04: AC https://acm.hdu.edu.cn/contes」
 ↪  t/view-code?cid=1233&rid=5589
 * HDU 2026 多校 5 1010「链上 Nim」：给出 K 组
 ↪  (位置集合 mask，该集合对应的值 S)，
 * Q 次询问某个位置集合的值 -- 即在 F2 上把询问
 ↪  mask 用已知 mask 线性表出，
 * 答案为对应 S 的异或；表不出来返回 -1。
 * 与普通异或线性基的差别：主元向量是 bitset<N>（N
 ↪  可远大于 64），且每行额外挂一个权值 val。
 */
template<int N>
struct LinearBasis {
  bitset<N> d[N]; // d[i]: 以第 i 位为主元的行
  ↪  (已消去所有更高位)
  ll val[N]; // 该行对应的异或值

  void clear() {
    for (int i = 0; i < N; ++i) {
      d[i].reset();
      val[i] = 0;
    }
  }

  void insert(bitset<N> mask, ll v) {
    for (int i = N - 1; i >= 0; --i) {
      if (mask[i]) {
        if (d[i].none()) {
          d[i] = mask;
          val[i] = v;
          break; // [KEY] 落位即停：mask 连同 v
          ↪  成为第 i 位的主元行
        } else {
          mask ^= d[i];
          v ^= val[i];
        }
      }
    }
    // 走完循环没 break：mask 被消成 0，
    ↪  说明它能被已有行表出，连同 v 一起丢弃
  }

  ll query(bitset<N> mask) {
    ll res = 0;
    for (int i = N - 1; i >= 0; --i) {
      if (mask[i]) {
        if (d[i].any()) {
          mask ^= d[i];
          res ^= val[i];
        }
      }
    }
    if (mask.any()) {
      return -1; // 消不干净：mask
      ↪  不在已知集合张成的子空间里
    }
    return res;
  }
};
```

2.6 区间异或线性基（prefix linear basis）

处理「区间内任取若干元素异或最大值」的问题。朴素做法是对每个区间重建线性基（$O(n\log V)$ 单次），但不能在线。下面这套**前缀线性基 + 贪心保留下标靠右元素**的写法可以做到 $O(\log V)$ 单次查询，且天然支持「末尾追加」操作（强制在线场景）。

模板中各核心操作的理论基础如下：

- **追加（append）**：对每个前缀 $[1..i]$ 维护一个线性基 $L[i]$，由 $L[i-1]$ 增量得到。插入新元素 (val,idx) 时，从高位到低位走，对每一个 val 仍存在的位 i：若 d_i 为空，直接落位；否则比较两者下标，将下标更大的那一对存入 d_i、pos_i，把下标更小的那一对继续往低位消。这样 $L[r]$ 的每条基底都尽可能用下标靠右的元素来表达。
- **区间查询（query_max）**：直接取 $L[r]$，从高位到低位贪心，但只接受 $\text{pos}_i \ge l$ 的基底。正确性来源于「靠右优先」的不变量：$L[r]$ 中所有 $\text{pos}_i \ge l$ 的基底，恰好张成 $a[l..r]$ 的异或子空间（每条基底都可写成只用 $a[l..r]$ 内元素的某个异或组合）。
- **空间**：每个 $L[i]$ 是 BITS 位的基底数组（再加一份 pos 数组），整体 $O(n\cdot\text{BITS})$。值域 $\le 2^{30}$ 时单层 240 字节，$n+m=10^6$ 约 240 MB；若值域更小可压成 `uint16_t` 进一步节省。

```
/*
 * 2026-04-29: AC
 ↪  https://vjudge.net/solution/69555764
 * 区间异或线性基模板题：HDU 6579 Operation
 *   op 0 l r：区间 [l, r] 内子集异或最大值
 ↪  (强制在线)
 *   op 1 x  ：在数组末尾追加 x（强制在线）
 */
struct PrefixLinearBasis {
  static constexpr int BITS = 30;
  using T = unsigned;

  struct Layer {
    T   d  [BITS] = {};   // d[i]：最高位为 i
    ↪  的基底
    int pos[BITS] = {};   // pos[i]：当前 d[i]
    ↪  对应的元素下标（1-indexed）
  };

  vector<Layer> L;            // L[k]：a[1..k]
  ↪  的前缀线性基

  PrefixLinearBasis()         {
  ↪  L.emplace_back(); }   // L[0] 为空基
  void reserve(int n)         { L.reserve(n +
  ↪  1); }
  void clear()                { L.clear();
  ↪  L.emplace_back(); }
  int  size()  const          { return (int)
  ↪  L.size() - 1; }
```

```
  // 末尾追加 val，自动成为 a[size()+1]
  void append(T val) {
    Layer cur = L.back();
    int idx   = (int) L.size();
    for (int i = BITS - 1; i >= 0; --i) {
      if (!((val >> i) & 1)) continue;
      if (!cur.d[i]) {                    //
      ↪  该位空，落位即可
        cur.d[i] = val;
        cur.pos[i] = idx;
        break;
      }
      // 让 d[i] 始终保留下标更靠右的那一对
      if (cur.pos[i] < idx) {
        swap(cur.d[i], val);
        swap(cur.pos[i], idx);
      }
      val ^= cur.d[i];
    }
    L.push_back(cur);
  }

  // a[l..r] 子集异或最大值（1-indexed，l <= r）
  T query_max(int l, int r) const {
    T ans = 0;
    const Layer &b = L[r];
    for (int i = BITS - 1; i >= 0; --i) {
      if (b.pos[i] < l) continue;         //
      ↪  该基底用到了 l 之前的元素，跳过
      if ((ans ^ b.d[i]) > ans) ans ^= b.d[i];
    }
    return ans;
  }
};
```

2.7 n 进制异或线性基

n 进制异或线性基（模 p 域上的线性基）用于处理在 $\mathbb{F}_p$（其中 p 为质数）域上的向量空间问题。与普通的 $\mathbb{F}_2$ 域（二进制异或）类似，在 $\mathbb{F}_p$ 域上，每个数都可以分解为 p 进制的各位数字，构成一个向量。数字间的“p 进制不进位加法”等价于向量的逐元模 p 加法。

模板中各核心操作的理论基础如下：

- **转化与消元（insert & query）**：首先将数字转化为 p 进制向量。利用类似高斯消元法从高到低进行处理，对于每一位，若当前位非零：
  - **插入时**：若该位不存在基底，说明找到了一个新的线性无关向量。为了方便后续消去其他向量的该位，乘以当前最高位元素的乘法逆元，将最高位“归一化”为 1 后存入矩阵（mat），插入成功。若已存在基底，则减去当前位数字与对应基底的乘积，从而消去当前位的非零值。
  - **查询时**：逻辑类似。若消元过程中遇到当前位非零但对应位无基底，说明包含无法由当前基表出的独立分量，返回不可表出；若最终整个向量被消为全 0，则说明该数字可由基底完全线性表出。

```
/*
 * AC
 * https://acm.hdu.edu.cn/contest/view-code?cid
 ↪  =1199&rid=7820&from=rank
 * n 进制异或线性基（模 p 域）
 */
class modn_XOR_base {
private:
  vector<ll> base; // base[i] 记录最高位为 i
  ↪  的那个原始数字
  vector<vector<ll> > mat; // mat[i] 记录最高位为
  ↪  i 的归一化后的 p 进制向量
  ll mod; // 模数 p（必须为质数）

  // 快速幂用于求逆元
  ll power(ll a, ll b) const {
    ll res = 1;
    a %= mod;
    while (b > 0) {
      if (b & 1) res = (res * a) % mod;
      a = (a * a) % mod;
      b >>= 1;
    }
    return res;
  }

  // 费马小定理求乘法逆元（mod 必须为质数）
  ll modInverse(ll n) const {
    return power(n, mod - 2);
  }

public:
  modn_XOR_base(ll Mod) {
    mod = Mod;
    base.assign(64, 0);
    mat.assign(64, vector<ll>(64, 0));
  }

  // 向极大线性无关组中添加新向量（p 进制数字）
  bool insert(ll x) {
    ll orig_x = x;
    vector<ll> v(64, 0);

    // 1. 将数字转化为 p 进制向量（低位到高位）
    for (int i = 0; i < 64; ++i) {
      v[i] = x % mod;
      x /= mod;
    }

    // 2. 从高位到低位遍历，进行高斯消元
    for (int i = 63; i >= 0; --i) {
      if (v[i] == 0) continue; // 当前位为 0，
      ↪  不需要消元

      // 若不存在该位的基底，则作为新基底存入
      if (base[i] == 0) {
        // 求出最高位数字的逆元，
        ↪  用于将最高位归一化为 1
        ll inv = modInverse(v[i]);
        // 整个向量乘以逆元，存入 mat[i]
        for (int j = 0; j <= i; ++j) {
          mat[i][j] = (v[j] * inv) % mod;
        }
```

```
        base[i] = orig_x; // 记录原始数字
        return true;
      }

      // 若已存在该位基底，
      ↪ 利用对应基底消去当前位的非零值
      ll k = v[i];
      for (int j = 0; j <= i; ++j) {
        // 减去基底对应位的 k 倍（k
        ↪ 刚好是当前最高位的值，
        ↪ 因为基底最高位已被归一化为 1）
        v[j] = (v[j] - k * mat[i][j]) % mod;
        if (v[j] < 0) v[j] += mod; //
        ↪ 保证结果为正
      }
    }
    // 若最终向量变为全 0，说明可由已有基底线性表出
    return false;
  }

  // 判断数字 x 能否由当前线性基表出（即 x
  ↪ 是否在张成空间内）
  bool query(ll x) const {
    vector<ll> v(64, 0);

    // 将数字转化为 p 进制向量
    for (int i = 0; i < 64; ++i) {
      v[i] = x % mod;
      x /= mod;
    }

    // 从高位到低位进行消元
    for (int i = 63; i >= 0; --i) {
      if (v[i] == 0) continue;

      // 若当前位不为 0，但基底中对应位为空，
      ↪ 说明带有新的独立分量，无法被表出
      if (base[i] == 0) return false;

      // 若存在基底，则继续消元
      ll k = v[i];
      for (int j = 0; j <= i; ++j) {
        v[j] = (v[j] - k * mat[i][j]) % mod;
        if (v[j] < 0) v[j] += mod;
      }
    }

    // 整个向量被成功消为全 0，说明可被线性表出
    return true;
  }
};
```

2.8 MEX (Minimum Excluded Value)

MEX（Minimum Excluded Value）通常指一个非负整数集合中未出现的最小非负整数。

2.9 中位数 (Median)

本节约定 **长度奇数** N 的数组，中位数即排序后的第 $\lceil N/2 \rceil$ 个元素（也等于第 $(N+1)/2$ 个）。中位数题的"特征气味"：题面里提到 **排序后取中间值 / 划分若干段、每段中位数相同 / 操作不改变中位数**之类。

2.9.1 中位数的双向判定不等式

"m 是数组 A 的中位数" 的等价条件是**两条对称的下界同时成立**：

$$\#\{a \le m\} \ge \lceil N/2 \rceil \quad \text{且} \quad \#\{a \ge m\} \ge \lceil N/2 \rceil$$

为什么是"$\le$" 和"$\ge$"，而不是"$<$" 和"$>$"：要让 m 这个具体的值**出现在中间位置**，必须**同时**有 $\ge \lceil N/2 \rceil$ 个不超过它的、$\ge \lceil N/2 \rceil$ 个不少于它的——两条等号都包含 m 自己，正是它落在中间的根据。如果换成严格不等式，只能限定中位数 $\ge m$ 或 $\le m$ 单边，得不到 $= m$。

推论：包含性自动达成。两条相加，等号情形给出

$$\#\{a \le m\} + \#\{a \ge m\} \;=\; N + \#\{a = m\},$$

右边减 N 等于左边减 $N \ge 2\lceil N/2 \rceil - N = 1$（奇 N），所以 $\#\{a = m\} \ge 1$。**即"m 在数组里出现至少一次"是这两条下界的推论，不是独立条件**。判区间中位数等于 m 时**不必单独判 m 在不在**，套这两条不等式即可——这是常踩的坑。

2.9.2 ± 1 编码 + 前缀和：$O(1)$ 区间中位数判定

要 $O(1)$ 判 "$\mathrm{med}(a[l..r]) = m$"（区间长度奇），定义两套 ± 1 编码：

$$b_1[k] = \begin{cases} +1, & a_k \le m \\ -1, & a_k > m \end{cases}, \qquad b_2[k] = \begin{cases} +1, & a_k \ge m \\ -1, & a_k < m \end{cases}$$

分别取前缀和 B_1, B_2。对于奇长 $\mathrm{len} = r - l + 1$：

$$\begin{aligned} B_1[r] - B_1[l-1] \;&=\; \#\{a \le m\} - \#\{a > m\} \\ &=\; 2\,\#\{a \le m\} - \mathrm{len}. \end{aligned}$$

所以 $\#\{a \le m\} \ge \lceil \mathrm{len}/2 \rceil = (\mathrm{len}+1)/2$ 当且仅当 B_1 差值 ≥ 1。B_2 同理。综合：

$$\begin{aligned} \mathrm{med}(a[l..r]) = m \iff\; & B_1[r] - B_1[l-1] \ge 1 \\ & \text{且}\ B_2[r] - B_2[l-1] \ge 1 \end{aligned}$$

预处理 $O(N)$，单次判 $O(1)$。这个 ± 1 + 双前缀和的组合是中位数题的**瑞士军刀**：除了等于判定，还能拓展到" 区间中位数 $\ge m$ 个数"（只要一条 B_1 差 ≥ 1）和" 区间中位数 $\le m$ 个数"（只要一条 B_2 差 ≥ 1）。

2.9.3 多段奇长度划分公共中位数 = 全局中位数

命题：长度奇 N 的数组 a 被划分成 p 段奇长度子数组，且**每段中位数都等于公共值** m，则 m 必等于 a 的整体中位数。

一行证明：每段 S_i 长 ℓ_i（奇）、中位数 m，由判定不等式 $\#_{S_i}(\le m) \ge (\ell_i+1)/2$。对所有 p 段相加：

$$\#_a(\le m) \;\ge\; \sum_{i=1}^{p} \frac{\ell_i+1}{2} \;=\; \frac{N+p}{2} \;\ge\; \lceil N/2 \rceil.$$

$\#_a(\ge m)$ 同理。两条都成立 $\Rightarrow m$ 是 a 的中位数；奇 N 中位数唯一，所以 m 即整体中位数。**推论**：处理这类划分题时**不必枚举公共中位数候选**——直接 $O(N\log N)$ 排序取出 $a_{\lceil N/2 \rceil}$ 当 m 即可，问题立刻退化成“已知 m，最大化奇长度段数”。

实战范例（CF 1094C *Median Partition*）：把奇长 N 数组划分成最多奇长度子数组、每段中位数相同。先求出全局中位数 m（上一条命题），再做 $O(N^2)$ DP：$f[i] = \max\{f[j]+1 : 0 \le j < i,\ (i-j)\ \text{奇},\ \text{med}(a[j{+}1..i]) = m\}$，区间中位数判定走 ±1 双前缀和（上一条工具）。两件套合用，$\sum N^2 \le 5000^2$ 直接通过。**这道题的“难点”完全集中在“公共中位数 = 全局中位数”和“±1 双前缀和”这两个性质上——把性质内化进板子后，类似题型可以“看到中位数 = 公共值”就直接套出 DP 框架，不再需要赛时重新发明判定。**

2.9.4 L_1 最优化是中位数（对比 L_2 平方和最优化是平均数）

命题：对实数集 $\{a_1,\dots,a_n\}$，函数 $f(x)=\sum_{i=1}^{n}|a_i-x|$ 在 x 取 $\{a_i\}$ 的**中位数**时达到最小值。

直觉证明：把 x 从极小往极大扫，f 的导数（除若干断点外存在）是 $\#\{a_i<x\}-\#\{a_i>x\}$。导数 $\le 0 \Leftrightarrow \#(a_i>x) \ge \#(a_i<x)$ 即 x 在中位数下方；导数 ≥ 0 同理。两侧夹逼出“中位数处导数变号”。n 偶时整段 $[a_{n/2}, a_{n/2+1}]$ 都是最小值点。

怎么用：题面看到“选一个 x 让 $\sum|a_i-x|$ 最小”/“把所有人移到同一个位置，移动总距离最小”/“涂色 / 校准 / 对齐成同一值的最小代价”**都是中位数**。$O(N\log N)$ 排序取中位即可，不要傻乎乎地三分 / 二分。

对比 L_2（平方和）：$\sum(a_i-x)^2$ 最小值点是**平均值**。中位数 vs 平均的根本差异就在 L_1 vs L_2。

实战范例：NBUT 1569 */HELLO - WORLD/*（最小化 $\sum|a_i-x|$ 模板题）、POJ 3579 *Median*（求所有数对绝对差的中位数，二分答案 + 双指针）、Gym 104386B *Random Array*（同模型变体）。

2.9.5 流式 / 增量中位数：对顶堆

要支持**在线 insert + 实时查询当前中位数**的场景，标准做法是**大根堆 + 小根堆**对顶维护：

- **下半堆** L：大根堆，存较小的 $\lceil n/2 \rceil$ 个数；堆顶就是中位数（n 奇时）或下中位数（n 偶时）。
- **上半堆** R：小根堆，存较大的 $\lfloor n/2 \rfloor$ 个数；堆顶是上中位数。
- **不变量**：$|L|-|R| \in \{0,1\}$ 且 $\max L \le \min R$。

insert 流程：先把新元素和 $\max L$ 比较丢进对应堆；若两堆 size 失衡（差 ≥ 2）就把多的那堆堆顶搬到另一堆。每次 $O(\log n)$。**查询中位数**：直接 $\max L$（n 奇）或 $\frac{\max L+\min R}{2}$（n 偶），$O(1)$。

进阶：滑动窗口中位数也是同样的对顶堆 + 延迟删除（lazy delete），或者用 `multiset` + 迭代器维护。复杂度 $O(N\log K)$。

实战范例：Baekjoon 1655 *说出中间值*（流式中位数模板）、HackerRank *Find the Running Median*、FZU 1702 *query*。

2.9.6 区间第 k 大（中位数是 $k=\lceil \text{len}/2 \rceil$ 的特例）

静态区间中位数 / 第 k 大查询，多次询问、不带修改：

主席树（可持久化权值线段树） 预处理 $O(N\log N)$、单次 $O(\log N)$，最常用。

整体二分 二分答案 mid，把所有 $\le$ mid 的数加进 BIT 算每个询问区间内 $\le$ mid 的个数，按是否 $\ge k$ 分流。$O((N+Q)\log^2 N)$ 但常数小。

莫队 + 值域分块 $O((N+Q)\sqrt{N})$，写起来短，对询问规模友好。

动态（带修改的区间中位数）则要带修主席树 / 树套树，复杂度 $O(\log^2 N)$。

实战范例：SPOJ MKTHNUM *K-th Number*（主席树模板）、Baekjoon 7469 *第 K 个数*（同款）。

2.10 常见结论（conclusion）

2.10.1 约数个数定理

设正整数 n 的质因数分解为：

$$n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$$

则 n 的约数个数 $d(n)$ 为：

$$d(n) = (e_1+1)(e_2+1)\cdots(e_k+1)$$

例题：ABC-383-D 9 Divisors

要让 n 恰好有九个约数，由于 $9 = 9\times1 = 3\times3$，只有两种形式：

1. $n = p^8$：只有一个质因子，指数加一等于 9。
2. $n = p^2\cdot q^2$ $(p \ne q)$：两个质因子，$(2{+}1)(2{+}1) = 9$。

2.10.2 Legendre 公式（$n!$ 的质因子指数）

对质数 p，$n!$ 中 p 的指数为

$$v_p(n!) = \sum_{i=1}^{\infty} \left\lfloor \frac{n}{p^i} \right\rfloor$$

求和到 $p^i > n$ 自动截断，实际只跑 $O(\log_p n)$ 项。

常见特例：

- **阶乘末尾零的个数**：$n!$ 末尾 0 的个数 $= \min(v_2(n!), v_5(n!)) = v_5(n!)$（因为 $v_2 \ge v_5$），即 $\lfloor n/5 \rfloor + \lfloor n/25 \rfloor + \lfloor n/125 \rfloor + \cdots$。

- **组合数中某质因子的指数**：$v_p\left(\binom{n}{k}\right) = v_p(n!) - v_p(k!) - v_p((n-k)!)$。结合 **Kummer 定理**：$v_p\left(\binom{n}{k}\right)$ 等于 p 进制下 $k + (n-k)$ 的进位次数。可用于快速判断 $\binom{n}{k}$ 的奇偶性（$p = 2$ 时进位次数 $= 0 \iff \binom{n}{k}$ 是奇数，进一步等价于 $k \,\&\, (n-k) = 0$，即 k 的二进制是 n 的子集）。

闭式（Legendre 等价形式）：$v_p(n!) = \dfrac{n - s_p(n)}{p-1}$，其中 $s_p(n)$ 是 n 在 p 进制下各位数字之和。

2.10.3 二次方程根与二阶线性递推

若 α, β 都是某个二次方程 $x^2 = px + q$ 的根，则 $S_n = k_1\alpha^n + k_2\beta^n$ 满足二阶线性递推：

$$S_n = p \cdot S_{n-1} + q \cdot S_{n-2}$$

经典例子：Fibonacci 数列 $F_n = \varphi^n + \psi^n$（$\varphi, \psi = \frac{1 \pm \sqrt{5}}{2}$），对应 $x^2 = x + 1$，递推为 $F_n = F_{n-1} + F_{n-2}$。

例：$x_1, x_2 = 11 \pm 2\sqrt{30}$ 满足 $x^2 = 22x - 1$，因此 $S_n = x_1^n + x_2^n$ 的递推为

$$S_n = 22S_{n-1} - S_{n-2}$$

2.10.4 逆序对奇偶性与置换环

交换一定改变逆序对数量的奇偶性。长度为 len 的循环移动一格等价于 len − 1 次交换。

例题：第 49 届 ICPC 区域赛沈阳站 D. Dot Product Game

```
FOR(i, 1, N - 1) {
  char ch;
  ll l, r, d;
  cin >> ch >> l >> r >> d;
  ll len = r - l + 1;
  // 向左循环移动 d 次 = 交换 d*(len-1) 次
  if (d != 0)
    inva += (len - 1) * d;
  out();
}
```

快速判断排列逆序对的奇偶性：将排列分解为置换环。一个长度为 k 的环需要 $k-1$ 次交换复位，因此逆序对数与 n 减去环的数量奇偶性相同。设排列有 c 个环，则逆序对数的奇偶性等于 $n - c$ 的奇偶性。比直接用树状数组求逆序对好写。

3 数论

3.1 线性筛

完全理解线性筛

线性筛的作用就是为了保证每个合数都只被其【最小的】那个质因数标记一次。

对于一个将被标记的合数，我们记为 h。在遍历过程中，它由当前的数字 i 和枚举到的素数 p 相乘得到：

$$h \leftarrow i \times p$$

线性筛的核心要求是，我们希望 p 恰好是 h 的最小素因数。

假设将 i 分解素因数，其分解形式为 $i = p_{i1} \times p_{i2} \times \ldots$，其中 p_{i1} 是 i 的最小素因数，也就是代码中记录的 `minp[i]`。那么合数 h 的因数分解可以表示为：

$$h \leftarrow \underbrace{(p_{i1} \times p_{i2} \times \ldots)}_{i} \times p$$

为了使 p 确实是 h 的最小素因数，必然要求 $p \le p_{i1}$，即要求：

$$p \le \text{minp}[i]$$

因此，在线性筛的内层循环中（`for p in primes`），我们首先执行：

```
minp[i * p] = p;
```

当遇到满足 `p == minp[i]` 的情况时，说明当前的素数 p 已经达到了 i 的最小素因数边界。由于 `primes` 列表是递增的，如果继续往后枚举素数，后面的素数必定严格大于 `minp[i]`，这样新枚举的素数就不再是所生成合数（$i \times p'$）的最小素因数了。因此我们必须在此处执行 `break`，从而保证每个合数仅被其最小素因数筛除。

qoj 995868

cp-algorithms 线性筛

```
//  minp[i] 存储的是 i 的最小素因数
// https://www.luogu.com.cn/record/221554377
vector<ll> minp,primes;
void sieve(ll n){
  minp.resize(n+5);
  // minp[1]=1;  // 加了这句话可能把 1
  ↪ 误判成素数，要小心
  for(int i=2;i<=n;++i){
    if(minp[i]==0){    // 是素数
      minp[i]=i;
      primes.push_back(i);
    }
    for(auto p:primes){
      if(i*p>n){  // 超范围了
        break;
      }
      minp[i*p]=p;
      // 为了保证每个合数都只被其【最小的】
      ↪ 那个质因数标记一次 break
      if(p==minp[i]){
        break;
      }
    }
  }
}

// 沿 minp 拆出 n 的素因子列表（计重）；
↪ 长度=Omega，sort+unique 后=omega
vector<ll> pFacLs(ll n) {
  if (n <= 0) return {};
  vector<ll> pLs;
  while (n != 1) {
    pLs.push_back(minp[n]);
    n /= minp[n];
  }
  return pLs;
}
```

使用线性筛的 minp 分解素因数更快，效率更高。用法（如 CF2238D，答案 $\Omega + \omega - 1$）：

```
  vector<ll> pLs = pFacLs(N);          //
  ↪ 素因子列表（计重）
  ll pCt = pLs.size();             // Omega =
  ↪ 列表长度
  sort(all(pLs));
  ll sz = unique(all(pLs)) - pLs.begin();    //
  ↪ omega = 去重后长度
```

3.2 Miller-Rabin 与 Pollard's Rho

模板题：洛谷 P4718【模板】Pollard-Rho（AC 提交）。

`is_prime(n)` 用前 7 个固定底数做确定性 Miller-Rabin，覆盖整个 `unsigned long long` 范围。`factor(n)` 用 Pollard's Rho 把 n 拆成质因子，期望复杂度 $O(n^{1/4})$。

速度关键点：(a) 浮点估商 mulmod 绕开 `__int128` 的硬件 DIV（x86-64 上快 $\sim$ 2x，要求 $n < 2^{63}$）；(b) Pollard 进入前先小质数试除 ≤ 23，把"含小因子的合数"秒掉；(c) $f(x) = x^2 + 1$ Floyd 双指针，差值批乘 256 步再 gcd 一次（gcd 比 mulmod 慢一个量级，必须摊）；(d) 撞环用计数器 `tot++` 重启，不用随机库；(e) 累乘归零时保留旧 `p`。

```
// 直接用模板头的 ll / ull，不另外 using
ull mulmod(ull a, ull b, ull m) {  // 要求 m <
↪ 2^63
  ull q = (long double)a * b / m;
  ll r = (ll)(a * b - m * q);
  return r < 0 ? r + m : ((ull)r >= m ? r - m :
  ↪ r);
}
ull powmod(ull a, ull b, ull m) {
  ull r = 1 % m; a %= m;
  for (; b; b >>= 1, a = mulmod(a, a, m)) if (b
  ↪ & 1) r = mulmod(r, a, m);
  return r;
}
bool is_prime(ull n) {
  if (n < 2) return false;
```

```
  for (ull p :
  ↪  {2,3,5,7,11,13,17,19,23,29,31,37})
    if (n % p == 0) return n == p;
  int s = __builtin_ctzll(n - 1);
  ull d = (n - 1) >> s;
  for (ull a : {2ULL,325ULL,9375ULL,28178ULL,45
  ↪  0775ULL,9780504ULL,1795265022ULL}) {
    ull x = powmod(a % n, d, n);
    if (x <= 1 || x == n - 1) continue;
    bool composite = true;
    for (int i = 0; i < s - 1; ++i) {
      x = mulmod(x, x, n);
      if (x == n - 1) { composite = false;
      ↪  break; }
    }
    if (composite) return false;
  }
  return true;
}
ull pollard(ull n) {
  for (ull p : {2,3,5,7,11,13,17,19,23}) if (n
  ↪  % p == 0) return p;
  static ull tot = 0;
  auto f = [&](ull x) { x = mulmod(x, x, n) +
  ↪  1; return x >= n ? x - n : x; };
  ull x = 0, y = 0, p = 1, q, g;
  for (int i = 0; (i & 255) || (g = __gcd(p,
  ↪  n)) == 1; ++i, x = f(x), y = f(f(y))) {
    if (x == y) { x = tot++; y = f(x); }  //
    ↪  撞环，换起点重启
    q = mulmod(p, x > y ? x - y : y - x, n);
    if (q) p = q;                          //
    ↪  累乘归零则保留旧 p
  }
  return g;
}
void factor(ull n, vector<ull>& res) {  // n < 2
↪  时什么都不写入；res 计重、无序、不清空
  if (n < 2) return;
  if (is_prime(n)) { res.push_back(n); return; }
  ull d = pollard(n);
  factor(d, res); factor(n / d, res);
}
```

用法（P4718 原题：n 是素数输出 `Prime`，否则输出最大质因子）：

```
if (is_prime(N)) puts("Prime");
else {
  vector<ull> res;          // 出参：计重、无序，
  ↪  多测每组自己 clear
  factor(N, res);
  printf("%llu\n", *max_element(all(res)));
}
```

拿到 `res` 后：长度即 $\Omega(n)$（计重），排序去重后的长度即 $\omega(n)$；用 `map<ull,int>` 数出各质因子指数 c_p 后，约数个数 $d(n)=\prod(c_p+1)$，欧拉函数 $\varphi(n)=\prod p^{c_p-1}(p-1)$。

注意：`n < 2^63` 是硬约束（浮点估商会崩），要覆盖 $[2^{63},2^{64})$ 就把 `mulmod` 换成 `return (__int128)a * b % m;`，慢约 2 倍但无范围限制；$n\le 10^6$ 或要对整段区间分解时，直接用上面线性筛的 `minp` 更快。

3.3 欧拉定理，扩展欧拉定理，欧拉函数的求解（线性筛，试除法）

完全理解欧拉函数的计算和欧拉定理（使用线性筛求解）（使用试除法求解）

定义：欧拉函数 $\varphi(n)$ 表示小于或等于 n 且与 n 互质的正整数的个数。

性质与定理：

1. **质数的欧拉函数**：若 p 是质数，则 $\varphi(p)=p-1$。

   解释：因为 p 是质数，所以在 $1\sim p$ 中，除了 p 本身以外，其余的 $p-1$ 个数都与 p 互质。

2. **质数次幂的欧拉函数**：若 p 是质数，则 $\varphi(p^k)=p^k-p^{k-1}=p^{k-1}(p-1)$。

   解释：在 $1\sim p^k$ 中，与 p^k 不互质的数只能是 p 的倍数。由于 $p^k=p^{k-1}\times p$，所以在该范围内 p 的倍数共有 p^{k-1} 个（包含 p^k 本身）。将其减去，剩下的就是与 p^k 互质的数。

3. **积性函数性质**：若 $\gcd(m,n)=1$，则 $\varphi(mn)=\varphi(m)\varphi(n)$。

   解释：由中国剩余定理（CRT）可知，任意一个与 mn 互质的数，都可以唯一映射到一个与 m 互质的数和与 n 互质的数构成的二元组。因此满足条件的数的个数就是这两者的乘积。

4. **欧拉函数通式**：若 n 的质因数分解为 $n=p_1^{k_1}p_2^{k_2}\dots p_m^{k_m}$，则 $\varphi(n)=n\times(1-\frac{1}{p_1})\times(1-\frac{1}{p_2})\cdots\times(1-\frac{1}{p_m})$。

   解释：由积性函数性质和质数次幂的公式直接推导得出。这是我们在使用试除法（$O(\sqrt{n})$ 求单个数的欧拉函数）时的理论基础。

5. **欧拉定理**：若 $\gcd(a,m)=1$，则 $a^{\varphi(m)}\equiv 1 \pmod m$。

   解释：当 m 为质数时，$\varphi(m)=m-1$，该定理即退化为常见的费马小定理 $a^{m-1}\equiv 1 \pmod m$。

6. **扩展欧拉定理（降幂公式）**：

$$a^b\equiv\begin{cases}a^{b\bmod\varphi(m)} \pmod m, & \gcd(a,m)=1\\ a^b \pmod m, & \gcd(a,m)\neq 1, b<\varphi(m)\\ a^{b\bmod\varphi(m)+\varphi(m)} \pmod m, & \gcd(a,m)\neq 1, b\ge\varphi(m)\end{cases}$$

   解释：常用于解决指数极大的取模降幂问题（如计算高塔求幂 $a^{b^c} \pmod m$），此公式在 a,m 不互质的情况下依然成立，非常实用。

3.3.1 欧拉定理与费马小定理的证明

对于任意正整数 m 和整数 a，若 $\gcd(a,m)=1$，则

$$a^{\varphi(m)} \equiv 1 \pmod{m}$$

其中 $\varphi(m)$ 是欧拉函数，表示 1 到 m 中与 m 互素的正整数个数。

证明：考虑模 m 的缩系 S（1 到 m 中所有与 m 互素的数，共 $\varphi(m)$ 个）。当 $\gcd(a,m)=1$ 时，S 中每个元素乘以 a 后模 m 仍是 S 的一个排列（双射），于是

$$\prod_{x\in S}(ax) \equiv \prod_{x\in S} x \pmod{m}$$

左边提出 a：

$$a^{\varphi(m)} \cdot \prod_{x\in S} x \equiv \prod_{x\in S} x \pmod{m}$$

因为 $\prod_{x\in S} x$ 与 m 互素，存在逆元，两边消掉即得 $a^{\varphi(m)} \equiv 1 \pmod{m}$。

费马小定理：当 $m=p$ 为素数时，$\varphi(p)=p-1$，即 $a^{p-1} \equiv 1 \pmod{p}$。

线性筛求欧拉函数的原理：

利用欧拉函数的几条重要性质，我们可以在 $O(n)$ 的线性时间内筛出 $1\sim n$ 所有数的欧拉函数，递推原理如下：

1. **当 i 为质数时**：根据性质 1 可知 $\varphi(i)=i-1$。
2. **当 $i \bmod p \neq 0$ 时**：由于 p 是质数，说明 i 和 p 互质。根据积性函数性质，有 $\varphi(i\times p)=\varphi(i)\times\varphi(p)=\varphi(i)\times(p-1)$。
3. **当 $i \bmod p = 0$ 时**：说明 p 已经是 i 的一个质因子，因此 $i\times p$ 的质因子种类与 i 完全相同。根据欧拉函数通式：

$$\varphi(i)=i\prod\left(1-\frac{1}{p_j}\right)$$

因为质因子种类没变，所以通式后面的连乘部分 $\prod\left(1-\frac{1}{p_j}\right)$ 完全一样，只是把前面的系数 i 变成了 $i\times p$：

$$\varphi(i\times p)=i\times p\prod\left(1-\frac{1}{p_j}\right)=\varphi(i)\times p$$

```cpp
#include <bits/stdc++.h>
using namespace std;

struct LinearSievePhi {
  int n;
  vector<int> primes, minp, phi;

  LinearSievePhi(int n = 0) {
    if (n > 0) init(n);
  }

  void init(int m) {
    n = m;
    primes.clear();
    minp.assign(n + 1, 0);
    phi.assign(n + 1, 0);

    phi[1] = 1;
    for (int i = 2; i <= n; i++) {
      if (minp[i] == 0) { // i 是一个质数
        minp[i] = i;
        primes.push_back(i);
        phi[i] = i - 1; // 质数的欧拉函数性质
      }
      for (int p : primes) {
        if (1LL * i * p > n) break;
        minp[i * p] = p;
        if (i % p == 0) {
          // p 已经是 i 的质因子，i*p 和 i 的质因子集合完全相同
          phi[i * p] = phi[i] * p;
          break; // 保证每个数只被其最小质因子筛去
        } else {
          // i 和 p 互质，利用积性函数性质：phi(i*p) = phi(i) * phi(p)
          phi[i * p] = phi[i] * (p - 1);
        }
      }
    }
  }
};
```

```cpp
/*
 * 试除法求一个数的欧拉函数（求 [1,n] 内与 n 互质的数的个数）
 * 2026-03-13: AC https://codeforces.com/gym/106380/submission/366492238
 */
long long euler_phi(long long n) {
  long long res = n;
  // 枚举可能存在的质因子，直到 sqrt(n)
  for (long long p = 2; p * p <= n; p++) {
    // 由于合数因子必定被其更小的质因子提前除尽，因此能进入 if 的 p 必然是质数
    if (n % p == 0) {
      // 根据通式：res = res * (1 - 1/p) = res / p * (p - 1)
      res = res / p * (p - 1);
      // 剔除 n 中所有质因子 p
      while (n % p == 0) n /= p;
    }
  }
  // 若循环结束 n 仍大于 1，说明存在唯一一个大于 sqrt(n) 的质因子
  if (n > 1) {
    res = res / n * (n - 1);
  }
  return res;
}
```

3.4 gcd（迭代与递归）

不过其实我们在 g++ 下有 __gcd 函数，可以直接使用。如果是在 C++17 以及之后，我们也有这个 std::gcd 函数，以及 std::lcm 函数。

```cpp
int gcd(int a, int b) {
  if (b == 0) return a;
  return gcd(b, a % b);
}
```

```cpp
inline ll gcd(ll a,ll b){
  if(a<b) swap(a,b);
  while(b){
    a%=b;
    swap(a,b);
  }
  return a;
}
```

3.5 exgcd 与逆元

求解 $ax + by = \gcd(a, b)$。利用 exgcd 可以求 a 在模 p 意义下的逆元（要求 $\gcd(a, p) = 1$）。

验证题目：P2613 【模板】有理数取余

```cpp
// AC https://www.luogu.com.cn/record/273130120
void exgcd(ll a, ll b, ll &x, ll &y) {
  if (!b) { x = 1; y = 0; return; }
  exgcd(b, a % b, y, x);
  y -= a / b * x;
}

ll inv_exgcd(ll a, ll mod) {
  ll x = 0, y = 0;
  exgcd(a % mod, mod, x, y);
  x %= mod;
  if (x < 0) x += mod;
  return x;
}
```

3.6 在线快速逆元（$O(1)$ 查询）

QOJ XXII 赛前模板训练赛 Problem 5

基于 Stern-Brocot 树的 $O(1)$ 在线模逆元查询。预处理 $O(2^{21})$ 时间与空间（约 20 MB），每次查询 $O(1)$。

原理：设模数为 p。对于输入 $x \in [1, p-1]$，利用 Stern-Brocot 树预处理的分数近似表，找到分数 a/b（分母 $b < 2048$），使得 $a/b \approx x/p$。令 $r = xb - a \cdot p$，因为 $a \cdot p \equiv 0$，有 $xb \equiv r \pmod p$，从而：

$$x^{-1} \equiv b \cdot r^{-1} \pmod p$$

$|r| \le 2^{21}$，故 r^{-1} 可从预计算的小值逆元表中 $O(1)$ 查找。

约束：模数 p 必须为质数且 $2^{21} < p < 2^{30}$（覆盖 998244353 和 $10^9 + 7$）。

```cpp
// AC https://qoj.ac/submission/2213840
constexpr bool is_prime_constexpr(int n) {
  if (n == 998244353 || n == 1000000007) return
  ↪  true;
  for (int i = 2; (ll)i * i <= n; i++)
    if (n % i == 0) return false;
  return true;
}

template<int mod>
struct FastInv {
  static_assert(2 * (1 << 20) < mod && mod < (1
  ↪  << 30));
  static_assert(is_prime_constexpr(mod));

  // Stern-Brocot 树构建分数近似表
  static vector<pair<uint16_t, uint16_t>>
  ↪  build_frac() {
    vector<pair<uint16_t, uint16_t>> res(1 <<
    ↪  20);
    array<tuple<int, int, int, int>, 2048> st;
    int p = 0;
    st[p++] = {0, 1, 1, 1};
    while (p) {
      auto [a, b, c, d] = st[--p];
      if (b + d < 2048) {
        st[p++] = {a + c, b + d, c, d};
        st[p++] = {a, b, a + c, b + d};
      } else {
        int s = (ll)a * mod / (1024 * b);
        int t = (ll)c * mod / (1024 * d);
        res[s] = {a, b};
        res[t] = {c, d};
        if (a > c) a = c;
        if (b > d) b = d;
        for (int i = s + 1; i < t; i++)
          res[i] = {a, b};
      }
    }
    return res;
  }

  // 线性递推求小值逆元（同时覆盖正负值）
  static vector<uint32_t> build_sinv() {
    constexpr int M = 2 * (1 << 20);
    vector<uint32_t> res(4 * (1 << 20) + 1);
    res[M + 1] = 1;
    res[M - 1] = mod - 1;
    for (int i = 2; i <= M; i++) {
      uint32_t x = (ll)(mod - res[M + (mod %
      ↪  i)])
        * (mod / i) % mod;
      res[M + i] = x;
      res[M - i] = mod - x;
    }
    return res;
  }

  inline static vector<pair<uint16_t,
  ↪  uint16_t>> frac
    = build_frac();
  inline static vector<uint32_t> sinv =
  ↪  build_sinv();

  static int inv(unsigned x) {
    auto [a, b] = frac[x >> 10];
    return (ll)sinv[2 * (1 << 20) + x * b
      - (unsigned)a * mod] * b % mod;
  }
};
```

```
// 使用：FastInv<998244353>::inv(x)
// 或   FastInv<(int)1e9+7>::inv(x)
```

调用示例：静态成员在 `main` 之前自动完成预处理，无需手动初始化。

```
// 直接调用，无需 init
ll ans = FastInv<mod>::inv(x);     // x 的逆元

// 也可以包一层方便使用
inline ll inv(ll x) {
  return FastInv<mod>::inv(x);
}

// 除法：a / b (mod p) = a * inv(b) % p
ll res = a % mod * inv(b) % mod;
```

适用场景：当题目中需要大量求逆元，且快速幂的 $O(\log p)$ 常数成为瓶颈时，可以用本模板替换。典型场景包括：

- **在线逆元查询**：交互题中每次给定 x 要求实时返回 x^{-1}，不允许离线预处理所有值。
- **NTT 内部加速**：NTT 蝴蝶变换中需要大量求逆元（如求原根的逆元、长度的逆元等），用 $O(1)$ 逆元可降低常数。
- **组合数 / 多项式运算的内层循环**：当逆元调用次数达到 $10^7 \sim 10^8$ 量级时，快速幂的 30 次乘法会成为瓶颈，而本模板仅需 1 次查表 +1 次乘法。
- **无法预处理阶乘逆元的情形**：如果逆元的输入没有规律（不是连续的 $1 \sim n$），无法用阶乘逆元表一次性预处理，此时本模板是最优选择。

与其他求逆元方法的对比：

- 快速幂 $O(\log p)$：无需预处理，但单次查询慢，适合逆元调用次数少的场景。
- 线性递推逆元 $O(n)$ 预处理 $+O(1)$ 查询：只能处理 $1 \sim n$ 的连续逆元，且 n 不能太大。
- 阶乘逆元表：适合组合数场景，但只能求阶乘的逆元，不能直接求任意数的逆元。
- **本模板**：$O(2^{21})$ 预处理后任意 $x \in [1, p-1]$ 均可 $O(1)$ 查询，代价是约 20 MB 内存。

3.7 中国剩余定理（CRT）

给定 k 个两两互素的正整数 $m_1, m_2, \ldots, m_k$，以及同余方程组：

$$\begin{cases} x \equiv r_1 \pmod{m_1} \\ x \equiv r_2 \pmod{m_2} \\ \quad\vdots \\ x \equiv r_k \pmod{m_k} \end{cases}$$

则在模 $M = m_1 \times m_2 \times \cdots \times m_k$ 意义下，x 有唯一解：

$$x \equiv \sum_{i=1}^{k} r_i \cdot M_i \cdot M_i^{-1} \pmod{M}$$

其中 $M_i = M/m_i$，M_i^{-1} 是 M_i 在模 m_i 意义下的逆元（即 $M_i \cdot M_i^{-1} \equiv 1 \pmod{m_i}$）。

直觉：$M_i \cdot M_i^{-1}$ 就是那个”只在模 m_i 下为 1、在其他模下为 0”的基础解，公式 $\sum r_i \cdot M_i \cdot M_i^{-1}$ 是把每个余数 r_i 按对应基础解线性组合。

```
// 依赖 exgcd 求逆元
void exgcd(ll a, ll b, ll &x, ll &y) {
  if (!b) { x = 1; y = 0; return; }
  exgcd(b, a % b, y, x);
  y -= a / b * x;
}

ll inv_mod(ll a, ll m) {
  ll x = 0, y = 0;
  exgcd(a % m, m, x, y);
  return (x % m + m) % m;
}

// r[i]: 余数，m[i]: 模数（两两互素）
ll crt(vector<ll> &r, vector<ll> &m) {
  int k = r.size();
  ll M = 1;
  for (int i = 0; i < k; ++i) M *= m[i];
  ll x = 0;
  for (int i = 0; i < k; ++i) {
    ll Mi = M / m[i];
    ll ti = inv_mod(Mi, m[i]);
    x = (x + r[i] % M * (Mi % M)
      % M * (ti % M)) % M;
  }
  return (x + M) % M;
}
```

3.7.1 CRT 双射形式（一维循环 $\leftrightarrow$ 二维网格）

$\gcd(n, m) = 1$ 时，映射 $i \mapsto (i \bmod n, i \bmod m)$ 是 $\mathbb{Z}/(nm) \to \mathbb{Z}/n \times \mathbb{Z}/m$ 的双射，即让 i 跑遍 $[0, nm)$，二元组 $(i \bmod n, i \bmod m)$ 恰好覆盖网格 $[0, n) \times [0, m)$ 每个格点一次。这就是 CRT 的同构 $\mathbb{Z}/(nm) \cong \mathbb{Z}/n \times \mathbb{Z}/m$ 反过来读——常用于把一维 $O(nm)$ 循环拆成两个独立的 $O(n) + O(m)$ 循环（如欧拉函数积性 $\varphi(nm) = \varphi(n)\varphi(m)$ 的证明）。详细证明、几何图示与 $\gcd > 1$ 时像集的刻画见 `TemplateDetailedExplain/CRT-双射形式.tex`。

例题：QOJ 9799 *Magical Palette*（乘积分量化）、QOJ 2735 *Shifty Grid*（行列循环位移）。

3.7.2 扩展中国剩余定理（exCRT）

当模数 $m_1, m_2, \ldots, m_k$ **不一定两两互素**时，普通 CRT 不再适用。exCRT 通过逐步合并方程来求解：每次将两个同余方程合并为一个，直到只剩一个方程。

设当前已合并的结果为 $x \equiv r_1 \pmod{M_1}$，下一个方程为 $x \equiv r_2 \pmod{M_2}$。令 $x = M_1 \cdot t + r_1$，代入得：

$$M_1 \cdot t \equiv r_2 - r_1 \pmod{M_2}$$

设 $g=\gcd(M_1,M_2)$，$c=r_2-r_1$。若 $g\nmid c$ 则无解；否则两边除以 g，用 exgcd 求出 t，合并后新的模数为 $\mathrm{lcm}(M_1,M_2)$。

```
// 返回 {r, M}，表示同余 x = r (mod M)
// 无解返回 {-1, -1}
pair<ll,ll> excrt(vector<ll> &r, vector<ll> &m) {
  ll cur = r[0], M = m[0];
  for (int i = 1; i < (int)r.size(); ++i) {
    ll r2 = r[i], m2 = m[i];
    ll g = __gcd(M, m2);
    ll c = (r2 - cur % m2 + m2) % m2;
    if (c % g) return {-1, -1};
    // 解同余 M/g * t = c/g (mod m2/g)
    ll x0 = 0, y0 = 0;
    exgcd(M / g, m2 / g, x0, y0);
    ll mod2 = m2 / g;
    ll t = (__int128)x0 * (c / g) % mod2;
    t = (t + mod2) % mod2;
    cur = cur + (__int128)M * t % (M / g * m2);
    M = M / g * m2; // lcm(M, m2)
    cur = ((cur % M) + M) % M;
  }
  return {cur, M};
}
```

3.7.3 CRT 的典型应用

1. 对多个质数分别取模后还原：当答案可能很大（如高精度），但已知上界 B 时，选取若干质数 $p_1,p_2,\dots$ 使得 $\prod p_i>B$，对每个 p_i 分别计算答案 r_i，最后用 CRT 还原出真实答案。

2. Lucas 定理 + CRT：当模数 m 不是质数但可以分解为 $m=p_1^{e_1}\times p_2^{e_2}\times\cdots$ 时，对每个 $p_i^{e_i}$ 分别求 $C_n^k \bmod p_i^{e_i}$，再用 CRT 合并。

3. 同余方程组求解：直接的应用场景，如" 一个数除以 3 余 2，除以 5 余 3，除以 7 余 2，求这个数"。

3.8 裴蜀定理（Bézout's identity）

裴蜀定理（Bézout's identity）：对整数 a,b（不全为零），方程 $ax+by=c$ 存在整数解 (x,y) 当且仅当 $\gcd(a,b)\mid c$。

多元推广：对整数 $a_1,a_2,\dots,a_n$（不全为零），方程 $\sum_{i=1}^{n}a_ix_i=c$ 存在整数解 $(x_1,\dots,x_n)$ 当且仅当 $\gcd(a_1,a_2,\dots,a_n)\mid c$。

等价表述：$\{a_1,\dots,a_n\}$ 的全体整数线性组合恰好等于 $\gcd(a_1,\dots,a_n)$ 的全体整数倍：

$$\left\{\sum_{i=1}^{n}k_ia_i : k_i\in\mathbb{Z}\right\}=\gcd(a_1,\dots,a_n)\cdot\mathbb{Z}.$$

证明梗概（二元）：

- ($\Rightarrow$) 设 $g=\gcd(a,b)$。由 $g\mid a$、$g\mid b$ 得 $g\mid(ax+by)=c$。
- ($\Leftarrow$) 由欧几里得算法构造性给出（即 exgcd）。先求 $ax_0+by_0=g$ 的一组解，两边乘 c/g 即得 $ax+by=c$ 的一组解。

多元情形可对 n 归纳：$\gcd(a_1,\dots,a_n)=\gcd(\gcd(a_1,\dots,a_{n-1}),a_n)$，逐对调用二元结论即可。

3.8.1 裴蜀定理的典型应用

1. 判定一次丢番图方程是否有解：方程 $ax+by=c$ 有整数解 $(x,y)\in\mathbb{Z}^2$ 当且仅当 $\gcd(a,b)\mid c$。n 元推广：方程

$$a_1x_1+a_2x_2+\cdots+a_nx_n=c$$

有整数解 $(x_1,\dots,x_n)\in\mathbb{Z}^n$ 当且仅当 $\gcd(a_1,a_2,\dots,a_n)\mid c$。这是最直接的判定层应用，常用作其它构造题的可行性预判。例题：HDU 2026 春季联赛 7 1001 - Card（图上选点喝药水拼魔力值 V，可达性归约为" 路径上点的 gcd 整除 V"，配合 (节点, gcd) 分层图 Dijkstra 求最短路）。

3.9 莫比乌斯反演（Möbius inversion）

3.9.1 莫比乌斯函数 μ

莫比乌斯函数 $\mu(n)$ 定义为：

$$\mu(n)=\begin{cases}1, & n=1,\\ (-1)^k, & n=p_1p_2\cdots p_k\ (k\text{ 个互异质因子，无平方因子}),\\ 0, & n\text{ 含平方因子（存在质数 }p\text{ 使 }p^2\mid n）.\end{cases}$$

μ 是积性函数（$\gcd(a,b)=1\Rightarrow\mu(ab)=\mu(a)\,\mu(b)$），可在线性筛里顺带在 $O(n)$ 内预处理：遇到质数 p 令 $\mu(p)=-1$；筛到 $i\cdot p$ 时，若 $p\mid i$ 则 $i\cdot p$ 含平方因子 p^2，$\mu(i\cdot p)=0$，否则 $\mu(i\cdot p)=-\mu(i)$。

```cpp
const int N = 1e6 + 5;
int mu[N], primes[N], cnt;
bool notPrime[N];
void sieve(int n) {
    mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!notPrime[i]) { primes[cnt++] = i; mu[i] = -1; }
        for (int j = 0; j < cnt && 1LL * i * primes[j] <= n; j++) {
            notPrime[i * primes[j]] = true;
            if (i % primes[j] == 0) { mu[i * primes[j]] = 0; break; }
            mu[i * primes[j]] = -mu[i];
        }
    }
}
```

3.9.2 反演公式

因数（约数）形式：若对任意正整数 n 都有

$$F(n)=\sum_{d\mid n}f(d),$$

则

$$f(n)=\sum_{d\mid n}\mu\Big(\frac{n}{d}\Big)\,F(d)=\sum_{d\mid n}\mu(d)\,F\Big(\frac{n}{d}\Big).$$

倍数形式：若在有限（或收敛）范围内对任意正整数 d 都有 $F(d)=\sum_{d\mid n}f(n)$（n 取遍 d 的所有倍数），则

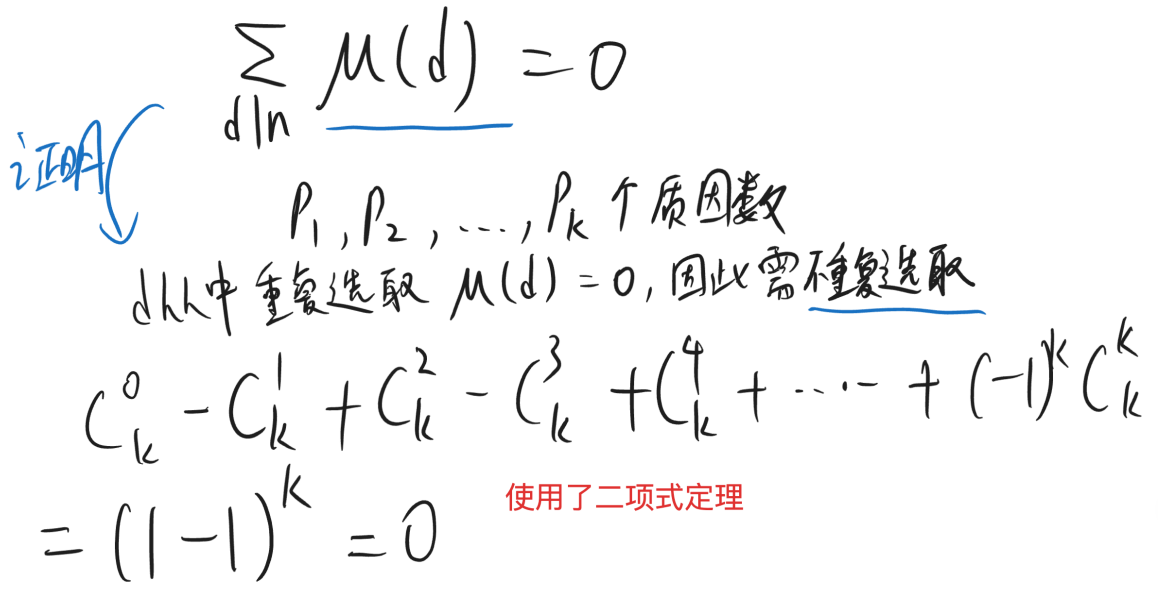
$$f(d)=\sum_{d\mid n}\mu\Big(\frac{n}{d}\Big)\,F(n).$$

下面给出证明（主体推导为手写稿）。核心是关键引理与一次**求和次序交换**；后者另配一张 $n=12$ 的示意图辅助理解。

3.9.3 关键引理：$\sum_{d\mid n}\mu(d)=[n=1]$

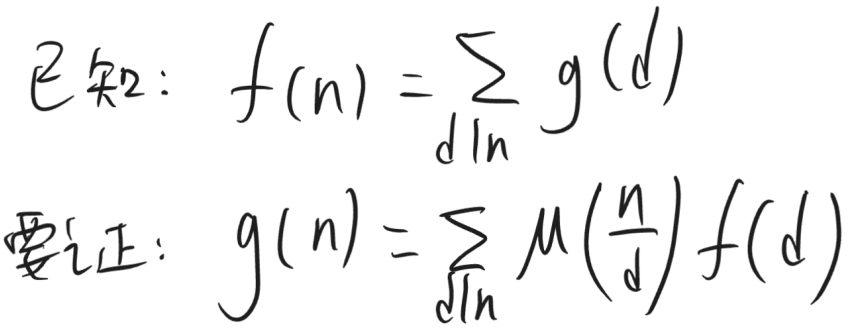
$$\sum_{d\mid n}\mu(d)=[\,n=1\,]=\begin{cases}1, & n=1,\\ 0, & n>1.\end{cases}$$

$n=1$ 显然成立。$n>1$ 时设其有 k 个互异质因子，只有**无平方因子**的约数（从 k 个质因子中不重复选取）贡献非零 μ，按选取个数分组并用二项式定理即得 $(1-1)^k=0$：



3.9.4 反演公式（因数形式）的证明

命题. 已知 $f(n) = \sum_{d|n} g(d)$，要证 $g(n) = \sum_{d|n} \mu(n/d)\, f(d)$：



代入并交换求和次序. 把右边的 $f(d)$ 代入为 $\sum_{k|d} g(k)$，交换求和次序后令 $x = n/d$，内层中括号化为 $\sum_{x|n/k} \mu(x)$：

$$\text{右边} = \sum_{d\mid n} \mu\left(\frac{n}{d}\right)\left[\sum_{k\mid d} g(k)\right]$$

$$= \sum_{k\mid n} g(k)\left[\sum_{d\mid n \,\&\, k\mid d} \mu\left(\frac{n}{d}\right)\right]$$

令 $x = \frac{n}{d}$，$x = \frac{n}{c\cdot k}$，即 $cx = \frac{n}{k}$，$x \mid \frac{n}{k}$

中括号 $= \sum_{x\mid \frac{n}{k}} \mu(x)$

用 μ 的性质收束. 由关键引理 $\sum_{x\mid n/k} \mu(x) = [n/k = 1] = [k = n]$，于是右边 $= \sum_{k\mid n} g(k)\,[k = n] = g(n) =$ 左边，得证：

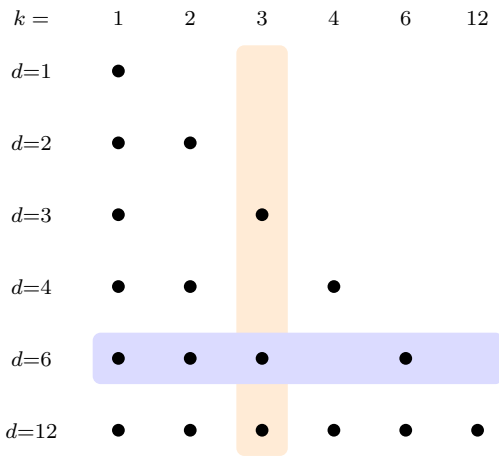
使用 μ 的性质

右边 $= \sum_{k\mid n} g(k) \times \begin{cases} 1 & \text{当 } k = n \text{ 即 } \frac{n}{k} = 1 \\ 0 & \text{当 } k < n \text{ 即 } \frac{n}{k} \neq 1 \end{cases}$

右边 $= g(n) =$ 左边

得证

求和次序交换示意（$n = 12$）. 上面「交换求和次序」一步，可用约数 incidence 网格直观理解——原式与交换后覆盖的是同一批 (k, d) 点对：



求和次序交换（$n=12$，约数 $\{1,2,3,4,6,12\}$）：实心点表示 $k \mid d$。蓝行是原式「固定 $d=6$、横向对 $k \mid d$ 求和」；橙列是交换后「固定 $k=3$、纵向对满足 $k \mid d \mid n$ 的 d 求和」。两种方式覆盖的是同一批点，故可交换。

4 计数与组合

4.1 Catalan 数（合法括号串数量）

Catalan 数是一类专门计数“前缀不越界”结构的经典数列。第 n 个 Catalan 数为：

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}.$$

其中组合数相减的写法可以用反射法理解：先不管前缀限制，只要求 n 个左括号和 n 个右括号，总数是 $\binom{2n}{n}$。坏串是某个前缀中右括号第一次严格多于左括号的串。找到第一次使前缀和变为 -1 的位置，把从开头到这个位置的括号全部翻转，例如 `())(()` 在第 3 位第一次出错，翻转前三位得到 `)((()`。这样坏串就会一一对应到含有 $n+1$ 个左括号、$n-1$ 个右括号的长度 $2n$ 串。反过来，对任意这样的串，找到第一次前缀和变为 1 的位置并翻转前缀，就能还原唯一的坏串。因此坏串数量是 $\binom{2n}{n+1}$，合法串数量即为 $\binom{2n}{n} - \binom{2n}{n+1}$。

最经典的含义是：长度为 $2n$ 的合法括号串数量。这里合法括号串要求恰好有 n 个左括号和 n 个右括号，并且任意前缀中左括号数量都不少于右括号数量。换成前缀和语言，就是把左括号看成 $+1$、右括号看成 -1，要求总和为 0 且任意前缀和都不为负。

例如 $n=3$ 时，合法括号串共有 $C_3 = 5$ 个：

`((()))`，`(()())`，`(())()`，`()(())`，`()()()`.

竞赛中遇到类似模型时，不要只凭“像括号”就直接套 Catalan 数；先检查三个条件：对象能否编码成 $+1/-1$ 序列，最终总和是否固定，合法性是否只由前缀和下界决定。若终点为 0、下界为 0，就是标准 Catalan 数；若终点或下界发生偏移，通常要改用 Ballot 数或反射法的推广。合法括号串、栈的入栈/出栈过程、不过对角线的网格路径，都是同一个前缀约束的不同外壳。

4.2 求组合数（递推法）

会推这个杨辉三角吗？那你就会推组合数，一样的东西。

```
// c[a][b] 表示从 a 个苹果中选 b 个的方案数
for (int i = 0; i < N; i ++ )
    for (int j = 0; j <= i; j ++ )
  if (!j) c[i][j] = 1;
  else c[i][j] = (c[i - 1][j] + c[i - 1][j -
  ↪  1]) % mod;
```

4.3 求组合数（逆元）

增加了这个如果传入参数有误（比如说顺序错了），输出报错信息的功能。

Comb(x) 初始化不能够传入 mod，最大传入 `Comb(mod-1)`，传入 mod 会导致 `frac[mod] = 0`，进而导致所有的 `infrac` 变成 0。

关于阶乘逆元的递推求法：由阶乘的定义可知：

$$frac_{i+1} = frac_i \times (i+1)$$

在模 mod 意义下，等式两边同乘 $infrac_{i+1} \times infrac_i$ 进行化简，直接可得递推式：

$$infrac_i \equiv infrac_{i+1} \times (i+1) \pmod{mod}$$

因此，只需要用费马小定理先算出最大项 $n!$ 的逆元，便可 $O(n)$ 倒推算出所有的阶乘逆元。

```
namespace Comb{    // 组合数加逆元，
↪  依赖于这个MAXN（初始化）
  vector<ll> frac,infrac;
  int ln=1;
  bool initialized=false;
  // 在 k=0 或者更小的时候 return 1
  inline ll binpow(ll a, ll k) {
    ll res = 1;
    a %= mod;
    while (k > 0) {
      if (k & 1) res = (res * a) % mod;
      a = (a * a) % mod;
      k >>= 1;
    }
    return res;
  }

  void Comb(int n) {
    ln = n;
    frac.assign(n + 5, 1);
    infrac.assign(n + 5, 1);
    for (int i = 1; i <= ln; ++i) {
      frac[i] = (frac[i - 1] * i) % mod;
    }
    infrac[ln] = binpow(frac[ln], mod - 2);
    for (int i = ln - 1; i >= 0; --i) {
      infrac[i] = (infrac[i + 1] * (i + 1)) %
      ↪  mod;
    }
  }

  inline ll CC(ll n, ll k) {
    if (k < 0 || k > n) {
#ifdef LOCAL
      cerr<<"CC 组合数传入参数不合法，
      ↪  可能是传入顺序错了\n";
#endif
      return 0; // Return 0 if k is out of range
      ↪  [0, n]
    }
    if(!initialized){
      Comb(MAXN);
      initialized=true;
    }
    ll res = frac[n];
```

```
    res = (res * infrac[n - k]) % mod;
    res = (res * infrac[k]) % mod;
    return res;
  }

  inline ll PP(ll n, ll k) {
    if (k < 0 || k > n) {
#ifdef LOCAL
      cerr<<"PP 排列数传入参数不合法，
      ↪ 可能是传入顺序错了\n";
#endif
      return 0;
    }
    if(!initialized){
      Comb(MAXN);
      initialized=true;
    }
    ll res = frac[n] * infrac[n - k];
    res %= mod;
    return res;
  }
}
using namespace Comb;
```

4.4 插板法（stars and bars）求非负整数解的数量

插板法（stars and bars）求非负整数解的数量
求解方程

$$x_1 + x_2 + \cdots + x_n = A$$

的解的个数（A 为给定的正整数，n 为变量个数）。

先看好理解的情形： $x_i \geq 1$。把和 A 看成排成一行的 A 个小球，相邻小球之间共有 $A-1$ 个空隙。我们要把这一排球切成 n 段，第 i 段的球数就是 x_i。因为每段至少有一个球（对应 $x_i \geq 1$），所以只需在这 $A-1$ 个空隙里插入 $n-1$ 块隔板、每个空隙至多插一块即可。于是方案数就是从 $A-1$ 个空隙里选 $n-1$ 个的组合数：

$$C_{A-1}^{n-1}$$

这一步非常直观，所以插板法的“本体”其实就是 $x_i \geq 1$ 这个版本。

真正不好理解的是 $x_i \geq 0$。此时允许某一段为空，没法直接套“在空隙里插板”的图像。不过我们可以做一个代数变形，令

$$y_i = x_i + 1 \quad (i = 1, 2, \cdots, n)$$

由 $x_i \geq 0$ 立刻得到 $y_i \geq 1$；同时原方程两边各加上 n（共 n 个 $+1$），右边变成 $A+n$：

$$y_1 + y_2 + \cdots + y_n = A + n$$

这样就把“$x_i \geq 0$、和为 A”的问题，**化归**成了刚刚已经解决的“$y_i \geq 1$、和为 $A+n$”的问题。直接套用上面 $x_i \geq 1$ 的结论（把那里的 A 换成 $A+n$）即得：

$$C_{(A+n)-1}^{n-1} = C_{A+n-1}^{n-1} = C_{A+n-1}^{A}$$

4.5 Lucas 定理

Lucas 定理
注意，**如果 p 不是质数，那么 Lucas 定理不成立**，而且这个 **质数 p 还是要求比较小的，一般就是 10^6 以内。**

对于质数 p 以及任意整数 $1 \leq m \leq n$，有如下同余式成立：

$$C_n^m \equiv C_{n \bmod p}^{m \bmod p} \times C_{\lfloor n/p \rfloor}^{\lfloor m/p \rfloor} \pmod{p}$$

代码实现如下：

```
// Lucas 定理递归求解，依赖于上方的组合数模板 CC
↪ 以及全局 mod
inline ll lucas(ll a, ll b) {
  // 如果 a, b 都小于 mod，直接通过预处理的 CC
  ↪ 求解组合数
  if (a < mod && b < mod) return CC(a, b);
  // 否则，根据 Lucas 定理递归求解
  return CC(a % mod, b % mod) * lucas(a / mod,
  ↪ b / mod) % mod;
}
```

注意，在解决 Lucas 定理的时候，余数最大为 mod-1，所以只用预处理到 mod-1。**绝对不能预处理到 mod**，否则 `frac[mod] = 0`，会导致所有的 `infrac` 变成 0。

```
inline ll CC(ll n, ll k) {
  if (k < 0 || k > n) return 0;
  if(!initialized){
    // 【核心细节】：Lucas定理的余数最大为 mod-1，
    ↪ 所以只用预处理到 mod-1。
    // 绝对不能预处理到 mod，否则 frac[mod] = 0，
    ↪ 会导致所有的 infrac 变成 0。
    Comb(mod - 1);
    initialized = true;
  }
  ll res = frac[n];
  res = (res * infrac[n - k]) % mod;
  res = (res * infrac[k]) % mod;
  return res;
}
```

4.6 组合相关公式以及技巧

4.6.1 离散变量概率变期望

当一个非负随机变量 X 只取整数值时，其期望可以通过如下方式计算：

$$E[X] = \sum_{i=1}^{\infty} P(X \geq i)$$

即先对 $i = 1, 2, \ldots$ 分别计算 $X \geq i$ 的概率，将这些概率相加即为期望。

直觉上，期望的定义是 $E[X] = \sum_{k=1}^{\infty} k \cdot P(X = k)$，也就是每个取值 k 贡献了 k 份 $P(X = k)$。而 $P(X \geq i)$ 会把所有 $k \geq i$ 的概率加起来，所以 $\sum_{i=1}^{\infty} P(X \geq i)$ 中，取值 k 恰好被 $i = 1, 2, \ldots, k$ 这 k 个求和项各计入一次，合计贡献正好是 $k \cdot P(X = k)$，与期望定义吻合。

这个技巧在概率题中非常常用：当直接求 $E[X]$ 困难时，转而对每个阈值 i 求 $P(X \geq i)$ 往往更容易。

4.6.2 朱世杰恒等式（Hockey-stick identity）

$$C_i^i + C_{i+1}^i + \cdots + C_n^i = C_{n+1}^{i+1}$$

等价形式：

$$\sum_{j=0}^{r} C_{i+j}^i = C_{i+r+1}^{i+1}$$

$$\begin{array}{ccccccccccc}
 & & & & & & 1 & & & & \\
 & & & & & 1 & & 1 & & & \\
 & & & & 1 & & 2 & & 1 & & \\
 & & & 1 & & 3 & & 3 & & 1 & \\
 & & 1 & & 4 & & 6 & & 4 & & 1 \\
 & 1 & & 5 & & 10 & & 10 & & 5 & & 1 \\
1 & & 6 & & 15 & & 20 & & 15 & & 6 & & 1 \\
 & 1 & & 7 & & 21 & & 35 & & 35 & & 21 & & 7 & & 1
\end{array}$$

杨辉三角，第 0 行到第 7 行。曲棍球棒恒等式确认，例如：对于 $n=6$，$r=2$：$1+3+6+10+15=35$。

应用：当需要对连续上指标的组合数求和时（如 $\sum_{k=r}^{n} C_k^r$），可直接用朱世杰恒等式化为单个组合数 C_{n+1}^{r+1}，将 $O(n)$ 的求和降为 $O(1)$。

4.7 二项式反演

两种最常用的形式（下标形式与上标形式）：

$$f_n = \sum_{i=0}^{n} \binom{n}{i} g_i \iff g_n = \sum_{i=0}^{n} (-1)^{n-i} \binom{n}{i} f_i,$$

$$f_k = \sum_{i=k}^{n} \binom{i}{k} g_i \iff g_k = \sum_{i=k}^{n} (-1)^{i-k} \binom{i}{k} f_i.$$

什么时候想到它：题目要「**恰好** k 个」，而「**至多/至少** k 个」好算的时候。「至多」对应第一式（f 是至多、g 是恰好），「至少」对应第二式。

例：错排数 D_n。设 g_i 为恰好 i 个不动点的方案数，那么「钦定 i 个位置不动、其余随意」的方案数是 $\binom{n}{i}(n-i)!$，它恰好等于 $\sum_{j\ge i}\binom{j}{i} g_j$，套第二式得

$$D_n = g_0 = \sum_{i=0}^{n} (-1)^i \binom{n}{i} (n-i)! = n! \sum_{i=0}^{n} \frac{(-1)^i}{i!}.$$

这也解释了为什么 $D_n/n! \to e^{-1}$。

4.8 范德蒙德卷积

$$\sum_i \binom{m}{i}\binom{n}{k-i} = \binom{m+n}{k}.$$

组合意义：从 m 男 n 女里选 k 人，按选了几个男的分类。两个常用变体：

$$\sum_i \binom{m}{i}\binom{n}{i} = \binom{m+n}{m}, \qquad \sum_i \binom{n}{i}^2 = \binom{2n}{n}.$$

（第一式把 $\binom{n}{i}$ 换成 $\binom{n}{n-i}$ 即可。）见到「两个组合数相乘再对下标求和」就应该先试着往这里凑。

「左右各选 r 个」式：左边 a 个、右边 b 个，两边选**一样多**的方案总数

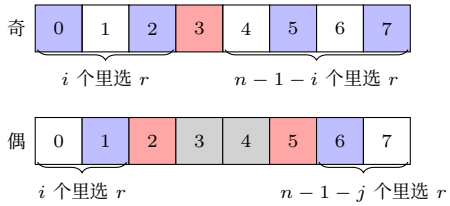
$$\sum_r \binom{a}{r}\binom{b}{r} = \binom{a+b}{a}.$$

一一对应：「左边选中的」并上「右边**没**选中的」恰好是 a 个，反过来也唯一还原。典型场景：数组排序后**固定中位数所在位置**，两侧必须等量选。两侧数量差恰好为 d 时同理（平移版）：

$$\sum_r \binom{a}{r}\binom{b}{r+d} = \binom{a+b}{a+d}.$$

4.8.1 例：中位数为 X（2026 CCPC 网络赛 M）

给长为 n 的序列和 x，数中位数恰好为 x 的非空子序列个数（偶数个取中间两数平均；不同下标集合算不同）。$n \le 2\times 10^5$。



图注：排序后 $n=8$ 的下标。红 = 中心，蓝 = 选中，灰 = 偶数情形两中心之间不能选。奇数中心 $i=3$ 两侧各选 $r=2$；偶数中心对 $(2,5)$ 两侧各选 $r=1$。

1. **去分数、恰好转不超过**：比 $2\cdot\text{med}$（必为整数）。设 $F(Y)$ 为 $2\cdot\text{med}\le Y$ 的集合数，答案 $= F(2x) - F(2x-1)$。
2. **排序后按中心计数**（重复值也当不同位置，下标集合自然区分）：奇数中心 i（0-based）：$\sum_r \binom{i}{r}\binom{n-1-i}{r} = \binom{n-1}{i}$，条件 $2a_i \le Y$；偶数中心对 $i<j$：$\sum_r \binom{i}{r}\binom{n-1-j}{r} = \binom{i+n-1-j}{i}$，条件 $a_i + a_j \le Y$。
3. **双指针 + 朱世杰**：i 增大时最大合法 j（记 q）单调不增；一整段 $j\in(i,q]$ 合并为

$$\sum_{j=i+1}^{q} \binom{i+n-1-j}{i} = \binom{n-1}{i+1} - \binom{i+n-q-1}{i+1}.$$

总复杂度 $O(n\log n)$（排序）。

```
// 依赖上文 namespace Comb (CC)，mod = 998244353，MAXN >= n
// F(Y) = 「2·中位数 <= Y」的非空下标集合数，a 已排序
ll F(const vector<ll> &a, ll Y) {
  int n = SZ(a); ll res = 0;
  for (int i = 0; i < n; i++)          // 奇数：中心 i，两侧各选 r 个
    if (2 * a[i] <= Y) res += CC(n - 1, i);
```

```cpp
  for (int i = 0, q = n - 1; i < n; i++) {  // 偶数：中心对 i<j，q 单调左移
    while (q > i && a[i] + a[q] > Y) q--;
    if (q <= i) break;
    res += CC(n - 1, i + 1) - CC(i + n - q - 1, i + 1) + mod; // j in (i, q] 朱世杰合并
  }
  return res % mod;
}
void Solve() {
  int n; ll x; cin >> n >> x;
  vector<ll> a(n); for (ll &v : a) cin >> v;
  sort(all(a));
  cout << (F(a, 2 * x) - F(a, 2 * x - 1) + mod) % mod << '\n';  // 恰好 = 不超过 2x 减不超过 2x-1
}
// 对拍：与官方 std、Python 暴力随机 3000 组一致
```

4.9 斯特林数与球盒模型

第二类斯特林数 $S(n,k)$（也写作 $\left\{ {n \atop k} \right\}$）：把 n 个**不同**的球放进 k 个**相同**的盒子、每盒非空的方案数。

$$S(n,k) = k\,S(n-1,k) + S(n-1,k-1)$$

（第 n 个球：放进已有的 k 个盒子之一，或自己开一个新盒子。）通项是一个卷积：

$$S(n,k) = \sum_{i=0}^{k} \frac{(-1)^i}{i!} \cdot \frac{(k-i)^n}{(k-i)!},$$

所以一**整行**可以用一次 NTT 在 $O(n \log n)$ 内求出（进阶数学一章）。

第一类斯特林数（无符号） $c(n,k)$（也写作 $\left[{n \atop k} \right]$）：把 n 个元素排成 k 个**非空轮换**的方案数。

$$c(n,k) = (n-1)c(n-1,k) + c(n-1,k-1)$$

（第 n 个元素：插到已有排列里 $n-1$ 个位置之一，或自成一个轮换。）

两组连接普通幂与阶乘幂的公式（做「k 次幂求和」类题目的核心工具）：

$$x^n = \sum_k S(n,k)\,x^{\underline{k}}, \qquad x^{\overline{n}} = \sum_k c(n,k)\,x^k,$$

其中 $x^{\underline{k}} = x(x-1)\cdots(x-k+1)$ 是下降幂。把 i^k 拆成下降幂之后，$\binom{i}{k}$ 型求和就能用朱世杰恒等式一步合并。

球盒模型十二路（n 球放 k 盒）：

球	盒	限制	方案数
异	异	无	k^n
异	异	每盒 ≤ 1	$k^{\underline{n}}$
异	异	每盒 ≥ 1	$k!\,S(n,k)$
异	同	无	$\sum_{i\le k} S(n,i)$
异	同	每盒 ≤ 1	$[n \le k]$
异	同	每盒 ≥ 1	$S(n,k)$
同	异	无	$\binom{n+k-1}{k-1}$
同	异	每盒 ≤ 1	$\binom{k}{n}$
同	异	每盒 ≥ 1	$\binom{n-1}{k-1}$
同	同	无	$\sum_{i\le k} p_i(n)$
同	同	每盒 ≤ 1	$[n \le k]$
同	同	每盒 ≥ 1	$p_k(n)$

最后两行的 $p_k(n)$ 是「把 n 拆成恰好 k 个正整数之和」的方案数，见下一节；「同球异盒」三行就是插板法。

4.10 整数分拆

$p(n)$ 表示把 n 拆成若干个正整数之和（不计顺序）的方案数，其生成函数是

$$\sum_{n\ge 0} p(n)x^n = \prod_{i\ge 1} \frac{1}{1-x^i}.$$

直接对它做多项式求逆是 $O(n \log n)$，但**完全没必要**——由五边形数定理

$$\prod_{i\ge 1}(1-x^i) = \sum_{k\in\mathbb{Z}} (-1)^k x^{k(3k-1)/2},$$

右边只有 $O(\sqrt{n})$ 项非零，两边同乘 $\sum p(n)x^n$ 后比较系数，得到一个每项只需 $O(\sqrt{n})$ 次加减的递推：

$$p(n) = \sum_{k\ge 1} (-1)^{k-1} \Big(p\big(n - \tfrac{k(3k-1)}{2}\big) + p\big(n - \tfrac{k(3k+1)}{2}\big) \Big),$$

总复杂度 $O(n\sqrt{n})$，常数极小，$n \le 10^5$ 时比多项式求逆快得多。

恰好 k 份的分拆数 $p_k(n)$ 用二维递推：$p_k(n) = p_{k-1}(n-1) + p_k(n-k)$（分类讨论「最小的那一份是不是 1」；不是的话每份都减 1）。

4.11 伯努利数与自然数幂和

伯努利数 B_i 的指数生成函数是 $\dfrac{x}{e^x - 1}$，实用的定义是递推

$$\sum_{j=0}^{n} \binom{n+1}{j} B_j = 0 \quad (n \ge 1), \qquad B_0 = 1,$$

于是 $B_1 = -\frac{1}{2}$，$B_2 = \frac{1}{6}$，$B_4 = -\frac{1}{30}$，且**所有奇数下标（≥ 3）的伯努利数都是** 0。它的用处是幂和的封闭式：

$$\sum_{i=0}^{n-1} i^k = \frac{1}{k+1} \sum_{j=0}^{k} \binom{k+1}{j} B_j\, n^{k+1-j}.$$

和拉格朗日插值怎么选：这个式子的复杂度是 $O(k^2)$（递推 B）或 $O(k\log k)$（多项式求逆，因为 B 的 EGF 就是 $\frac{x}{e^x-1}$ 的系数），**与 n 无关**，适合「k 很大、要对很多个 n 求值」的场合；若只求单个 n、且 k 不大，直接用拉格朗日插值取 $k+2$ 个点值更省事，代码也短得多（见进阶数学一章）。

4.12 常用组合恒等式

- 对称：$\binom{n}{k}=\binom{n}{n-k}$；
- 吸收：$k\binom{n}{k}=n\binom{n-1}{k-1}$ ——见到组合数前面乘着下指标就用它；
- 二项式定理的两个特例：$\sum_i\binom{n}{i}=2^n$，$\sum_i(-1)^i\binom{n}{i}=[n=0]$；
- 带权和（都由吸收恒等式导出）：

$$\sum_i i\binom{n}{i}=n2^{n-1},\quad \sum_i i^2\binom{n}{i}=n(n+1)2^{n-2};$$

- 上指标求和即朱世杰恒等式（见上文）；
- 平方和即范德蒙德卷积：$\sum_i\binom{n}{i}^2=\binom{2n}{n}$（「左右各选 r 个」与平移版见范德蒙德卷积一节）；
- 奇偶项和：$\sum_i\binom{n}{2i}=\sum_i\binom{n}{2i+1}=2^{n-1}$（$n\ge1$）；
- 行前缀和 $S(n,k)=\sum_{i\le k}\binom{n}{i}$ 没有封闭式，但能 $O(1)$ 移动：

$$S(n+1,k)=2S(n,k)-\binom{n}{k},$$
$$S(n,k+1)=S(n,k)+\binom{n}{k+1},$$

  多组询问 (n,k) 用莫队；
- 交错前缀和：$\sum_{i\le k}(-1)^i\binom{n}{i}=(-1)^k\binom{n-1}{k}$；
- 三项式换位：$\binom{n}{k}\binom{k}{m}=\binom{n}{m}\binom{n-m}{k-m}$ ——「先选 k 个再从中挑 m 个」交换求和顺序时用；
- 上指标卷积：$\sum_k\binom{k}{a}\binom{n-k}{b}=\binom{n+1}{a+b+1}$（枚举第 $a+1$ 个被选元素的位置）；
- 斜线和：$\sum_i\binom{n-i}{i}=F_{n+1}$（$F_1=F_2=1$，即 1×2 骨牌铺 $1\times n$）；
- 多重集排列 $\dfrac{n!}{\prod a_i!}$，环排列 $(n-1)!$（项链可翻转再 $\div2$，$n\ge3$）；
- 有上界的插板：$x_1+\cdots+x_k=n$，$0\le x_i\le c$ 的解数，容斥钦定 j 个超界：

$$\sum_{j:\,j(c+1)\le n}(-1)^j\binom{k}{j}\binom{n-j(c+1)+k-1}{k-1};$$

- 格路反射：$(0,0)\to(n,m)$ 只走右 / 上、**不碰**直线 $y=x+t+1$（$m\le n+t$）的路径数 $\binom{n+m}{n}-\binom{n+m}{n+t+1}$（起点关于该直线反射到 $(-t-1,t+1)$）；$t=0$，$m=n$ 即 Catalan；
- 错排递推：$D_n=(n-1)(D_{n-1}+D_{n-2})$，$D_1=0$，$D_2=1$。

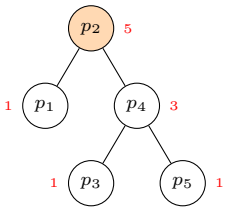
4.13 Hook-length 公式（树版本）：递增标号计数

有根树 T（n 节点）上把 $1,\dots,n$ 填到节点、要求**每个非根节点标号 > 父节点标号**（即整体是 min-heap / 递增树），合法方案数

$$\#\{\text{递增标号}\}=\frac{n!}{\prod_{v\in T}\text{size}(v)}.$$

节点 v 的「钩」就是它的整棵子树，钩长 $h(v)=\text{size}(v)=|T_v|$。

直观概率法证明。把 $1,\dots,n$ **均匀随机**分派到节点上（$n!$ 种等可能）。合法 min-heap $\iff$ 每个 v 是其子树 T_v 的最小标号；单事件概率 $=1/\text{size}(v)$（$\text{size}(v)$ 个位置地位对等）。这些事件**相互独立**——事件「v 是 T_v 最小」只看 T_v 内部相对顺序，不同子树的相对顺序在均匀随机下互不影响。故合法概率 $\prod 1/\text{size}(v)$，方案数 $=n!\cdot\prod 1/\text{size}(v)$。■



图注：$p=[3,1,4,2,5]$ 的笛卡尔树；橙 = 根（全局最小）；红字 = 子树 size。答案 $=5!/(5\cdot1\cdot3\cdot1\cdot1)=8$。

与笛卡尔树。若能从 a 唯一恢复 prev_smaller/next_smaller（即唯一定出笛卡尔树**形态**），则合法排列数 $=n!/\prod\text{size}_i$，其中 $\text{size}_i=\text{left}_i+\text{right}_i-1$。**例题**：CF 2234E Vlad, Misha and Two Arrays。

拓展 1：杨表 Hook-length（原版，Frame–Robinson–Thrall 1954）

分拆 $\lambda=(\lambda_1\ge\lambda_2\ge\dots)\vdash n$ 对应 Young 图；标准杨表（SYT）是把 $1,\dots,n$ 填进 Young 图使得**每行严格递增、每列严格递增**。SYT 计数

$$f^\lambda=\frac{n!}{\prod_{c\in\lambda}h(c)},\qquad h(c)=\underbrace{r(c)}_{\text{同行右侧}}+\underbrace{d(c)}_{\text{同列下方}}+1.$$

本节的树版本就是这个公式在**树**上的类比（把 Young 图换成树、$h(c)$ 换成子树大小）。CP 里出现的场合：

- n **阶置换的 RSK 对应**：$n! = \sum_{\lambda \vdash n} (f^\lambda)^2$（每个 λ 贡献 $(f^\lambda)^2$ 对 (P, Q)）。
- **长度 $\le k$ 的最长上升子序列** = 第一行长度 $\le k$ 的杨表；配合 hook-length 可数计数。
- **格路计数**：从 $(0,0)$ 到 (a,b)、不越对角线的路径数 $= f^{(b,a)}$/显式化简 = Catalan / Ballot 数（Catalan 的另一种视角）。

拓展 2：森林 / 有序 vs 无序 / q-类

- **森林版**：n 节点森林（k 棵根无序的树）的递增标号数 $= n!/\prod_v \text{size}(v)$，跟单树公式一致（等价于加一个虚根统一）。
- **有序 vs 无序树**：本公式的树是「有孩子顺序」的（如笛卡尔树左右子树有别）。若树是无序的（同构算一次），除以每个节点的孩子置换群阶 $\prod_v |\text{children}(v)|!$。
- **q-类版**：对树的**降序列**（descent）生成函数 $\sum_T q^{\text{maj}(T)} = [n]_q!/\prod[\text{size}(v)]_q$，其中 $[m]_q = (1-q^m)/(1-q)$。CP 少用但组合数学味浓，遇到"q-类比" / "maj 生成函数" 题目时可想。
- **路径 / 链退化**：若树是一条链（n 个节点从上到下），size $= n, n-1, \ldots, 1$，公式给 $n!/n! = 1$——链上只有唯一递增填法，符合直觉。
- **完全二叉树**：$n = 2^h - 1$ 层完全二叉树，$\prod$size 有闭式 $\prod_{k=0}^{h-1} (2^{h-k}-1)^{2^k}$，可直接查表验证。

4.14 Erdős–Szekeres 定理（单调子序列）

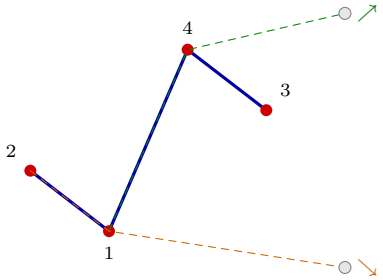
任意 n 个**互不相同**的实数构成的序列，只要长度

$$n \ge (r-1)(s-1)+1,$$

就必含一个长度为 r 的（严格）递增子序列，**或**一个长度为 s 的（严格）递减子序列。

取 $r = s = 3$：长度 $\ge (3-1)(3-1)+1 = 5$ 的序列必含长度 3 的单调子序列；反过来，**不含长度 3 单调子序列的序列最长只有 4**，临界长度恰好是 5。

鸽巢证明：给每个位置记一对数 (u, v)——u 是以它结尾的最长（非严格）递增子序列长度，v 是最长递减长度。若全程不出现长度 3 的单调子序列，则 $u, v \in \{1, 2\}$，组合只有 $2 \times 2 = 4$ 种；第 5 个位置必与前面某位置的 (u, v) 相同，而两位置标签相同会强制其中一维 $+1$，矛盾。故最长为 4。



图注：蓝 = 锯齿序列 2 1 4 3（长度 4，任取 3 个都不构成非严格单调，合法）；补第 5 个点（灰）无论放高放低，都与前两点凑出长度 3 单调子序列（绿 $1 \le 4 \le 4.6$ 递增 / 橙 $2 \ge 1 \ge 0.4$ 递减），故临界长度为 5。

应用：统计「不含长度 3 非严格单调子序列」的连续子段个数时，由本定理合法子段长度 ≤ 4，对每个右端点向左只需枚举长度 $1 \sim 4$ 暴力判定，$O(n)$ 完成。例题：牛客河南萌新联赛 2025（一）M 无聊的子序列（$n \le 10^6$）。该题是**非严格**版（允许相等），合法子段长度同样 ≤ 4；定理原版针对严格单调（元素互异），可与 Dilworth 定理、最长上升子序列对照理解。

5 博弈论

5.1 Nim 博弈

规则：桌上有 n 堆石子，第 i 堆有 $a_i \geq 0$ 颗。两人轮流操作，每回合选一堆**非空**的石子，从中拿走任意正整数颗（至少 1 颗，不能跨堆）。**无法操作**（所有堆都空）的玩家落败。问先手能否取胜。

结论（Bouton 定理）：先手胜当且仅当

$$a_1 \oplus a_2 \oplus \cdots \oplus a_n \neq 0.$$

把这个异或值称为局面的 **Nim 和**（Nim-sum）。Nim 和为零的局面是 **P 局面**（Previous，前一手获胜，即**当前轮到的人必败**）；非零是 **N 局面**（Next，当前轮到的人必胜）。

为什么是异或？三段闭合性：

1. **终止 = P**：全 0 局面 Nim 和为 0，确实无路可走、当前人输。
2. **P 走完必入 N**（封死 P 出口）：从 Nim 和 $= 0$ 出发，任何合法操作只动一堆且必使该堆严格变小，单堆变化必然改变异或——**改一堆就一定让 Nim 和从 0 变非 0**。所以 P 局面下你怎么走都把回合白送给对手。
3. **N 总能走回 P**（构造 N 通道）：从 Nim 和 $s \neq 0$ 出发，找 s 的最高位 k。s 在第 k 位为 1 意味着 a_i 中至少有一个在第 k 位为 1，挑这样的一堆 a_j，把它改为 $a_j \oplus s$。两个关键事实：(i) a_j 第 k 位被 s 翻成 0、更高位不动，所以 $a_j \oplus s < a_j$，是**合法的"减一个正数"操作**；(ii) 操作后总异或变成 $s \oplus s = 0$。因此 N 局面总能一步把 P 局面"踢回"对手。

三段闭合后再做归纳：终止 P 局面下手必败 $\Rightarrow$ 一切 P 局面下手必败（因为只能进入 N，再被对手踢回 P 直至终止）；一切 N 局面下手必胜（一步进入 P，把上述循环转嫁给对手）。

Nim 走法（构造性）：判负只是判定，但 **遇到 N 局面时必须知道"该动哪一堆、动到几"**。算法照搬上面 (3)：

```
// 返回 (堆下标 j，改后的新值)；保证 a_j 严格减小
pair<int, ll> nim_move(const vector<ll>& a) {
    ll s = 0;
    for (ll x : a) s ^= x;
    if (s == 0) return {-1, -1}; // P 局面，
    ↪ 无必胜走法
    for (int j = 0; j < (int) a.size(); ++j) {
        ll t = a[j] ^ s;
        if (t < a[j]) return {j, t}; //
        ↪ 找到含最高位的那一堆
    }
    return {-1, -1}; // 不会到这
}
```

注意"含 s 最高位"的判定可以**不显式取最高位**：直接看 $a_j \oplus s < a_j$ 是否成立——等价于 a_j 在 s 的最高位上为 1。这个写法少踩位运算坑。

推广：Sprague–Grundy 定理。对任意**公平组合游戏**（双方走法对称、有限步必终止、无随机），定义局面 u 的 **SG 值** $\mathrm{SG}(u) = \mathrm{mex}\{\mathrm{SG}(v) : u \to v\}$。则：

- 单局游戏 u 是 P 局面 $\iff \mathrm{SG}(u) = 0$。
- 多个独立游戏**并行**进行（每回合任选一局走一步），整体 SG 值 $= \bigoplus_i \mathrm{SG}(u_i)$。

Nim 单堆的 SG 值就是堆里石子数（堆 a 一步可达 $\{0, 1, \ldots, a-1\}$，mex 即 a），所以多堆 Nim 的"异或为零判 P"正是 SG 定理在 Nim 上的特化。**遇到看似复杂的博弈，先尝试拆成若干独立子游戏 + 求每个 SG + 异或起来**——下一节的"三角形博弈"就是把"非标准的单游戏"通过换元映回标准 Nim 的最简案例。

5.1.1 三角形博弈（带约束的 Nim）

给定能构成非退化三角形的三整数 a, b, c（即满足 $a + b > c$、$b + c > a$、$a + c > b$）。两人轮流操作，每回合选一个数减去任意正整数；操作后若三数无法构成非退化三角形，则该玩家落败。先手能否获胜？

结论：先手胜当且仅当

$$(a-1) \oplus (b-1) \oplus (c-1) \neq 0.$$

核心观察——把"边长"换元为"石子堆"：令 $x_i = s_i - 1$（s_i 是边长）。每条边至少为 1，对应每堆石子至少为 0；"减去任意正整数"在两套语言下完全一致。终止局面 $(1, 1, 1)$ 对应 $(0, 0, 0)$，与标准 Nim 的零局面重合。**换言之，剥掉三角约束之后，这就是三堆 Nim**。剩下要论证的只是：三角约束**不会挡掉**从 N 局面走向 P 局面的那一步（沿用上一节的 Nim 走法）。

为什么三角约束不挡 Nim 走法：从异或非零的 (a, b, c) 出发，设 $s = (a-1) \oplus (b-1) \oplus (c-1)$，找到 $(a-1, b-1, c-1)$ 中含 s 最高位的那一堆，标准 Nim 操作把它改成"它 $\oplus s$"——这是一次**严格变小**的修改。不妨设被改的是 $c-1$，新值 $c' - 1 = (c-1) \oplus s = (a-1) \oplus (b-1)$。此时 $|a-b| \leq c'-1 \leq a+b-2 < a+b$，新三角不等式自动成立（最长边假设依然成立时同理），所以"修复异或"的 Nim 走法**始终是合法着**。

反向也封死：从异或为零的 P 局面出发，任何合法操作只动一堆，异或必然变成非零；对手回到 N 局面继续兑现获胜策略。

实战提示：实现一行就够——读入三个数后输出 `((a-1)^(b-1)^(c-1)) ? "Win" : "Lose"`。范例题：QOJ 4545 Triangle Game（凯特和艾米丽科，$1 \leq a, b, c \leq 10^9$，$T \leq 10^4$，$O(1)$ 一次输出）。**推广视角**：所有"看似有附加约束的 Nim"都值得先做这套换元——**把约束的下界吸收到偏移里、把"零状态"对齐成 $(0, 0, \ldots)$、再核对"最优 Nim 走法是否被约束挡掉"，三步通吃**；本题正是 -1 偏移加"最长边可调"的最简版本。

5.1.2 Moore's Nim-K（每回合可同时动 K 堆）

规则：n 堆石子，每回合**至多选** K **堆**，每选中的堆都取走任意正整数颗（不同堆取数可以不同），**取到最后一颗者胜**。$K=1$ 即标准 Nim。

结论：把每堆数写成二进制，记第 i 位上 1 的个数为 ones_i。**先手败** $\iff$ **每个** ones_i **都是** $(K+1)$ **的倍数**。$K=1$ 退化成" 每位 ones_i 都是偶数"，即 XOR $=0$。

直觉：标准 Nim 的" 模 2 异或" 是 $K=1$ 的特例，因为每回合只能动 1 堆，每一位上的 1 个数变化量 $\in\{-1,0,+1\}$，对手可恢复奇偶。Nim-K 下变化量 $\in\{-K,\dots,K\}$（每选中堆贡献 ±1），对手可恢复" 模 $K+1$" 残差。

5.1.3 反 Nim（Anti-Nim / Misère Nim）

规则：与标准 Nim 完全相同，但**取到最后一颗者输**（拿到最后一颗的" 出局"）。

结论：**先手胜** $\iff$ **满足下列其一** ——

- 所有堆都 ≤ 1 且 Nim 和 $=0$（等价于" 全是 1 且堆数为偶数"）；
- 至少一堆 >1 且 Nim 和 $\neq 0$。

直觉：当所有堆都 ≤ 1 时游戏退化成" 轮流取走单颗"，胜负由堆数奇偶决定（misère 下偶数堆先手胜）。当存在 >1 的" 大堆" 时，先手始终能控制把局面引入" 全 1 局面，且堆数对自己有利"——具体策略类似标准 Nim，只是终态判定反过来。">1 堆 + Nim 和非零" 是先手能掌控终态奇偶的标志。

5.1.4 阶梯 Nim

规则：n 级台阶（编号 $1\dots n$），第 i 级有 a_i 颗石子。每回合从某级 $i\ge 1$ 拿任意正数颗放到第 $i-1$ 级（第 1 级的石子被放到地面 0 级，地面石子不能再动），**取走最后一颗者胜**（即让对手面对" 无可动石子" 局面者胜）。

结论：**只看奇数级台阶上的石子做标准 Nim**——先手败 $\iff a_1\oplus a_3\oplus a_5\oplus\cdots=0$。

直觉——" 偶数级是镜子"：偶数级上的动作可以被对手**原样模仿**——你从第 $2k$ 级搬 x 颗到 $2k-1$ 级，对手就从 $2k-1$ 级搬同样 x 颗到 $2k-2$ 级，把你" 白送" 到偶数级的石子继续往下推，整体偶数级局面回到等价状态。所以**偶数级上的动作不影响胜负**，等价 Nim 只发生在奇数级。

5.1.5 Lasker's Nim（可拆分的 Multi-SG）

规则：n 堆，每回合二选一——**(a)** 从某堆取若干颗（标准 Nim 操作）；**(b)** 把某堆 ≥ 2 颗的拆成两堆非空。取到最后一颗者胜。

结论：单堆 SG 函数为

$$\mathrm{SG}(x)=\begin{cases}x-1, & x \bmod 4=0\\ x, & x \bmod 4\in\{1,2\}\\ x+1, & x \bmod 4=3\end{cases}$$

多堆答案 $=\bigoplus_i \mathrm{SG}(a_i)$，非零时先手胜。

打表速记：$x=0,1,2,3,4,5,6,7$ 时 SG 为 $0,1,2,4,3,5,6,8$——**每 4 个一组、组内除了 $4k$ 与 $4k+3$ 互换以外都是恒等**。背不住时直接用通用 SG 暴力打表（见下节）现场跑。

5.2 巴什博弈（单堆有上限取）

规则：单堆 N 颗石子，每回合取 $1\sim M$ 颗，**取到最后一颗者胜**。

结论：先手败 $\iff N \bmod (M+1)=0$。

为什么是 $M+1$ 周期——" 凑齐 $M+1$" 的控制术：双方可以**稳定地**把每一回合的总取数控制为 $M+1$（一方取 $X\in[1,M]$，另一方立即取 $M+1-X$，二者必然合法）。所以 N **被** $M+1$ **整除的局面是 P 局面**：当前人无论怎么取，对手都能" 补齐成 $M+1$"，让 N 严格按 $M+1$ 递减直到 0（最后一颗永远被对手拿走）。N 不被整除时，先手取走余数 $R=N \bmod (M+1)$ 把局面调成整除留给对手，化身" 控盘者" 反将一军。

扩展巴什博弈：每回合取走 $X\in[a,b]$ 颗，剩余不足 a 时不能再取，**取到最后一颗者胜**（注意：剩余 $R<a$ 时游戏自然结束，" 最后一颗" 指 R 中是否含取数边界——按惯例，无法再取的当前玩家算作" 未取到最后一颗"，即输家）。

结论：以 $a+b$ 为周期划分，先手败 $\iff N \bmod (a+b)<a$。

$a+b$ **周期的直觉**：把上面" 凑齐 $M+1$" 换成" 凑齐 $a+b$"——一方取 $X\in[a,b]$，对手取 $a+b-X\in[a,b]$ 始终合法。余数 $R\in[0,a)$ 是 P 局面：先手取走 $X\ge a$ 后剩下 $N-X$，对手补成 $a+b$ 周期把局面继续锁在余数 $<a$ 区段，最终把 $<a$ 的" 垃圾尾巴" 留给先手取不动。余数 $R\in[a,b]$ 时先手直取 R；余数 $R\in(b,a+b)$ 时先手取 $X=R-(a-1)\in(b-a+1,b]$（边界容易验合法），把局面调到余数 $a-1$ 留给对手。

5.3 SG 函数的暴力打表与 Anti / Every 变体

通用 SG 暴力（适合状态数 $\le 10^6$ 的题目，没法找闭式时打表观察规律）：

```cpp
// 单堆从 x 颗一步可达的下一状态集合由 trans(x) 给出
// SG(x) = mex{ SG(y) : y in trans(x) }
int sg[N];
void compute_sg(int upTo) {
    for (int x = 0; x <= upTo; ++x) {
        vector<int> nxt = trans(x);
        vector<bool> seen(nxt.size() + 1, false);
        for (int y : nxt) if (sg[y] < (int) seen.size()) seen[sg[y]] = true;
        int m = 0; while (m < (int) seen.size() && seen[m]) ++m;
        sg[x] = m;
    }
}
```

用法：把所有子游戏的 SG 异或起来；非零先手胜。**打表观察规律**是博弈题最常用的找通项手段——先暴力

跑出 SG(0 . . . 100) 看是否周期 / 等于 x / 等于 $\lfloor x/k \rfloor$ 之类的简单形式，再去证。

5.3.1 反 SG（Anti-SG）

规则：与 SG 游戏一致，但**无法行动者胜**（最先不能动者赢）。

结论（与反 Nim 同形）：先手胜 $\iff$ 满足其一——

- 所有子游戏 SG ≤ 1 且总 XOR $= 0$；
- 至少一个子游戏 SG > 1 且总 XOR $\neq 0$。

5.3.2 Every-SG（每回合走”所有还能动的石子”）

规则：DAG 上若干石子，每回合**必须把所有还能动的石子各走一步**（不能选择不动），无法行动者输。

结论：对每个起点求 step(v)——**在该子游戏中博弈能持续的回合数**（必胜方尽量缩短、必败方尽量拖长）：

- 终止节点：step $= 0$。
- SG$(v) > 0$（必胜）：step$(v) = 1 + \min\{\text{step}(w) : v \to w, \text{SG}(w) = 0\}$。
- SG$(v) = 0$（必败）：step$(v) = 1 + \max\{\text{step}(w) : v \to w\}$。

先手胜 $\iff$ 各子游戏 step 的最大值是奇数。直觉：所有子游戏**并行推进**，最长那局决定整盘何时结束；该局轮到谁动谁就输，奇偶反转后对应先手胜。

5.3.3 实战范例：CCPC 2025 网络赛 F「连线博弈」——XOR 哈希切子游戏

题意（第十一届 CCPC 网络预选赛 F）：单位圆周上等距 n 个点 ($2 \le n \le 10^9$)，初始有 m 条两两**严格不交且不共端点**的弦 ($0 \le m \le 10^5$, $\sum m \le 10^5$)。Alice/Bob 轮流操作，每回合选**两个未被任何弦端点占用**的”自由点”画一条不与已有弦相交的新弦，无法操作者输。问 Alice 是否必胜。

子游戏分解：每条弦把圆盘切成两半，自由点被弦端点序列划分成若干**独立子游戏**。x 个自由点的子游戏一步操作占掉 2 个点，并把段切成大小为 a, b ($a + b = x - 2$) 的两个新子游戏。故

$$\text{SG}(x) = \max_{a+b=x-2} \big(\text{SG}(a) \oplus \text{SG}(b)\big), \quad \text{SG}(0) = \text{SG}(1) = 0.$$

暴力打表至 500 后发现 $x \ge 36$ 起呈周期 34，但 idx $= 52$ **是异常项**（sg[52] $= 2$，但 $x \ge 70$ 落到 idx $= 52$ 的真实周期值是 sg[86] $= 9$），**必须特判**，否则会被 hack 数据 $n = 8$, $m = 2$, $(0, 1)(4, 5)$ 卡掉（朴素周期答 NO，正解 YES）。**教训**：找到看似规整的周期**务必本地用暴力对拍校验首尾几百项**，不要看一眼”前几行循环对了”就上交。

XOR 哈希切子游戏（关键技巧）：题面要求”识别哪些自由点共属一个子游戏”，传统做法（按弦端点排序，相邻端点之间一段算一个子游戏）会把”两条互不相交的弦同时切到的中间区段”误判为多个独立段。**用随机 XOR 标记代替几何判定**——给每条弦 (x, y) 生成随机权 r，记 mp$[x]$ ^= r、mp$[y]$ ^= r。按下标升序扫描端点，维护前缀异或 cur；cur **相同的下标段就属于同一子游戏**（因为弦的两个端点同时翻转 cur 两次，段内自由点的 cur 等于”包住它的所有未闭合弦”的随机权异或，几何上”被同一组弦包裹”等价于”在同一子游戏”）。最后对每个 cur 桶累加自由点数，再异或所有 SG(桶大小)。

```cpp
// 预处理 SG 至 500
sg[0] = sg[1] = 0;
for (int i = 2; i <= 500; ++i) {
    set<int> s;
    for (int j = 0; j <= i - 2; ++j)
    ↪  s.insert(sg[j] ^ sg[i - 2 - j]);
    int mex = 0; while (s.count(mex)) ++mex;
    sg[i] = mex;
}

// 周期 34（从 i=36 起），idx==52 异常项特判
auto SG = [&](ll x) -> int {
    if (x <= 200) return sg[x];
    int idx = 36 + (x - 36) % 34;
    return idx == 52 ? sg[idx + 34] : sg[idx];
};

// 单组询问：XOR 哈希 + 扫描分桶
void solve() {
    cin >> n >> m;
    map<ll, ll> mp;            // 端点 → 该点累计的随机权异或
    mt19937_64 rng(time(0));
    for (int i = 0; i < m; ++i) {
        ll x, y; cin >> x >> y;
        ll r = rng();
        mp[x] ^= r; mp[y] ^= r;   // 弦两端各异或同一个 r
    }
    if (m == 0) { cout << (SG(n) ? "YES\n" : "NO\n"); return; }

    map<ll, ll> bucket;        // cur → 该组子游戏的自由点总数
    ll up = -1, cur = 0;
    mp[n] = 0;                 // 末尾哨兵：把最大端点之后到 n-1 的尾段也结算
    for (auto& [u, v] : mp) {
        bucket[cur] += max(0LL, u - up - 1);  // [up+1, u-1] 的自由点数
        up = u; cur ^= v;
    }
    ll ans = 0;
    for (auto& [_, sz] : bucket) ans ^= SG(sz);
    cout << (ans ? "YES\n" : "NO\n");
}
```

为什么 XOR 哈希这一招通用：本质是**用差分维护”被哪些区间覆盖”**——每条弦 (x, y) 的”覆盖范围”在排序后的端点序列上是一段，而 mp[x] ^= r; mp[y] ^= r; 正是**异或差分**（cur 在 x 之后翻入 r，在 y 之后翻出 r，$[x, y)$ 段的 cur 多含 r）。任何”按区间覆盖签名分组”的题都可套用这套：把”判断两个位置是否属于同一组”翻译成”它们的 cur 哈希是否相等”，免掉显式区间扫线 / 并查集合并。**相关题**：HDU/CodeForces

上” 求被不同区间集合覆盖的位置数 / 等价类数” 系列均可一键套用。

5.4 威佐夫博弈（双堆联动）

规则：两堆石子 (a,b)（设 $a \le b$）。每回合二选一——**(a)** 从单堆取任意正数颗；**(b)** 从两堆同时取走**相同**的正数颗。取到最后一颗者胜。

结论：奇异（P）局面是 $(a_k,b_k)=(\lfloor k\varphi\rfloor,\lfloor k\varphi\rfloor+k)$，$k=0,1,2,\dots$，其中 $\varphi=\frac{1+\sqrt{5}}{2}$ 是黄金分割比。即 $(0,0),(1,2),(3,5),(4,7),(6,10),(8,13),\dots$。**当前 (a,b) 是 P 局面** $\iff \lfloor (b-a)\cdot\varphi\rfloor = a$。

Beatty 定理背景：两序列 $\{\lfloor k\varphi\rfloor\}$ 与 $\{\lfloor k\varphi^2\rfloor\}$ 由 Beatty 定理给出”$\mathbb{N}^+$ 的不交划分”，恰好把每个正整数唯一指派为某个 a_k 或 $b_k=a_k+k$。这保证了奇异局面集合” 覆盖且只覆盖一次”。

大数精度警告：$N \le 10^9$ 时 `double` 误差刚好够呛，建议直接用高精度常数 $\varphi \approx 1.6180339887498948482$，或直接打表黄金比的两个 Beatty 序列（项数 $\le 10^9/\varphi \approx 6\times10^8$ 不行，所以现场公式判定）：

```
const double PHI = 1.6180339887498948482L;
bool first_loses(long long a, long long b) {
    if (a > b) swap(a, b);
    return (long long)((b - a) * PHI) == a;
}
```

5.5 斐波那契博弈（前手两倍约束）

规则：单堆 N 颗。先手第 1 回合可取 $1 \sim N-1$ 颗（不能一次拿光），此后每回合的取数上限为**上一回合所取数的 2 倍**。取到最后一颗者胜。

结论：**先手败** $\iff$ **N 是斐波那契数**（$N \in \{1,2,3,5,8,13,21,\dots\}$）。

直觉——齐肯多夫（Zeckendorf）分解：每个非斐波那契数都能唯一拆成**若干不相邻**斐波那契数之和 $N=F_{k_1}+F_{k_2}+\cdots+F_{k_t}$（$k_1>k_2>\cdots>k_t$，$k_i-k_{i+1}\ge 2$）。先手取走最小项 F_{k_t}，由 Fibonacci 递推 $F_{k_{t-1}}>2F_{k_t}$，对手无法一次拿完下一项 $F_{k_{t-1}}$ 段，必然被迫” 破坏” 它，先手再取该段最小项继续逼迫……层层剥到只剩 F_{k_1}，此时正好对手取一颗、先手取一颗交替到底。N 本身是斐波那契数时分解只有一项，先手第一手就被” 取数 $\le N-1$ 不能一次拿完 + 任取必让对手反吃 $\le 2X$ 区间” 的双重约束封死。

```
bool first_loses_fib(long long n) {
    long long a = 1, b = 2;
    while (b <= n) { long long c = a + b; a =
    ↪  b; b = c; }
    // 此时 a 是 <= n 的最大 Fibonacci
    return a == n;
}
```

5.6 图上删边游戏

5.6.1 树上删边（Colon Principle）

规则：有根树，每回合删一条边，与根失去连接的部分整体被删除（即删去子树）。**无边可删者输**。

SG 公式（Colon Principle，Smith 定理）：以节点 v 为根的子树 SG 值 $g(v)$ 满足

$$g(\text{叶子})=0,\qquad g(v)=\bigoplus_{c\ \text{是}v\ \text{的孩子}} \big(g(c)+1\big).$$

注意 $+1$ 是**普通整数加法**（含义：” 孩子子树的 SG 等价为一堆 $g(c)$ 颗石子，挂上连父边后再加一颗”），异或才是组合多个独立分支。

先手胜 $\iff$ **$g(\text{根})\neq 0$**（与 SG 通则一致）。

```
// 返回以 u 为根的子树 SG 值
function<int(int, int)> dfs = [&](int u, int
↪  fa) -> int {
    int s = 0;
    for (int v : adj[u]) if (v != fa) s ^=
    ↪  (dfs(v, u) + 1);
    return s;
};
cout << (dfs(root, -1) != 0 ? "Win\n" :
↪  "Lose\n");
```

5.6.2 无向图删边（Fusion Principle）

规则：连通无向图（可含环、重边、自环），指定一个根，每回合删一条边，与根失联的部分整体删除。无边可删者输。

Fusion Principle（环约简 $\to$ 化为树上删边）：

- **奇环**（含奇数条边的环，含自环）：缩成**一个新点 + 一条挂在它上面的悬空边**。
- **偶环**：缩成**一个新点**（不留多余边）。
- 原本连接到环上任意点的所有边，全部接到这个新缩出来的点上。

反复约简到没有环为止，剩下的就是一棵树，套用上一节的 Colon Principle 求 SG。

直觉：环上任意一条边被删后剩下一条路径——” 环作为整体” 的 SG 等于” 等价路径的 SG”，奇数 / 偶数环的等价物正好是”1 条边” / ”0 条边”（这是 Hackenbush 经典结论）。所以缩点保留这条等价残留即可。

6 进阶数学

6.1 FFT 多项式乘法（非递归）

采用非递归版本实现 FFT（蝴蝶变换），能显著减小常数并优化内存使用。这里通过 `multiply` 演示了如何运用 `fft` 对两个多项式（系数数组表示）进行乘法操作。

```
// 理论讲解
↪ https://www.bilibili.com/video/BV1za411F76U/
// 参考（抄的）
↪ https://cp-algorithms.com/algebra/fft.html
// 多项式乘法
↪ https://www.luogu.com.cn/record/229525769
using CD = complex<double>;

void fft(vector<CD> &a, bool invert) {
  int n = a.size();
  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len <<= 1) {
    double ang = 2 * pi / len * (invert ? -1 :
    ↪ 1);
    CD wlen(cos(ang), sin(ang));
    for (int i = 0; i < n; i += len) {
      CD w(1);
      for (int j = 0; j < len / 2; j++) {
        CD u = a[i + j], v = a[i + j + len / 2]
        ↪ * w;
        a[i + j] = u + v;
        a[i + j + len / 2] = u - v;
        w *= wlen;
      }
    }
  }
  if (invert) {
    for (CD &x : a) x /= n;
  }
}

vector<ll> multiply(vector<ll> &a, vector<ll>
↪ &b) {
  vector<CD> fa(a.begin(), a.end()),
  ↪ fb(b.begin(), b.end());
  int n = 1;
  while (n < (int)(a.size() + b.size())) {
    n <<= 1;
  }
  fa.resize(n); fb.resize(n);
  fft(fa, false); fft(fb, false);
  for (int i = 0; i < n; i++) fa[i] *= fb[i];
  fft(fa, true);
  vector<ll> res(n);
  // 避免精度损失
  for (int i = 0; i < n; i++) res[i] =
  ↪ llround(fa[i].real());
  return res;
}
```

6.1.1 高精度乘法（基于 FFT）

https://www.luogu.com.cn/record/229526218

大整数乘法的核心思想：将大整数的每一位看作多项式的一个系数。例如 1234 可以表示为多项式 $4 + 3x + 2x^2 + x^3$（低位在前）。两个大整数相乘等价于两个多项式相乘，乘完后对结果数组统一处理进位。

```
inline void __() {
  string A, B;
  cin >> A >> B;
  vector<ll> a, b;
  for (int i = (int)A.size() - 1; i >= 0; --i) {
    a.push_back(A[i] - '0');
  }
  for (int i = (int)B.size() - 1; i >= 0; --i) {
    b.push_back(B[i] - '0');
  }

  vector<ll> ans = multiply(a, b);
  ll carry = 0;
  for (int i = 0; i < (int)ans.size(); ++i) {
    ans[i] += carry;
    carry = ans[i] / 10;
    ans[i] %= 10;
  }
  while (!ans.empty() && ans.back() == 0) {
    ans.pop_back();
  }
  if (ans.empty()) {
    cout << 0;
    return;
  }
  for (int i = (int)ans.size() - 1; i >= 0;
  ↪ --i) {
    cout << ans[i];
  }
}
```

6.1.2 FFT 的使用方法与常见应用

`multiply(a, b)` 的本质是:给定多项式 $A(x) = \sum a_i x^i$ 和 $B(x) = \sum b_i x^i$, 返回乘积多项式 $C(x) = A(x) \cdot B(x)$ 的系数数组, 其中 $c_k = \sum_{i+j=k} a_i \cdot b_j$。因此, **只要问题能转化为这种卷积形式, 就能用 FFT 加速到** $O(n \log n)$。

1. 加法卷积计数

最典型的场景：有两组数，问它们各取一个数相加能得到每个和的方案数。

例如掷两颗骰子，求每种点数之和的方案数。将骰子 A 的各面出现次数存入 `a[]`（`a[k]` 表示点数 k 出现的次数），骰子 B 同理存入 `b[]`，则 `multiply(a, b)` 的结果 `c[k]` 就是和为 k 的方案数。

```
// 例：两个数组各取一个数，统计每种和的方案数
// a[i] = 数 i 在第一组中出现的次数
// b[j] = 数 j 在第二组中出现的次数
// c[k] = 和为 k 的方案数 = sum(a[i]*b[k-i])
vector<ll> c = multiply(a, b);
```

2. 字符串匹配（含通配符）

给定文本串 T（长度 n）和模式串 P（长度 m），对于 T 的每个起始位置 t，定义失配度：

$$D(t) = \sum_{j=0}^{m-1} (T_{t+j} - P_j)^2 \cdot w_j$$

其中 w_j 是通配符权重（通配符处 $w_j = 0$）。$D(t) = 0$ 表示位置 t 完全匹配。

将 P 翻转为 $P'_j = P_{m-1-j}$，展开后各项都可化为卷积形式，分别用 FFT 计算后合并。

```
// 无通配符的简单情形：检查 T 和 P
↪  是否在每个位置匹配
// 构造 a[i] = T[i], b[j] = P[m-1-j]（翻转 P）
// 则 c[t+m-1] = sum(T[t+j] * P[m-1-j]) 即为位置
↪  t 的内积
// 更一般地，含通配符时需要构造多组卷积分别计算
vector<ll> a(n), b(m);
for (int i = 0; i < n; i++) a[i] = T[i] - 'a' +
↪  1;
for (int j = 0; j < m; j++) b[j] = P[m - 1 - j]
↪  - 'a' + 1;
vector<ll> c = multiply(a, b);
// c[t + m - 1] 即为位置 t 的匹配内积
```

3. 高精度乘法

如上一节所述，将大整数每一位作为多项式系数，调用 `multiply` 后统一处理进位。

4. 多项式求值与插值的中间步骤

分治 FFT 可以加速多项式多点求值、多点插值等操作，核心都是递归地调用 `multiply` 合并子问题的结果多项式。

6.2 NTT 卷积（ac-library 精简移植）

洛谷 P3803 多项式乘法

本节到「FWT 与子集卷积」为止的**全部代码**，都在 `template-check/src/poly_check.cpp` 里与朴素实现随机对拍过（装了 ac-library 时还会与官方 NTT 逐点比对）；改动其中任何一段后请重跑它。

上一节的复数 FFT 只能算整数卷积，一旦要在模 998244353 意义下做多项式运算就得换成 NTT。本节直接移植 ac-library 的 `convolution.hpp`，它是目前公开实现里常数最小的一档，理由有三：

- **radix-4 蝶形**：一次推进两层，乘法次数比常见的 radix-2 少约三成；
- **不需要位逆序置换数组**：正变换故意把结果留在位逆序上，逆变换再吃回去，于是那个 `rev` 数组和它的预处理全都省掉了；
- **小规模自动降级**：短的一侧不超过 60 时直接朴素乘，避开两次变换的固定开销——多项式求逆、`exp` 这类倍增算法会反复在很短的长度上调用卷积，这个降级的收益比想象中大。

代价是 `butterfly` 的输出不是自然序。只做点乘的场景（卷积、转置卷积、多点求值）完全不受影响；但如果哪天想在频域上「按下标做事」（比如手动取某一位的单位根），必须先想清楚下标已经被位逆序打乱了。

模数与原根：NTT 要求模数形如 $c \cdot 2^k + 1$，且 2^k 不小于变换长度。常用的几个如下（`rk` 即 k，决定了最长能做多长）：

模数	原根	rk	备注
998244353	3	23	最常用，$= 119 \cdot 2^{23} + 1$
1004535809	3	21	备用，与上者互质
167772161	3	25	三模数之一
469762049	3	26	三模数之一
754974721	**11**	24	三模数之一，**原根不是 3**
2281701377	3	27	超 `int` 范围，乘法要用 `__int128`

最后一行的坑很值得记：754974721 的原根是 11，抄板子时顺手写成 3，单模数的题照样能过（因为根本没用到它），一上三模数就会 WA 到怀疑人生。所以下面的模板把原根做成了**编译期自动求解**的默认模板参数，换模数时不用查表，也就没有写错的机会。

```
// ===== NTT 卷积：ac-library atcoder/convolution.hpp 精简移植 =====
// 参考（抄的） https://github.com/atcoder/ac-library/blob/master/atcoder/convolution.hpp
// 模板题 https://www.luogu.com.cn/problem/P3803
// 编译期快速幂 + 最小原根：换模数时不用查表，也就不会写错原根
constexpr ull powConst(ull b, ull e, ull m) {
    ull r = 1;
    for (b %= m; e; e >>= 1, b = b * b % m)
        if (e & 1) r = r * b % m;
    return r;
}
constexpr uint32_t primitiveRoot(uint32_t m) {
    uint32_t divs[20] = {2}, x = (m - 1) / 2;
    int cnt = 1;
    while (x % 2 == 0) x /= 2;
    for (uint32_t i = 3; (ull)i * i <= x; i += 2)
        if (x % i == 0) {
            divs[cnt++] = i;
            while (x % i == 0) x /= i;
        }
    if (x > 1) divs[cnt++] = x;
    for (uint32_t g = 2;; g++) {
        bool ok = true;
        for (int i = 0; i < cnt && ok; i++)
            ok = powConst(g, (m - 1) / divs[i], m) != 1;
        if (ok) return g;
    }
}

// MOD 必须形如 c * 2^k + 1（k 要够大），G 默认取最小原根
```

```cpp
template <uint32_t MOD, uint32_t G = primitiveRoot(MOD)>
struct NTT {
    using mint = modint<uint32_t, MOD>;
    static constexpr int rk = __builtin_ctz(MOD - 1);   // 最长支持 2^rk
    struct Info {
        mint root[rk + 1], iroot[rk + 1];               // root[i]^(2^i) == 1
        mint rate2[rk - 1], irate2[rk - 1];
        mint rate3[rk - 2], irate3[rk - 2];
        Info() {
            root[rk] = mint::powmod(mint(G), (MOD - 1) >> rk);
            iroot[rk] = mint::inv(root[rk]);
            for (int i = rk - 1; i >= 0; i--) {
                root[i] = root[i + 1] * root[i + 1];
                iroot[i] = iroot[i + 1] * iroot[i + 1];
            }
            mint prod = 1, iprod = 1;
            for (int i = 0; i + 2 <= rk; i++) {
                rate2[i] = root[i + 2] * prod;
                irate2[i] = iroot[i + 2] * iprod;
                prod *= iroot[i + 2], iprod *= root[i + 2];
            }
            prod = 1, iprod = 1;
            for (int i = 0; i + 3 <= rk; i++) {
                rate3[i] = root[i + 3] * prod;
                irate3[i] = iroot[i + 3] * iprod;
                prod *= iroot[i + 3], iprod *= root[i + 3];
            }
        }
    };
    static const Info &info() { static Info x; return x; }

    // 正变换：一次推两层（radix-4），出来是位逆序，因此不需要 rev 置换数组
    static void butterfly(vector<mint> &a) {
        const Info &w = info();
        int n = a.size(), h = __builtin_ctz((unsigned)n), len = 0;
        while (len < h) {
            if (h - len == 1) {                         // 只剩一层，走 radix-2
                int p = 1 << (h - len - 1);
                mint rot = 1;
                for (int s = 0; s < (1 << len); s++) {
                    int off = s << (h - len);
                    for (int i = 0; i < p; i++) {
                        mint l = a[i + off], r = a[i + off + p] * rot;
                        a[i + off] = l + r, a[i + off + p] = l - r;
                    }
                    if (s + 1 != (1 << len))
                        rot *= w.rate2[__builtin_ctz(~(unsigned)s)];
                }
                len++;
            } else {                                    // radix-4
                int p = 1 << (h - len - 2);
                mint rot = 1, imag = w.root[2];
                for (int s = 0; s < (1 << len); s++) {
                    mint rot2 = rot * rot, rot3 = rot2 * rot;
                    int off = s << (h - len);
                    for (int i = 0; i < p; i++) {
                        ull m2 = (ull)MOD * MOD;
                        ull a0 = a[i + off].val();
                        ull a1 = (ull)a[i + off + p].val() * rot.val();
                        ull a2 = (ull)a[i + off + 2 * p].val() * rot2.val();
                        ull a3 = (ull)a[i + off + 3 * p].val() * rot3.val();
                        ull t = (ull)mint(a1 + m2 - a3).val() * imag.val();
                        ull na2 = m2 - a2;
```

```
                    a[i + off] = a0 + a2 + a1 + a3;
                    a[i + off + p] = a0 + a2 + (2 * m2 - (a1 + a3));
                    a[i + off + 2 * p] = a0 + na2 + t;
                    a[i + off + 3 * p] = a0 + na2 + (m2 - t);
                }
                if (s + 1 != (1 << len))
                    rot *= w.rate3[__builtin_ctz(~(unsigned)s)];
            }
            len += 2;
        }
    }
}
// 逆变换：吃位逆序输入，还原自然序（未除以 n，交给 conv 统一处理）
static void butterflyInv(vector<mint> &a) {
    const Info &w = info();
    int n = a.size(), h = __builtin_ctz((unsigned)n), len = h;
    while (len) {
        if (len == 1) {
            int p = 1 << (h - len);
            mint irot = 1;
            for (int s = 0; s < (1 << (len - 1)); s++) {
                int off = s << (h - len + 1);
                for (int i = 0; i < p; i++) {
                    mint l = a[i + off], r = a[i + off + p];
                    a[i + off] = l + r;
                    a[i + off + p] =
                        (ull)((uint32_t)(l.val() - r.val()) + MOD) * irot.val();
                }
                if (s + 1 != (1 << (len - 1)))
                    irot *= w.irate2[__builtin_ctz(~(unsigned)s)];
            }
            len--;
        } else {
            int p = 1 << (h - len);
            mint irot = 1, iimag = w.iroot[2];
            for (int s = 0; s < (1 << (len - 2)); s++) {
                mint irot2 = irot * irot, irot3 = irot2 * irot;
                int off = s << (h - len + 2);
                for (int i = 0; i < p; i++) {
                    ull a0 = a[i + off].val(), a1 = a[i + off + p].val();
                    ull a2 = a[i + off + 2 * p].val();
                    ull a3 = a[i + off + 3 * p].val();
                    ull t = mint((MOD + a2 - a3) * iimag.val()).val();
                    a[i + off] = a0 + a1 + a2 + a3;
                    a[i + off + p] = (a0 + (MOD - a1) + t) * irot.val();
                    a[i + off + 2 * p] =
                        (a0 + a1 + (MOD - a2) + (MOD - a3)) * irot2.val();
                    a[i + off + 3 * p] =
                        (a0 + (MOD - a1) + (MOD - t)) * irot3.val();
                }
                if (s + 1 != (1 << (len - 2)))
                    irot *= w.irate3[__builtin_ctz(~(unsigned)s)];
            }
            len -= 2;
        }
    }
}
static vector<mint> naive(const vector<mint> &a, const vector<mint> &b) {
    vector<mint> c(a.size() + b.size() - 1);
    for (size_t i = 0; i < a.size(); i++)
        for (size_t j = 0; j < b.size(); j++) c[i + j] += a[i] * b[j];
    return c;
}
```

```
    // 短的一侧不超过 60 时直接朴素乘，省掉两次变换的常数
    static vector<mint> conv(vector<mint> a, vector<mint> b) {
        int n = a.size(), m = b.size();
        if (!n || !m) return {};
        if (min(n, m) <= 60) return naive(a, b);
        int z = 1;
        while (z < n + m - 1) z <<= 1;
        assert((MOD - 1) % z == 0);   // 长度超过 2^rk：换模数或走三模数版本
        a.resize(z), b.resize(z);
        butterfly(a), butterfly(b);
        for (int i = 0; i < z; i++) a[i] *= b[i];
        butterflyInv(a);
        a.resize(n + m - 1);
        mint iz = mint::inv(mint(z));
        for (auto &x : a) x *= iz;
        return a;
    }
};

constexpr uint32_t P = 998244353;   // 原根 3，rk = 23（最长做到 2^23）
using Z = modint<uint32_t, P>;
vector<Z> convolution(const vector<Z> &a, const vector<Z> &b) {
    return NTT<P>::conv(a, b);
}
```

用法：`convolution(a, b)` 返回 $c_k = \sum_{i+j=k} a_i b_j$，长度是 `a.size() + b.size() - 1`。依赖基础数学一章的自动取模类 `modint`。

```
vector<Z> a{1, 2, 3}, b{4, 5};        // (1+2x+3x^2)(4+5x)
vector<Z> c = convolution(a, b);      // c = {4, 13, 22, 15}
// 换模数只要改这两行，原根会自己算出来：
// constexpr uint32_t P = 1004535809;
// using Z = modint<uint32_t, P>;
```

6.3 多项式全家桶（Poly）

把系数数组包成一个继承自 `vector<Z>` 的结构体，所有操作都建立在上面那个 `convolution` 之上。约定：下标即次数，`f[0]` 是常数项；所有带参数 `m` 的运算都只保证**前 m 项正确**（幂级数运算本来就是在 $\bmod x^m$ 意义下做的）。

基础部分：取前 k 项、平移、加减、数乘、求导、积分。`get(i)` 越界返回 0，这样加减法不用先对齐长度。

```
// ===== 多项式全家桶：系数下标从低到高，f = a[0] + a[1] x + a[2] x^2 + ... =====
struct Poly : vector<Z> {
    using vector<Z>::vector;
    Poly(const vector<Z> &a) : vector<Z>(a) {}
    Z get(int i) const { return i < 0 || i >= (int)size() ? Z(0) : (*this)[i]; }
    int deg() const {               // 最高非零次数，零多项式返回 -1
        int i = (int)size() - 1;
        while (i >= 0 && (*this)[i] == Z(0)) i--;
        return i;
    }
    Poly &norm() { resize(deg() + 1); return *this; }
    Poly trunc(int k) const {       // 取前 k 项，不足补零
        Poly f = *this;
        f.resize(k);
        return f;
    }
    Poly shift(int k) const {       // 乘上 x^k，k 为负则右移（丢掉低位）
        if (k >= 0) {
            Poly f = *this;
            f.insert(f.begin(), k, Z(0));
            return f;
```

```
        }
        if ((int)size() <= -k) return Poly();
        return Poly(begin() - k, end());
    }
    friend Poly operator+(const Poly &a, const Poly &b) {
        Poly c(max(a.size(), b.size()));
        for (int i = 0; i < (int)c.size(); i++) c[i] = a.get(i) + b.get(i);
        return c;
    }
    friend Poly operator-(const Poly &a, const Poly &b) {
        Poly c(max(a.size(), b.size()));
        for (int i = 0; i < (int)c.size(); i++) c[i] = a.get(i) - b.get(i);
        return c;
    }
    friend Poly operator-(const Poly &a) {
        Poly c = a;
        for (auto &x : c) x = -x;
        return c;
    }
    friend Poly operator*(const Poly &a, Z k) {
        Poly c = a;
        for (auto &x : c) x *= k;
        return c;
    }
    friend Poly operator*(Z k, const Poly &a) { return a * k; }
    friend Poly operator*(const Poly &a, const Poly &b) {
        return Poly(convolution(a, b));
    }
    Poly &operator+=(const Poly &b) { return *this = *this + b; }
    Poly &operator-=(const Poly &b) { return *this = *this - b; }
    Poly &operator*=(const Poly &b) { return *this = *this * b; }
    Poly &operator*=(Z k) { return *this = *this * k; }
```

牛顿迭代五件套。它们的骨架是同一个：设已经求出 $\bmod x^k$ 的解 x_0，要把精度翻倍到 $\bmod x^{2k}$，就对方程 $G(x)=0$ 做一步牛顿法

$$x \equiv x_0 - \frac{G(x_0)}{G'(x_0)} \pmod{x^{2k}}.$$

代入不同的 G 就得到不同的运算：求逆取 $G(x)=1/x-f$，开方取 $G(x)=x^2-f$。因为每一步的规模翻倍，总复杂度是 $T(n)=T(n/2)+O(n\log n)=O(n\log n)$——**和一次卷积同阶**，这也是为什么多项式科技能在 10^5 级别的题上跑得动。

```
    Poly deriv() const {           // 求导
        if (empty()) return Poly();
        Poly f(size() - 1);
        for (int i = 1; i < (int)size(); i++) f[i - 1] = (*this)[i] * Z(i);
        return f;
    }
    Poly integr() const {          // 积分，常数项取 0
        Poly f(size() + 1);
        for (int i = 0; i < (int)size(); i++) f[i + 1] = (*this)[i] / Z(i + 1);
        return f;
    }
    // 乘法逆，要求 a[0] != 0    模板题 https://www.luogu.com.cn/problem/P4238
    Poly inv(int m) const {
        Poly x{Z::inv((*this)[0])};
        for (int k = 1; k < m; k <<= 1)
            x = (x * (Poly{Z(2)} - trunc(k << 1) * x)).trunc(k << 1);
        return x.trunc(m);
    }
    // ln，要求 a[0] == 1        模板题 https://www.luogu.com.cn/problem/P4725
    Poly log(int m) const { return (deriv() * inv(m)).integr().trunc(m); }
    // exp，要求 a[0] == 0       模板题 https://www.luogu.com.cn/problem/P4726
```

```
    Poly exp(int m) const {
        Poly x{Z(1)};
        for (int k = 1; k < m; k <<= 1)
            x = (x * (Poly{Z(1)} - x.log(k << 1) + trunc(k << 1))).trunc(k << 1);
        return x.trunc(m);
    }
    // 开方，这里只处理 a[0] == 1；a[0] 是别的二次剩余时先解出 sqrt(a[0]) 再整体缩放
    // 模板题 https://www.luogu.com.cn/problem/P5205
    Poly sqrt(int m) const {
        Poly x{Z(1)};
        Z inv2 = Z::inv(Z(2));
        for (int k = 1; k < m; k <<= 1)
            x = (x + (trunc(k << 1) * x.inv(k << 1)).trunc(k << 1)) * inv2;
        return x.trunc(m);
    }
    // k 次幂，允许最低非零位不在 0 处、也允许 a[i] != 1
    // 模板题 https://www.luogu.com.cn/problem/P5245
    Poly pow(ll k, int m) const {
        if (k == 0) { Poly r(m); if (m) r[0] = 1; return r; }
        int i = 0;
        while (i < (int)size() && (*this)[i] == Z(0)) i++;
        if (i == (int)size() || (__int128)i * k >= m) return Poly(m);
        Z v = (*this)[i];
        Poly f = shift(-i) * Z::inv(v);
        int rest = m - i * (int)k;
        return (f.log(rest) * Z(k % (ll)P)).exp(rest).shift(i * (int)k)
               * Z::powmod(v, k);
    }
```

使用前提别记错：inv 要 $f_0 \neq 0$；log 要 $f_0 = 1$（否则 $\ln f$ 的常数项不存在）；exp 要 $f_0 = 0$（否则 e^f 的常数项是个超越数）；sqrt 这份只处理 $f_0 = 1$，f_0 是别的二次剩余时先用 Cipolla 解出 $\sqrt{f_0}$ 再整体缩放。pow 已经处理好了「最低非零项不在 0 次」的情况：先把 x^i 提出来，剩下的部分归一化成 $f_0 = 1$ 再走 $\exp(k \ln f)$。

转置卷积与多点求值。mulT 取 $f \cdot \mathrm{rev}(b)$ 的高位，它算的是「相关」而不是卷积；多点求值就是拿它在分治树上把 $\prod(1 - x_i t)$ 一层层往下推，$O(n \log^2 n)$。

```
    // 转置卷积：取 (*this) * reverse(b) 的高位，多点求值的核心零件
    Poly mulT(const Poly &b) const {
        if (b.empty()) return Poly();
        Poly c = b;
        reverse(c.begin(), c.end());
        return (*this * c).shift(-((int)b.size() - 1));
    }
    // 多点求值 O(n log^2 n)   模板题 https://www.luogu.com.cn/problem/P5050
    vector<Z> eval(vector<Z> x) const {
        if (empty()) return vector<Z>(x.size(), Z(0));
        int n = max((int)x.size(), (int)size());
        vector<Poly> q(n << 2);
        vector<Z> ans(x.size());
        x.resize(n);
        auto build = [&](auto &self, int p, int l, int r) -> void {
            if (r - l == 1) { q[p] = Poly{Z(1), -x[l]}; return; }
            int m = (l + r) >> 1;
            self(self, p << 1, l, m), self(self, p << 1 | 1, m, r);
            q[p] = q[p << 1] * q[p << 1 | 1];
        };
        build(build, 1, 0, n);
        auto work = [&](auto &self, int p, int l, int r, const Poly &f) -> void {
            if (r - l == 1) { if (l < (int)ans.size()) ans[l] = f.get(0); return; }
            int m = (l + r) >> 1;
            self(self, p << 1, l, m, f.mulT(q[p << 1 | 1]).trunc(m - l));
            self(self, p << 1 | 1, m, r, f.mulT(q[p << 1]).trunc(r - m));
        };
        work(work, 1, 0, n, mulT(q[1].inv(n)));
```

```
        return ans;
    }
};
```

带余除法。设 $\deg f = n$、$\deg g = m$，令 f^R 表示系数翻转，由 $f = qg + r$ 两边翻转可得

$$f^R \equiv q^R g^R \pmod{x^{n-m+1}},$$

于是 $q^R = f^R \cdot (g^R)^{-1} \bmod x^{n-m+1}$，再翻回去、回代求 r。

```
// 带余除法 f = q * g + r, deg(r) < deg(g)
// 模板题 https://www.luogu.com.cn/problem/P4512
pair<Poly, Poly> divmod(Poly f, Poly g) {
    int n = f.deg(), m = g.deg();
    f.norm(), g.norm();
    if (n < m) return {Poly(), f};
    Poly fr = f, gr = g;
    reverse(fr.begin(), fr.end()), reverse(gr.begin(), gr.end());
    Poly q = (fr.trunc(n - m + 1) * gr.inv(n - m + 1)).trunc(n - m + 1);
    reverse(q.begin(), q.end());
    Poly r = (f - q * g).trunc(max(m, 1));
    return {q.norm(), r.norm()};
}
```

快速插值是多点求值的对偶：给 n 个点值还原出 $n-1$ 次多项式。记 $M(x) = \prod(x - x_i)$，则

$$f(x) = \sum_i \frac{y_i}{M'(x_i)} \prod_{j \neq i} (x - x_j),$$

分母那一堆 $M'(x_i)$ 用一次多点求值批量算出来，分子在分治树上自底向上合并。

```
// 快速插值：给 n 个点值还原 n-1 次多项式 O(n log^2 n)
// 模板题 https://www.luogu.com.cn/problem/P5158
Poly interpolate(const vector<Z> &x, const vector<Z> &y) {
    int n = x.size();
    vector<Poly> m(n << 2);
    auto build = [&](auto &self, int p, int l, int r) -> void {
        if (r - l == 1) { m[p] = Poly{-x[l], Z(1)}; return; }
        int mid = (l + r) >> 1;
        self(self, p << 1, l, mid), self(self, p << 1 | 1, mid, r);
        m[p] = m[p << 1] * m[p << 1 | 1];
    };
    build(build, 1, 0, n);
    vector<Z> d = m[1].deriv().eval(x);      // d[i] = M'(x_i)
    auto solve = [&](auto &self, int p, int l, int r) -> Poly {
        if (r - l == 1) return Poly{y[l] / d[l]};
        int mid = (l + r) >> 1;
        return self(self, p << 1, l, mid) * m[p << 1 | 1]
             + self(self, p << 1 | 1, mid, r) * m[p << 1];
    };
    return solve(solve, 1, 0, n);
}
```

操作	复杂度	模板题
operator*	$O(n \log n)$	P3803
inv	$O(n \log n)$	P4238
log / exp	$O(n \log n)$	P4725 / P4726
sqrt	$O(n \log n)$	P5205
pow	$O(n \log n)$	P5245
divmod	$O(n \log n)$	P4512
eval	$O(n \log^2 n)$	P5050
interpolate	$O(n \log^2 n)$	P5158

6.4 任意模数卷积（三模数 NTT + Garner）

洛谷 P4245 任意模数多项式乘法

题目给的模数是 10^9+7 这类**不支持 NTT** 的数时，常见有两条路：

- **拆系数 FFT（MTT）**：把每个系数拆成 $a=a_1\cdot 2^{15}+a_0$，做三到四次复数 FFT 再合并。代码短，但全程靠 `double` 的 53 位尾数硬扛，系数和长度一起上去就会掉精度，而且掉了不报错。
- **三模数 NTT + Garner**：在三个 NTT 友好的模数下各算一遍，再用 CRT 把真值还原出来，最后对题目模数取模。**全程整数运算，没有精度问题**，代价是常数约为单次 NTT 的三倍。

这里取后者。三个模数的乘积约 5.9×10^{25}，足够容纳「长度 10^6、系数 10^9」量级卷积的真值（上界约 10^{24}）。

```cpp
// ===== 任意模数 / 大整数卷积：三模数 NTT + Garner 合并 =====
// 模板题 https://www.luogu.com.cn/problem/P4245
// 三个模数乘积 ~5.9e25，足够容纳任意 int 系数、长度 1e6 级别的卷积真值
// 注意原根各不相同（167772161 与 469762049 是 3，754974721 是 11），
// 这里靠 primitiveRoot 编译期算出来，不必手写
constexpr uint32_t M1 = 167772161, M2 = 469762049, M3 = 754974721;

template <uint32_t MOD>
vector<ll> convRaw(const vector<ll> &a, const vector<ll> &b) {
    using mint = modint<uint32_t, MOD>;
    vector<mint> A(a.size()), B(b.size());
    for (size_t i = 0; i < a.size(); i++) A[i] = a[i];
    for (size_t i = 0; i < b.size(); i++) B[i] = b[i];
    auto C = NTT<MOD>::conv(A, B);
    vector<ll> c(C.size());
    for (size_t i = 0; i < C.size(); i++) c[i] = C[i].val();
    return c;
}

// p > 0：结果对 p 取模，p 可以是 1e9+7 这种非 NTT 模数
// p <= 0：不取模，要求真实值 < 2^63（否则请自行换成 __int128 返回）
vector<ll> convolutionAnyMod(const vector<ll> &a, const vector<ll> &b, ll p) {
    if (a.empty() || b.empty()) return {};
    auto c1 = convRaw<M1>(a, b), c2 = convRaw<M2>(a, b), c3 = convRaw<M3>(a, b);
    ll i12 = modint<uint32_t, M2>::inv(modint<uint32_t, M2>(M1)).val();
    ll i3 = modint<uint32_t, M3>::inv(
                modint<uint32_t, M3>((ll)((ull)M1 * M2 % M3))).val();
    vector<ll> c(c1.size());
    for (size_t i = 0; i < c1.size(); i++) {
        ll k1 = (c2[i] - c1[i]) % (ll)M2 * i12 % (ll)M2;
        if (k1 < 0) k1 += M2;
        ll x12 = c1[i] + (ll)M1 * k1;                  // < M1*M2 ~ 7.9e16
        ll k2 = (c3[i] - x12) % (ll)M3 * i3 % (ll)M3;
        if (k2 < 0) k2 += M3;
        __int128 x = (__int128)x12 + (__int128)M1 * M2 * k2;
        c[i] = p > 0 ? (ll)(x % p) : (ll)x;
    }
    return c;
}
```

两个用途：`p > 0` 时是任意模数卷积；`p <= 0` 时不取模，直接给出真实的整数结果（要求真值 $<2^{63}$），可以当作**高精度乘法**或「大整数卷积」用——比上一节的复数 FFT 版本精度可靠得多。

6.5 分治 FFT

洛谷 P4721 分治 FFT

要解的是这类自指递推：f_0 已知，且

$$f_i = \sum_{j=1}^{i} f_{i-j}\, g_j.$$

麻烦在于 f 的每一项都依赖前面所有项，不能一次卷积算完。CDQ 分治的思路是「先把左半边算对，再把左半边对右半边的贡献一次卷积推过去」。

```
// ===== 分治 FFT：f[0] 已知，f[i] = sum_{j=1..i} f[i-j] * g[j] =====
// 模板题 https://www.luogu.com.cn/problem/P4721
// 也可以直接写成 f = f0 / (1 - g)，即 Poly{1} - g 求逆，常数更小、代码更短
vector<Z> divideConquerFFT(const vector<Z> &g, int n, Z f0 = 1) {
    vector<Z> f(n);
    if (!n) return f;
    f[0] = f0;
    auto cdq = [&](auto &self, int l, int r) -> void {
        if (r - l == 1) return;
        int m = (l + r) >> 1;
        self(self, l, m);
        vector<Z> a(f.begin() + l, f.begin() + m);
        vector<Z> b(g.begin(), g.begin() + min((int)g.size(), r - l));
        vector<Z> c = convolution(a, b);
        for (int i = m; i < r; i++)
            if (i - l < (int)c.size()) f[i] += c[i - l];
        self(self, m, r);
    };
    cdq(cdq, 0, n);
    return f;
}
```

但更该记住的是它的等价形式。把递推写成生成函数：$F = FG + f_0$，于是

$$F = \frac{f_0}{1 - G}.$$

直接一次多项式求逆就完了，代码只有一行，常数还比分治小：

```
// 与 divideConquerFFT(g, n) 等价，但只需一次求逆
Poly f = (Poly{Z(1)} - Poly(g)).inv(n) * f0;
```

那还留着分治版做什么？因为**当 g 本身依赖 f 时（两个数列互相牵扯），生成函数写不出封闭形式，分治仍然是唯一可行的写法**。「先看看能不能写成求逆」应该是遇到这类递推的第一反应，写不成再退回分治。

6.6 线性递推：Berlekamp-Massey 与 Bostan-Mori

洛谷 P5487 BM · 洛谷 P4723 常系数线性递推

这两个板子是一对，合起来能做一件很赖皮的事：**打出前几十项，直接问第 10^{18} 项是多少**。

Berlekamp-Massey 从一段数列里找出最短的线性递推式。经验口径：若怀疑答案满足 k 阶递推，喂给它至少 $2k$ 项（保险起见 $2k+10$），返回的系数 c 满足 $s_i = \sum_j c_j\, s_{i-1-j}$。

```
// ===== Berlekamp-Massey：求最短线性递推式 O(n^2) =====
// 模板题 https://www.luogu.com.cn/problem/P5487
// 返回 c，满足 s[i] = sum_{j} c[j] * s[i-1-j]
vector<Z> berlekampMassey(const vector<Z> &s) {
    vector<Z> ls, cur;
    int lf = 0;
    Z ld = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        Z t = 0;
        for (int j = 0; j < (int)cur.size(); j++) t += cur[j] * s[i - 1 - j];
        if (t == s[i]) continue;
```

```
        if (cur.empty()) {                    // 还没有任何递推式
            cur.resize(i + 1);
            lf = i, ld = t - s[i];
            continue;
        }
        Z k = (t - s[i]) / ld;
        vector<Z> c(i - lf - 1);
        c.push_back(k);
        for (int j = 0; j < (int)ls.size(); j++) c.push_back(-ls[j] * k);
        if (c.size() < cur.size()) c.resize(cur.size());
        for (int j = 0; j < (int)cur.size(); j++) c[j] += cur[j];
        if (i - (int)cur.size() > lf - (int)ls.size()) ls = cur, lf = i, ld = t - s[i];
        cur = c;
    }
    return cur;
}
```

Bostan-Mori 求有理分式的第 n 项系数 $[x^n]\dfrac{P(x)}{Q(x)}$。核心恒等式是分母有理化：

$$\frac{P(x)}{Q(x)}=\frac{P(x)Q(-x)}{Q(x)Q(-x)},$$

右边的分母 $Q(x)Q(-x)$ 是 x^2 的多项式，于是按 n 的奇偶性抽取分子的奇/偶次项，问题规模从 n 降到 $n/2$，共 $O(\log n)$ 轮。

```
// ===== Bostan-Mori: 求 [x^n] P(x)/Q(x), O(k log k log n) =====
Z bostanMori(Poly p, Poly q, ll n) {
    while (n) {
        Poly qm = q;                          // Q(-x)
        for (int i = 1; i < (int)qm.size(); i += 2) qm[i] = -qm[i];
        Poly u = p * qm, v = q * qm;          // V(x^2) = Q(x)Q(-x), 只剩偶次项
        Poly np(u.size() / 2 + 1), nq(v.size() / 2 + 1);
        for (int i = (int)(n & 1); i < (int)u.size(); i += 2) np[i >> 1] = u[i];
        for (int i = 0; i < (int)v.size(); i += 2) nq[i >> 1] = v[i];
        p = np, q = nq, n >>= 1;
    }
    return p.get(0) / q[0];
}

// 常系数齐次线性递推：给出前 k 项 a[0..k-1] 与系数 c（BM 的输出可直接喂进来），
// 求 a[n]。模板题 https://www.luogu.com.cn/problem/P4723
Z linearRecurrence(const vector<Z> &a, const vector<Z> &c, ll n) {
    int k = c.size();
    if (n < (ll)a.size()) return a[n];
    Poly q(k + 1);
    q[0] = 1;
    for (int i = 0; i < k; i++) q[i + 1] = -c[i];
    Poly p = (Poly(a) * q).trunc(k);
    return bostanMori(p, q, n);
}
```

典型用法：矩阵快速幂只能做到 $O(k^3\log n)$，k 一大就没救；这套是 $O(k\log k\log n)$，$k=10^4$、$n=10^{18}$ 也能秒。

```
vector<Z> s = {1, 1, 2, 3, 5, 8, 13, 21};   // 打表出来的前几项
vector<Z> c = berlekampMassey(s);            // c = {1, 1}, 即 f[i]=f[i-1]+f[i-2]
Z ans = linearRecurrence(s, c, 100000000000000000LL);
```

6.7 拉格朗日插值

洛谷 P4781 拉格朗日插值

已知 n 个点值求 $n-1$ 次多项式在某点的取值：

$$f(k)=\sum_i y_i\prod_{j\neq i}\frac{k-x_j}{x_i-x_j}.$$

```
// ===== 拉格朗日插值 =====
// 任意 n 个点值求 f(k)，O(n^2)  模板题 https://www.luogu.com.cn/problem/P4781
Z lagrange(const vector<Z> &x, const vector<Z> &y, Z k) {
    int n = x.size();
    Z ans = 0;
    for (int i = 0; i < n; i++) {
        Z num = y[i], den = 1;
        for (int j = 0; j < n; j++)
            if (j != i) num *= k - x[j], den *= x[i] - x[j];
        ans += num / den;
    }
    return ans;
}
```

横坐标取连续整数 $0,1,\dots,n-1$ 时能降到 $O(n)$：分母是 $i!\,(n-1-i)!$ 带个符号，分子 $\prod_{j\neq i}(k-j)$ 用前后缀积。这是竞赛里最常用的形态：

```
// 横坐标是 0,1,...,n-1 时可以做到 O(n)：前后缀积 + 阶乘逆元
// 典型用途：自然数幂和 sum_{i=1..m} i^t 是 t+1 次多项式，取 t+2 个点值即可
Z lagrangeConsecutive(const vector<Z> &y, ll k) {
    int n = y.size();
    if (k < n) return y[k];
    vector<Z> pre(n + 1), suf(n + 1), fac(n), ifac(n);
    pre[0] = suf[n] = 1;
    for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * (Z(k) - Z(i));
    for (int i = n - 1; i >= 0; i--) suf[i] = suf[i + 1] * (Z(k) - Z(i));
    fac[0] = 1;
    for (int i = 1; i < n; i++) fac[i] = fac[i - 1] * Z(i);
    ifac[n - 1] = Z::inv(fac[n - 1]);
    for (int i = n - 1; i >= 1; i--) ifac[i - 1] = ifac[i] * Z(i);
    Z ans = 0;
    for (int i = 0; i < n; i++) {
        Z t = y[i] * pre[i] * suf[i + 1] * ifac[i] * ifac[n - 1 - i];
        ans += ((n - 1 - i) & 1) ? -t : t;
    }
    return ans;
}
```

最典型的应用是自然数幂和。$\sum_{i=1}^{m} i^t$ 关于 m 是一个 $t+1$ 次多项式（可由伯努利数或差分法证），所以**只要暴力算出前 $t+2$ 个点值，插值就能得到任意 m 处的答案**，m 可以大到 10^{18}：

```
// 求 sum_{i=1..m} i^t  (t 很小，m 很大)
vector<Z> y(t + 2);
for (int i = 1; i <= t + 1; i++) y[i] = y[i - 1] + Z::powmod(Z(i), t);
Z ans = lagrangeConsecutive(y, m);
```

6.8 FWT 与子集卷积

洛谷 P4717 FWT ·洛谷 P6097 子集卷积

FFT 处理的是**加法卷积** $c_k=\sum_{i+j=k}a_ib_j$；把「$i+j$」换成位运算就是 FWT 的地盘：

$$c_k=\sum_{i\,\mathrm{op}\,j=k}a_ib_j,\qquad \mathrm{op}\in\{\text{or},\ \text{and},\ \text{xor}\}.$$

三者的变换矩阵不同，但骨架完全一样：按位分治，每层把 $(a_j,\ a_{j+len})$ 换成两个线性组合。或卷积做的是子集和（高维前缀和），与卷积是超集和，异或卷积则是每一位上的 $\pm$ 组合。

```cpp
// ===== FWT: 位运算卷积 =====
// 模板题 https://www.luogu.com.cn/problem/P4717
// type: 0 = 或 (OR)  1 = 与 (AND)  2 = 异或 (XOR); 数组长度必须是 2 的幂
void fwt(vector<Z> &a, int type, bool inv) {
    int n = a.size();
    for (int len = 1; len < n; len <<= 1)
        for (int i = 0; i < n; i += len << 1)
            for (int j = i; j < i + len; j++) {
                Z u = a[j], v = a[j + len];
                if (type == 0) a[j + len] = inv ? v - u : v + u;
                else if (type == 1) a[j] = inv ? u - v : u + v;
                else a[j] = u + v, a[j + len] = u - v;
            }
    if (type == 2 && inv) {
        Z iv = Z::inv(Z(n));
        for (auto &x : a) x *= iv;
    }
}

vector<Z> bitwiseConv(vector<Z> a, vector<Z> b, int type) {
    fwt(a, type, false), fwt(b, type, false);
    for (size_t i = 0; i < a.size(); i++) a[i] *= b[i];
    fwt(a, type, true);
    return a;
}
```

子集卷积要求的是**不交并**：$c_S = \sum_{T \subseteq S} a_T\, b_{S \setminus T}$。直接用或卷积会把 T 与 $S \setminus T$ 有交的情况一起算进去。经典的修补办法是**占位多项式**：给每个集合额外记一维 popcount，只有当 $|T| + |S \setminus T| = |S|$ 时才是真正的不交并，于是在「或卷积」的基础上让这一维做加法卷积即可，复杂度 $O(n^2 2^n)$。

```cpp
// ===== 子集卷积: c[S] = sum_{T subset S} a[T] * b[S\T], O(n^2 2^n) =====
// 模板题 https://www.luogu.com.cn/problem/P6097
// 做法: 给或卷积补一维「popcount」, 占位后维度相加才是真正的不交并
vector<Z> subsetConv(const vector<Z> &a, const vector<Z> &b, int n) {
    int N = 1 << n;
    vector<vector<Z>> fa(n + 1, vector<Z>(N)), fb(n + 1, vector<Z>(N));
    vector<vector<Z>> fc(n + 1, vector<Z>(N));
    for (int s = 0; s < N; s++) {
        fa[__builtin_popcount(s)][s] = a[s];
        fb[__builtin_popcount(s)][s] = b[s];
    }
    for (int i = 0; i <= n; i++) fwt(fa[i], 0, false), fwt(fb[i], 0, false);
    for (int i = 0; i <= n; i++)
        for (int j = 0; i + j <= n; j++)
            for (int s = 0; s < N; s++) fc[i + j][s] += fa[i][s] * fb[j][s];
    for (int i = 0; i <= n; i++) fwt(fc[i], 0, true);
    vector<Z> c(N);
    for (int s = 0; s < N; s++) c[s] = fc[__builtin_popcount(s)][s];
    return c;
}
```

6.9 拉格朗日反演

洛谷 P5825 排列计数（典型应用）

生成函数题里经常出现这种局面：G **只满足一个方程、写不出封闭形式**，但它的**复合逆** F（即 $F(G(x)) = x$）反而形式简单。拉格朗日反演说，这时候不必真的把 G 解出来：

$$[x^n]\, G(x) = \frac{1}{n}\, [x^{n-1}] \left(\frac{x}{F(x)} \right)^n ,$$

更常用的是带一个外层函数的扩展形式

$$[x^n]\,H(G(x)) = \frac{1}{n}\,[x^{n-1}]\,H'(x)\left(\frac{x}{F(x)}\right)^n.$$

成立条件是 $F_0 = 0$ 且 $F_1 \neq 0$（这正是复合逆存在的条件）。

```
// ===== 拉格朗日反演：由 F 直接取出它复合逆 G 的某一项系数 =====
// F(G(x)) = x，要求 F[0] = 0 且 F[1] != 0
// 模板题 https://www.luogu.com.cn/problem/P5825 （常用于 GF 只写得出方程的场合）
// [x^n] G = (1/n) [x^{n-1}] (x / F(x))^n
Z lagrangeInversion(const Poly &f, int n) {
    Poly t = f.shift(-1).inv(n);            // t = x / F(x)
    return t.pow(n, n).get(n - 1) / Z(n);
}
// 扩展形式：[x^n] H(G(x)) = (1/n) [x^{n-1}] H'(x) (x / F(x))^n
Z lagrangeInversion(const Poly &f, const Poly &h, int n) {
    Poly t = f.shift(-1).inv(n);
    return (h.deriv() * t.pow(n, n)).get(n - 1) / Z(n);
}
```

怎么用：设未知的计数序列是 G，从组合意义列出它满足的方程（常见形如 $G = x\,\Phi(G)$），把它改写成 $F(G) = x$ 的样子读出 F，再套上面的式子。例如卡特兰数：由 $G = x + G^2$ 得 $F(x) = x - x^2$，于是

$$[x^n]G = \frac{1}{n}[x^{n-1}](1-x)^{-n} = \frac{1}{n}\binom{2n-2}{n-1},$$

正是 C_{n-1}。这条路子的价值在于：**哪怕 Φ 复杂到 G 根本解不出来，只要能写出 F，单点系数依然是 $O(n \log n)$ 可算的**。

6.10 生成函数常用结论

代码只解决「算得快」，真正卡人的是**把计数问题翻译成幂级数**这一步。这一节是给赛场上现查的对照表。

两种生成函数。普通型（OGF）与指数型（EGF）：

$$F(x) = \sum_{n\geq 0} a_n x^n, \qquad \hat{F}(x) = \sum_{n\geq 0} a_n \frac{x^n}{n!}.$$

选哪个取决于**对象之间有没有编号**：组合出来的结构**无标号**（如整数拆分、括号串）用 OGF，因为两个 OGF 相乘正好是「左边取一部分、右边取一部分」；**有标号**（如排列、带编号的图）用 EGF，因为

$$\hat{F}\hat{G} = \sum_{n}\left(\sum_{k}\binom{n}{k} f_k\, g_{n-k}\right)\frac{x^n}{n!}$$

里的 $\binom{n}{k}$ 恰好完成了「把 n 个编号分给两边」这件事。

OGF 常见封闭形式：

- $\sum_{n\geq 0} x^n = \dfrac{1}{1-x}$，更一般地 $\sum_{n\geq 0} x^{kn} = \dfrac{1}{1-x^k}$（每种物品可取任意多个）；
- $1 + x + \cdots + x^{m-1} = \dfrac{1-x^m}{1-x}$（每种物品最多取 $m-1$ 个）；
- $\sum_{n\geq 0}(n+1)x^n = \dfrac{1}{(1-x)^2}$，一般地 $\sum_{n\geq 0}\binom{n+m}{m}x^n = \dfrac{1}{(1-x)^{m+1}}$；
- $\sum_{n\geq 0}\binom{m}{n}x^n = (1+x)^m$（二项式定理）；
- **广义二项式定理**：对任意实数 α，$(1+x)^\alpha = \sum_{n\geq 0}\binom{\alpha}{n}x^n$，其中 $\binom{\alpha}{n} = \dfrac{\alpha(\alpha-1)\cdots(\alpha-n+1)}{n!}$；

- $\displaystyle\sum_{n\ge1}\frac{x^n}{n}=-\ln(1-x)$；
- **斐波那契**：$\displaystyle\sum_{n\ge0}F_nx^n=\frac{(F_1-F_0)x+F_0}{1-x-x^2}$（由 $G=xG+x^2G+(F_1-F_0)x+F_0$ 解出）；
- **卡特兰数**：$\displaystyle\sum_{n\ge0}C_nx^n=\frac{1-\sqrt{1-4x}}{2x}$（由 $C=xC^2+1$ 解出）；
- **前缀和算子**：若 $s_n=\sum_{i\le n}a_i$，则 $S(x)=\dfrac{F(x)}{1-x}$；乘 $1-x$ 就是差分；
- **五边形数定理**：$\displaystyle\prod_{i\ge1}(1-x^i)=\sum_{k\in\mathbb{Z}}(-1)^kx^{k(3k-1)/2}$。分拆数 $p(n)$ 的 OGF 是它的倒数 $\prod(1-x^i)^{-1}$，于是 $p(n)$ 可以用这个只有 $O(\sqrt{n})$ 项的稀疏级数在 $O(n\sqrt{n})$ 内递推，不需要多项式求逆。

EGF 与 exp 的组合意义。这是 EGF 最有威力的地方：

$$\hat{F}=\exp\hat{P}\iff\text{「}F\text{ 是若干个无序的}P\text{ 拼起来」}.$$

直观理解是 $\exp\hat{P}=\sum_k\hat{P}^k/k!$：取 k 个 P 结构，除以 $k!$ 消掉它们之间的顺序。反过来取 ln 就是「从任意结构里剥出连通结构」。由此可得一串对偶：

- $\displaystyle\sum_{n\ge0}\frac{x^n}{n!}=e^x$（每个元素只有一种选择）；
- 长度为 n 的**轮换**有 $(n-1)!$ 种，$\displaystyle\hat{P}=\sum_{n\ge1}\frac{x^n}{n}=-\ln(1-x)$；而排列是若干轮换的无序拼接，故排列数的 EGF $\exp\hat{P}=\dfrac{1}{1-x}$，即 $n!$，自洽；
- 限定轮换长度 ≥2（即无不动点）得 **错排**：$\hat{P}=-\ln(1-x)-x$，错排数的 EGF 为 $\exp\hat{P}=\dfrac{e^{-x}}{1-x}$；
- n 个点的**有标号无向图**有 $2^{\binom{n}{2}}$ 个，其 EGF 记为 $\hat{G}$；**连通图**的 EGF 就是 $\ln\hat{G}$——这是「连通图计数」标准做法的全部内容；
- n 个点的**有标号树**有 n^{n-2} 个（Cayley 公式），森林的 EGF 即 exp 掉它。

单位根反演。要从卷积结果里只取下标为 k 倍数的项：

$$[k\mid n]=\frac{1}{k}\sum_{j=0}^{k-1}\omega_k^{jn},\qquad\omega_k=e^{2\pi i/k}\ \text{(模意义下取}g^{(p-1)/k}\text{)}.$$

于是 $\displaystyle\sum_{k\mid n}a_n=\frac{1}{k}\sum_{j=0}^{k-1}F(\omega_k^j)$。$k$ 很小（2、3、4）时这招最好使，常见于「计数模 k 余 r 的方案」。

解题套路：把「每个物品能选多少个」写成一个多项式，所有物品的多项式**相乘**就是总方案的生成函数；若干个同类物品用 exp（EGF，无序）或 $\frac{1}{1-x^k}$（OGF）合并；最后要么化成封闭形式直接展开，要么用本章的多项式全家桶把它算出来。

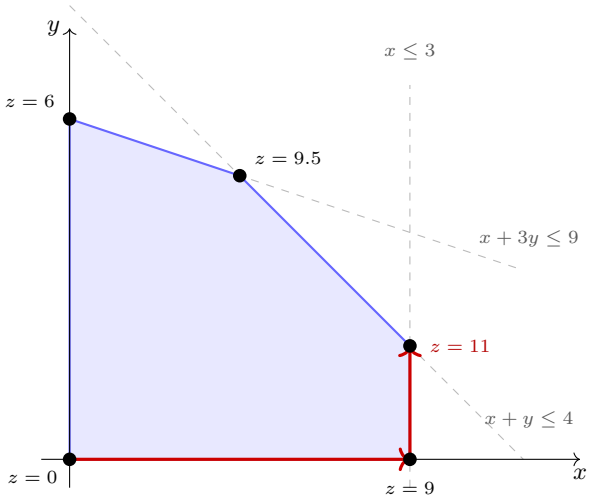
6.11 线性规划（两阶段单纯形 Simplex）

求解标准型

$$\max\ c^{\mathsf{T}}z\quad\text{s.t.}\quad Az\le b,\ z\ge0.$$

三样输入分别是：A 是**约束系数**（$A_{ij}=$ 第 i 条约束里变量 j 的系数），b 是每条约束的**上限**，c 是**要最大化的那个式子的系数**。代码里就直接叫 `coef` / `limit` / `profit`，解向量叫 `x`。

可行域是若干半空间交出的凸多面体，最优值必在某个**顶点**处取到。单纯形法就是从一个顶点出发，每次沿一条棱走到相邻的更优顶点（一次 `pivot` = 一个非基变量入基、一个基变量出基），直到目标行里没有改进方向为止。



图中目标为 $\max z = 3x + 2y$，路径 $(0,0) \to (3,0) \to (3,1)$ 即两次 pivot 后到达最优顶点。

实现要点：

- **字典表连续存储**：$(rows+2) \times (cols+2)$ 一维数组行主序，比 `vector<vector<double>>` 少一层间接寻址，内层循环无分支可向量化。
- **两阶段**：b 有负分量时原点不可行，先引入人工变量 x_0 跑第一阶段 $\min x_0$；若最优 $x_0 > 0$ 则原问题**不可行**。$b \geq 0$ 时直接跳过第一阶段。
- **Devex 定价**：入基列挑 d_j^2/w_j 最大者（d_j 为目标行系数、w_j 为参考权重），比朴素 Dantzig 规则（挑最负的 d_j）迭代次数少很多。
- **防循环**：退化时目标值长期不推进，连续 `stallLimit` 次后切 Bland 最小下标规则（保证有限步终止），推进后自动切回 Devex。
- **出基行**：最小比值 b_i/a_{is}；比值持平时选主元绝对值更大的一行，数值更稳。

返回值：最优值；无界返回 $+\infty$，不可行返回 $-\infty$；`solution` 非空时回填一组最优解。

建模转换：等式约束拆成 $\leq$ 与 $\geq$ 两条；$\geq$ 约束两边取负变 $\leq$；min 问题把 `profit` 整体取负、答案再取负；变量可正可负时拆成 $z = z^+ - z^-$ 两个非负变量。

```
// 两个变量 x、y，都 >= 0；要最大化 3x + 2y
// 三条约束：x + y <= 4，x + 3y <= 9，x <= 3
vector<vector<double>> coef = {{1, 1},      // 第 1 条约束里 x、y 的系数
                               {1, 3},      // 第 2 条
                               {1, 0}};     // 第 3 条（y 不出现，系数写 0）
vector<double> limit = {4, 9, 3};           // 每条约束不等号右边的上限
vector<double> profit = {3, 2};             // 目标 3x + 2y 里 x、y 的系数
SimplexLP lp;
lp.init(coef, limit, profit);
vector<double> x;                           // 取到最优时每个变量的值
double best = lp.solve(&x);          // best = 11, x = {3, 1}
```

```
// 变量 x[0..cols) 全都 >= 0；每条约束形如 coef[i][0]*x[0] + ... <= limit[i]；
// 目标是让 profit[0]*x[0] + profit[1]*x[1] + ... 尽量大。
struct SimplexLP {
    // 核心就是一张表 tab，(rows + 2) 行 x (cols + 2) 列，拍平成一维存（比二维 vector 少一层寻址）：
    //   每行一条约束，每列一个变量，最后一列放这条约束还剩多少余量
    //   多出的倒数第 2 行 = 目标（每个变量再加一单位能赚多少），最后一行只在" 先找可行点 " 时用
    //   多出的倒数第 2 列 = 辅助变量 x0，同样只在" 先找可行点 " 时用
    int rows = 0;               // 约束条数
    int cols = 0;               // 变量个数
    int stride = 0;             // 一行占多少个 double (= cols + 2)
    vector<double> tab;
    vector<int> basis;          // basis[i] = 第 i 行当前解出来的那个变量的编号
```

```
    vector<int> nonbasis;       // nonbasis[j] = 第 j 列对应的变量编号，这些变量当前取 0
    double eps = 1e-9;
    long long pivots = 0;       // 换基次数，调试看规模用
    vector<double> weight;      // Devex 参考权重，挑变量时用

    double* row(int i) { return tab.data() + static_cast<size_t>(i) * stride; }
    const double* row(int i) const { return tab.data() + static_cast<size_t>(i) * stride; }

    // coef[i][j]: 第 i 条约束里变量 j 的系数     limit[i]: 第 i 条约束的上限
    // profit[j] : 变量 j 每多一单位，目标增加多少
    void init(const vector<vector<double>>& coef,
              const vector<double>& limit,
              const vector<double>& profit) {
        rows = static_cast<int>(limit.size());
        cols = static_cast<int>(profit.size());
        stride = cols + 2;
        tab.assign(static_cast<size_t>(rows + 2) * stride, 0.0);
        basis.resize(rows);
        nonbasis.resize(cols + 1);
        for (int i = 0; i < rows; ++i) {
            double* p = row(i);
            for (int j = 0; j < cols; ++j) p[j] = coef[i][j];
            p[cols] = -1.0;          // 辅助变量 x0 那一列，只在找可行点时起作用
            p[cols + 1] = limit[i];
            basis[i] = cols + i;     // 一开始所有变量取 0，这一行剩的余量就是 limit[i]
        }
        double* obj = row(rows);
        // 目标行存的是 -profit: 以后看到负数就说明" 这个变量还能加，目标还能变大"
        for (int j = 0; j < cols; ++j) { obj[j] = -profit[j]; nonbasis[j] = j; }
        nonbasis[cols] = -1;
        row(rows + 1)[cols] = 1.0;   // 找可行点那一阶段的目标：把 x0 压到 0
        weight.assign(cols + 1, 1.0);
        pivots = 0;
    }

    // 让第 s 列的变量从 0 开始往大加，加到第 r 条约束把它卡住为止；这两个变量交换身份
    void pivot(int r, int s) {
        ++pivots;
        double* pr = row(r);
        const double inv = 1.0 / pr[s];
        {   // Devex: 顺手更新参考权重，供下次挑变量用 (O(cols))
            const double as = pr[s], ws = weight[s];
            for (int j = 0; j <= cols; ++j) {
                if (j == s) continue;
                const double ratio = pr[j] / as;
                const double cand = ratio * ratio * ws;
                if (cand > weight[j]) weight[j] = cand;
            }
            const double nw = ws / (as * as);
            weight[s] = nw > 1.0 ? nw : 1.0;
        }
        const int total = rows + 2, width = cols + 2;
        for (int i = 0; i < total; ++i) {   // 用第 r 行把其余每行的第 s 列消成 0
            if (i == r) continue;
            double* pi = row(i);
            const double f = pi[s] * inv;
            if (f == 0.0) continue;
            for (int j = 0; j < width; ++j) pi[j] -= pr[j] * f;   // 内层无分支，编译器可向量化
            pi[s] = -f;                                           // 第 s 列要单独修正
        }
        for (int j = 0; j < width; ++j) pr[j] *= inv;
        pr[s] = inv;
        swap(basis[r], nonbasis[s]);
```

```
}

// phase = 1: 先找一个满足所有约束的点（把 x0 压到 0）
// phase = 2: 在可行的范围内把目标做到最大
bool run(int phase) {
    const int objRow = (phase == 1) ? rows + 1 : rows;
    // 目标值长时间不动（退化）就换 Bland 规则：它慢，但保证不会绕圈死循环
    long long stall = 0;
    const long long stallLimit = 4LL * (rows + cols) + 64;
    while (true) {
        const double* obj = row(objRow);
        int s = -1;                                     // 先挑" 加谁": 哪个变量还能让目标变大
        if (stall < stallLimit) {                       // Devex: 挑性价比最高的
            double best = 0.0;
            for (int j = 0; j <= cols; ++j) {
                if (phase == 2 && nonbasis[j] == -1) continue;
                const double d = obj[j];
                if (d > -eps) continue;                 // 不是负的 => 加它目标也不会变大
                const double score = d * d / weight[j];
                if (score > best) { best = score; s = j; }
            }
        } else {                                        // Bland: 老实挑编号最小的
            for (int j = 0; j <= cols; ++j) {
                if (phase == 2 && nonbasis[j] == -1) continue;
                if (obj[j] < -eps && (s == -1 || nonbasis[j] < nonbasis[s])) s = j;
            }
        }
        if (s == -1) return true;                       // 没变量能让目标变大 => 这一阶段做完

        // 再看" 加到哪停": 哪条约束最先被顶满，即余量 / 系数 最小的那一行
        int r = -1;
        double bestRatio = 0.0, bestPiv = 0.0;
        for (int i = 0; i < rows; ++i) {
            const double* pi = row(i);
            if (pi[s] < eps) continue;                  // 系数 <= 0，这条约束拦不住它
            const double ratio = pi[cols + 1] / pi[s];
            if (r == -1 || ratio < bestRatio - 1e-12 ||
                // 两行一样紧时选系数绝对值更大的那行，算起来数值更稳
                (ratio < bestRatio + 1e-12 && pi[s] > bestPiv)) {
                r = i; bestRatio = ratio; bestPiv = pi[s];
            }
        }
        if (r == -1) return false;                      // 没人拦得住它 => 目标能到无穷大

        const double before = row(objRow)[cols + 1];
        pivot(r, s);
        const double after = row(objRow)[cols + 1];
        stall = (after > before + 1e-12) ? 0 : stall + 1;
    }
}

// 返回目标能达到的最大值；无界返回 +inf，一个可行点都没有返回 -inf
// 传了 solution 就顺便把最优时每个变量的取值填回去
double solve(vector<double>* solution = nullptr) {
    int r = 0;
    for (int i = 1; i < rows; ++i)
        if (row(i)[cols + 1] < row(r)[cols + 1]) r = i;
    if (row(r)[cols + 1] < -eps) {                      // 有约束上限是负的 => 全取 0 都不合法，
    ↪ 先找可行点
        pivot(r, cols);
        if (!run(1) || row(rows + 1)[cols + 1] < -eps)
            return -numeric_limits<double>::infinity();   // x0 压不到 0 => 无解
        for (int i = 0; i < rows; ++i) {
            if (basis[i] != -1) continue;               // 把还赖在解里的 x0 换出去
```

```
                int s = -1;
                for (int j = 0; j <= cols; ++j)
                    if (s == -1 || row(i)[j] < row(i)[s] ||
                        (row(i)[j] == row(i)[s] && nonbasis[j] < nonbasis[s])) s = j;
                pivot(i, s);
            }
        }
        if (!run(2)) return numeric_limits<double>::infinity();
        if (solution) {                                    // 没被解出来的变量取值就是 0
            solution->assign(cols, 0.0);
            for (int i = 0; i < rows; ++i)
                if (basis[i] < cols) (*solution)[basis[i]] = row(i)[cols + 1];
        }
        return row(rows)[cols + 1];
    }
};
```

7 位运算相关恒等式

在处理位运算相关的数学推导时，往往需要从其本质意义出发来理解其与四则运算的关系。为了书写清晰，以下使用 $\oplus$ 表示按位异或，使用 & （即 &）表示按位与，使用 | （即 |）表示按位或，使用 NOT 表示按位取反。

7.1 位运算辅助函数（GNU 与 C++20）

先分清「个数」「下标」「位数」：二进制下标从右往左、从 0 开始。例如 $40 = (101000)_2$，有 2 个 1，最低位 1 的下标为 3，最高位 1 的下标为 5，表示它需要 6 位。若存成 32 位无符号数，高位补 26 个 0；clz 数的是这些前导 0，不是最高位下标。

7.1.1 GNU 常用函数：popcount、clz、ctz、__lg

__builtin_ 系列是 GCC / Clang 的编译器扩展；std::__lg 是 libstdc++ 的内部辅助函数，通常由 <algorithm> 或 <bits/stdc++.h> 引入，**不是标准 C++ 接口**。以下按竞赛常见的 unsigned int 为 32 位、unsigned long long 为 64 位说明。

```
unsigned x = 40;                          // 0b101000
__builtin_popcount(x);                    // 2：
↪ 二进制中 1 的个数
__builtin_clz(x);                         // 26：
↪ 从最高位开始连续的 0
__builtin_ctz(x);                         // 3：
↪ 从最低位开始连续的 0
std::__lg(x);                             // 5：
↪ floor(log2(x))，最高位 1 的下标
// x > 0 时：最高位下标 = 31 - clz(x)，位数 = 32
↪ - clz(x)
unsigned long long y = 1ULL << 40;
__builtin_popcountll(y);                  // 1
__builtin_clzll(y);                       // 23，
↪ 最高位下标 = 63 - clzll(y)
__builtin_ctzll(y);                       // 40
std::__lg(y);                             // 40：
↪ 按参数类型选重载
```

参数宽度不能靠赋值后的结果类型补救：无后缀版本吃 unsigned int，后缀 ll 吃 unsigned long long，后缀 l 吃 unsigned long（宽度随平台变化）。把 64 位的 y 传给 __builtin_popcount(y) 会先截成 32 位，可能丢掉高位；clz/ctz 也一样，甚至截成 0 后触发未定义行为。

零值与用途：popcount(0) 返回 0；GNU 的 clz(0)、ctz(0) 是未定义行为（UB），__lg 也必须保证输入为正。ctz(x) 在 $x > 0$ 时就是最低位 1 的下标，也就是 __lg(lowbit(x))；常用于逐个取出集合元素。__lg(n) 常用于 ST 表选层，避免浮点 log2 的舍入误差。

7.1.2 C++20：<bit> 的标准接口

加 #include <bit>，编译选 -std=c++20。**下面这些整数位操作只接受无符号整数**：用 40u、40ULL，或先转换到对应无符号类型；std::popcount(40) 的 int 参数不能匹配。返回的计数是 int，幂取整和旋转的返回类型与参数相同。

```
#include <bit>
#include <cstdint>
unsigned x = 40;                          //
↪ 0b101000，按 32 位计算
std::popcount(x);                         // 2：1
↪ 的总个数
std::countl_zero(x);                      // 26：前导
↪ 0（对应 clz）
std::countr_zero(x);                      // 3：尾随
↪ 0（对应 ctz）
std::countl_one(x);                       // 0：
↪ 从最高位开始连续的 1
std::countr_one(7u);                      // 3：
↪ ...000111，尾随 1
std::bit_width(x);                        // 6：表示
↪ x 所需位数
std::has_single_bit(x);                   // false：
↪ 是否恰有一个 1，即为 2 的幂
std::has_single_bit(32u);                 // true
std::bit_floor(x);                        // 32：
↪ 不大于 x 的最大 2 的幂
```

```cpp
std::bit_ceil(x);                          // 64: 不小于 x 的最小 2 的幂
std::bit_ceil(32u);                        // 32: 已是 2 的幂就保持原值
std::rotl(std::uint8_t{0b10000001}, 1);// 0b00000011: 循环左移
std::rotr(std::uint8_t{0b10000001}, 1);// 0b11000000: 循环右移
```

C++20 对 0 有定义：`popcount(0u)` 返回 0；`countl_zero(0u)` 和 `countr_zero(0u)` 均返回 32，64 位参数则返回 64；`countl_one(0u)` 和 `countr_one(0u)` 均返回 0。`bit_width(0u)` 和 `bit_floor(0u)` 均返回 0；`bit_ceil(0u)` 返回 1；`has_single_bit(0u)` 返回 `false`。全 1 的无符号数，连续 1 计数返回类型位宽。不过 0 没有最低 / 最高位 1，**不能把这些零值结果直接当有效下标**。

幂取整与旋转的边界：`bit_ceil(x)` 的结果必须能被参数类型表示，否则是 UB。例如 32 位无符号数的 $x > 2^{31}$、64 位无符号数的 $x > 2^{63}$ 都越界；增大接收变量的类型无效，须在调用前扩大参数类型。`rotl/rotr` 移出去的位会绕回另一端，按**参数类型的整个位宽**旋转，次数对位宽取模，负数次数表示反向旋转；并非只在有效位或自定义的低 k 位内转。

7.1.3 竞赛中直接套用的写法

```cpp
unsigned long long mask = 40;
int cnt = std::popcount(mask);          // 集合大小
int hi = mask ? std::bit_width(mask) - 1 : -1;
int lo = mask ? std::countr_zero(mask) : -1;
for (auto rem = mask; rem; rem &= rem - 1) {
    int i = std::countr_zero(rem);      // 按下标递增枚举所有 1
    // 使用元素 i；此时 rem != 0
}
// NTT 等补齐到 2 的幂：先转 64 位，保证结果能表示
unsigned n = 1000;
auto len = std::bit_ceil(static_cast<unsigned long long>(n)); // 1024
```

记忆法：`pop` 数总数；`l/r` 表示从左 / 右端开始数；`width` 是位数，最高位下标要减 1；`floor/ceil` 是向下 / 向上取 2 的幂。功能定义参见 GCC 位操作文档与 C++ 标准草案 <bit>。

7.2 加法与位运算转换

$$a + b = (a \oplus b) + 2 \times (a \,\&\, b)$$

意义解释：$a \oplus b$ **本质上是“不进位加法”**，它计算了每一位上独立相加的结果；而 $a \,\&\, b$ **则找出了所有产生进位的位。因为进位会向高位进一**（即在数值上相当于**乘以** 2），所以真实加法等于不进位加法加上两倍的进位值。

推广到三个数：令 $S = (X \oplus Y) \,\&\, Z + X \,\&\, Y$，则

$$X + Y + Z = (X \oplus Y \oplus Z) + 2S$$

推导：先对 X, Y 使用两数恒等式得 $X + Y = (X \oplus Y) + 2(X \,\&\, Y)$，再令 $a = X \oplus Y$，对 a 与 Z 使用恒等式得 $a + Z = (a \oplus Z) + 2(a \,\&\, Z)$，合并即得。

S **的两项没有公共位**：第一阶段 $X + Y$ 产生进位 $X \,\&\, Y$，同时把这些位在异或残差 $X \oplus Y$ 中清零了（两个 1 对冲变为 0）；第二阶段拿残差与 Z 相加，但那些位已经是 0，$(X \oplus Y) \,\&\, Z$ 在那里不可能为 1。因此两项的加法等于按位或，不会再产生新的进位，即 $S = (X \oplus Y) \,\&\, Z \mid (X \,\&\, Y)$，$S$ 是逐位运算的结果。

应用场景：当问题同时约束三个数的加法和与异或和时（如 CodeChef “Sum and XOR”），该恒等式可将全局加法约束分解为逐位独立的计数问题。

7.3 或、与、异或的关系

$$a \mid b = (a \oplus b) + (a \,\&\, b)$$

意义解释：$a \mid b$ 表示两数中至少有一个为 1 的位。这些位可以拆分成两类：一类是只有其中一个数为 1 的位（即 $a \oplus b$），另一类是两个数都为 1 的位（即 $a \,\&\, b$）。由于这两类位的集合互不相交，因此它们的加法可以直接等价于按位或（相加过程没有任何进位）。

推论：将上式代入加法等式，可得两数之和也等于它们的按位或与按位与之和：

$$a + b = (a \mid b) + (a \,\&\, b)$$

$a \mid b$ **相当于丢失了需要进位的位**，当然，相比于异或，只丢失了一份。因此就**加上一份这个** $a \,\&\, b$ **就可以了**。

下面这个就是一个移项：

$$a \oplus b = (a \mid b) - (a \,\&\, b)$$

7.4 减法与位运算

$$a - b = (a \oplus b) - 2 \times ((\mathrm{NOT}\, a) \,\&\, b)$$

意义解释：与加法同理，减法可以理解为“借位减法”。**只有当被减数** a **当前位为** 0 **且减数** b **当前位为** 1 **时**（即 $(\mathrm{NOT}\, a) \,\&\, b$），才会产生借位。借位意味着需要向高位借一（相当于减去当前位权重的 2 倍）。

7.5 & 与 | 的分配律

$$a \,\&\, (b \mid c) = (a \,\&\, b) \mid (a \,\&\, c)$$

$$a \mid (b \,\&\, c) = (a \mid b) \,\&\, (a \mid c)$$

意义解释（集合观点）：把一个整数看作它” 所有为 1 的位置” 组成的集合，那么 & 就是集合的交 $\cap$，| 就是集合的并 $\cup$。这两条分配律正好对应集合论里熟知的

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C), \quad A \cup (B \cap C) = (A \cup B) \cap (A \cup C),$$

常见用途：**按位拆贡献**时，如果贡献式里混用了 & 和 |，可以用分配律把表达式转成“只含一种操作 + 其他操作套在外层”的形式，便于按位独立处理。

7.6 lowbit 的推导

$$\text{lowbit}(a) = a \,\&\, ((\text{NOT}\, a) + 1) = a \,\&\, (-a)$$

意义解释：由于计算机中负数采用补码表示，$-a$ 等价于 $\text{NOT}\, a + 1$。将 a 按位取反后，原来最低位的 1 变成了 0，其右侧的所有 0 都变成了 1。此时再加 1，会导致进位一直传递，直到遇到那个变为了 0 的原始最低位，将其变回 1，且进位停止。因此，原来最低位的 1 及其右侧的所有位在 $-a$ 中与在 a 中完全相同，而更高位则全部相反。取按位与后，就只剩下了最低位的 1。

7.7 去除最低位的 1

$$a \,\&\, (a-1) = a - \text{lowbit}(a)$$

意义解释：将 a 减去 1，相当于把 a 中最低位的 1 变成 0，并将该位右侧所有的 0 变成 1。更高位保持不变。因此 a 和 $a-1$ 在最低位 1 的左侧完全相同，但在最低位 1 及其右侧的位全都不一致。进行按位与操作后，最低位 1 及其右侧被清零，达到了去除最低位 1 的效果。

其实 `x-lowbit(x)` 去除这个最低位更好用，`__lg(lowbit(x))`，这个获得这个最低位 1 编号（0-based）也很好用。

如果要获得最低位 0 编号，取反一下即可，注意不要使用这个 `signed`，`signed` 类型取反以后是负数。

```
int i = __lg(lowbit(rev_s));
int rest = rev_s - lowbit(rev_s);
for (int j = rest; j; j -= lowbit(j)) {
  int jb = __lg(lowbit(j));
  for (int k = j - lowbit(j); k; k -=
  ↪ lowbit(k)) {
    int kb = __lg(lowbit(k));
    ll val = maze[i][jb];
    val += maze[i][kb];
    val += maze[jb][kb];
    int to_sta = sta;
    to_sta |= 1 << i;
    to_sta |= 1 << jb;
    to_sta |= 1 << kb;
    dp[to_sta] = max(dp[to_sta], dp[sta] + val);
  }
}
```

7.8 连续异或前缀和

定义 $F(n) = 0 \oplus 1 \oplus 2 \oplus \cdots \oplus n$，则 $F(n)$ 以 4 为周期循环：

$$F(n) = \begin{cases} n & n \bmod 4 = 0 \\ 1 & n \bmod 4 = 1 \\ n+1 & n \bmod 4 = 2 \\ 0 & n \bmod 4 = 3 \end{cases}$$

意义解释：每四个连续整数 $4k, 4k+1, 4k+2, 4k+3$ 的异或和为 0（因为 $4k$ 和 $4k+1$ 只在最低位不同，异或得 1；$4k+2$ 和 $4k+3$ 也只在最低位不同，异或得 1；$1 \oplus 1 = 0$）。因此 $F(n)$ 只取决于 n 除以 4 的余数：余数为 3 时恰好凑满完整的四元组，结果为 0；余数为 0 时多出一个 n，结果为 n；余数为 1 时多出 $n-1$ 和 n，它们异或为 1（仅最低位不同）；余数为 2 时在此基础上再异或 n，得 $n+1$。

应用（CF2225D）：已知 n, x，$1 \le x \le n \le 10^{18}$，求在序列 $[1, 2, \ldots, n]$ 中，求有多少个子段 $[l, r]$ 满足：(1) 包含位置 x（即 $l \le x \le r$）；(2) 异或和为 0（即 $l \oplus (l+1) \oplus \cdots \oplus r = 0$）。

第一步：区间异或和可用前缀异或表示：$l \oplus \cdots \oplus r = F(r) \oplus F(l-1)$。因此条件” 区间异或和为 0” 等价于 $F(r) = F(l-1)$。

第二步：约束 $l \le x \le r$ 意味着 $l - 1 \in [0, x-1]$，$r \in [x, n]$，两个范围不重叠（$l - 1 < x \le r$）。回顾 F 的四种取值：$\bmod 4 = 0$ 时 $F = n$，$\bmod 4 = 2$ 时 $F = n + 1$——这两种是**位置相关**的，F 的值随 n 变化而变化。由于 $l - 1 \ne r$，这类取值不可能跨范围匹配。而 $\bmod 4 = 1$ 时 $F = 1$，$\bmod 4 = 3$ 时 $F = 0$——这两种是**常数**，与位置无关，能够跨范围匹配。特别地，$F(0) = 0$（虽然 $0 \bmod 4 = 0$，但恰好 $F = 0$ 也是常数 0），也可以与 $\bmod 4 = 3$ 的项匹配。

第三步：问题化归为：分别统计 $[0, x-1]$ 中有多少个 a 满足 $F(a) = 0$ 或 $F(a) = 1$，以及 $[x, n]$ 中有多少个 b 满足 $F(b) = 0$ 或 $F(b) = 1$，然后将对应的计数相乘再求和。每个计数只需数区间内 $\bmod 4$ 各残基的个数，$O(1)$ 完成。

7.9 对相邻元素异或的操作转化为前缀异或交换

模型：给定数组 A，每次操作可以选择一个位置 i，同时更新：

$$A_{i-1} \leftarrow A_{i-1} \oplus A_i$$

$$A_i \leftarrow A_i$$

$$A_{i+1} \leftarrow A_{i+1} \oplus A_i$$

恒等式 / 结论：构造前缀异或数组 $B_i = A_1 \oplus A_2 \oplus \cdots \oplus A_i$。每次对位置 i 进行上述操作，**本质上等价于交换前缀异或数组中 B_{i-1} 和 B_i 的值**，而其他所有的 B_k 都不变。

意义解释：直观来看，操作使得 A_{i-1} 和 A_{i+1} 都多异或了一个 A_i，我们直接观察前缀异或的变化：

- B'_{i-1} 的前缀和中多出了 1 个 A_i：$B'_{i-1} = B_{i-1} \oplus A_i = B_i$。
- B'_i 的前缀和中也只多出了 1 个 A_i（来自 A_{i-1}）：$B'_i = B_i \oplus A_i = B_{i-1}$。
- B'_{i+1} 的前缀和中多出了 2 个 A_i（分别来自 A_{i-1} 和 A_{i+1}），两者互相抵消，故 $B'_{i+1} = B_{i+1}$ 不变。

可见只有 B_{i-1} 和 B_i 发生了交换，后续的前缀异或值不受影响（由于 A_i 被异或了 3 次，相当于异或了 1 次）。

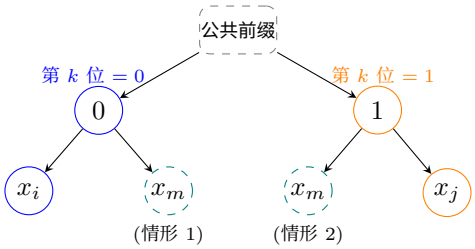
应用场景：若允许对 $2 \le i \le N-1$ 任意操作，那么 $B_1, \ldots, B_{N-1}$ 就可以通过类似冒泡排序的方式**任意排列**，而 B_N 是固定的。利用这一点可以将对原数组的复杂区间操作转化为集合元素的选择或排序问题（如 CodeChef Echo）。

7.10 集合内两两最小异或和

结论：在一个集合中，任意两个元素之间的最小异或和，必定出现在将该集合内的元素按从小到大排序后的**相邻元素对**中。

意义解释：假设将集合升序排序为 $x_1 \le x_2 \le \cdots \le x_n$。考虑任意不相邻的两个数 $x_i < x_j$（即 $j - i > 1$）。设 x_i 和 x_j 的二进制表示在从高到低第 k 位第一次出现不同，则 x_i 的第 k 位必然为 0，x_j 的第 k 位必然为 1。由于它们中间存在元素 x_m（$x_i \le x_m \le x_j$），x_m 的第 k 位要么是 0，要么是 1：

- 若 x_m 的第 k 位为 0，则 $x_i \oplus x_m$ 在最高差异位及以上都为 0，但在第 k 位上也为 0（而 $x_i \oplus x_j$ 在第 k 位为 1），故 $x_i \oplus x_m < x_i \oplus x_j$。
- 若 x_m 的第 k 位为 1，则 $x_m \oplus x_j$ 在第 k 位上为 0，同理可得 $x_m \oplus x_j < x_i \oplus x_j$。



图注：你可以想象这个是一颗异或字典树

因此，跨越元素的异或和永远不会严格优于某对距离更近（最终归结为相邻）元素的异或和。

应用场景：求数组中任意两数最小异或和，只需排序后遍历相邻元素计算即可，时间复杂度 $O(N \log N)$。常与”前缀异或交换”等允许任意重排的性质结合使用。

8 基础算法

8.1 二维前缀和

二维前缀和递推公式。

```
Sum[i][j]=Sum[i-1][j]+Sum[i][j-1]+(G[i][j]=='g'
↪ )-Sum[i-1][j-1];
```

```
ll lans=Sum[xr][yr]+Sum[xl-1][yl-1]-Sum[xl-1][y
↪ r]-Sum[xr][yl-1];
```

或者调用库函数计算二维前缀和。

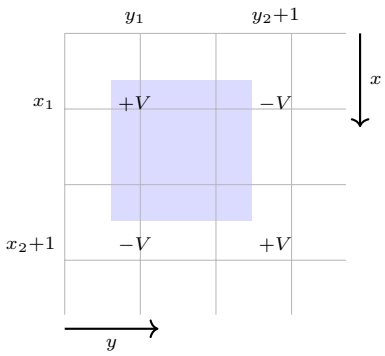
```
class NumMatrix {
public:
    vector<vll> Sum;
    ll N,M;
    NumMatrix(vector<vector<int>>& matrix) {
        N=SZ(matrix),M=SZ(matrix[0]);
        Sum.assign(N+5,vll(M+5,0));
        FOR(i,0,N-1){
            partial_sum(all(matrix[i]),Sum[i+1]
            ↪ .begin()+1);
        }
        FOR(i,2,N){
            FOR(j,1,M){
                Sum[i][j]=Sum[i][j]+Sum[i-1][j];
            }
        }
    }
    int sumRegion(int row1, int col1, int row2,
    ↪ int col2) {
        row2++;col2++;
        ll ans=Sum[row2][col2]+Sum[row1][col1]-
        ↪ Sum[row1][col2]-Sum[row2][col1];
        return ans;
    }
};
```

8.2 二维差分

```
class Solution {
public:
    vector<vector<int>> rangeAddQueries(int n,
    ↪ vector<vector<int>>& queries) {
        vector<vii> diff(n,vii(n));
        FOR(i,0,SZ(queries)-1){
            ll l1=queries[i][0],r1=queries[i][1
            ↪ ],l2=queries[i][2],
            r2=queries[i][3];
            diff[l1][r1]+=1;
            if(r2+1<n) diff[l1][r2+1]-=1;
            if(l2+1<n)diff[l2+1][r1]-=1;
            if(l2+1<n && r2+1<n)
            ↪ diff[l2+1][r2+1]+=1;
        }
        FOR(i,0,n-1){
            partial_sum(all(diff[i]),diff[i].be
            ↪ gin());
        }
        FOR(i,1,n-1){
            FOR(j,0,n-1){
                diff[i][j]=diff[i][j]+diff[i-1]
                ↪ [j];
            }
        }
        return diff;
    }
};
```

原理解释（容斥原理）：差分矩阵的特性是——对 $D[x][y]$ 加上 V 后，在后续求前缀和还原矩阵时，以 (x,y) 为左上角、直到矩阵右下角的所有元素都会被加上 V。所以想让矩形 $[x_1,x_2]\times[y_1,y_2]$ 恰好加 V，就是拿四个这样的「右下四分之一平面」互相抵消：

1. $D[x_1][y_1]\ +=V$：让目标区域及右下方的所有点都加 V。
2. $D[x_2+1][y_1]\ -=V$：把多加的下方区域减掉。
3. $D[x_1][y_2+1]\ -=V$：把多加的右方区域减掉。
4. $D[x_2+1][y_2+1]\ +=V$：右下方区域被减了两次，所以要加回一次来抵消。



蓝色是目标矩形，四个格子是打标记的位置：每个标记的作用范围都是「自己往右下无限延伸」，四者叠加后只有蓝色区域净得 $+V$。

代码里 `l1,l2` 是行（对应 x_1,x_2）、`r1,r2` 是列（对应 y_1,y_2）；四个 `if` 是越界保护：差分数组没有多开一圈，落在 n 之外的标记本来就影响不到答案，直接丢弃即可。

8.3 双指针模板

参考来源：AcWing

```
for (int l = 0, r = 0; r < n; r++) {
    add(a[r]);     // 1 扩展：a[r] 进入窗口
    while (l < r && check(l, r)) l++; // 2 收缩
    // 更新答案     // 3 记录
}
```

三段顺序的硬约束：

1. 步骤 1 **必须**在步骤 2 之前——`check(l, r)` 要用到 `a[r]` 已在窗口的事实。

2. 步骤 3 **必须**在步骤 2 之后——只有收缩到位的 $[l, r]$ 才是当前 r 下的最优左端。

AcWing 原始模板把扩展和记录都塞进「具体问题的逻辑」一行注释里，容易让人误以为收缩在记录前面就够了；实际上扩展、收缩、记录三步都不能省、相对位置也不能换。

常见问题分类：

1. 对于一个序列，用两个指针维护一段区间。
2. 对于两个序列，维护某种次序，比如归并排序中合并两个有序序列的操作。

8.4 整数二分

```
bool check(int x) {/* ... */} //
↪  检查x是否满足某种性质

// 区间[l, r]被划分成[l, mid]和[mid + 1,
↪  r]时使用:
int bsearch_1(int l, int r)
{
    while (l < r)
    {
        int mid = l + r >> 1;
        if (check(mid)) r = mid;    //
        ↪  check()判断mid是否满足性质
        else l = mid + 1;
    }
    return l;
}
// 区间[l, r]被划分成[l, mid - 1]和[mid,
↪  r]时使用:
int bsearch_2(int l, int r)
{
    while (l < r)
    {
        int mid = l + r + 1 >> 1;
        if (check(mid)) l = mid;
        else r = mid - 1;
    }
    return l;
}
```

另一种更常见、也更不容易记混的写法是**闭区间 + ans 记录**：循环条件写 `l <= r`，每次 `check(mid)` 通过就把 `mid` 抄进 `ans`，再按「答案还想更大 / 更小」挪端点。答案不依赖「最后返回 `l` 还是 `r`」，也不用为了防死循环去记 `mid = l + r + 1 >> 1` 这套上取整；最大化与最小化只差循环体里两行的方向。

```
// 最大化: check 为 T,T,T,F,F, 求最后一个 T
int l = L, r = R, ans = -1;  // -1 = 无解
while (l <= r) {
    int mid = l + (r - l) / 2;
    if (check(mid)) ans = mid, l = mid + 1;
    else r = mid - 1;
}
return ans;

// 最小化: check 为 F,F,F,T,T, 求第一个 T
// 循环体只需把上面两行反向:
    if (check(mid)) ans = mid, r = mid - 1;
    else l = mid + 1;
```

- `ans` 初值取一个不可能的值（`-1` / `L - 1` / `INF`），循环跑完仍是初值就说明整个区间无解，不必额外判断。
- 用 `mid = l + (r - l) / 2` 而不是 `(l + r) / 2`，避免 `l + r` 溢出。
- 循环结束时必有 `r + 1 == l`，两端各自停在分界线两侧；但答案已经落在 `ans` 里，不需要再去分辨该返回哪个端点。

8.4.1 partition_point 实现二分答案（C++20）

可以知道 `T,T,T,T,F,F,F,F` 中的第一个 `F` 出现位置，`iota` 实现的是这个左闭右开区间，如果想反过来，就是像 `range2` 一样使用这个 `reverse` 视图即可。

```
auto check_bs=[&](ll i) {
    return
    ↪  query_is_a_gt_b(i,sec_down_start,memo);
};
auto range1=views::iota(sec_down_start+1,end_of
↪  _down);
auto it1=ranges::partition_point(range1,check_b
↪  s);
auto range2=views::iota(end_of_down,N+1)|views:
↪  :reverse;
auto it2=ranges::partition_point(range2,check_b
↪  s);

it1=ranges::prev(it1);

ll to_be_select=*it1;
if (it2!=range2.begin()) {
    it2=ranges::prev(it2);
    if (query_is_a_lt_b(*it2,*it1,memo)) {
        to_be_select=*it2;
    }
}
```

8.5 三分（实数）

凸函数。

```
while(R-L>eps){    // 保证为凸性函数在 [l,r] 内
    DB mid=L+(R-L)/2;
    if(F(mid-eps)>F(mid+eps)){
        R=mid;
    }else{
        L=mid;
    }
}
```

还有一种实践方式：使用固定迭代次数代替 `eps` 阈值。

```
// 方案二：使用固定迭代次数（100次）进行三分搜索。
// 这种方法不依赖eps，更稳定，
↪ 且100次迭代足以保证精度远高于1e-8。
for(int _=0; _<100; ++_){   // 凹函数，
↪ 求最小值用的
    DB mid1 = l + (r-l)/3.0; // 推荐用 l +
    ↪ (r-l)/3 的形式，数值上更稳定
    DB mid2 = r - (r-l)/3.0;
    if(fx(mid1) > fx(mid2)){
        l=mid1;
    }else{
        r=mid2;
    }
}
printf("%.11f\n",fx(l));
```

8.6 三分整数

注意**整数三分遇到平台都会出现问题**，调整 `r-l>?` 这个判定阈值也只是缓兵之计，只能保证在小于这个平台的数据上得到正确答案。

因此正确的想法应该是消除平台（尽可能），比如设计这个 f 函数，**使其返回值是一个浮点数**，但是仍能反映增减关系（如 f 原来是**向下取整除法**，可改成**浮点数除法**）。

```
l=0,r=N-1;
while (r-l>2) {
    ll mid1=l+(r-l)/3;
    ll mid2=r-(r-l)/3;
    if (f(mid1)<=f(mid2)) {
        l=mid1;
    }else {
        r=mid2;
    }
}
FOR(i,l,r) ans=max(ans,f(i));
```

参考提交：AtCoder ABC 421。

另一种处理思路：将 f 函数反转（比如加一个符号）使凸函数变为凹函数（相当于沿 X 轴翻折下来），再用普通二分定位极值。

```
ll l=1,r=mn;
while (l<r) {
    ll mid=l+r>>1;
    // if(f(mid)<=f(mid+1)){
    // 从 mid 点（或其前）开始，函数开始单调递增
    if(f(mid+1)-f(mid)>=0){
        r=mid;
    }else{
        l=mid+1;
    }
}
```

8.7 倍增法

参考题目：Matiji 提交记录。

首先求出 `nextPos` 数组（注意：该数组定义是**结束位置**，所以 `break` 以后需要 `-1`），然后从 `N+1` 开始往前递推。

```
vector<vector<ll> >
↪ jump(N+5,vector<ll>(mx_k+2,0));
ROF(i,N+1,1){
    jump[i][0]=i<=N?nextPos[i]+1:N+1;
    FOR(j,1,mx_k){
        jump[i][j]=jump[jump[i][j-1]][j-1];
    }
}
```

调用方式：

```
FOR(i,1,N){
    ll lans=0,idx=i;
    FOR(j,0,mx_k){
        if(bik[j]){
            idx=jump[idx][j];
        }
    }
    lans=idx-i;
    ans=max(ans,lans);
}
```

8.8 LIS 最长上升子序列及其还原

0-based 输入，构造时传 `strict` 开关：`true` 求严格递增 LIS，`false` 求单调不降 LNDS（最长非降子序列）。算法核心是 patience sort：严格用 `lower_bound` 替换首个 $\geq a_i$，非严格用 `upper_bound` 替换首个 $> a_i$。还原阶段反向贪心，严格判 `<`，非严格判 `<=`。

参考提交：Yosupo Library Checker（严格）、OpenJudges 单调不降（长度，非严格还原已通过本地大规模对拍 $O(N^2)$ 暴力 DP 验证 512/512）。

```
// 最长（严格 / 非严格）上升子序列 + 还原值与下标
// 0-based 输入；strict=true → 严格递增，
↪ strict=false → 单调不降
struct LIS {
    vector<ll> a;        // 输入序列
    int n;
    bool strict;
    vector<ll> tail;     // patience sort: 长度-i
    ↪ 的子序列最小末值
    vector<int> i_len;   // 以下标 i 结尾的 LIS
    ↪ 长度
    vector<ll> seq;      // 还原的递增子序列值
    vector<int> pos;     // 还原的下标 (0-based,
    ↪ 严格递增)

    LIS(const vector<ll>& v, bool strict_ =
    ↪ true)
        : a(v), n((int)v.size()),
        ↪ strict(strict_), i_len(v.size(), 0)
        ↪ {
        for (int i = 0; i < n; ++i) {
            // 严格 → lower_bound（替换首个 >=
            ↪ a[i]）
            // 非严格 → upper_bound（替换首个 >
            ↪ a[i]）
            auto it = strict
```

```
                ? lower_bound(tail.begin(),
                ↪  tail.end(), a[i])
                : upper_bound(tail.begin(),
                ↪  tail.end(), a[i]);
            if (it == tail.end()) {
                tail.push_back(a[i]);
                i_len[i] = (int)tail.size();
            } else {
                int idx = (int)(it -
                ↪  tail.begin());
                tail[idx] = a[i];
                i_len[i] = idx + 1;
            }
        }
        // 反向贪心还原一组解
        int need = (int)tail.size();
        ll cur = LLONG_MAX;
        for (int i = n - 1; i >= 0 && need > 0;
        ↪  --i) {
            bool ok = strict ? (a[i] < cur) :
            ↪  (a[i] <= cur);
            if (ok && i_len[i] >= need) {
                seq.push_back(a[i]);
                pos.push_back(i);
                cur = a[i];
                --need;
            }
        }
        reverse(seq.begin(), seq.end());
        reverse(pos.begin(), pos.end());
    }
    int size() const { return (int)tail.size();
    ↪  }
};

void Solve(){
    int N; cin >> N;
    vector<ll> P(N);
    for (int i = 0; i < N; ++i) cin >> P[i];
    LIS lis(P);                    // 默认严格;
    ↪  非严格写 LIS lis(P, false);
    cout << lis.size() << "\n";
    for (int p : lis.pos) cout << p << " ";
    cout << "\n";
}
```

8.8.1 Dilworth 定理（最小链覆盖 / 偏序划分）

偏序集 $(S, \preceq)$ 上，两两可比的子集叫**链**，两两不可比的子集叫**反链**。

Dilworth 定理：把 S 划分成链的最少链数 = 最长反链的元素个数。**对偶（Mirsky 定理）**：把 S 划分成反链的最少反链数 = 最长链的长度。

落到序列上（与上面的 LIS 互通），下标先后即时间序：

- 划分成**最少条不升**子序列 = 最长**严格上升**子序列长度；
- 划分成**最少条不降**子序列 = 最长**严格下降**子序列长度；
- 对偶：最长**不降**子序列长度 = 划分成**最少条严格下降**子序列的数目。

用法：说白了，从“最少链覆盖”翻到“最长子序列”，只是同时拨两个开关——**严格↔非严格**、**上升↔下降**，再套上面那个 LIS 即可。

8.9 高精度大整数类

什么时候才用它：答案必须精确且超出 `__int128` 的 $\approx 1.7 \times 10^{38}$。先按这个顺序排除——题目允许取模就取模、$10^{38}$ 以内直接上 `__int128`（§14.1 有它的读写重载）、只是本地算个常数就用 Python（§18）。都不行，再搬这个类。

实现：10^9 压位存进 `vector<int>`（低位在前），零的唯一表示是空 `vector` 且 `sign = 1`，`trim()` 负责维持这个不变量。加减 $O(n)$；乘法先把位宽拆到 10^6 再跑 Karatsuba，$O(n^{\log_2 3}) = O(n^{1.585})$——**拆到 10^6** 是因为 Karatsuba 内层是 `res[i+j] += x[i] * y[j]` 的裸累加，位宽 10^9 时几步就爆 `long long`；除法是 Knuth 归一化后逐位试商，$O(n(n-m+1))$。符号语义与内建整数完全一致：**除法向零取整，余数符号跟随被除数**。

实测（`-O2`，本机）：6 万位 $\times$ 6 万位 5 ms，12 万位 $\div$ 6 万位 350 ms。

六个坑（本版都已经处理掉，抄的时候别改回去）：

- `divmod` 的**除数不能为** 0——它上来就读 `y.a.back()`，除数为零时那是空 `vector`，直接 UB。调用方自己保证；`gcd` 内部已经回避了。
- `bigint(int)` 那条构造**不能删**。字面量 `0` 同时是空指针常量，少了它，`b < 0` 会在 `bigint(long long)` 与 `bigint(const char *)` 之间二义，直接 CE。
- 和 `long long` 的乘除只在 $|v| \le 9 \times 10^9$ 时走 $O(n)$ 快路径，再大会自动退化成大整数乘除——不会算错，但心里要有数。
- 输出走 `str()`，**不要**改回流传版本的 `setw + setfill('0')`：那个写法不还原 fill 字符，打完一个大整数之后，同一个流里后面所有 `setw` 的填充都变成 `0`。
- 流传版本把「乘负数时翻符号」抽成了 `void check(int v)`，**传值**——函数里的 `v = -v` 出不去，于是 `x *= -3` 静默算错。本版直接写在 `operator*=` 里。
- Karatsuba 合并结果时的进位**必须是 `long long`**：`c[i]` 量级可达 $n \cdot 10^{12}$，一万位往上进位就爆 `int`。

随机对拍自测入口：`template-check/src/bigint_check.cpp`（与 `__int128` 逐点比对 + 与竖式乘法比对 + 代数恒等式，改这一节的代码后跑一遍）。

```
// 高精度大整数 bigint：1e9 压位 + Karatsuba 乘法 + 试商长除法
//   加减 O(n)，乘 O(n^1.585)，divmod O(n(n-m+1))，n/m 为压位后的位数（1 位 = 9 个十进制位）
//   语义与 C++ 内建整数一致：除法向零取整，余数符号跟随被除数
//   零的唯一表示是 a 为空且 sign = 1（trim() 维持这个不变量）
struct bigint {
    static constexpr int BASE = 1000000000, W = 9;  // 压 9 位十进制
    // 与小整数运算的 O(n) 快路径上界：a[i] * v 不能爆 long long（9e9 * 1e9 < 9.22e18）
    static constexpr long long SMALL = 9000000000LL;
    vector<int> a;  // 低位在前，每位取值 [0, BASE)
    int sign = 1;   // +1 / -1

    bigint() {}
    bigint(long long v) { *this = v; }
    // 这条 int 重载不能删：字面量 0 同时是空指针常量，
    // 少了它，b < 0 这种写法会在 bigint(long long) 与 bigint(const char*) 之间二义
    bigint(int v) : bigint((long long)v) {}
    bigint(const string &s) { read(s); }
    bigint(const char *s) { read(s); }

    void operator=(long long v) {
        a.clear();
        sign = v < 0 ? -1 : 1;
        // 先转 unsigned 再取反，否则 v = LLONG_MIN 时 -v 溢出（UB）
        unsigned long long u = v < 0 ? -(unsigned long long)v : (unsigned long long)v;
        for (; u; u /= BASE) a.push_back(int(u % BASE));
        if (a.empty()) sign = 1;
    }
    void trim() {  // 去前导零，顺手把 -0 归一成 +0
        while (!a.empty() && a.back() == 0) a.pop_back();
        if (a.empty()) sign = 1;
    }
    bool isZero() const { return a.empty(); }
```

```
    bigint abs() const { bigint r = *this; r.sign = 1; return r; }
    bigint operator+() const { return *this; }
    bigint operator-() const { bigint r = *this; if (!r.isZero()) r.sign = -sign; return r; }

    // ---------- 比较 ----------
    static int cmpAbs(const bigint &x, const bigint &y) {  // 只比绝对值
        if (x.a.size() != y.a.size()) return x.a.size() < y.a.size() ? -1 : 1;
        for (int i = int(x.a.size()) - 1; i >= 0; --i)
            if (x.a[i] != y.a[i]) return x.a[i] < y.a[i] ? -1 : 1;
        return 0;
    }
    static int cmp(const bigint &x, const bigint &y) {
        if (x.sign != y.sign) return x.sign < y.sign ? -1 : 1;
        return cmpAbs(x, y) * x.sign;
    }
    // 六个比较运算符。写成 friend 是为了让两侧都能吃隐式转换，
    // b < 0 和 0 < b 都合法（写成成员函数就只有左侧能转）
    friend bool operator<(const bigint &x, const bigint &y) { return cmp(x, y) < 0; }
    friend bool operator>(const bigint &x, const bigint &y) { return cmp(x, y) > 0; }
    friend bool operator<=(const bigint &x, const bigint &y) { return cmp(x, y) <= 0; }
    friend bool operator>=(const bigint &x, const bigint &y) { return cmp(x, y) >= 0; }
    friend bool operator==(const bigint &x, const bigint &y) { return cmp(x, y) == 0; }
    friend bool operator!=(const bigint &x, const bigint &y) { return cmp(x, y) != 0; }

    // ---------- 加减 ----------
    static bigint addAbs(const bigint &x, const bigint &y) {  // |x| + |y|
        bigint r;
        r.a = x.a;
        for (int i = 0, carry = 0; i < int(y.a.size()) || carry; ++i) {
            if (i == int(r.a.size())) r.a.push_back(0);
            r.a[i] += carry + (i < int(y.a.size()) ? y.a[i] : 0);
            carry = r.a[i] >= BASE;
            if (carry) r.a[i] -= BASE;
        }
        return r;
    }
    static bigint subAbs(const bigint &x, const bigint &y) {  // |x| - |y|，要求 |x| >= |y|
        bigint r;
        r.a = x.a;
        for (int i = 0, borrow = 0; i < int(y.a.size()) || borrow; ++i) {
            r.a[i] -= borrow + (i < int(y.a.size()) ? y.a[i] : 0);
            borrow = r.a[i] < 0;
            if (borrow) r.a[i] += BASE;
        }
        r.trim();
        return r;
    }
    friend bigint operator+(const bigint &x, const bigint &y) {
        bigint r;
        if (x.sign == y.sign) r = addAbs(x, y), r.sign = x.sign;
        else if (cmpAbs(x, y) >= 0) r = subAbs(x, y), r.sign = x.sign;
        else r = subAbs(y, x), r.sign = y.sign;
        r.trim();
        return r;
    }
    friend bigint operator-(const bigint &x, const bigint &y) { return x + (-y); }

    // ---------- 与小整数运算（O(n) 快路径，divmod 内部靠它） ----------
    bigint &operator*=(long long v) {
        if (v > SMALL || v < -SMALL) return *this = *this * bigint(v);  // 超出快路径就退化成大整数乘
        if (v < 0) sign = -sign, v = -v;
        long long carry = 0;
        for (int i = 0; i < int(a.size()) || carry; ++i) {
```

```
            if (i == int(a.size())) a.push_back(0);
            long long cur = a[i] * v + carry;
            a[i] = int(cur % BASE);
            carry = cur / BASE;
        }
        trim();
        return *this;
    }
    bigint &operator/=(long long v) {  // 向零取整
        if (v > SMALL || v < -SMALL) return *this = *this / bigint(v);
        if (v < 0) sign = -sign, v = -v;
        long long rem = 0;
        for (int i = int(a.size()) - 1; i >= 0; --i) {
            long long cur = a[i] + rem * BASE;
            a[i] = int(cur / v);
            rem = cur % v;
        }
        trim();
        return *this;
    }
    int operator%(int v) const {  // 余数符号跟随被除数；v 取任意 int（含 INT_MIN）
        long long d = v < 0 ? -(long long)v : v, m = 0;
        for (int i = int(a.size()) - 1; i >= 0; --i) m = (a[i] + m * BASE) % d;
        return int(m) * sign;
    }
    friend bigint operator*(bigint x, long long v) { return x *= v; }
    friend bigint operator*(long long v, bigint x) { return x *= v; }
    friend bigint operator/(bigint x, long long v) { return x /= v; }

    // ---------- 大整数乘：拆成 1e6 压位后跑 Karatsuba ----------
    // 拆到 1e6 是为了让 karatsuba 内层 res[i+j] += a[i]*b[j] 累加不爆 long long
    static vector<int> convertBase(const vector<int> &v, int oldW, int newW) {
        vector<long long> p(max(oldW, newW) + 1);
        p[0] = 1;
        for (int i = 1; i < int(p.size()); ++i) p[i] = p[i - 1] * 10;
        vector<int> res;
        long long cur = 0;
        int curW = 0;
        for (int x : v) {
            cur += x * p[curW];
            curW += oldW;
            while (curW >= newW) {
                res.push_back(int(cur % p[newW]));
                cur /= p[newW];
                curW -= newW;
            }
        }
        res.push_back(int(cur));
        while (!res.empty() && res.back() == 0) res.pop_back();
        return res;
    }
    static vector<long long> karatsuba(const vector<long long> &x, const vector<long long> &y) {
        int n = int(x.size());
        vector<long long> res(n + n);
        if (n <= 32) {  // 小规模直接暴力，常数远小于递归
            for (int i = 0; i < n; ++i)
                for (int j = 0; j < n; ++j) res[i + j] += x[i] * y[j];
            return res;
        }
        int k = n >> 1;
        vector<long long> x1(x.begin(), x.begin() + k), x2(x.begin() + k, x.end());
        vector<long long> y1(y.begin(), y.begin() + k), y2(y.begin() + k, y.end());
        vector<long long> x1y1 = karatsuba(x1, y1), x2y2 = karatsuba(x2, y2);
```

```
        for (int i = 0; i < k; ++i) x2[i] += x1[i], y2[i] += y1[i];
        vector<long long> r = karatsuba(x2, y2);  // (x1+x2)(y1+y2) - x1y1 - x2y2
        for (int i = 0; i < int(x1y1.size()); ++i) r[i] -= x1y1[i];
        for (int i = 0; i < int(x2y2.size()); ++i) r[i] -= x2y2[i];
        for (int i = 0; i < int(r.size()); ++i) res[i + k] += r[i];
        for (int i = 0; i < int(x1y1.size()); ++i) res[i] += x1y1[i];
        for (int i = 0; i < int(x2y2.size()); ++i) res[i + n] += x2y2[i];
        return res;
    }
    friend bigint operator*(const bigint &x, const bigint &y) {
        vector<int> x6 = convertBase(x.a, W, 6), y6 = convertBase(y.a, W, 6);
        vector<long long> p(x6.begin(), x6.end()), q(y6.begin(), y6.end());
        while (p.size() < q.size()) p.push_back(0);
        while (q.size() < p.size()) q.push_back(0);
        while (p.size() & (p.size() - 1)) p.push_back(0), q.push_back(0);  // 补到 2 的幂
        vector<long long> c = karatsuba(p, q);
        bigint r;
        r.sign = x.sign * y.sign;
        long long carry = 0;  // 必须 long long: c[i] 量级可达 n*1e12, 进位早就爆 int
        for (int i = 0; i < int(c.size()); ++i) {
            long long cur = c[i] + carry;
            r.a.push_back(int(cur % 1000000));
            carry = cur / 1000000;
        }
        for (; carry; carry /= 1000000) r.a.push_back(int(carry % 1000000));
        r.a = convertBase(r.a, 6, W);
        r.trim();
        return r;
    }

    // ---------- 大整数除: Knuth 归一化 + 逐位试商 ----------
    // 返回 {商, 余数}; 商向零取整, 余数符号跟随被除数。除数为 0 是 UB, 调用方自己保证
    friend pair<bigint, bigint> divmod(const bigint &x, const bigint &y) {
        int norm = BASE / (y.a.back() + 1);  // 归一化让最高位 >= BASE/2, 试商误差 <= 2
        bigint u = x.abs() * norm, v = y.abs() * norm, q, r;
        q.a.assign(u.a.size(), 0);
        for (int i = int(u.a.size()) - 1; i >= 0; --i) {
            // r = r * BASE + u[i]: 把被除数的下一位拽下来。
            // 直接在低位插一格是 O(n) 的 memmove, 写成 r = r * BASE + u.a[i] 会慢一个数量级
            r.a.insert(r.a.begin(), u.a[i]);
            r.trim();
            int m = int(v.a.size());
            int s1 = int(r.a.size()) <= m ? 0 : r.a[m];
            int s2 = int(r.a.size()) <= m - 1 ? 0 : r.a[m - 1];
            int d = int((1LL * BASE * s1 + s2) / v.a.back());  // 试商, 至多偏大 2
            r = r - v * d;
            for (; r < 0; --d) r += v;
            q.a[i] = d;
        }
        q.sign = x.sign * y.sign;
        r.sign = x.sign;
        q.trim();
        r.trim();
        return {q, r / norm};
    }
    friend bigint operator/(const bigint &x, const bigint &y) { return divmod(x, y).first; }
    friend bigint operator%(const bigint &x, const bigint &y) { return divmod(x, y).second; }

    bigint &operator+=(const bigint &v) { return *this = *this + v; }
    bigint &operator-=(const bigint &v) { return *this = *this - v; }
    bigint &operator*=(const bigint &v) { return *this = *this * v; }
    bigint &operator/=(const bigint &v) { return *this = *this / v; }
    bigint &operator%=(const bigint &v) { return *this = *this % v; }
```

```
    friend bigint gcd(bigint x, bigint y) {  // 写成迭代，避免大数递归爆栈
        x = x.abs(), y = y.abs();
        while (!y.isZero()) { bigint t = x % y; x = y, y = t; }
        return x;
    }
    friend bigint lcm(const bigint &x, const bigint &y) { return (x / gcd(x, y) * y).abs(); }

    // ---------- 读写 ----------
    void read(const string &s) {
        sign = 1;
        a.clear();
        int pos = 0;
        while (pos < int(s.size()) && (s[pos] == '-' || s[pos] == '+')) {
            if (s[pos] == '-') sign = -sign;
            ++pos;
        }
        for (int i = int(s.size()) - 1; i >= pos; i -= W) {  // 从低位往高位每 W 个十进制位打一包
            int x = 0;
            for (int j = max(pos, i - W + 1); j <= i; ++j) x = x * 10 + (s[j] - '0');
            a.push_back(x);
        }
        trim();
    }
    string str() const {  // 不用 setw/setfill：那会把 cout 的 fill 字符永久改成 '0'
        string s = sign < 0 ? "-" : "";
        s += a.empty() ? "0" : to_string(a.back());
        for (int i = int(a.size()) - 2; i >= 0; --i) {
            string t = to_string(a[i]);
            s += string(W - t.size(), '0') + t;
        }
        return s;
    }
    friend istream &operator>>(istream &is, bigint &v) { string s; is >> s; v.read(s); return is; }
    friend ostream &operator<<(ostream &os, const bigint &v) { return os << v.str(); }
};
```

用法:

```
void Solve() {
    bigint a, b;
    cin >> a >> b;                  // 支持前导零与正负号
    cout << a + b << "\n";
    cout << a * b << "\n";
    if (!b.isZero()) {
        auto [q, r] = divmod(a, b);  // 一次拿到商和余数
        cout << q << " " << r << "\n";
        cout << gcd(a, b) << " " << lcm(a, b) << "\n";
    }
    bigint f = 1;
    for (int i = 2; i <= 100; ++i) f *= i;
    cout << f.str().size() << "\n";  // 100! 有 158 位
}
```

9 数据结构

9.1 单调栈

求右边第一个大于自己的数的下标，没有则记 0。

```
deque<ll> q;
FOR(i, 1, N) {
    while (q.empty() == false
           && A[i] > A[q.back()]) {
        Ans[q.back()] = i;
        q.pop_back();
    }
    q.push_back(i);
}
FOR(i, 1, N) cout << Ans[i] << " ";
cout << "\n";
```

什么时候能用单调栈? 求解条件和单调条件一**致**（或至少有关联，比如相反）即可。例如已知一组上界 up[i]（满足 up[i] < A[i]），想为每个 i 找最近的 $j > i$ 使得 up[j] - j >= -i + A[i]：可以让单调栈维护 $-j + A_j$ 的单调性，把比 up[i] - i 大的那些 $-j + A_j$ 都弹掉——因为 up[j] < A[j]，所以 up[j] - j >= -i + A[i] 时 A[j] - j >= -i + A[i] 也成立。

```
q.clear();
FOR(i, 1, N) {
    // 维护 -j + A[j] 的单调性
    while (q.empty() == false
           && up[i] - i < -q.back() +
           ↪ A[q.back()]) {
        seb[q.back()] = i;
        q.pop_back();
    }
    q.push_back(i);
}
```

9.2 离散化

将值域很大但数量有限的数据映射到连续的小区间上，常用于线段树、树状数组等需要以值作为下标的场景。

使用方法:

1. 创建对象并用 add 添加所有需要离散化的值，然后调用 init() 完成排序去重。
2. get_i(x)：返回 x 在离散化后的新下标。
3. operator[](idx)：根据新下标反查原始值。
4. projectLr(l, r)：将原始区间 $[l, r]$ 投影到离散化空间，返回新区间 $[li, ri]$ 以及是否合法。
5. contains(x)：查询 x 是否在离散化集合中。
6. 通过 shift 控制下标基准（0 为 0-based，1 为 1-based）。

参考来源：hh2048/XCPC 数据结构模板

```
/* 参考了 hh2048/XCPC
 * AC https://acm.hdu.edu.cn/contest/view-code?
 ↪  cid=12006&rid=14607
 * 上面这个测试了这个 projectLr,get_i
 ↪  应该是没什么问题
 */
template<typename T>
struct Compress_ {
  // shift=0 就是采用 0-based 下标
  // shift=1 就是采用 1-based 下标
  int n, shift = 0;
  vector<T> liLs;

  Compress_() {
  }

  // 预估大小
  Compress_(ll n_) {
    liLs.reserve(n_);
  }

  Compress_(vector<T> in) : liLs(in) {
    init();
  }

  void add(T x) {
    liLs.emplace_back(x);
  }

  template<typename... Args>
  void add(T x, Args... args) {
    add(x), add(args...);
  }

  void init() {
    liLs.emplace_back(numeric_limits<T>::max());
    sort(liLs.begin(), liLs.end());
    liLs.resize(unique(liLs.begin(),
    ↪  liLs.end()) - liLs.begin());
    this->n = liLs.size();
  }

  int size() {
    return n;
  }

  int get_i(T x) {
    // 返回 x 元素的新下标
    return lower_bound(liLs.begin(),
    ↪  liLs.end(), x) - liLs.begin() + shift;
  }

  struct ProjectResult {
    bool ok;
    int l, r;
  };

  ProjectResult projectLr(T l, T r) {
    // 将区间 [l, r] 投影到离散化空间
    // li: 第一个 >= l 的离散化位置
```

```
    // ri: 最后一个 <= r 的离散化位置
    int li = (int) (lower_bound(liLs.begin(),
    ↪  liLs.end(), l) - liLs.begin()) + shift;
    int ri = (int) (upper_bound(liLs.begin(),
    ↪  liLs.end(), r) - liLs.begin()) - 1 +
    ↪  shift;
    return {li <= ri, li, ri};
  }

  T operator[](int idx) {
    // 根据新下标返回原来元素
    assert(idx - shift < n);
    return idx - shift < n ? liLs[idx - shift]
    ↪  : -1;
  }

  bool contains(T x) {
    // 查找元素 x 是否存在
    return binary_search(liLs.begin(),
    ↪  liLs.end(), x);
  }

  void reserve(ll n_) {
    liLs.reserve(n_);
  }

  friend auto &operator<<(ostream &o, const
  ↪  auto &j) {
    cout << "{";
    for (auto it: j.alls) {
      o << it << " ";
    }
    return o << "}";
  }
};
```

典型用法示例：

```
Compress_<int> compress;
// 添加需要离散化的值
compress.add(100, 200, 500);
compress.init();

// 查询离散化后的下标
int idx = compress.get_i(200); // 1 (0-based)

// 反查原始值
int val = compress[1]; // 200

// 区间投影
auto [ok, l, r] = compress.projectLr(100, 500);
// ok=true, l=0, r=2
```

9.3 ST 表（Sparse Table）

ST 表用于 $O(n \log n)$ 预处理、$O(1)$ 回答静态区间**可重复贡献查询**（如区间最值、区间 GCD 等）。模板通过模板参数 `F` 接受任意二元运算，支持函数对象和 lambda。

```
/*
 * AC https://codeforces.com/contest/2175/submi
 ↪  ssion/352780861
 * 实现了 op 函数的传入（可传入 lambda）
 * AC https://codeforces.com/contest/2205/submi
 ↪  ssion/365029208
 */
template<class T, class F>
struct ST {
  ll N;
  vector<vector<T> > dp;
  F op;

  ST(ll N, const vector<T> &a, F fun)
    : N(N), dp(__lg(N) + 1,
      vector<T>(N + 5)), op(fun) {
    for (int i = 1; i <= N; ++i) {
      dp[0][i] = a[i];
    }
    for (int lay = 1; lay <= __lg(N); ++lay) {
      for (int i = 1;
        i <= N - (1 << (lay - 1)); ++i) {
        dp[lay][i] = op(dp[lay - 1][i],
          dp[lay - 1][i + (1 << (lay - 1))]);
      }
    }
  }

  T query(ll l, ll r) {
    ll lay = __lg(r - l + 1);
    return op(dp[lay][l],
      dp[lay][r - (1 << lay) + 1]);
  }
};
```

定长数组版（全局定义，防爆栈）

上面的 `vector` 版每次构造都要动态分配；下面这版把 `dp` 换成定长二维数组、运算用**函数指针模板参数** `T (*op)(T, T)`（编译期确定，可内联，零开销），必须**全局或 static 定义**，否则栈上放不下。`init` 只要求 `a[i]` 可下标访问，`vector` / 裸数组 / 指针都能传。

```
/* 2026/8/1 定长数组版，全局定义防爆栈
 * MAXN 传最大下标即可（内部已 +5）
 * 下标 1..N，闭区间 query(l, r)
 * 用法：const int N = 200005;
 *   int mx(int a, int b) { return max(a, b); }
 *   ST<int, mx, N> st;    <- 必须全局 / static
 *   st.init(n, a);        <- a 是 vector 或 int
 ↪  a[N]
 */
template<class T, T (*op)(T, T), int MAXN>
struct ST {
private:
  static constexpr int LOG = __lg(MAXN) + 1;
  ll N;
  T dp[LOG][MAXN + 5];

public:
  ST() {}

  template<class A>
  ST(ll N, const A &a) { init(N, a); }

  // a 只需支持 a[i]，i 取 1..n
```

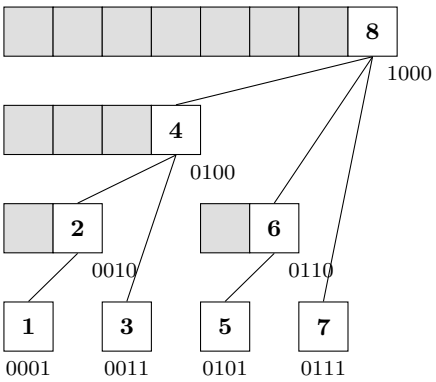
```
  // vector / T[] / T* / array 均可
  template<class A>
  void init(ll n, const A &a) {
    N = n;
    for (int i = 1; i <= N; ++i) {
      dp[0][i] = a[i];
    }
    for (int lay = 1; lay <= __lg(N); ++lay) {
      for (int i = 1;
        i + (1 << lay) - 1 <= N; ++i) {
        dp[lay][i] = op(dp[lay - 1][i],
          dp[lay - 1][i + (1 << (lay - 1))]);
      }
    }
  }

  T query(ll l, ll r) {
    ll lay = __lg(r - l + 1);
    return op(dp[lay][l],
      dp[lay][r - (1 << lay) + 1]);
  }
};
```

9.4 树状数组（BIT / Fenwick Tree）

按支持的「修改方式 + 查询方式」分成三个版本，从简单到复杂依次是：单点加区间和、区间加单点查、区间加区间和。所有版本下标都从 1 开始，且都依赖 `lowbit(x) = x & (-x)`。

下图是 $n = 8$ 时 BIT 的经典结构示意图：第 i 格的条形宽度就是 `lowbit(i)`，最右那格（白色）标号为 i，下面标它的二进制。连线表示「合并关系」，即 `tr[i]` 由哪几个更小的块拼出来。



例如 `tr[8]` 覆盖区间 $[1, 8]$，它正好由 `tr[4]`（管 $[1, 4]$）、`tr[6]`（管 $[5, 6]$）、`tr[7]`（管 $\{7\}$）和 a_8 本身拼成；而 `tr[4]` 又由 `tr[2]`、`tr[3]` 和 a_4 拼成。所有 BIT 操作（add 沿 `i += lowbit(i)` 向上、query 沿 `i -= lowbit(i)` 向左跳）都能从这张图里直接读出来。

9.4.1 单点加，区间求和

最基础的 BIT：`add(p, x)` 给位置 p 加上 x，`query(l, r)` 返回 `a[l]+a[l+1]+...+a[r]`。下面这份在常规功能之外还带两个小工具，前提是数组里只存 0/1（常用于维护一个动态可重集的占位表）：

- `findkth(k)`：从左往右数第 k 个 1 所在的位置。
- `kthspace(k)`：从左往右数第 k 个 0 所在的位置。

两者思路一样：在 BIT 上按位倍增地往右走，每到一步判断「这个块里 1（或 0）的总数够不够吃掉剩下的 k」，够就跨过，不够就留在当前位置继续往下走。整个过程 $O(\log n)$。

```
// by znzryb
// 普通的单点修改区间查询树状数组，
// 含有查找第 K 个空位，第 K 个元素的功能
// https://www.codechef.com/viewsolution/117989」
↪  3708
//   查找第 K 个元素测试
// 第K个空位的测试应该是一道abc的题目，
↪  但是当时没保存
struct BIT {
  vector<ll> tr;
  int n;

  inline int lowbit(int x) {
    return x & (-x);
  }

  BIT(int ln) {
    n = ln;
    tr.resize(n + 5, 0);
  }
  BIT() {}

  inline void add(int p, ll x) {
    if (p == 0) return;
    while (p <= n) {
      tr[p] += x;
      p += lowbit(p);
    }
  }

  inline ll query(int l, int r) {
    ll lres = 0, rres = 0;
    l -= 1;
    while (l > 0) {
      lres += tr[l];
      l -= lowbit(l);
    }
    while (r > 0) {
      rres += tr[r];
      r -= lowbit(r);
    }
    return rres - lres;
  }

  // 找到第 k 个元素，要求只能出现 0 和 1
  inline int findkth(int k) {
    int x = 0, sum = 0;
    for (int i = __lg(n); ~i; --i) {
      x += 1 << i;
      if (x >= n || sum + tr[x] >= k) {
        x -= 1 << i;
      } else {
        sum += tr[x];
      }
    }
```

```
    // 始终让 cx < k, +1 就是新元素出现的位置
    return x + 1;
  }

  // 找到第 k 个空位，要求只能出现 0 和 1
  inline int kthspace(int k) {
    int cx = 0;
    // 不断缩小步长的搜索
    for (int i = 1 << 20; i > 0; i >>= 1) {
      if (cx + i <= n &&
        lowbit(cx + i) - tr[cx + i] < k) {
        // lowbit(cx+i) - tr[cx+i]
        //   即该 BIT 块内的空位数
        k -= lowbit(cx + i) - tr[cx + i];
        cx += i;
      }
    }
    return cx + 1;
  }
};
```

9.4.2 区间加，单点查询

BIT 不再维护原数组 a，而是维护差分数组 d[i] = a[i] - a[i-1]。这样：

- **区间加** a[l..r] += x：只改差分的两个位置，d[l] += x; d[r+1] -= x;
- **单点查** a[p]：就是差分的前缀和 d[1] + d[2] + ... + d[p]，BIT 天然支持。

所以这一版的 add(l, r, x) 就是对差分做两次单点加，query(p) 就是差分的前缀和查询。

```
struct BIT {
  // 支持区间修改，单点查询（使用差分实现）
  // https://www.luogu.com.cn/record/221012815
  vector<ll> tr;
  int n;

  ll lowbit(ll x) {
    return x & (-x);
  }

  BIT(int n_) {
    n = n_;
    tr.resize(n, 0);
  }

  // 差分的单点加，内部用
  void add(ll p, ll x) {
    while (p <= n) {
      tr[p] += x;
      p += lowbit(p);
    }
  }

  // 区间加：a[l..r] += x
  //   等价于 d[l] += x, d[r+1] -= x
  void add(ll l, ll r, ll x) {
    r += 1;
    add(l, x);
    add(r, -x);
  }

  // 单点查 a[p] = d[1] + d[2] + ... + d[p]
  ll query(ll p) {
    ll ret = 0;
    while (p) {
      ret += tr[p];
      p -= lowbit(p);
    }
    return ret;
  }
};
```

9.4.3 区间加，区间求和

继续沿用差分 d[i] = a[i] - a[i-1]。区间加依然是两次 d[l] += x; d[r+1] -= x。难点在区间和查询，先看怎么算前缀和 a[1] + a[2] + ... + a[P]，把每一项用差分展开：

```
a[1..P] = d[1]
        + d[1] + d[2]
        + d[1] + d[2] + d[3]
        + ...
        + d[1] + d[2] + ... + d[P]

数一下每个 d[i] 出现了几次：
  d[1] 出现 P 次
  d[2] 出现 P-1 次
  ...
  d[i] 出现 (P - i + 1) 次

所以合并起来：
a[1..P] = sum{ d[i] * (P - i + 1), i = 1..P }
        = (P+1) * sum(d[1..P])
            - sum(i * d[i], i = 1..P)
```

右边两项都是普通的前缀和，各用一个 BIT 就行：tr1 存 d[i]，tr2 存 i * d[i]。于是 query(P) = (P+1) * sum(tr1, P) - sum(tr2, P)，区间和就是 query(r) - query(l-1)。

区间加 [l, r] += x 时，两个 BIT 要同步改：

```
对 tr1 (存 d[i]):
  tr1 在 l   处 += x
  tr1 在 r+1 处 -= x

对 tr2 (存 i * d[i]):
  i = l   时: d[l]   += x  =>  i*d[i] +=   l  * x
  i = r+1 时: d[r+1] -= x  =>  i*d[i] -= (r+1) *
  ↪  x
```

注意乘的系数 l、r+1 在修改时就是已知常数，所以能直接塞进 tr2 的单点加里。

```
/* 2026-04-19 luogu 线段树模板题
 * AC https://www.luogu.com.cn/record/274702532
 * 进一步添加了 public,private 规定了接口的可见性
```

```cpp
 * 2026-04-19 增强了对 0 的鲁棒性
 * AC https://acm.hdu.edu.cn/contest/view-code?cid=1201&rid=9803
 */
class BIT {
 // 支持区间修改，区间查询
 // 维护两个差分：d[i] 和 i*d[i]
 // 前缀和 sum(p) = (p+1)*sum(d[1..p])
 //              - sum(i*d[i], 1..p)
 vector<ll> tr1, tr2;
 int n;

 ll lowbit(ll x) {
   return x & (-x);
 }

public:
 BIT(int n_) {
   n = n_;
   // 这个 n+1 就够了，虽然涉及到这个差分
   tr1.assign(n + 2, 0);
   tr2.assign(n + 2, 0);
 }

private:
 void add(vector<ll> &tr, ll p, ll x) {
   // 防止 0 死循环
   if (p <= 0) return;
   while (p <= n) {
     tr[p] += x;
     p += lowbit(p);
   }
 }

 ll sum(vector<ll> &tr, ll p) {
   ll ret = 0;
   while (p) {
     ret += tr[p];
     p -= lowbit(p);
   }
   return ret;
 }

 // 单点加
 void add(ll p, ll x) {
   add(tr1, p, x);
   add(tr2, p, p * x);
 }

public:
 // 区间加，[l, r] 每个位置加 x
 void add(ll l, ll r, ll x) {
   add(tr1, l, x);
   add(tr1, r + 1, -x);
   add(tr2, l, l * x);
   add(tr2, r + 1, -(r + 1) * x);
 }

private:
 // 前缀和 a[1] + a[2] + ... + a[p]
 ll query(ll p) {
   return (p + 1) * sum(tr1, p) - sum(tr2, p);
 }

public:
 // 区间和 a[l] + ... + a[r]
 ll query(ll l, ll r) {
   return query(r) - query(l - 1);
 }
};
```

技巧：提前结算避免树状数组清零 从这道题目学到的技巧

树状数组不支持高效的区间清零（需要逐点撤销）。但如果我们知道某段区间 $[l,r]$ 上累积的值最终要**贡献给谁**，就可以在需要“清零”的时刻，先把 BIT 上 $[l,r]$ 的当前和查出来、结算到对应的答案里，然后**不做任何清零操作**。之后 BIT 上再产生的增量会叠加在旧值之上，但因为旧值已经被结算走了，所以只要在最终统计时再查一次并减去” 已结算部分”，效果就等价于清零后重新累加。

典型场景：维护若干组的区间贡献，当某段区间的归属从旧组切换到新组时：

```cpp
// 将 [l, r] 的归属从 originV 切换到 val，不清零 BIT
ll add = tr->query(l, r);   // 查出 [l, r] 上的已有增量
anss[originV] += add;         // 旧归属结算
anss[val] -= add;             // 新归属预扣（之后查询时会连带查出这部分旧值）
```

9.4.4 单点改，区间最值查询（BIT 实现 RMQ）

和求和不同，min / max 这类运算**只有结合律、没有逆运算**（即没有「区间减法」），所以不能像前缀和那样用 $\mathrm{pre}(r)-\mathrm{pre}(l-1)$ 直接拿到任意区间的结果。不过 BIT 的分块结构只依赖结合律就够用了：每个 `tr[i]` 管辖 `a[i - lowbit(i) + 1 .. i]` 这个区间的最值，因此整个骨架仍然可以复用，只是查询和更新都必须按块拼凑，复杂度从 $O(\log n)$ 退化到 $O(\log^2 n)$。相对于 ST 表的优势是**支持在线单点修改**。

查询 `query(l, r)`：从右端 `r` 往左跳，每次能吃掉一整个 BIT 块就吃（`tr[r]`，跨度 `lowbit(r)`），吃不动的那一格（即 `r - lowbit(r) < l` 的时候）退化为单独看 `a[r]`，然后 `r--` 继续：

```python
res = dummy
while r >= l:
    res = op(res, a[r])          // 单独处理 a[r]
    r -= 1
    while r - lowbit(r) >= l:    // 尽量用 BIT 块跳
        res = op(res, tr[r])
        r -= lowbit(r)
```

更新 `upd(p, x)`：把 `a[p]` 覆盖为 `x` 之后，所有「管辖 `p` 的 BIT 块」都要重算。这些块的下标就是

```
i = p,
    p + lowbit(p),
    p + lowbit(p) + lowbit(p + lowbit(p)),
    ...
```

也就是外层循环 `i += lowbit(i)` 依次枚举的那些位置。由于没有区间减法，无法从旧 `tr[i]` 里扣掉旧 `a[p]` 的贡献，所以只能**从头合并整块**。

好在 BIT 的管辖区间有一个漂亮的自相似分解——它能切成若干长度是 2 的幂、两两不相交的子段，而这些子段本身就是已有的更小的 `tr`：

```
tr[i] 管辖 [i - lowbit(i) + 1 .. i]，长度
↪ lowbit(i)
可拆成：
    tr[i - lowbit(i)/2]     长度 lowbit(i)/2   //
    ↪ 左半
    tr[i - lowbit(i)/4]     长度 lowbit(i)/4   //
    ↪ 再右半的一半
    ...
    tr[i - 2]               长度 2
    tr[i - 1]               长度 1
    a[i]                    长度 1             //
    ↪ 自己这一格

验证：1 + 1 + 2 + 4 + ... + lowbit(i)/2 =
↪ lowbit(i)
      刚好铺满管辖区间。
```

所以重算就是：

```
tr[i] = op(a[i],
           tr[i - 1],
           tr[i - 2],
           tr[i - 4],
           ...,
           tr[i - lowbit(i)/2])
```

对应代码里的内层循环——让 `j` 从 1 按 2 的幂翻倍直到 `lowbit(i)/2`。

```
// by Gospel_rock znzryb
// 参考了（抄了）OIwiki
// AC https://www.luogu.com.cn/record/228807838
//    （ST 表模版题，max 版本）
// AC https://www.luogu.com.cn/record/228809172
//    （打乱了插入顺序，同题）
// AC https://codeforces.com/contest/2184/submi」
↪  ssion/358028437
// 进行了现代化改造，更加符合现在我们的习惯
struct RMQ {
  const ll dummy = INF;
  ll n;
  vector<ll> tr;
  vector<ll> origin;

  inline ll lowbit(const ll &x) {
    return x & -x;
  }

private:
  ll op(const ll &a, const ll &b) {
    return min(a, b);
  }

public:
  RMQ(ll n) {
    this->n = n + 2;
    tr.resize(n + 5, INF);
    origin.resize(n + 5);
  }

  ll query(ll l, ll r) {
    ll res = dummy;
    while (r >= l) {
      res = op(origin[r], res);
      r--;
      for (; r - lowbit(r) >= l; r -=
      ↪  lowbit(r)) {
        res = op(res, tr[r]);
      }
    }
    return res;
  }

  // 注意：语义是「把位置 p 覆盖为 x」，不是增加
  void upd(ll p, ll x) {
    origin[p] = x;
    for (int i = p; i <= n; i += lowbit(i)) {
      tr[i] = origin[i];
      for (int j = 1; j < lowbit(i); j <<= 1) {
        tr[i] = op(tr[i], tr[i - j]);
      }
    }
  }
};
```

切换 `min/max`：把 `op` 改成 `max`、同时把 `dummy` 从 `INF` 改成 `-INF` 即可。如果要泛化成任意幂等运算（操作一次和操作十次结果一样如，gcd、按位与/或等），需要选一个对该运算「吸收」的恒元作为 `dummy`。

9.5 珂朵莉树（ODT / Chtholly Tree）

珂朵莉树用 `std::map` 维护若干不相交的「颜色段」`[l, r, val]`。核心思路：**每次区间赋值把覆盖到的所有旧段一次性合并删除**，再插入一段新段。在随机数据或有大量区间覆盖操作的题目中，段数期望 $O(n \log n)$，因此总复杂度可接受。

两个核心操作：

- `split(pos)`：把包含 `pos` 的段在 `pos` 处切开，保证 `pos` 是某段的左端点。
- `assign(la, ra, val)`：先 `split` 两端，然后一次性抹掉 $[la, ra]$ 内所有旧段，插入新段。在抹掉每段时可顺带做任意聚合统计（如下方用 BIT 查区间权重和并累到 `anss` 上）。

```
/*
 * 2026-04-19 内部式的 ODT
 * AC https://acm.hdu.edu.cn/contest/view-cod」
 ↪  e?cid=1201&rid=9850
 */
struct RVal {
  ll r;
  ll val;
};
```

```
  map<ll, RVal> mpODT;

  void split(ll pos) {
    if (mpODT.empty()) return;
    auto it = mpODT.upper_bound(pos);
    if (it == mpODT.begin()) return;
    it--;
    // pos 就是左端点          pos 不在区间内
    if (it->first == pos || it->second.r < pos)
    ↪  {
      return;
    }
    mpODT[pos] = {it->second.r, it->second.val};
    it->second.r = pos - 1;
  }

  void assign(ll la, ll ra, ll val) {
    split(la);
    split(ra + 1);
    auto it = mpODT.lower_bound(la);
#ifdef LOCAL
    assert(it->first==la);
#endif
    for (; it != mpODT.end();) {
      if (it->first > ra) {
        break;
      }
      ll originV = it->second.val;
      ll l = it->first, r = it->second.r;
      // 这里是做一些事情，比如说累加答案
      // ll add = tr->query(l, r);
      // anss[originV] += add;
      // anss[val] -= add;
      it = mpODT.erase(it);
    }
    mpODT[la] = {ra, val};
  }
```

9.6 带权可撤销并查集（判二分图）

带权可撤销并查集（队友的模版），可以用来判断图是否是二分图。**一般和线段树分治绑定**：分治进入子区间前 `snapshot()`，离开时 `rollback()` 回到该快照，省去为每个时间区间重建图的代价。`val[x]` 维护 `x` 到 `fa[x]` 的边权异或值，`find` 不做路径压缩（路径压缩与回滚不兼容），靠按秩合并保 $O(\log n)$ 单次。`badCnt` 计数当前累计的「奇环」（合并时 `valx ^ valy != w` 即一条奇环），`isBipartiteGraph()` 即 `badCnt == 0`。

```
class RollbackDSU
{
  struct Info
  {
    int x, y, oldSizeY, oldBadCnt, oldValX;
    bool isMerged;
  };

  int badCnt;
  vector<int> fa, sz, val;
  stack<Info> history;

public:
  RollbackDSU(int n) : badCnt(0), fa(n + 1),
  ↪  sz(n + 1, 1), val(n + 1, 0)
  {
    iota(fa.begin(), fa.end(), 0);
  }

  int snapshot()
  {
    return (int)history.size();
  }

  void rollback(int snapshot)
  {
    while ((int)history.size() > snapshot)
    {
      auto [x, y, oldSizeY, oldBadCnt, oldValX,
      ↪  isMerged] = history.top();
      history.pop();

      badCnt = oldBadCnt;

      if (!isMerged)
      {
        continue;
      }

      val[x] = oldValX;
      sz[y] = oldSizeY;
      fa[x] = x;
    }
  }

  pair<int, int> find(int x)
  {
    int cur = 0;

    while (fa[x] != x)
    {
      cur ^= val[x];
      x = fa[x];
    }

    return {x, cur};
  }

  bool merge(int x, int y, int w)
  {
    auto [fx, valx] = find(x);
    auto [fy, valy] = find(y);

    if (fx == fy)
    {
      history.push({0, 0, 0, badCnt, 0, false});

      if ((valx ^ valy) != w)
      {
        badCnt++;
      }

      return false;
    }

    if (sz[fx] > sz[fy])
```

```
    {
      swap(fx, fy);
      swap(valx, valy);
    }

    history.push({fx, fy, sz[fy], badCnt,
    ↪  val[fx], true});

    sz[fy] += sz[fx];
    val[fx] = valx ^ valy ^ w;
    fa[fx] = fy;

    return true;
  }

  bool isBipartiteGraph() const
  {
    return badCnt == 0;
  }
};
```

9.7 线段树分治

线段树分治模板（队友的模版），常和上面的可撤销带权并查集绑定，用来**离线处理「在时间轴上每条边只在区间 $[l,r]$ 内存在」类型的问题**。把每条边按其存活时间区间挂到线段树对应的若干 $O(\log q)$ 个节点上；DFS 整棵线段树，进入节点前依次 `merge` 该节点上挂的所有边，离开节点前 `rollback` 到进入时的 `snapshot`，到叶子时即可拿到该时间点的并查集状态（如查二分图：`isBipartiteGraph()`）。

```
struct SegmentTreeDivide {
  int q;
  vector<vector<pair<int, int>>> seg;

  SegmentTreeDivide() {}

  SegmentTreeDivide(int q) {
    init(q);
  }

  void init(int q_) {
    q = q_;
    seg.assign(4 * q + 5, {});
  }

  void add(int node, int l, int r, int ql, int
  ↪  qr, pair<int, int> edge) {
    if (ql <= l && r <= qr) {
      seg[node].push_back(edge);
      return;
    }

    int mid = (l + r) >> 1;

    if (ql <= mid) {
      add(node << 1, l, mid, ql, qr, edge);
    }

    if (qr > mid) {
      add(node << 1 | 1, mid + 1, r, ql, qr,
      ↪  edge);
    }
  }

  void add(int l, int r, pair<int, int> edge) {
    if (l > r) {
      return;
    }

    add(1, 1, q, l, r, edge);
  }
};
```

9.8 ACL segtree（单点修改）

维护满足**结合律**的二元运算 op 与其**单位元** e（即幺半群 monoid）上的序列，支持单点赋值、区间求积与线段树上二分，单次操作均为 $O(\log n)$。

接口一览（区间一律 $[l, r)$ 左闭右闭；seg(n) 的合法位置是 $0 \dots n$，共 $n+1$ 个叶子）：

- segtree<S, op, e> seg(n)：全部初始化为 e()。
- segtree<S, op, e> seg(v)：由 vector<S> 在 $O(n)$ 内建树。
- set(p,x) / get(p)：单点赋值 / 单点读取。
- prod(l,r) / all_prod()：闭区间 / 全区间求积；$r = l - 1$ 表示空区间。
- max_right(l,f)：返回最大的 r，使 $f(op(a_l, \dots, a_r))$ 为真；取不到时返回 $l - 1$。
- min_left(r,f)：返回最小的 l，使 $f(op(a_l, \dots, a_r))$ 为真；取不到时返回 $r + 1$。

注意：op 与 e 必须是自由函数或函数指针；e() 必须是真单位元。max_right/min_left 的谓词必须满足 f(e()) == true，且沿扩展方向只能从真变假一次。

9.8.1 模板本体

```
/* AtCoder Library v1.6 atcoder/segtree 自包含移植 + 闭区间
 * segtree(n) 合法位置 0..n; 区间 [l, r] 左闭右闭
 * 1-based 只碰 1..n; 0-based 只碰 0..n-1
 * 从 vector 建树时 v[i] 对应位置 i, 长度由 vector 决定
 * 空区间写 r == l - 1, prod 返回 e()
 */
template<class S, S (*op)(S, S), S (*e)()>
struct segtree {
public:
    segtree() : segtree(0) {}

    explicit segtree(int n)
        : segtree(vector<S>(n + 1, e())) {}

    explicit segtree(const vector<S> &v)
        : _n(int(v.size())) {
        size = (int) bit_ceil((unsigned) _n);
        log = countr_zero((unsigned) size);
        d = vector<S>(2 * size, e());
        for (int i = 0; i < _n; i++)
            d[size + i] = v[i];
        for (int i = size - 1; i >= 1; i--)
            update(i);
    }

    void set(int p, S x) {
        assert(0 <= p && p < _n);
        p += size;
        d[p] = x;
        for (int i = 1; i <= log; i++)
            update(p >> i);
    }

    S get(int p) const {
        assert(0 <= p && p < _n);
        return d[p + size];
    }

    S prod(int l, int r) const {
        r++;
        assert(0 <= l && l <= r && r <= _n);
        S sml = e(), smr = e();
```

```
        l += size, r += size;
        while (l < r) {
            if (l & 1) sml = op(sml, d[l++]);
            if (r & 1) smr = op(d[--r], smr);
            l >>= 1, r >>= 1;
        }
        return op(sml, smr);
    }

    S all_prod() const { return d[1]; }

    template<class F>
    int max_right(int l, F f) const {
        assert(0 <= l && l <= _n);
        assert(f(e()));
        if (l == _n) return _n - 1;
        l += size;
        S sm = e();
        do {
            while (l % 2 == 0) l >>= 1;
            if (!f(op(sm, d[l]))) {
                while (l < size) {
                    l = 2 * l;
                    if (f(op(sm, d[l]))) {
                        sm = op(sm, d[l]);
                        l++;
                    }
                }
                return (l - size) - 1;
            }
            sm = op(sm, d[l]);
            l++;
        } while ((l & -l) != l);
        return _n - 1;
    }

    template<class F>
    int min_left(int r, F f) const {
        r++;
        assert(0 <= r && r <= _n);
        assert(f(e()));
        if (r == 0) return 0;
        r += size;
        S sm = e();
        do {
            r--;
            while (r > 1 && (r % 2)) r >>= 1;
            if (!f(op(d[r], sm))) {
                while (r < size) {
                    r = 2 * r + 1;
                    if (f(op(d[r], sm))) {
                        sm = op(d[r], sm);
                        r--;
                    }
                }
                return r + 1 - size;
            }
            sm = op(d[r], sm);
        } while ((r & -r) != r);
        return 0;
    }

private:
```

```
    int _n, size, log;
    vector<S> d;

    static unsigned bit_ceil(unsigned n) {
        unsigned x = 1;
        while (x < n) x *= 2;
        return x;
    }

    static int countr_zero(unsigned n) {
        return __builtin_ctz(n);
    }

    void update(int k) {
        d[k] = op(d[2 * k], d[2 * k + 1]);
    }
};
```

9.8.2 示例：单点赋值 + 区间最大值

负数序列中，最大值的单位元必须是足够小的数，不能写成 0。

```
// ACL_EXAMPLE: seg_point_max
using S = ll;

S op_max(S a, S b) { return max(a, b); }
S e_max() { return -4e18; }

using SegMax = segtree<S, op_max, e_max>;

// vector<S> a(n + 1, e_max()); // 1-based
// SegMax seg(a);
// seg.set(p, x);
// ll ans = seg.prod(l, r);
```

9.8.3 示例：单点修改 + 最大子段和

节点保存总和、最大前缀、最大后缀和最大子段和。额外的 empty 让空区间成为真正单位元，也避免用哨兵参与加法。

```
// ACL_EXAMPLE: seg_max_subarray
struct S {
    ll sum, pre, suf, best;
    bool empty;
};

S op(S a, S b) {
    if (a.empty) return b;
    if (b.empty) return a;
    return {
        a.sum + b.sum,
        max(a.pre, a.sum + b.pre),
        max(b.suf, b.sum + a.suf),
        max({a.best, b.best, a.suf + b.pre}),
        false
    };
}

S e() { return {0, 0, 0, 0, true}; }
S make_leaf(ll x) { return {x, x, x, x, false}; }
↪ }

using SegMaxSubarray = segtree<S, op, e>;

// vector<S> a(n + 1, e());
// FOR(i, 1, n) a[i] = make_leaf(x[i]);
// SegMaxSubarray seg(a);
// seg.set(p, make_leaf(x));
// ll ans = seg.prod(l, r).best;
```

9.8.4 示例：第 k 个 1 与左右边界

当叶子只有 0/1 时，前缀计数具有单调性。max_right 找到“计数仍小于 k”的最后位置，再加一就是第 k 个 1。

```
// ACL_EXAMPLE: seg_kth_boundary
using S = ll;
S op(S a, S b) { return a + b; }
S e() { return 0; }
using SegCount = segtree<S, op, e>;

int kth_one(const SegCount &seg, ll k) {
    if (k <= 0 || seg.all_prod() < k) return -1;
    return seg.max_right(1, [&](S cnt) {
        return cnt < k;
    }) + 1;
}

int farthest_right(const SegCount &seg, int l,
↪ ll x) {
    // 元素非负时，区间和 <= x 才沿右扩单调。
    return seg.max_right(l, [&](S sum) {
        return sum <= x;
    });
}

int farthest_left(const SegCount &seg, int r,
↪ ll x) {
    int l = seg.min_left(r, [&](S sum) {
        return sum <= x;
    });
    return max(1, l); // 1-based 时排除额外的 0
    ↪ 号单位元
}
```

9.8.5 示例：含负数时找首个前缀和 $\geq x$

有负数时不能拿区间和判定单调性；节点必须维护“这段能达到的最大前缀和”pre。若累计信息的 pre < x，说明还没遇到答案。

```
// ACL_EXAMPLE: seg_negative_prefix
struct S {
    ll sum, pre;
    bool empty;
};

S op(S a, S b) {
    if (a.empty) return b;
    if (b.empty) return a;
    return {a.sum + b.sum,
            max(a.pre, a.sum + b.pre), false};
}

S e() { return {0, 0, true}; }
S make_leaf(ll x) { return {x, x, false}; }
using SegPrefix = segtree<S, op, e>;

int first_prefix_at_least(
        const SegPrefix &seg, int l, int n, ll
        ↪ x) {
    if (seg.prod(l, n).pre < x) return -1;
    int last_bad = seg.max_right(l, [&](S cur) {
        return cur.empty || cur.pre < x;
    });
    return last_bad + 1;
}
```

9.9 ACL `lazy_segtree`（区间修改）

在 `segtree` 的幺半群 (S, op, e) 上再定义作用幺半群 $(F, composition, id)$，即可支持区间修改、区间查询和树上二分，单次操作均为 $O(\log n)$。

五件套约束：`op` 必须满足结合律；`e` 是其单位元；`mapping(f,x)` 表示把作用 f 施加到 x；`composition(f,g)` 表示“先 g 后 f”；`id` 是恒等作用。作用还必须满足对 `op` 的分配律。

精简写法：`up(k)` 写成 `d[k] = mapping(lz[k], op(左, 右))`，即不变式「节点值 = 自身标记作用在两儿子之积上」，所以区间两端的下推 / 上拉不必像 ACL 那样判断边界是否对齐，无条件做即可；代价是每次上拉多一次 `mapping`，复杂度不变。

最大的坑：`mapping` 拿不到区间边界。凡是更新公式需要长度，都要把 `len` 存进 S，让它随 `op` 一起合并。

接口：除 `segtree` 的 `set/get/prod/all_prod/max_right/min_left` 外，新增闭区间 `apply(l,r,f)`；单点修改写 `apply(p,p,f)`（精简版删了 ACL 的 `apply(p,f)` 重载）。因为查询前可能需要下推标记，`get/prod` 与两个二分接口不再是 `const`。

9.9.1 模板本体

```
/* ACL atcoder/lazysegtree 精简移植 + 闭区间
 * lazy_segtree(n) 合法位置 0..n; 区间 [l, r] 左闭右闭
 * 单点修改写 apply(p, p, f); 空区间写 r == l - 1
 * 不变式 d[k] = mapping(lz[k], 两儿子之积), 故下推/上拉可无条件做
 */
template<class S, S (*op)(S, S), S (*e)(),
         class F, S (*mapping)(F, S),
         F (*composition)(F, F), F (*id)()>
struct lazy_segtree {
    int n, sz = 1, lg = 0;
    vector<S> d;
    vector<F> lz;
    lazy_segtree(int n = 0) : lazy_segtree(vector<S>(n + 1, e())) {}
    lazy_segtree(const vector<S> &v) : n(v.size()) {
        while (sz < n) sz *= 2, lg++;
        d.assign(2 * sz, e()), lz.assign(sz, id());
        copy(v.begin(), v.end(), d.begin() + sz);
        for (int i = sz - 1; i; i--) up(i);
    }
    void up(int k) { d[k] = mapping(lz[k], op(d[2 * k], d[2 * k + 1])); }
    void all(int k, F f) {
        d[k] = mapping(f, d[k]);
        if (k < sz) lz[k] = composition(f, lz[k]);
    }
    void push(int k) { all(2 * k, lz[k]), all(2 * k + 1, lz[k]), lz[k] = id(); }
    void set(int p, S x) {
        p += sz;
        for (int i = lg; i; i--) push(p >> i);
        d[p] = x;
        for (int i = 1; i <= lg; i++) up(p >> i);
    }
    S get(int p) { return prod(p, p); }
    S all_prod() { return d[1]; }
    S prod(int l, int r) {
        S sl = e(), sr = e();
        if (l > r) return sl;
        l += sz, r += sz + 1;
        for (int i = lg; i; i--) push(l >> i), push((r - 1) >> i);
        for (; l < r; l >>= 1, r >>= 1) {
            if (l & 1) sl = op(sl, d[l++]);
            if (r & 1) sr = op(d[--r], sr);
        }
        return op(sl, sr);
    }
    void apply(int l, int r, F f) {
        if (l > r) return;
        l += sz, r += sz + 1;
```

```
        for (int i = lg; i; i--) push(l >> i), push((r - 1) >> i);
        for (int a = l, b = r; a < b; a >>= 1, b >>= 1) {
            if (a & 1) all(a++, f);
            if (b & 1) all(--b, f);
        }
        for (int i = 1; i <= lg; i++) up(l >> i), up((r - 1) >> i);
    }
    template<class G> int max_right(int l, G g) {
        if (l == n) return n - 1;
        l += sz;
        for (int i = lg; i; i--) push(l >> i);
        S sm = e();
        do {
            while (l % 2 == 0) l >>= 1;
            if (!g(op(sm, d[l]))) {
                while (l < sz) {
                    push(l), l *= 2;
                    if (g(op(sm, d[l]))) sm = op(sm, d[l++]);
                }
                return l - sz - 1;
            }
            sm = op(sm, d[l++]);
        } while ((l & -l) != l);
        return n - 1;
    }
    template<class G> int min_left(int r, G g) {
        if (++r == 0) return 0;
        r += sz;
        for (int i = lg; i; i--) push((r - 1) >> i);
        S sm = e();
        do {
            r--;
            while (r > 1 && r % 2) r >>= 1;
            if (!g(op(d[r], sm))) {
                while (r < sz) {
                    push(r), r = 2 * r + 1;
                    if (g(op(d[r], sm))) sm = op(d[r--], sm);
                }
                return r + 1 - sz;
            }
            sm = op(d[r], sm);
        } while ((r & -r) != r);
        return 0;
    }
};
```

9.9.2 示例：区间加 + 区间和

区间长度必须存进 S；空出来的 0 号位置长度为 0，因此不会影响 1-based 查询。

```
// ACL_EXAMPLE: lazy_range_add_sum
struct S {
    ll sum;
    int len;
};
using F = ll;

S op(S a, S b) {
    return {a.sum + b.sum, a.len + b.len};
}
S e() { return {0, 0}; }
S mapping(F f, S x) {
    return {x.sum + f * x.len, x.len};
}
F composition(F f, F g) { return f + g; }
F id() { return 0; }

using SegAddSum = lazy_segtree<S, op, e, F,
        mapping, composition, id>;

// vector<S> a(n + 1, e());
// FOR(i, 1, n) a[i] = {x[i], 1};
// SegAddSum seg(a);
// seg.apply(l, r, add);
// ll ans = seg.prod(l, r).sum;
```

9.9.3 示例：仿射修改 + 一至三次幂和

用 $x \leftarrow ax + b$ 统一表达乘法、加法与赋值：乘 v 是 {v,0}，加 v 是 {1,v}，赋值 v 是 {0,v}。节点同时维护 $\sum x$、$\sum x^2$、$\sum x^3$。

```
// ACL_EXAMPLE: lazy_affine_moments
const ll MOD = 998244353;

ll mul_mod(ll a, ll b) {
    return (ll)((__int128)a * b % MOD);
}

struct S {
    ll s1, s2, s3;
    int len;
};
struct F {
    ll mul, add; // x <- mul * x + add
};

S op(S x, S y) {
    return {(x.s1 + y.s1) % MOD,
            (x.s2 + y.s2) % MOD,
            (x.s3 + y.s3) % MOD,
            x.len + y.len};
}
S e() { return {0, 0, 0, 0}; }

S mapping(F f, S x) {
    ll m = f.mul, a = f.add;
    ll m2 = mul_mod(m, m), m3 = mul_mod(m2, m);
    ll a2 = mul_mod(a, a), a3 = mul_mod(a2, a);
    ll old1 = x.s1, old2 = x.s2, old3 = x.s3;
    ll n = x.len % MOD;
    ll s1 = (mul_mod(m, old1) + mul_mod(a, n))
    ↪  % MOD;
    ll s2 = (mul_mod(m2, old2)
           + mul_mod(2 * mul_mod(m, a) % MOD,
           ↪  old1)
           + mul_mod(a2, n)) % MOD;
    ll s3 = (mul_mod(m3, old3)
           + mul_mod(3 * mul_mod(m2, a) % MOD,
           ↪  old2)
           + mul_mod(3 * mul_mod(m, a2) % MOD,
           ↪  old1)
           + mul_mod(a3, n)) % MOD;
    return {s1, s2, s3, x.len};
}

F composition(F f, F g) {
    // 先 g 后 f
    return {mul_mod(f.mul, g.mul),
            (mul_mod(f.mul, g.add) + f.add) %
            ↪  MOD};
}
F id() { return {1, 0}; }

using SegMoments = lazy_segtree<S, op, e, F,
        mapping, composition, id>;

// add v:    seg.apply(l, r, {1, v});
// multiply: seg.apply(l, r, {v, 0});
// assign v: seg.apply(l, r, {0, v});
```

9.9.4 示例：赋值/加 + 和、最值与位置

这一份合并“扶苏的问题”和“最大值位置”板子。最大值并列时取最左位置；整段赋值后，最左最大值就是该段左端点。

```
// ACL_EXAMPLE: lazy_assign_add_stats
struct S {
    ll sum, mn, mx;
    int mx_pos, left, len;
};
struct F {
    bool has_assign;
    ll assign, add;
};

S op(S a, S b) {
    if (a.len == 0) return b;
    if (b.len == 0) return a;
    int pos = a.mx >= b.mx ? a.mx_pos :
    ↪  b.mx_pos;
    return {a.sum + b.sum, min(a.mn, b.mn),
            max(a.mx, b.mx), pos, a.left,
            a.len + b.len};
}
S e() { return {0, 0, 0, 0, 0, 0}; }

S mapping(F f, S x) {
    if (x.len == 0) return x;
    if (f.has_assign) {
        ll v = f.assign + f.add;
        return {v * x.len, v, v,
                x.left, x.left, x.len};
    }
    x.sum += f.add * x.len;
    x.mn += f.add;
    x.mx += f.add;
    return x;
}

F composition(F f, F g) {
    if (f.has_assign) return f;
    g.add += f.add;
    return g;
}
F id() { return {false, 0, 0}; }

using SegStats = lazy_segtree<S, op, e, F,
        mapping, composition, id>;

// leaf i: {x, x, x, i, i, 1}
// assign: seg.apply(l, r, {true, v, 0});
// add:    seg.apply(l, r, {false, 0, v});
```

9.9.5 示例：最小代价位置的权值和

区间加只平移最小值，不改变哪些位置达到最小值，因此对应权值和保持不变。常用于只统计“当前代价最小”转移点的 DP。

```cpp
// ACL_EXAMPLE: lazy_min_weight
const ll MOD = 998244353;

struct S {
    ll mn, weight;
    bool empty;
};
using F = ll;

S op(S a, S b) {
    if (a.empty) return b;
    if (b.empty) return a;
    if (a.mn < b.mn) return a;
    if (b.mn < a.mn) return b;
    return {a.mn, (a.weight + b.weight) % MOD,
    ↪  false};
}
S e() { return {0, 0, true}; }
S mapping(F f, S x) {
    if (!x.empty) x.mn += f;
    return x;
}
F composition(F f, F g) { return f + g; }
F id() { return 0; }

using SegMinWeight = lazy_segtree<S, op, e, F,
        mapping, composition, id>;

// 真实叶子必须显式 set 成 {cost, weight, false}
// seg.apply(l, r, delta);
// S best = seg.prod(l, r);

// leaf i: {0, compressed_values[i], false}
// seg.apply(left_value_id, right_value_id,
↪  delta);
// auto [frequency, value, unused] =
↪  seg.all_prod();
```

9.9.6 示例：权值线段树求最小众数

离散化后，每个叶子代表一个权值。区间加维护一段权值的出现次数；合并时先取最高频次，频次相同取较小权值。

```cpp
// ACL_EXAMPLE: lazy_value_mode
struct S {
    ll cnt, value;
    bool empty;
};
using F = ll;

S op(S a, S b) {
    if (a.empty) return b;
    if (b.empty) return a;
    if (a.cnt != b.cnt) return a.cnt > b.cnt ?
    ↪  a : b;
    return a.value < b.value ? a : b;
}
S e() { return {0, 0, true}; }
S mapping(F f, S x) {
    if (!x.empty) x.cnt += f;
    return x;
}
F composition(F f, F g) { return f + g; }
F id() { return 0; }

using SegMode = lazy_segtree<S, op, e, F,
        mapping, composition, id>;
```

9.10 动态开点线段树（区间加，区间求和）

AC https://www.luogu.com.cn/record/266006147

```
/*
 * 2026-03-08  P13825 【模板】线段树 1.5 动态开点线段树
 * AC https://www.luogu.com.cn/record/266006147
 *
 */
struct Tag {
    // 在母串中的这个位置
    ll add=0;
    bool need_apply=false;
    void apply(const Tag &t) {
        add+=t.add;
        need_apply=true;
    }
};

struct Info {
    ll l=0,r=0;
    i128 sum=0;
    static Info merge(const Info &a,const Info &b) {
        Info res=a;
        res.r=b.r;
        res.sum+=b.sum;
        return res;
    }
    void apply(const Tag&t) {
        if (!t.need_apply) {
            return;
        }
        ll len=r-l+1;
        sum+=len*t.add;
    }
};

// 我认为还是用 C++14 的写法，把你要写的东西传进去比较好，不容易写错
template<class I=Info,class T=Tag>
struct SEG {
    struct Node {
        I info;
        T tag;
        Node* rs=nullptr;
        Node* ls=nullptr;
    };

    void init_node(Node *u,ll l,ll r) {
        u->info.l=l;
        u->info.r=r;
        u->info.sum=i128(u->info.l+u->info.r)*(u->info.r-u->info.l+1)/2;
    }

    Node* root=nullptr;
    ll N;

    static bool isIntersect(ll l1, ll r1, ll l2, ll r2) {
        if (!(l1 > r2 || r1 < l2)) return true;
        return false;
    }

    SEG(ll N):N(N) {
        root=new Node();
        root->info.l=1;
        root->info.r=N;
        init_node(root,1,N);
    }

    void extend(Node *u) {
        if (u->ls==nullptr) {
            u->ls=new Node();
            init_node(u->ls,u->info.l,(u->info.l+u->info.r)>>1);
        }
        if (u->rs==nullptr) {
            u->rs=new Node();
            init_node(u->rs,((u->info.l+u->info.r)>>1)+1,u->info.r);
        }
    }

    void push(Node *u) {
        if (u->info.l==u->info.r) {
            return;
        }
        extend(u);
        if (u->tag.need_apply) {
            u->ls->info.apply(u->tag);
            u->rs->info.apply(u->tag);
            u->ls->tag.apply(u->tag);
            u->rs->tag.apply(u->tag);
            u->tag={};
        }
    }
    void pull(Node *u) {
        if (u->info.l==u->info.r) {
            return;
        }
        u->info=I::merge(u->ls->info,u->rs->info);
    }
    void modify(Node*u,ll l,ll r,const T& t) {
        if (u->info.l>=l && u->info.r<=r) {
            u->info.apply(t);
            u->tag.apply(t);
            return;
        }
        push(u);
        if (isIntersect(u->ls->info.l,u->ls->info.r,l,r)) {
            modify(u->ls,l,r,t);
        }
        if (isIntersect(u->rs->info.l,u->rs->info.r,l,r)) {
            modify(u->rs,l,r,t);
        }
        pull(u);
    }

    Info query(Node *u,ll l,ll r) {
        if (u->info.l>=l && u->info.r<=r) {
            return u->info;
```

```
        }
        Info res;
        bool is_get=false;
        push(u);
        if (isIntersect(u->ls->info.l,u->ls->in
        ↪ fo.r,l,r)) {
            res=query(u->ls,l,r);
            is_get=true;
        }
        if (isIntersect(u->rs->info.l,u->rs->in
        ↪ fo.r,l,r)) {
            if (!is_get) {
                res=query(u->rs,l,r);
            }else {
                res=I::merge(res,query(u->rs,l,
                ↪ r));
            }
        }
        return res;
    }
};
```

9.11 rope（O(1) 拷贝的可持久化序列）

__gnu_cxx::rope 与 pb_ds 同属 GCC libstdc++ 扩展（MSVC 无），底层是 **平衡 concat 树**，节点 immutable + 引用计数。复制赋值仅共享根指针，是 **O(1)**；修改走 **path copying**：只复制根到目标位置的 $O(\log n)$ 个节点，其他子树原样共享。保留历史版本不需要额外 API——把 rope 对象本身存起来即可，每个新版本均摊 $O(\log n)$ 空间，q 次操作总空间 $O(q \log n)$，与可持久化线段树同档。

基本用法（位置寻址，下标 0-based）：

```
#include <ext/rope>
using namespace __gnu_cxx;

rope<int> R;
R.push_back(7);             // 末尾追加
R.insert(2, 99);            // 在下标 2 插入，
↪ O(log n)
R.erase(1, 3);              // 从下标 1 起删 3 个，
↪ O(log n)
int x = R[0];               // 单点访问 O(log n)，
↪ 不是 O(1)
auto S = R.substr(2, 5);    // 取 [2, 5)，O(log
↪ n)，共享子树
size_t n = R.size();
```

持久化 = 普通 copy / 赋值。把每个版本装进数组即拿到版本树：

```
vector<rope<int>> ver;
ver.push_back(rope<int>());                 // 版本
↪ 0：空
for (int i = 1; i <= q; i++) {
    ver.push_back(ver[parent[i]]);          //
    ↪ 从父版本 fork，O(1)
    ver.back().insert(pos[i], val[i]);      //
    ↪ 在新版本上改
}
int ans = ver[k][i];                        //
↪ 查询版本 k 的下标 i
```

9.12 FHQ-treap（无旋 Treap）

无旋 Treap 用 splitVal / merge 两个 $O(\log n)$ 操作做底，实现 insert / erase / rank_of_key / find_by_order / pre / nxt 全套**平衡树标准接口**。下方实现**用 struct + vector 存点**：可以**同时开多棵**（多组数据、按颜色分裂合并等场景直接开一个 vector<FHQ>），节点按需增长而**不吃 maxn 静态数组的固定内存**；已知规模先调 reserve 把扩容开销省掉。

节点优先级取自 rnd()——mt19937_64 的输出再过一层 splitmix64 非线性混合，比裸 rng() 更抗针对性构造。merge **只看优先级不看权值**，所以合并前必须保证左树权值全部小于右树；splitVal(x, k) 则按权值把子树切成「$\le k$」和「$> k$」两半，全部接口都是这两个操作的组合。

边界约定：find_by_order 越界返回 -INF；但 pre / nxt 无解时返回的是 val[0] 即 0，**不是** -INF——需要哨兵语义的题先 insert 一对 ±INF 兜底。

```
// AC 洛谷 P3369【模板】普通平衡树
// 依赖竞赛模板头的 ll / SZ(a) / INF
// 已展开过 :mt19937 的话，
// 下面 rng/splitmix64/rnd 三段删掉别重复定义
std::mt19937_64 rng(
    std::chrono::steady_clock::now()
        .time_since_epoch().count());

// 把 rng() 输出再过一层非线性混合，
// 挡住 F2 上的 Berlekamp-Massey 攻击
uint64_t splitmix64(uint64_t x) {
    x ^= x >> 30;
    x *= 0xbf58476d1ce4e5b9ULL;
    x ^= x >> 27;
    x *= 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}
// XOR hashing 等场景也用这个，别用裸 rng()
uint64_t rnd() { return splitmix64(rng()); }

struct FHQ {
    using T = ll;
    vector<array<int, 2> > ch;
    vector<int> sz;
    vector<T> val;
    vector<ll> pri;
    int rt = 0;
    int n;

    FHQ() {
        ch.resize(1);
        sz.resize(1);
        val.resize(1);
        pri.resize(1);
    }

    int newNode(T v) {
        ch.push_back({0, 0});
```

```
        // 不要不小心 push_back 0 了
        sz.push_back(1);
        val.push_back(v);
        pri.push_back(rnd());
        n = SZ(sz) - 1;
        return n;
    }

    // 已知规模先 reserve，省掉 vector 反复扩容
    void reserve(int _n) {
        ch.reserve(_n + 2);
        sz.reserve(_n + 2);
        val.reserve(_n + 2);
        pri.reserve(_n + 2);
    }

    void pushup(int x) {
        sz[x] = sz[ch[x][0]] + sz[ch[x][1]] + 1;
    }

    // 按值切：返回 (<= k 的那棵，> k 的那棵)
    array<int, 2> splitVal(int x, T k) {
        if (x == 0) return {0, 0};
        if (val[x] <= k) {
            auto [l, r] = splitVal(ch[x][1], k);
            ch[x][1] = l;
            pushup(x);
            return {x, r};
        }
        auto [l, r] = splitVal(ch[x][0], k);
        ch[x][0] = r;
        pushup(x);
        return {l, x};
    }

    // a 中的权值必须全部小于 b 的权值
    // merge 的时候只考虑优先级，不考虑权值
    int merge(int a, int b) {
        if (!a || !b) return a | b;
        if (pri[a] >= pri[b]) {
            ch[a][1] = merge(ch[a][1], b);
            pushup(a);
            return a;
        }
        ch[b][0] = merge(a, ch[b][0]);
        pushup(b);
        return b;
    }

    void insert(T k) {
        auto [a, b] = splitVal(rt, k);
        rt = merge(a, merge(newNode(k), b));
    }

    void erase(T k) {
        auto [a, b] = splitVal(rt, k);
        auto [a_lt_k, a_eq_k]
            = splitVal(a, k - 1);
        a_eq_k = merge(ch[a_eq_k][0],
                       ch[a_eq_k][1]);
        rt = merge(merge(a_lt_k, a_eq_k), b);
    }

    // 比 k 小的数有几个？再加 1
    int rank_of_key(T k) {
        auto [a, b] = splitVal(rt, k - 1);
        int res = sz[a] + 1;
        rt = merge(a, b);
        return res;
    }

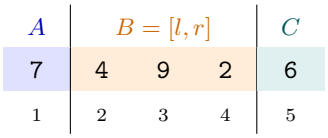
    // 返回 1-base 下第 k 小的数；越界返回 -INF
    T find_by_order(int k) const {
        int x = rt;
        while (true) {
            if (x == 0) return -INF;
            int l_sz = sz[ch[x][0]];
            if (l_sz + 1 == k) {
                return val[x];
            }
            if (l_sz >= k) {
                x = ch[x][0];
            } else {
                k -= (l_sz + 1);
                x = ch[x][1];
            }
        }
    }

    // 返回 k 的前驱（第一个比 k 小的数）
    T pre(T k) {
        auto [a, b] = splitVal(rt, k - 1);
        int x = a;
        T res = val[x];
        while (x != 0) {
            res = max(res, val[x]);
            x = ch[x][1];
        }
        merge(a, b);
        return res;
    }

    // 返回 k 的后继（第一个比 k 大的数）
    T nxt(T k) {
        auto [a, b] = splitVal(rt, k);
        int x = b;
        T res = val[x];
        while (x != 0) {
            res = min(res, val[x]);
            x = ch[x][0];
        }
        merge(a, b);
        return res;
    }
};
```

9.13 隐式 FHQ-treap（可区间维护的动态序列）

上一节的普通 FHQ-treap 按**权值**分裂，用来维护有序集合；这一版改为按**前缀长度**分裂，因此中序遍历顺序就是序列顺序，适合需要插入、删除、翻转和区间修改的动态数组。若当前位置为 1-based 闭区间 $[l, r]$，连续两次分裂即可得到三段：先取前 $l-1$ 个，再从余下部分取 $r-l+1$ 个。只对中段打标记，最后按原顺序合并回去。



图注：以序列 7 4 9 2 6、$[l, r] = [2, 4]$ 为例。第一次按 $l-1=1$ 切出蓝色 A；第二次按 $r-l+1=3$ 切出橙色 B；青色 C 是剩余后缀。区间操作只改 B，随后合并为 $A+B+C$。

模板本体如下。它自带区间赋值、区间加、区间翻转，以及区间和、最小值、最大值；所有操作期望复杂度均为 $O(\log n)$。代码使用自己的 `ll` 和随机数生成器，整段复制即可。

```cpp
#include <bits/stdc++.h>
using namespace std;

struct ImplicitTreap {
    using ll = long long;
    using ull = unsigned long long;
    static constexpr ll INF = (1LL << 60);

    // 外部位置：insert(pos, x) 在第 pos 个元素后插入，pos 为 0-based 间隙；
    // 其余位置和区间接口均为 1-based，区间均为闭区间 [l, r]。
    struct Node {
        int ch[2] = {0, 0};
        int sz = 1;
        ll val = 0, sum = 0, mn = 0, mx = 0;
        ull pri = 0;
        bool rev = false;
        bool has_set = false;
        ll set_tag = 0, add_tag = 0;
    };

    vector<Node> tr;
    int rt = 0;
    mt19937_64 rng;

    ImplicitTreap()
        : rng(chrono::steady_clock::now().time_since_epoch().count()) {
        tr.emplace_back();  // 0 号空节点
    }

    explicit ImplicitTreap(const vector<ll> &a)
        : rng(chrono::steady_clock::now().time_since_epoch().count()) {
        tr.emplace_back();
        build(a);
    }

    int size(int x) const { return x ? tr[x].sz : 0; }
    ll sum(int x) const { return x ? tr[x].sum : 0; }
    ll min_value(int x) const { return x ? tr[x].mn : INF; }
    ll max_value(int x) const { return x ? tr[x].mx : -INF; }

    int new_node(ll v) {
        Node t;
        t.val = t.sum = t.mn = t.mx = v;
        t.pri = splitmix64(rng());
        tr.push_back(t);
        return (int)tr.size() - 1;
    }
```

```
    void reserve(int n) { tr.reserve(n + 1); }

    void apply_set(int x, ll v) {
        if (!x) return;
        tr[x].val = tr[x].mn = tr[x].mx = v;
        tr[x].sum = v * tr[x].sz;
        tr[x].has_set = true;
        tr[x].set_tag = v;
        tr[x].add_tag = 0;
    }

    void apply_add(int x, ll v) {
        if (!x) return;
        tr[x].val += v;
        tr[x].sum += v * tr[x].sz;
        tr[x].mn += v;
        tr[x].mx += v;
        if (tr[x].has_set) tr[x].set_tag += v;
        else tr[x].add_tag += v;
    }

    void apply_reverse(int x) {
        if (!x) return;
        swap(tr[x].ch[0], tr[x].ch[1]);
        tr[x].rev ^= 1;
    }

    void pushup(int x) {
        if (!x) return;
        int l = tr[x].ch[0], r = tr[x].ch[1];
        tr[x].sz = size(l) + size(r) + 1;
        tr[x].sum = sum(l) + tr[x].val + sum(r);
        tr[x].mn = min({min_value(l), tr[x].val, min_value(r)});
        tr[x].mx = max({max_value(l), tr[x].val, max_value(r)});
    }

    // 赋值必须先于加法下传；翻转与二者可交换，这里统一最后下传。
    void pushdown(int x) {
        if (!x) return;
        if (tr[x].has_set) {
            apply_set(tr[x].ch[0], tr[x].set_tag);
            apply_set(tr[x].ch[1], tr[x].set_tag);
            tr[x].has_set = false;
        }
        if (tr[x].add_tag != 0) {
            apply_add(tr[x].ch[0], tr[x].add_tag);
            apply_add(tr[x].ch[1], tr[x].add_tag);
            tr[x].add_tag = 0;
        }
        if (tr[x].rev) {
            apply_reverse(tr[x].ch[0]);
            apply_reverse(tr[x].ch[1]);
            tr[x].rev = false;
        }
    }

    // 按数量切：左树为前 k 个元素，右树为其余元素。
    pair<int, int> split(int root, int k) {
        if (!root) return {0, 0};
        pushdown(root);
        int left_size = size(tr[root].ch[0]);
        if (left_size >= k) {
```

```
            auto [a, b] = split(tr[root].ch[0], k);
            tr[root].ch[0] = b;
            pushup(root);
            return {a, root};
        }
        auto [a, b] = split(tr[root].ch[1], k - left_size - 1);
        tr[root].ch[1] = a;
        pushup(root);
        return {root, b};
    }

    int merge(int a, int b) {
        if (!a || !b) return a ? a : b;
        if (tr[a].pri >= tr[b].pri) {
            pushdown(a);
            tr[a].ch[1] = merge(tr[a].ch[1], b);
            pushup(a);
            return a;
        }
        pushdown(b);
        tr[b].ch[0] = merge(a, tr[b].ch[0]);
        pushup(b);
        return b;
    }

    void clear() {
        tr.clear();
        tr.emplace_back();
        rt = 0;
    }

    void build(const vector<ll> &a) {
        clear();
        reserve((int)a.size());
        for (ll x : a) rt = merge(rt, new_node(x));
    }

    int root() const { return rt; }
    int length() const { return size(rt); }

    // 把当前序列切为 A=[1,l-1]、B=[l,r]、C=[r+1,n]。
    tuple<int, int, int> split_range(int l, int r) {
        auto [a, bc] = split(rt, l - 1);
        auto [b, c] = split(bc, r - l + 1);
        return {a, b, c};
    }

    void merge_range(int a, int b, int c) {
        rt = merge(a, merge(b, c));
    }

    // 在第 pos 个元素后插入；pos=0 表示插到最前面。
    void insert(int pos, ll v) {
        assert(0 <= pos && pos <= length());
        auto [a, b] = split(rt, pos);
        rt = merge(a, merge(new_node(v), b));
    }

    void push_back(ll v) { insert(length(), v); }
    void push_front(ll v) { insert(0, v); }

    void erase(int pos) {
        assert(1 <= pos && pos <= length());
```

```
        auto [a, bc] = split(rt, pos - 1);
        auto [b, c] = split(bc, 1);
        (void)b;  // 不回收节点；总插入次数应控制在可接受范围内。
        rt = merge(a, c);
    }

    void erase(int l, int r) {
        assert(1 <= l && l <= r && r <= length());
        auto [a, b, c] = split_range(l, r);
        (void)b;
        rt = merge(a, c);
    }

    ll kth(int k) {
        assert(1 <= k && k <= length());
        int x = rt;
        while (x) {
            pushdown(x);
            int left_size = size(tr[x].ch[0]);
            if (k == left_size + 1) return tr[x].val;
            if (k <= left_size) x = tr[x].ch[0];
            else k -= left_size + 1, x = tr[x].ch[1];
        }
        assert(false);
        return 0;
    }

    void set_value(int pos, ll v) {
        assert(1 <= pos && pos <= length());
        auto [a, b, c] = split_range(pos, pos);
        apply_set(b, v);
        merge_range(a, b, c);
    }

    ll get_value(int pos) { return kth(pos); }

    void range_set(int l, int r, ll v) {
        assert(1 <= l && l <= r && r <= length());
        auto [a, b, c] = split_range(l, r);
        apply_set(b, v);
        merge_range(a, b, c);
    }

    void range_add(int l, int r, ll v) {
        assert(1 <= l && l <= r && r <= length());
        auto [a, b, c] = split_range(l, r);
        apply_add(b, v);
        merge_range(a, b, c);
    }

    void range_reverse(int l, int r) {
        assert(1 <= l && l <= r && r <= length());
        auto [a, b, c] = split_range(l, r);
        apply_reverse(b);
        merge_range(a, b, c);
    }

    ll range_sum(int l, int r) {
        assert(1 <= l && l <= r && r <= length());
        auto [a, b, c] = split_range(l, r);
        ll ans = sum(b);
        merge_range(a, b, c);
        return ans;
```

```
    }

    ll range_min(int l, int r) {
        assert(1 <= l && l <= r && r <= length());
        auto [a, b, c] = split_range(l, r);
        ll ans = min_value(b);
        merge_range(a, b, c);
        return ans;
    }

    ll range_max(int l, int r) {
        assert(1 <= l && l <= r && r <= length());
        auto [a, b, c] = split_range(l, r);
        ll ans = max_value(b);
        merge_range(a, b, c);
        return ans;
    }

    struct Info {
        ll sum = 0, mn = INF, mx = -INF;
        int sz = 0;
    };

    Info range_info(int l, int r) {
        assert(1 <= l && l <= r && r <= length());
        auto [a, b, c] = split_range(l, r);
        Info ans{sum(b), min_value(b), max_value(b), size(b)};
        merge_range(a, b, c);
        return ans;
    }

    void dump_dfs(int x, vector<ll> &res) {
        if (!x) return;
        pushdown(x);
        dump_dfs(tr[x].ch[0], res);
        res.push_back(tr[x].val);
        dump_dfs(tr[x].ch[1], res);
    }

    vector<ll> to_vector() {
        vector<ll> res;
        res.reserve(length());
        dump_dfs(rt, res);
        return res;
    }

    void print(ostream &out = cout) {
        vector<ll> a = to_vector();
        for (int i = 0; i < (int)a.size(); ++i) {
            if (i) out << ' ';
            out << a[i];
        }
        out << '\n';
    }

private:
    static ull splitmix64(ull x) {
        x ^= x >> 30;
        x *= 0xbf58476d1ce4e5b9ULL;
        x ^= x >> 27;
        x *= 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
```

```
};
```

下标约定：除 `insert(pos,x)` 的 pos 表示“前面已有几个元素”外，其余位置均为 1-based；区间均为闭区间。于是 `insert(0,x)` 插到最前，`insert(length(),x)` 插到最后。

最常用的调用方式：

```
vector<long long> a{1, 2, 3, 4, 5};
ImplicitTreap t(a);

t.range_add(2, 4, 10);
// 1 12 13 14 5

t.range_reverse(2, 5);
// 1 5 14 13 12

t.insert(1, 99);
// 在第 1 个元素后插入：
// 1 99 5 14 13 12

t.erase(3);                 // 删除位置 3
auto info = t.range_info(2, 4);
cout << info.sum << " "
     << info.mn << " " << info.mx << "\n";
t.print();
```

接口速查：

- 建树：`ImplicitTreap t(a);` 已知后续总插入量时可先 `reserve`。
- 插删：`insert`、`push_front`、`push_back`、`erase`。
- 单点：`kth` / `get_value` / `set_value`。
- 区间改：`range_set`、`range_add`、`range_reverse`。
- 区间查：`range_sum`、`range_min`、`range_max`，或一次取全的 `range_info`。
- 调试输出：`to_vector`、`print`。

两个容易踩的坑：第一，`split` / `merge` 会改变树根与儿子，自己手动调用时必须接住返回值；日常做题优先使用封装好的公开接口。第二，删除只把节点从序列中摘掉，并不回收空间，所以内存按**整场程序累计插入次数**估算，而不是按当前序列长度估算。

9.14 主席树（可持久化权值线段树）

主席树即**可持久化权值线段树**，每次单点更新只新建 $O(\log n)$ 个节点，多保存一份「版本根」。处理**静态区间第 k 小**问题时，第 i 个版本对应前 i 个数构成的权值集合，区间 $[l,r]$ 的第 k 小按左子树差量 `tree[ls[v]].sum - tree[ls[u]].sum` 决定向左还是向右走。

空间按 $25n$ 预留（$\log_2 5\times10^5 \approx 19$，留余量）。`vector+reserve` 实现，可放局部作用域；**在线插入需保证新元素已在建树时出现在离散化表里**。

```
// 值域可持久化线段树
class Segment {
    struct node { int sum = 0, leftChild = 0,
    ↪  rightChild = 0; };
    vector<node> tree;
    vector<int> root;
    vector<ll> discreted, origin;
    int idx = 0, N;

    void insert(int prev, int &cur, int l, int
    ↪  r, int p) {
        cur = ++idx;
        tree.push_back(node());
        tree[cur] = tree[prev];
        tree[cur].sum++;
        if (l == r) return;
        int mid = (l + r) >> 1;
        if (p <= mid)
        ↪  insert(tree[prev].leftChild,
        ↪  tree[cur].leftChild,  l, mid, p);
        else
        ↪  insert(tree[prev].rightChild,
        ↪  tree[cur].rightChild, mid + 1, r,
        ↪  p);
    }
    ll queryKthNumber(int prev, int cur, int l,
    ↪  int r, int x) {
        if (l == r) return discreted[l - 1];
        int leftSum =
        ↪  tree[tree[cur].leftChild].sum -
        ↪  tree[tree[prev].leftChild].sum;
        int mid = (l + r) >> 1;
        if (leftSum < x)
            return queryKthNumber(tree[prev].ri
            ↪  ghtChild, tree[cur].rightChild,
            ↪  mid + 1, r, x - leftSum);
        else
            return queryKthNumber(tree[prev].le
            ↪  ftChild,  tree[cur].leftChild,
            ↪  l, mid, x);
    }
    ll queryRank(int prev, int cur, int l, int
    ↪  r, int p) {
        if (p < l) return 0;
        if (r <= p) return tree[cur].sum -
        ↪  tree[prev].sum;
        int mid = (l + r) >> 1;
        return queryRank(tree[prev].leftChild,
        ↪  tree[cur].leftChild,  l, mid, p)
             + queryRank(tree[prev].rightChild,
             ↪  tree[cur].rightChild, mid + 1,
             ↪  r, p);
    }
    int id(ll x) { return
    ↪  lower_bound(all(discreted), x) -
    ↪  discreted.begin() + 1; }

public:
    Segment(int n) : tree(1, node()), root(n +
    ↪  1), origin(n + 1) {
```

```
        tree.reserve(25ll * n + 5);
        for (int i = 1; i <= n; i++) {
            cin >> origin[i];
            discreted.push_back(origin[i]);
        }
        sort(all(discreted));
        discreted.erase(unique(all(discreted)),
        ↪  discreted.end());
        N = discreted.size();
        for (int i = 1; i <= n; i++)
            insert(root[i - 1], root[i], 1, N,
            ↪  id(origin[i]));
    }

    // 区间第 k 小元素
    ll queryKthNumber(int l, int r, int k) {
        return queryKthNumber(root[l - 1],
        ↪  root[r], 1, N, k);
    }
    // 区间 <= x 的数字个数（x 可未出现过，故走
    ↪  upper_bound 不走 id()）
    ll queryRank(int l, int r, ll x) {
        int p = upper_bound(all(discreted), x)
        ↪  - discreted.begin();
        if (p == 0) return 0;
        return queryRank(root[l - 1], root[r],
        ↪  1, N, p);
    }
};
```

典型用法（构造时读入 n 个数，区间第 k 小 / 区间 $\le x$ 计数）：

```
inline void solve() {
    int n, m; cin >> n >> m;
    Segment tr(n);  // 构造时一次性读入 n 个数
    FOR(_, 1, m) {
        int l, r, k; cin >> l >> r >> k;
        cout << tr.queryKthNumber(l, r, k) <<
        ↪  "\n";
    }
}
```

9.15 分块（单点修改）

分块以 $\sqrt{N}$ 为块长，把数组切成若干块，**块内保持有序**。查询区间 $[a,b]$ 内 $\ge c$（或 $\le c$）的元素个数：散块（左右两端不完整的块）暴力扫；整块用 `lower_bound` / `upper_bound` 二分。单点修改 $O(\sqrt{N})$，单次查询 $O(\sqrt{N}\log\sqrt{N})$。

```
// AC https://www.luogu.com.cn/record/220963190
// 单点修改 + 查询区间 [a, b] 内 >=c 的数的个数
ll L[MAXN], R[MAXN];
vector<arr2> g[MAXN];  // 记得清空

inline void solve() {
    cin >> N;
    ll sqn = sqrt(N), idx = 1;
    L[idx] = 1; R[idx] = sqn;
    FOR(i, 1, N) {
        ll a; cin >> a;
        if (i <= R[idx]) {
            g[idx].pb({a, i});
        } else {
            ++idx;
            L[idx] = i; R[idx] = i + sqn - 1;
            g[idx].pb({a, i});
        }
    }
    FOR(i, 1, idx) sort(all(g[i]));
    L[idx + 1] = INF;
    cin >> Q;
    FOR(_, 1, Q) {
        ll op, a, b, c;
        cin >> op;
        if (op == 0) {
            cin >> a >> b >> c;
            ll ida = upper_bound(L + 1,
                L + 1 + idx, a) - L;
            ida -= 1;
            ll idb = upper_bound(L + 1,
                L + 1 + idx, b) - L;
            idb -= 1;
            ll ans = 0;
            // 左散块
            FOR(i, 0, SZ(g[ida]) - 1) {
                if (g[ida][i][1] >= a
                    && g[ida][i][1] <= b
                    && g[ida][i][0] >= c)
                    ++ans;
            }
            // 右散块（若与左不同块）
            if (ida != idb) {
                FOR(i, 0, SZ(g[idb]) - 1) {
                    if (g[idb][i][1] >= a
                        && g[idb][i][1] <= b
                        && g[idb][i][0] >= c)
                        ++ans;
                }
            }
            // 整块二分
            FOR(i, ida + 1, idb - 1) {
                // <=c 版本：upper_bound(c, INF)
                // ll id =
                ↪  upper_bound(all(g[i]),
                //          (arr2){c, INF})
                //        - g[i].begin();
                // ans += id;
                ll id = lower_bound(all(g[i]),
                        (arr2){c, 0})
                      - g[i].begin();
                ans += SZ(g[i]) - id;
            }
            cout << ans << "\n";
        } else {  // 单点修改 a → b
            cin >> a >> b;
            ll id = upper_bound(L + 1,
                L + 1 + idx, a) - L;
            id -= 1;
            FOR(i, 0, SZ(g[id]) - 1) {
                if (g[id][i][1] == a) {
                    g[id][i][0] = b;
                    break;
                }
```

```
            }
            sort(all(g[id]));
        }
    }
}
```

9.16 回滚莫队

普通莫队需要支持**加 + 删**两种 update。当题目里「删」很难写（如众数：删掉最大频次后要回到第二大频次，没有方便的 $O(1)$ 回退）时，用**回滚莫队**：每个询问的左端点对应到所属块，按 (块号, r) 排序后，r **单调递增；左端点每次从块右端开始向左扩展**，扫描完后**只 minus 撤销**（不动 r）。如此整个过程只用 add，加完即丢，不需要写 del 的逆操作。

l, r **同块的情况**单独暴力处理（块内最多 $\sqrt{N}$ 个元素）。下方实现求区间众数 $\geq w$（LeetCode threshold-majority-queries）：

```
struct Que { ll bid, l, r, w, i; };
bool cmp(const Que& a, const Que& b) {
    if (a.bid != b.bid) return a.bid < b.bid;
    return a.r < b.r;
}

class Solution {
public:
    vector<int> subarrayMajority(
        vector<int>& nums,
        vector<vector<int>>& queries) {
        ll Q = SZ(queries);
        ll n = max(1ll,
            SZ(nums) / (ll)sqrtl(Q));
        vector<int> ans(Q);
        vector<Que> que;
        FOR(i, 0, Q - 1) {
            ll l = queries[i][0],
               r = queries[i][1],
               w = queries[i][2];
            ll bid  = l / n;
            ll rbid = r / n;
            if (bid == rbid) {
                // l, r 同块，暴力
                unordered_map<ll, ll> cnt;
                ll lans = INF, mxc = 0;
                FOR(j, l, r) {
                    cnt[nums[j]]++;
                    if (cnt[nums[j]] == mxc)
                        lans = min<ll>(lans,
                                       nums[j]);
                    else if (cnt[nums[j]] >
                    ↪  mxc) {
                        mxc = cnt[nums[j]];
                        lans = nums[j];
                    }
                }
                if (lans == INF || mxc < w)
                    ans[i] = -1;
                else ans[i] = lans;
            } else {
                que.EB(bid, l, r, w, i);
            }
        }
        sort(all(que), cmp);
        ll lans = INF, mxc = 0;
        unordered_map<ll, ll> freq;
        auto add = [&](ll num, ll cc) {
            freq[num] += cc;
            if (freq[num] == mxc)
                lans = min(lans, num);
            else if (freq[num] > mxc) {
                mxc = freq[num];
                lans = num;
            }
        };
        auto minus = [&](ll num, ll cc) {
            freq[num] -= cc;
        };
        ll up = -1, upr = -1;
        ll tmplans = INF, tmpmxc = 0;
        FOR(i, 0, SZ(que) - 1) {
            auto& [bid, l, r, w, id] = que[i];
            if (bid != up) {
                lans = INF; mxc = 0;
                freq.clear();
                up = bid;
                upr = (bid + 1) * n;
            }
            // r 单调右扩
            for (; upr <= r; ++upr)
                add(nums[upr], 1);
            // 回滚莫队的精髓：备份 + 临时左扩
            tmplans = lans; tmpmxc = mxc;
            FOR(j, l,
                min<ll>((bid + 1) * n - 1,
                        SZ(nums) - 1))
                add(nums[j], 1);
            if (lans == INF || mxc < w)
                ans[id] = -1;
            else ans[id] = lans;
            // 回滚
            lans = tmplans; mxc = tmpmxc;
            FOR(j, l,
                min<ll>((bid + 1) * n - 1,
                        SZ(nums) - 1))
                minus(nums[j], 1);
        }
        return ans;
    }
};
```

10 图论

10.1 倍增法求 LCA

倍增法 LCA 用 `Anc[u][k]` 存「u 的第 2^k 级祖先」。先把两点跳到同深度，再一起往上跳直到分叉，跳完返回 `Anc[a][0]` 就是 LCA。模板题 P3398 仓鼠找 sugar：判两条树上路径是否相交，技巧是用 LCA 把" 两条路径" 翻译成" 两个区间端点祖先关系"。

```
ll N,M,Q,K,T;
ll Deep[MAXN],Anc[MAXN][30];
bool vis[MAXN];
vector<ll> g[MAXN];

void dfs(ll x,ll fa){
    if(vis[x]) return;
    vis[x]=true;
    Deep[x]=Deep[fa]+1;
    Anc[x][0]=fa;
    FOR(i,0,SZ(g[x])-1){
        ll vnode=g[x][i];
        if(vis[vnode]) continue;
        dfs(vnode,x);
    }
}

inline ll LCA(ll a,ll b){    // 倍增法求 LCA
    if(Deep[a]<Deep[b]) swap(a,b);
    ROF(i,18,0){
        if(Deep[Anc[a][i]]>=Deep[b]){
            a=Anc[a][i];
        }
    }
    if(a==b) return a;
    ROF(i,18,0){
        if(Anc[a][i]!=Anc[b][i]){
            a=Anc[a][i],b=Anc[b][i];
        }
    }
    return Anc[a][0];
}

inline void solve(){
    cin>>N>>Q;
    FOR(i,1,N-1){
        ll u,v; cin>>u>>v;
        g[u].pb(v); g[v].pb(u);
    }
    dfs(1,0);
    FOR(j,1,18) FOR(i,1,N)
    ↪  Anc[i][j]=Anc[Anc[i][j-1]][j-1];

    FOR(i,1,Q){
        ll a,b,c,d; cin>>a>>b>>c>>d;
        if(Deep[a]<Deep[b]) swap(a,b);
        if(Deep[c]<Deep[d]) swap(c,d);
        ll lcaAB=LCA(a,b);
        ll lcaCD=LCA(c,d);
        // 保证共线：lcaCD 在 lcaAB 子树内 且
        ↪  lcaAB 在 c-d 路径上
        if(LCA(lcaAB,lcaCD)==lcaCD &&
        ↪  LCA(c,lcaAB)==lcaAB){ cout<<"Y\n";
        ↪  continue; }
        if(LCA(lcaAB,lcaCD)==lcaCD &&
        ↪  LCA(lcaAB,d)==lcaAB){ cout<<"Y\n";
        ↪  continue; }
        // 看 lcaCD 在不在 a->lcaAB 上
        if(LCA(lcaCD,lcaAB)==lcaAB &&
        ↪  LCA(lcaCD,a)==lcaCD){ cout<<"Y\n";
        ↪  continue; }
        if(LCA(lcaCD,lcaAB)==lcaAB &&
        ↪  LCA(lcaCD,b)==lcaCD){ cout<<"Y\n";
        ↪  continue; }
        cout<<"N\n";
    }
}
// AC https://www.luogu.com.cn/record/217989209
```

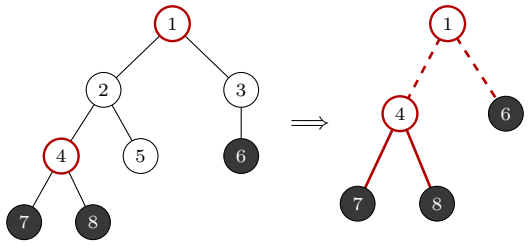
10.2 虚树（Virtual Tree）

适用场景：多组询问，每次给出一批**关键点**，要在树上做只跟这些关键点有关的 DP ／贪心。保证 $\sum k \le n$，但单次询问若还遍历整棵原树就是 $O(nq)$。虚树把每次的规模压到 $O(k)$：只保留**关键点**以及**任意两关键点的 LCA**，其余点全部缩掉，两点间的原树路径压成一条虚边。总复杂度 $O(\sum k \log k)$（LCA 用 dfn 序 ST 表做到 $O(1)$，瓶颈在按 dfn 排序）。

关键事实：所有关键点两两的 LCA 集合，等于「按 dfn 排序后**相邻**两点的 LCA」集合——所以只需算 $k-1$ 次 LCA，不必两两枚举。

构造步骤（本模板用的是" 排序 + 相邻连边" 版，不写单调栈，短且不易错）：

1. 关键点按 dfn 升序排序；
2. 对每对相邻关键点求 LCA，一并塞进点集；
3. 再按 dfn 排序并去重，得到虚树点集；
4. 对排好序的点集，把 $\mathrm{LCA}(\mathrm{keys}_i, \mathrm{keys}_{i+1})$ 连向 keys_{i+1}，即得虚树的所有边。



关键点（黑）$\{6,7,8\}$：相邻 LCA 补进 $4=\mathrm{LCA}(7,8)$ 与 $1=\mathrm{LCA}(8,6)$（红圈），点 $2,3,5$ 被缩掉。虚线虚边表示它在原树里对应一条多边路径（如 $1\to4$ 实为 1-2-4），做 DP 时边权取 dep 之差。

易错点：

- 清空虚树邻接表**只能清本次用到的点**（`for u in keys: _g[u].clear()`），整表清是 $O(n)$，多测直接退化。
- 虚树的根是 `keys.front()`（dfn 最小者），**不一定**是原树根 1；DFS 起点要用它。

- 关键点集合会被构造过程**就地扩充**，需要原始关键点判定时先拷一份（下面的 origin_keys）。
- 判「两个关键点在原树里是否直接相邻」要查原邻接表，不能看虚树边——虚边可能跨了很多原边。
- 黄色高亮的两处 init() 是惰性初始化（isInit 保证只跑一遍 dfs + ST 表预处理）：lca_ab 与 build_virtual_tree 各自开头都要调，才能**不管调用顺序**都拿到正确的 dfn / Dep / st。漏掉就是全 0 的静默 WA。

例题 CF 613D Kingdom and its Cities：每次给 k 个关键城市，问最少删几个**非关键**点使关键点两两不连通，两个关键点直接相邻则无解。虚树上贪心：dfs 返回子树里「还没被切断的关键点数」cnt——当前点是关键点就把子树里所有 cnt 都就地切掉（ans += cnt）并返回 1；否则 cnt ≥ 2 时删自己最划算（ans++，返回 0），cnt ≤ 1 时原样上传。

```cpp
struct Tree {
    vector<vector<int> > g; // 原树邻接表
    vector<int> dfn; // dfs 序，虚树排序的依据
    vector<int> rev; // rev[dfn[u]] = u，dfn
    ↪ 反查节点
    vector<int> Fa; // 父亲（替代原来的 Anc
    ↪ 倍增表）
    vector<ll> Dep; // 深度，LCA 不再需要，但树形
    ↪ DP 常用，保留
    vector<vector<int> > st; // st[j][i]：dfn
    ↪ 区间 [i, i+2^j) 内 dfn 最小的父亲
    int n, dfIdx = 0;
    const int LOG;
    bool isInit = false;

    Tree(int _n) : n(_n), LOG(__lg(n) + 1) {
        g.resize(n + 2);
        dfn.assign(n + 2, 0);
        rev.assign(n + 2, 0);
        Fa.assign(n + 2, 0);
        Dep.assign(n + 2, 0);
        // 只有 LOG+1 次堆分配；原来的
        ↪ Anc.resize(n+2, vector<int>(LOG+1))
        ↪ 是 n+2 次
        st.assign(LOG + 1, vector<int>(n + 2,
        ↪ 0));
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs(int u, int fa, ll dep) {
        Fa[u] = fa;
        dfn[u] = ++dfIdx;
        rev[dfIdx] = u;
        Dep[u] = dep;
        for (auto to: g[u]) {
            if (to == fa || to == u) {
                continue;
            }
            dfs(to, u, dep + 1);
        }
    }

    // 按 dfn 取较小者（ST 表的合并函数）
    inline int mn(int x, int y) const {
        return dfn[x] < dfn[y] ? x : y;
    }

    void init() {
        // 幂等：多次调用只跑一遍
        if (isInit) {
            return;
        }
        isInit = true;
        dfs(1, 0, 0);
        // st[0][i] = dfn 为 i 的点的父亲。i=1
        ↪ 是根，父亲是 0 号虚点（dfn=0 会污染
        ↪ min），
        // 但查询区间左端 >= 2 永远取不到它，
        ↪ 这里仍填根自己保证安全
        for (int i = 2; i <= dfIdx; ++i) {
            st[0][i] = Fa[rev[i]];
        }
        st[0][1] = rev[1];
        for (int j = 1; j <= LOG; ++j) {
            for (int i = 1; i + (1 << j) - 1 <=
            ↪ dfIdx; ++i) {
                st[j][i] = mn(st[j - 1][i],
                ↪ st[j - 1][i + (1 << (j -
                ↪ 1))]);
            }
        }
    }

    int lca_ab(int a, int b) {
        init();
        if (a == b) {
            return a;
        }
        int x = dfn[a], y = dfn[b];
        if (x > y) {
            swap(x, y);
        }
        ++x; // 区间 (dfn[a], dfn[b]] ——
        ↪ 左开右闭，长度 >= 1
        int j = __lg(y - x + 1);
        return mn(st[j][x], st[j][y - (1 << j)
        ↪ + 1]);
    }

    // keys 就地扩充成「虚树点集」；_g
    ↪ 是虚树邻接表（有向：父 -> 子）
    void build_virtual_tree(vector<vector<int>
    ↪ > &_g, vector<int> &keys) {
        init();
        auto byDfn = [&](const int &a, const
        ↪ int &b) {
            return dfn[a] < dfn[b];
        };
        sort(all(keys), byDfn);
        ll sz_key = SZ(keys);
        keys.reserve(sz_key * 2); //
        ↪ 避免扩充过程中反复 realloc
```

```
        for (int i = 0; i < sz_key - 1; ++i) {
            // 只需相邻两点的 LCA
            keys.push_back(lca_ab(keys[i],
            ↪ keys[i + 1]));
        }
        sort(all(keys), byDfn);
        keys.resize(unique(all(keys)) -
        ↪ keys.begin());
        for (auto u: keys) {
            _g[u].clear(); // 只清本次用到的点
        }
        for (int i = 0; i < SZ(keys) - 1; ++i) {
            _g[lca_ab(keys[i], keys[i +
            ↪ 1])].push_back(keys[i + 1]);
        }
    }
};

// ---- 例题 CF613D 的虚树 DP ----
int dfs(const vector<vector<int>> &g, int u,
↪ int fa, ll &ans,
        const vector<int> &origin_keys) {
    int cnt = 0;
    bool isOri =
    ↪ binary_search(all(origin_keys), u);
    for (auto to : g[u]) {
        if (to == fa) continue;
        cnt += dfs(g, to, u, ans, origin_keys);
    }
    if (isOri) { ans += cnt; return 1; }     //
    ↪ 关键点：切断子树里所有未切断的
    if (cnt >= 2) { ++ans; return 0; }       //
    ↪ 非关键点且是汇合处：删自己
    return cnt;
}

void Solve() {
    cin >> N;
    Tree g(N);
    for (int i = 1; i <= N - 1; ++i) { ll u, v;
    ↪ cin >> u >> v; g.add_edge(u, v); }
    for (int i = 1; i <= N; ++i)
    ↪ sort(all(g.g[i]));  // 为了
    ↪ binary_search 判相邻
    ll Q; cin >> Q;
    vector<vector<int>> _g(N + 2);           //
    ↪ 虚树邻接表复用，不重开
    while (Q--) {
        ll num; cin >> num;
        vector<int> origin_keys(num);
        for (auto &x : origin_keys) cin >> x;
        vector<int> keys = origin_keys;      //
        ↪ 拷一份，keys 会被就地扩充
        sort(all(origin_keys));
        g.build_virtual_tree(_g, keys);
        ll ans = 0; bool havAns = true;
        for (auto u : origin_keys) {         //
        ↪ 两关键点在原树直接相邻 -> 无解
            for (auto v : _g[u]) {
                if (binary_search(all(origin_ke
                ↪ ys), v) &&
                    binary_search(all(g.g[u]),
                    ↪ v)) {
                    havAns = false; cout << -1
                    ↪ << "\n"; break;
                }
            }
            if (!havAns) break;
        }
        // 根是 dfn 最小的点 keys.front(),
        ↪ 不一定是 1
        if (havAns) { dfs(_g, keys.front(), -1,
        ↪ ans, origin_keys); cout << ans <<
        ↪ "\n"; }
    }
}
// AC https://codeforces.com/contest/613/submis
↪ sion/384497446
```

10.3 并查集（DSU）与 DSU on next 技巧

朴素并查集（find 用**路径减半**，一行、非递归），外加 taglr(l, r) 把 $[l, r]$ 这段区间整段标记为「已处理」——具体做法是让区间里的每个点都不指向自己（fa[i] = i + 1，一路连到 $r+1$）。find(x) 因此天然返回 x 之后第一个未被标记的位置，扫描时常用来**跳过整段已处理区间**。fa 多开两格，n + base 是越界哨兵（全标完时 find 落在这里）。普通 merge 与 taglr 不要混在同一个对象上用（l、$l+1$ 已同块时会成环）。

```
/**   并查集（DSU）+ DSU on next：find(x) = x
↪ 之后第一个没被标记的位置
 *    base = 0 / 1：合法下标 [0, n) / [1, n]；n
 ↪ 要给准，否则 countFa 出错
 */
struct DSU {
    vector<int> fa; int n = 0, base;
    DSU(int n = 0, int base = 0) : base(base) {
    ↪ init(n); }
    void init(int _n) { n = _n; fa.resize(n +
    ↪ base + 2); iota(all(fa), 0); }
    int find(int x) { while (x != fa[x]) x =
    ↪ fa[x] = fa[fa[x]]; return x; } //
    ↪ 路径减半
    bool same(int x, int y) { return find(x) ==
    ↪ find(y); }
    bool merge(int x, int y) {           // 方向
    ↪ fa[x] = y，配合 dsu on next
        x = find(x), y = find(y);
        if (x == y) return false;
        fa[x] = y; return true;
    }
    void taglr(int l, int r) {           // [l,
    ↪ r] 整段标记，之后 find 会越过去
        r = min(r, (int)SZ(fa) - 2);
        for (l = find(max(1, base)); l <= r; l
        ↪ = find(l)) fa[l] = l + 1;
    }
    int countFa() { int c = 0; for (int i =
    ↪ base; i < base + n; i++) c += fa[i] ==
    ↪ i; return c; }
};
// 对拍：与旧版（ll + 两趟压缩）随机 2 万组一致
```

有时候这套思路别僵在数组上，**用 map 当并查集容器**也行，没出现过的 key 默认就是它自己当根。参考 CF Round 1077E 的提交：

```
map<ll,ll> fa;
auto find = [&](ll x) {
    ll root = x;
    while (fa.contains(root)) root = fa[root];
    ↪ // 没 contain 默认就是 root
    while (fa.contains(x)) {
    ↪ // 路径压缩
        ll p = fa[x];
        fa[x] = root;
        x = p;
    }
    return root;
};
auto merge = [&](ll x, ll y) {
    ll fax = find(x), fay = find(y);
    fa[fax] = fay;
};
```

维护到祖宗节点距离的并查集（带权并查集）：d[x] 存 x 到 p[x] 的距离，find 的同时累加距离。

```
int p[N], d[N];
// p[] 存每个点的祖宗节点，d[x] 存 x 到 p[x]
↪ 的距离

int find(int x) {
    if (p[x] != x) {
        int u = find(p[x]);
        d[x] += d[p[x]];
        p[x] = u;
    }
    return p[x];
}

// 初始化（节点 1..n）
for (int i = 1; i <= n; ++i) { p[i] = i; d[i] =
↪ 0; }

// 合并 a 与 b 所在的两个集合：
p[find(a)] = find(b);
d[find(a)] = distance;   // 根据具体问题，初始化
↪ find(a) 的偏移量
```

10.3.1 带权并查集（class 封装，维护到根距离与连通块大小）

把「路径压缩 + 维护 dis[x]（value[x] - value[fa[x]]）」与按大小合并打包成完整类。find 递归回程时把 fa[x] 直接压到 root、dis[x] 累成 value[x] - value[root]；merge(x, y, w) 加入约束 value[y] - value[x] = w,若和已有关系矛盾返回 false；query(x, y) 返回同一连通块下的 value[y] - value[x]。

```
// 带权 DSU 维护点到父节点的距离
class WeightedDSU
{
private:
    vector<int> fa, siz;
    vector<ll> dis; // dis[x] = value[x] -
    ↪ value[fa[x]]，find 后变成 value[x] -
    ↪ value[root]

public:
    WeightedDSU(int n)
    {
        fa.resize(n + 1);
        siz.assign(n + 1, 1);
        dis.assign(n + 1, 0);
        iota(fa.begin(), fa.end(), 0);
    }
    int find(int x)
    {
        if (fa[x] == x)
            return x;
        int oldFa = fa[x];
        int root = find(oldFa);
        dis[x] += dis[oldFa]; // x 到旧父亲 +
        ↪ 旧父亲到根 = x 到根
        fa[x] = root;
        return root;
    }
    bool same(int x, int y)
    {
        return find(x) == find(y);
    }

    // 加入约束：value[y] - value[x] = w
    // 返回 false 表示和已有关系矛盾
    bool merge(int x, int y, ll w)
    {
        int rootX = find(x);
        int rootY = find(y);

        if (rootX == rootY)
            return dis[y] - dis[x] == w;

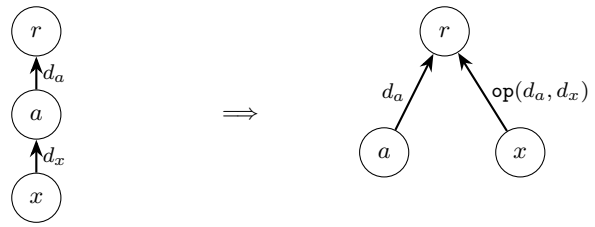
        // 按大小合并
        if (siz[rootX] > siz[rootY])
        {
            swap(x, y);
            swap(rootX, rootY);
            w = -w;
        }
        // 让 rootX 挂到 rootY 下面
        fa[rootX] = rootY;
        siz[rootY] += siz[rootX];
        // 推导：
        // value[y] - value[x] = w
        // value[x] = dis[x] + value[rootX]
        // value[y] = dis[y] + value[rootY]
        //
        // 所以：
        // dis[y] + value[rootY] - dis[x] -
        ↪ value[rootX] = w
        //
        // 得：
        // value[rootX] - value[rootY] = dis[y]
        ↪ - dis[x] - w
        dis[rootX] = dis[y] - dis[x] - w;

        return true;
```

```
    }
    // 查询 value[x] - value[root]
    ll distToRoot(int x)
    {
        find(x);
        return dis[x];
    }
    // 查询 value[y] - value[x]
    // 调用前最好保证 same(x, y)
    ll query(int x, int y)
    {
        find(x);
        find(y);
        return dis[y] - dis[x];
    }
};
```

10.3.2 带权并查集（群作用泛型版）+ 不动点钉死 pinned_dsu

把上一份 WeightedDSU 里写死的「ll 加法」换成抽象的群 S（给出 op（复合）/ e（单位元）/ inv（逆）三个函数指针即可），同一套代码就能吃加法、异或、模 k 加法、仿射变换 $v \mapsto s \cdot v + d$ 等各种约束。约定 d[x] 是映射「value[fa[x]] $\mapsto$ value[x]」，op(a, b) 表示**先 a 后 b**；leader 用两趟迭代代替递归（链长 10^5 时递归版会爆栈），顺手把整条路径压到根上。



压缩前　　压缩后

图中箭头是 fa 指针，边上标的映射方向与它相反（d_x 把父亲的取值映到 x 的取值）。压缩就是把路径上的映射按「先靠近根」的顺序复合后挂到根上；diff(x, y) = op(inv(d_x), d_y)，即先退回根再走到 y。

那层 pinned_dsu 解决什么：裸版同块时判 op(inv(dx), dy) == w，这只在群作用**自由**（非单位元一律没有不动点，如平移 / 异或）时才等价于「矛盾」。仿射群下就会误判：已知 $v_y = v_x$ 又来一条 $v_y = -v_x + 4$，两个变换确实不同，但并不矛盾——只是把整块钉死成了 $h(v) = -v + 4$ 的不动点 $v = 2$。所以 pinned_dsu 多存 det[根] / val[根]，冲突时改判 $h = \mathrm{op}(w, \mathrm{inv}(g_0))$ 有没有不动点：没有才是真矛盾，有就钉死或与已钉死的值核对。让 fix 恒返回 {false, *} 即退化成裸 weighted_dsu。

坑：示例的 fix 用 h.d / 2 求不动点，h.d 为奇数时其实无整数解，这种题要自己在这里返回矛盾，别被整除截断悄悄吞掉。

```
/* 泛型带权并查集：权是一个群 S，作用在取值集合 V
↪  上
 * 2026 杭电多校 7 1011 https://acm.hdu.edu.cn/」
 ↪  contest/problem?cid=12358&pid=1011
 * 约定：d[x] 是映射 value[fa[x]] |-> value[x]；
 ↪  op(a, b) 表示「先 a 后 b」
 */
template<class S, S (*op)(S, S), S (*e)(), S
↪  (*inv)(S)>
struct weighted_dsu {
public:
  weighted_dsu() : weighted_dsu(0) {
  }

  // 合法位置 0..n，内部开 n+1 个槽
  explicit weighted_dsu(int n)
    : _n(n + 1), fa(n + 1), siz(n + 1, 1), d(n
    ↪  + 1, e()) {
    iota(fa.begin(), fa.end(), 0);
  }

  // 找根 + 路径压缩；返回后 d[x] 就是映射
  ↪  value[root] |-> value[x]
  int leader(int x) {
    assert(0 <= x && x < _n);
    // 第一趟：往上走，新边排在前面（它更靠近根，
    ↪  先被走到）
    int r = x;
    S acc = e(); // 映射 value[r] |-> value[x]
    while (fa[r] != r) {
      acc = op(d[r], acc); // 先 d[r]，后 acc
      r = fa[r];
    }
    // 第二趟：求逆回填，沿途每个点直接挂到 r
    S cur = acc; // 映射 value[r] |->
    ↪  value[当前点]
    while (x != r) {
      int nxt = fa[x];
      S old = d[x]; // 必须先存旧边
      d[x] = cur;
      fa[x] = r;
      cur = op(cur, inv(old)); // 先 cur，后退掉
      ↪  x 这一步
      x = nxt;
    }
    return r;
  }

  bool same(int x, int y) { return leader(x) ==
  ↪  leader(y); }
  int size(int x) { return siz[leader(x)]; }

  // 映射 value[根] |-> value[x]
  S dist(int x) {
    leader(x);
    return d[x];
  }

  // 同块时返回 g，满足 value[y] = g(value[x])
  S diff(int x, int y) {
    int rx = leader(x), ry = leader(y); //
    ↪  显式压缩，不依赖 assert
    assert(rx == ry);
    (void)rx;
    (void)ry;
    // value[x] --inv(d[x])--> V --d[y]-->
    ↪  value[y]
    return op(inv(d[x]), d[y]);
```

```
  }

  // 加入约束 value[y] = w(value[x]); 返回 false
  ↪ 表示与已有约束不一致
  bool merge(int x, int y, S w) {
    int rx = leader(x), ry = leader(y);
    S dx = d[x], dy = d[y];
    if (rx == ry) return op(inv(dx), dy) == w;

    // 目标 g = 映射 V_rx |-> V_ry
    // V_rx --dx--> value[x] --w--> value[y]
    ↪ --inv(dy)--> V_ry
    S g = op(op(dx, w), inv(dy));
    if (siz[rx] < siz[ry]) {
      swap(rx, ry);
      g = inv(g);
    }
    fa[ry] = rx;
    d[ry] = g;
    siz[rx] += siz[ry];
    return true;
  }

private:
  int _n; // 槽数, 合法下标 0.._n-1
  vector<int> fa, siz;
  vector<S> d;
};

/* 在 weighted_dsu 之上加一层「不动点钉死」
 *
 * 用途: 当群在取值集合上的作用【不自由】
 ↪ 时(存在非单位元有不动点),
 *      裸 weighted_dsu 的 merge 返回 false
 ↪ 只表示「两个变换不同」,
 *      不等于矛盾 ——
 ↪ 它可能只是把某个块的取值钉死到了不动点上。
 *
 * 用法: pinned_dsu(n) 合法位置 0..n, 与
 ↪ weighted_dsu 一致
 *
 * 额外需要:
 *   V            : 被群作用的值类型,
 ↪ 需可默认构造且有 operator==
 *   act(f, v)    : 群作用, 返回 f(v)
 *   fix(h)       : h 的不动点集合; 返回 {true,
 ↪ p} 表示唯一不动点 p,
 *                  {false, *} 表示无不动点。
 *                  仅在 h != e 时被调用,
 ↪ 故不必处理「处处不动」的情形。
 *
 * 自由作用的群(加法/异或/模 k 加法)让 fix 恒返回
 ↪ {false, *} 即可,
 * 此时本结构退化成裸 weighted_dsu。
 */
template<class S, S (*op)(S, S), S (*e)(), S
↪ (*inv)(S),
  class V, V (*act)(S, V), pair<bool, V>
  ↪ (*fix)(S)>
struct pinned_dsu {
public:
  pinned_dsu() : pinned_dsu(0) {
  }

  explicit pinned_dsu(int n)
    : uf(n), det(n + 1, 0), val(n + 1, V{}) {
  }

  // 加入约束 value[y] = w(value[x]); 返回 false
  ↪ 表示与已有约束矛盾
  bool merge(int x, int y, S w) {
    int rx = uf.leader(x), ry = uf.leader(y);

    if (rx == ry) {
      // ---------- 同一个块 ----------
      S g0 = uf.diff(x, y); // 已知的 value[x]
      ↪ |-> value[y]
      if (g0 == w) return true; // 恒成立,
      ↪ 无新信息

      // 约束 <=> value[x] 是 h 的不动点
      S h = op(w, inv(g0));
      auto [has, p] = fix(h);
      if (!has) return false; // 无不动点 ->
      ↪ 真矛盾

      if (det[rx]) // 已钉死过, 验证是否一致
        return act(uf.dist(x), val[rx]) == p;
      det[rx] = 1; // 由 value[x] = p 反解出根
      val[rx] = act(inv(uf.dist(x)), p);
      return true;
    }

    // ---------- 跨块 ----------
    if (det[rx] && det[ry]) {
      // 两块数值全已知, 必须先验证
      V vx = act(uf.dist(x), val[rx]);
      V vy = act(uf.dist(y), val[ry]);
      if (!(act(w, vx) == vy)) return false;
    }
    // 合并前先取出「锚点」的真实值(合并后 dist
    ↪ 会变)
    bool anchored = det[rx] || det[ry];
    int src = det[rx] ? x : y;
    V sv = V{};
    if (anchored) sv = act(uf.dist(src),
    ↪ det[rx] ? val[rx] : val[ry]);

    uf.merge(x, y, w); // 挂载方向由内层自行决定

    if (anchored) {
      // 钉死状态传播到新根
      int nr = uf.leader(src);
      det[nr] = 1;
      val[nr] = act(inv(uf.dist(src)), sv);
    }
    return true;
  }

  bool same(int x, int y) { return uf.same(x,
  ↪ y); }
  int size(int x) { return uf.size(x); }

  // 该点所在块是否已被钉死;
  ↪ 是则一并返回该点的确定取值
  pair<bool, V> value(int x) {
    int r = uf.leader(x);
```

```
    if (!det[r]) return {false, V{}};
    return {true, act(uf.dist(x), val[r])};
  }

private:
  weighted_dsu<S, op, e, inv> uf;
  vector<char> det; // det[根]: 该块是否已被钉死
  vector<V> val; // val[根]: 钉死时根的取值
};

/* ---- 用法示例: 仿射约束 value[y] = s *
↪  value[x] + d, s 取 +1 / -1 ---- */
namespace T1 {
  struct S {
    int s;
    ll d;

    bool operator==(const S &o) const {
      return o.s == s && d == o.d;
    }
  };

  // 先 a 后 b
  S op(S a, S b) {
    //              一次项              常数项
    return {a.s * b.s, b.s * (a.d) + b.d};
  }

  S e() {
    return {1, 0};
  }

  // V=a.s*X+a.d;
  // a.s*X=V-a.d
  // X=a.s*V-a.d*a.s
  S inv(S a) {
    return {a.s, -a.d * a.s};
  }

  using V = ll;
  // 群作用: f(v) = f.s * v + f.d
  V act(S f, V v) { return (ll)f.s * v + f.d; }

  // 仅在 h != e 时调用
  //   h.s = +1 : h(v) = v + h.d 且 h.d != 0,
  ↪  纯平移, 无不动点
  //   h.s = -1 : h(v) = -v + h.d, 唯一不动点 h.d
  ↪  / 2
  pair<bool, V> fix(S h) {
    if (h.s == 1) return {false, 0};
    return {true, h.d / 2};
  }
}

using WDSU = pinned_dsu<T1::S, T1::op, T1::e,
↪  T1::inv, T1::V, T1::act, T1::fix>;
```

用法例子：n 个未知数 value[1..n]，逐条读入 m 条形如「1 x y d $\Rightarrow$ $v_y = v_x + d$」「2 x y d $\Rightarrow$ $v_y = -v_x + d$」的约束，输出第一条与前面矛盾的编号（没有则 0），再输出所有已被唯一确定的未知数。要点：merge 返回 false 时内部状态**不会被改动**，所以撞到矛盾之后还能接着往下加。

```
void Solve() {
  int n, m;
  cin >> n >> m;
  WDSU dsu(n); // 合法下标 0..n, 这里只用 1..n
  int bad = 0;
  for (int i = 1; i <= m; ++i) {
    int type, x, y;
    ll d;
    cin >> type >> x >> y >> d;
    int s = (type == 1 ? 1 : -1); // 群元素 w:
    ↪  value[y] = s * value[x] + d
    if (!dsu.merge(x, y, {s, d}) && bad == 0) {
      bad = i;
    }
  }
  cout << bad << "\n";
  for (int i = 1; i <= n; ++i) {
    auto [det, v] = dsu.value(i); // det=false
    ↪  表示这块还有自由度, 取值不唯一
    if (det) {
      cout << i << " = " << v << "\n";
    }
  }
}

/* 样例输入          样例输出
 * 4 4               4
 * 1 1 2 3           1 = 3
 * 2 1 2 9           2 = 6
 * 1 3 4 5
 * 1 1 2 100
 *
 * v2 = v1+3 与 v2 = -v1+9 联立 -> v1 钉死成 3,
 ↪  v2 = 6
 * v3 v4 只有相对关系, 没被钉死, 不输出
 * 第 4 条 v2 = v1+100 与前面冲突 -> bad = 4
 */
```

10.4 Borůvka 最小生成树

每轮让每个连通块各挑一条最小出边一次性合并，块数至少减半，故至多 $O(\log V)$ 轮、每轮 $O(E)$ 扫边，总 $O(E \log V)$。边以 (w, u, v) 数组给入，节点 0-based，函数自带精简 DSU（与上一节的 DSU 独立）。

```
struct DSU {
    vector<int> f;
    DSU(int n) : f(n) { iota(all(f), 0); }
    int find(int x) { return f[x] == x ? x :
    ↪  f[x] = find(f[x]); } // 真递归路径压缩
    bool unite(int a, int b) { a = find(a); b =
    ↪  find(b); if (a == b) return false; f[a]
    ↪  = b; return true; }
};

// 返回 MST 总权; edges: (w, u, v)
ll boruvka(int n, vector<array<ll, 3>> &edges) {
    DSU d(n);
    int comps = n;
    ll total = 0;
    while (comps > 1) {
```

```
        vector<int> cheap(n, -1);
        // cheap[块根] = 最优出边下标
        for (int i = 0; i < (int)edges.size();
        ++i) {
            auto [w, u, v] = edges[i];
            int ru = d.find(u), rv = d.find(v);
            if (ru == rv) continue;
            // 同块，不是出边
            if (cheap[ru] == -1 || w <
            edges[cheap[ru]][0]) cheap[ru]
            = i;
            if (cheap[rv] == -1 || w <
            edges[cheap[rv]][0]) cheap[rv]
            = i;
        }
        for (int r = 0; r < n; ++r) {
            if (cheap[r] == -1) continue;
            auto [w, u, v] = edges[cheap[r]];
            if (d.unite(u, v)) { total += w;
            --comps; }  //
            兜底跳过本轮已连通者
        }
    }
    return total;
}
```

10.5 树上背包

经典做法：每个点合并儿子时，外层倒序枚举背包总容量，内层正序枚举划分给当前儿子的容量。这样总复杂度可以做到 $O(n \cdot \text{budget})$。下面这份带"父亲买了 / 父亲没买"两种状态（来自 LeetCode-style 题 maxProfit），`dp[1][u][i]` = 父没买时 u 子树用预算 i 的最大收益，`dp[2][u][i]` = 父买了时（当前点享半价）。

```
class Solution {
public:
    int maxProfit(int n, vector<int>& present,
    vector<int>& future,
                  vector<vector<int>>&
                  hierarchy, int budget) {
        int dp[3][170][170];
        for(int i = 1; i <= 2; ++i)
            for(int j = 0; j <= n + 5; ++j)
                for(int k = 0; k <= budget + 5;
                ++k)
                    dp[i][j][k] = 0;
        vector<vector<int>> g(n + 5);
        for(auto &t : hierarchy)
        g[t[0]].push_back(t[1]);

        // ism == 1: 父亲没买; ism == 2:
        父亲买了（享半价）
        auto dfs = [&](this auto &&self, int u,
        int p) -> void {
            for(auto v : g[u]) { if(v == p)
            continue; self(v, u); }

            // memo: 暂时不考虑当前节点 u
            的儿子合并结果
            vector<array<int,3>> memo(budget +
            5, {0,0,0});
            for(auto &v : g[u]) {
                for(int i = budget; i >= 0;
                --i) {
                    for(int j = 0; j <= i; ++j)
                    {       // 内层必须正序
                        memo[i][1] =
                        max(memo[i][1],
                        memo[i-j][1] +
                        dp[1][v][j]);
                        memo[i][2] =
                        max(memo[i][2],
                        memo[i-j][2] +
                        dp[2][v][j]);
                    }
                }
            }
            for(int i = budget; i >= 0; --i) {
                for(int j = 1; j <= 2; ++j) {
                    int cost = present[u-1] / j;
                    dp[j][u][i] =
                    max(dp[j][u][i],
                    memo[i][1]);
                    if(i >= cost) {
                        dp[j][u][i] =
                        max(dp[j][u][i],
                          future[u-1] - cost +
                          memo[i-cost][2]);
                    }
                }
            }
        };
        dfs(1, 1);
        int ans = 0;
        for(int i = budget; i >= 0; --i) ans =
        max({ans, dp[1][1][i]});
        return ans;
    }
};
```

10.6 Floyd 全源最短路

三层循环，外层 i 是「中转点」，必须放最外面。复杂度 $O(n^3)$，写起来最简单。

```
const ll N = 110;
ll n, m, D[N][N];

inline void Floyd() {
    for(int i = 1; i <= n; ++i)             //
    中转点 i 必须在最外层
        for(int j = 1; j <= n; ++j) {
            if(j == i) continue;
            for(int k = 1; k <= n; ++k) {
                if(k == i) continue;
                D[j][k] = min(D[j][k], D[j][i]
                + D[i][k]);
            }
        }
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0);
    cout.tie(0);
```

```
    cin >> n >> m;
    memset(D, 0x3f, sizeof(D));
    for(int i = 1; i <= n; ++i) D[i][i] = 0;
    for(int i = 1; i <= m; ++i) {
        ll u, v, w; cin >> u >> v >> w;
        D[u][v] = min(D[u][v], w);          //
        ↪ 处理重边、自环
        D[v][u] = min(D[v][u], w);
    }
    Floyd();
}
// AC https://www.luogu.com.cn/record/186274091
```

10.7 Dijkstra（堆优化）

核心：小根堆存 {点，距离}，每次取距离最小的点。同一点可能以不同距离多份入堆，第一次出堆即为最短路、定为已固定点，靠 vis 懒删除跳过其余过期记录。松弛成功（能让邻居更短）才把邻居压回堆；堆开在函数里，多测天然清空。

```
struct Node {
    ll node, dis;
    bool operator<(const Node &o) const {
        return dis > o.dis;
        ↪ // dis 大的优先级低 -> 小根堆
    }
};

void Solve() {
    ll N, M, s;
    cin >> N >> M >> s;
    vector<vector<pair<ll,ll>>> g(N + 3);
    ↪ // g[u] = {邻点 v, 边权 w}
    for (int i = 1; i <= M; ++i) {
        ll u, v, w;
        cin >> u >> v >> w;
        g[u].emplace_back(v, w);
    }
    vector<ll> dist(N + 3, INF);
    vector<char> vis(N + 3);
    priority_queue<Node> pq;
    dist[s] = 0;
    pq.push({s, 0});
    while (!pq.empty()) {
        auto [u, dis] = pq.top();
        pq.pop();
        if (vis[u]) continue;
        ↪ // 过期记录，懒删除
        vis[u] = true;
        ↪ // 第一次出堆即定点
        for (auto [to, w] : g[u]) {
            if (dis + w < dist[to]) {
            ↪ // 松弛成功才入堆
                dist[to] = dis + w;
                pq.push({to, dis + w});
            }
        }
    }
}
// AC https://acm.hdu.edu.cn/contest/view-code?
↪ cid=1203&rid=10964
```

10.8 二分图判定（染色法）

DFS 给每个点染 1 或 2，相邻点必须异色；用 3 - c 在 1 和 2 之间翻转就够。注意图可能不连通，外层要对每个未染色的点都启动一次。

```
const ll N = 1e5 + 10;
ll n, m;
short int color[N];
vector<ll> g[N];
bool isErfen = true;

void dfs(int x, int c) {
    if(!isErfen) return;
    if(color[x] == 0) color[x] = c;
    else {
        if(color[x] != c) { isErfen = false;
        ↪ return; }
        return;
    }
    for(int i = 0; i < (int)g[x].size(); ++i) {
        dfs(g[x][i], 3 - c);                // 1
        ↪ 和 2 之间翻转的小 trick
    }
}

int main() {
    cin >> n >> m;
    for(int i = 1; i <= m; ++i) {
        int u, v; cin >> u >> v;
        g[u].push_back(v); g[v].push_back(u);
    }
    for(int i = 1; i <= n; ++i) {          //
    ↪ 处理不连通
        if(color[i] == 0) dfs(i, 1);
        if(!isErfen) break;
    }
    cout << (isErfen ? "Yes" : "No") << endl;
}
```

10.9 匈牙利算法（二分图最大匹配）

把 match[] 哨兵值设为 -1（不是 0），可以避免 0-based 编号造成歧义。每次给左侧某点 u 找匹配时清空 vis，DFS 沿增广路尝试腾位。

```
/* 二分图最大匹配，匈牙利算法
 * 2026-03-31 AC https://acm.hdu.edu.cn/contest
 ↪ /view-code?cid=1198&rid=18859
 */
struct Bi_graph_matcher {
    vector<vector<ll>> g;
    vector<char> vis;
    vector<ll> match;
    ll left_cnt;

    Bi_graph_matcher(vector<vector<ll>> _g, ll
    ↪ _left_cnt)
        : g(std::move(_g)), vis(SZ(g) + 2),
        ↪ match(SZ(g) + 2, -1),
        ↪ left_cnt(_left_cnt) {}
```

```
    bool find(ll u) {
        for (auto to : g[u]) {
            if (vis[to]) continue;
            vis[to] = true;
            // v 还没匹配 / v
            ↪ 原本匹配的人能腾出位置去找别人
            if (match[to] == -1 ||
            ↪ find(match[to])) {
                match[to] = u;
                return true;
            }
        }
        return false;
    }

    ll mx_match() {
        ll ans = 0;
        for (int i = 0; i <= left_cnt; ++i) {
            vis.assign(SZ(g) + 2, 0);
            if (find(i)) ++ans;
        }
        return ans;
    }
};
```

10.9.1 Hall 婚配定理（完美匹配判定）

给定二分图 $G = (L \cup R, E)$。L 存在完美匹配（即所有 L 中的点都被匹配）当且仅当：对 L 的任意子集 $S \subseteq L$，都有

$$|N(S)| \geq |S|$$

其中 $N(S)$ 是 S 在 R 中所有邻居构成的集合。

推论（亏量定理 / Deficiency form）：L 的最大匹配大小为

$$|L| - \max_{S \subseteq L} \big(|S| - |N(S)|\big)$$

右边的 max 项称为 Hall 亏量，刻画 L 中”必然落单的点数下界”。

10.10 差分约束

差分约束就是把一组形如 $x \leq y + a$ 的约束转化成最短路问题：把约束写成三角不等式 $\mathrm{dist}(v) \leq \mathrm{dist}(u) + w(u, v)$，每条约束就是一条有向边 $u \to v$ 边权 a，跑最短路得到的 `dist[]` 就是一组合法解（超级源点的 `dist` 值不重要，差值才重要）。

例题 ABC 395F Smooth Occlusion：相邻牙齿高度差 $\leq X$，转成 $\mathrm{dist}_v \leq \mathrm{dist}_u + X$ 跑 Dijkstra。

```
ll N, X;
arr ud[MAXN];

inline void solve(){
    cin >> N >> X;
    ll sumUd = 0;
    vector<ll> dist(N + 5, INF);
    vector<vector<ll>> g(N + 5);
    vector<bool> vis(N + 5, false);
    priority_queue<arr, vector<arr>, cmp> q;
    FOR(i, 1, N) {
        cin >> ud[i][0] >> ud[i][1];
        sumUd += (ud[i][0] + ud[i][1]);
        if(i != N) g[i+1].pb(i), g[i].pb(i+1);
        dist[i] = ud[i][0];
        q.push({i, dist[i]});
    }
    // 在差分约束图上跑 Dijkstra
    while(!q.empty()) {
        ll u = q.top()[0], dis = q.top()[1];
        ↪ q.pop();
        if(vis[u]) continue;
        vis[u] = true; dist[u] = dis;
        FOR(i, 0, SZ(g[u]) - 1) {
            ll v = g[u][i];
            if(!vis[v]) q.push({v, dis + X});
        }
    }
    ll H = INF;
    FOR(i, 1, N) H = min(H, dist[i] + ud[i][1]);
    cout << sumUd - N * H << "\n";
}
// AC https://atcoder.jp/contests/abc395/submis
↪ sions/64194370
```

10.11 Tarjan 缩点为边双连通分量（EBCC）

把求桥和缩点合在一起：先一遍 Tarjan 标出所有桥，再绕开桥跑一遍连通块染色，每个连通块就是一个 EBCC。输入是带边编号（eid）的无向图，重边用不同 eid 区分。

```
// Tarjan_EBCC: 在原图上(带边编号)求割边(桥)并缩
↪ 点为边双连通分量(edge-biconnected components)
// 下标约定: EBCC 数组下标采用 0-base,
↪ 但是点所在的 EBCC_ID(comp[u]) 采用 1-base
class Tarjan_EBCC {
    int n;
    vector<int> dfn, low;
    int timer;
    vector<int> bridges;
    vector<int> comp;
    int comp_cnt;

public:
    vector<vector<pair<int,int>>> g;
    Tarjan_EBCC(int n = 0) : n(n), g(n+1),
    ↪ dfn(n+1,0), low(n+1,0), timer(0),
    ↪ comp(n+1,0), comp_cnt(0) {}

    void add_edge(int u, int v, int id) {
        g[u].push_back({v,id});
        g[v].push_back({u,id});
    }

    void run() {
        for(int i=1;i<=n;i++){
            if(!dfn[i]) dfs_bridge(i,-1);
        }
        unordered_set<int> bridgeSet;
```

```
        bridgeSet.reserve(bridges.size()*2+10);
        for(int eid: bridges)
        ↪ bridgeSet.insert(eid);
        for(int i=1;i<=n;i++){
            if(!comp[i]){
                ++comp_cnt;
                dfs_comp(i, comp_cnt,
                ↪ bridgeSet);
            }
        }
    }

    vector<int> get_bridges() const { return
    ↪ bridges; }
    vector<int> get_components() const { return
    ↪ comp; }
    int get_comp_cnt() const { return comp_cnt;
    ↪ }

private:
    void dfs_bridge(int u, int peid) {
        dfn[u] = low[u] = ++timer;
        for(auto [v,eid] : g[u]){
            if(!dfn[v]){
                dfs_bridge(v,eid);
                low[u] = min(low[u], low[v]);
                if(low[v] > dfn[u])
                ↪ bridges.push_back(eid);
            } else if(eid != peid){
                low[u] = min(low[u], dfn[v]);
            }
        }
    }

    void dfs_comp(int u, int cid, const
    ↪ unordered_set<int> &bridgeSet) {
        comp[u] = cid;
        for(auto [v,eid] : g[u]){
            if(bridgeSet.count(eid)) continue;
            if(!comp[v]) dfs_comp(v, cid,
            ↪ bridgeSet);
        }
    }
};
```

10.12 Tarjan 求强连通分量（SCC）

单次 DFS 维护栈：入栈时打 `tin / low`，遇到栈中点用 `low[v]` 而非 `tin[v]` 更新（书上常见两种写法都对）。退栈条件 `tin[u] == low[u]` 时把栈顶到 `u` 全部弹出，组成一个 SCC。

```
vector<ll> tin(N + 5), low(N + 5);
vector<ll> stk;
vector<char> instk(N + 5);
vector<ll> scc(N + 5);
vector<ll> scca(N + 5);                    //
↪ 每个 scc 的点权之和（按需）
ll idx = 0, sccidx = 0;

auto dfs1 = [&](auto self, ll u) -> void {
    tin[u] = low[u] = ++idx;
    stk.PB(u);
    instk[u] = 1;
    for (auto v : g[u]) {
        if (!tin[v]) {
            self(self, v);
            low[u] = min(low[u], low[v]);
        } else if (instk[v]) {
            low[u] = min(low[u], low[v]);
        }
    }
    if (tin[u] == low[u]) {                // u
    ↪ 是当前 scc 的根
        ++sccidx;
        ll y = 0;
        do {
            y = stk.back(); stk.pop_back();
            instk[y] = 0;
            scc[y] = sccidx;
            scca[sccidx] += A[y];
        } while (y != u);
    }
};

FOR(i, 1, N) if(!tin[i]) dfs1(dfs1, i);
```

10.13 点分治

找重心 → 统计经过重心的路径 → 删掉重心递归。把当前重心记为 `c`，删掉 `c` 之后整棵树被分成若干棵子树，每棵子树的根是 `c` 的某个儿子 `v`。任意一条**跨子树路径**（比如从 `v1` 子树某点 x 走到 `v2` 子树某点 y）都一定经过 `c`。

标准套路：

1. 对每个儿子 `v`，跑一次 DFS：从 `c` 出发把”以 `c` 为起点到这棵子树任意点的路径信息”（长度 / 最值 / 计数等）收进 `bucket_v`。
2. 一棵子树一棵子树处理：当前子树 = `bucket_v`，之前已处理过的所有子树用一个全局桶维护（`map` / `vector` / Fenwick 都可）。
3. 处理某棵子树时，先用它的每条路径信息和”之前所有子树”配对，统计跨子树路径；配对完再把这棵子树的信息塞进全局桶，留给后面的子树用。
4. 重心 `c` 自己若也能作为路径端点，最后单独处理一下（如 `ans += cnt[A[root]]`）。

为什么不重不漏：每条路径的「最高层重心」（点分治那棵重心树最上面的那个）唯一，所以每条路径只在它的最高层重心处算一次；不经过 `c` 的路径，等递归下去到那棵子树自己的重心时再算。

模板（来自 Nowcoder 提交）：把”DFS 收集”和”在重心处合并”两个钩子分别叫 `collector / merger`，外面的题目逻辑通过 lambda 传进去。

```
const ll INF = (ll)1e17 + 9;

// 题意相关：每条"从当前重心某儿子出发"的路径上，
// 我们关心的只是"成为严格新最小值"的点的权值 A[x]
```

```cpp
struct Info { ll val; };

// 点分治结构体：负责树、找重心、递归分治。题意通过 collector / merger 传入。
struct CentroidDecomp {
    int n;
    vector<vector<int>> g;
    vector<int> sz;
    vector<char> vis;
    int centroid, best;

    CentroidDecomp(int n_ = 0) { init(n_); }
    void init(int n_) {
        n = n_;
        g.assign(n + 1, {}); sz.assign(n + 1, 0); vis.assign(n + 1, 0);
    }
    void add_edge(int u, int v) { g[u].push_back(v); g[v].push_back(u); }

    template <class InfoT, class Collector, class Merger>
    void run(int rt, Collector collector, Merger merger) {
        int tot = get_size(rt, 0);
        best = tot; centroid = rt;
        get_centroid(rt, 0, tot);
        decompose<InfoT>(centroid, collector, merger);
    }

private:
    int get_size(int u, int p) {
        sz[u] = 1;
        for (int v : g[u]) if (v != p && !vis[v]) sz[u] += get_size(v, u);
        return sz[u];
    }
    void get_centroid(int u, int p, int tot) {
        int mx = tot - sz[u];
        for (int v : g[u]) {
            if (v == p || vis[v]) continue;
            get_centroid(v, u, tot);
            mx = max(mx, sz[v]);
        }
        if (mx < best) { best = mx; centroid = u; }
    }

    template <class InfoT, class Collector, class Merger>
    void decompose(int c, Collector &collector, Merger &merger) {
        vis[c] = 1;
        // 1) 对当前重心 c 的每棵未删子树 dfs 收集信息
        vector<vector<InfoT>> buckets;
        buckets.reserve(g[c].size());
        for (int v : g[c]) {
            if (vis[v]) continue;
            buckets.push_back(collector(v, c, c));
        }
        // 2) 在重心 c 这里把这些子树的信息合并进答案
        merger(c, buckets);
        // 3) 对每棵子树递归
        for (int v : g[c]) {
            if (vis[v]) continue;
            int tot = get_size(v, c);
            best = tot; centroid = v;
            get_centroid(v, c, tot);
            decompose<InfoT>(centroid, collector, merger);
        }
    }
};

// 用法：求树上「严格递减最小值序列」的有序对 (u, v) 数（举例）
// auto collector = [&](int u, int p, int root) -> vector<Info> {
//     vector<Info> vec;
//     function<void(int,int,ll)> dfs = [&](int x, int fa, ll mn) {
//         if (mn > A[x]) vec.push_back({A[x]});
//         // 严格新最小值才收
//         mn = min(mn, A[x]);
//         for (int y : cd.g[x]) if (y != fa && !cd.vis[y]) dfs(y, x, mn);
//     };
//     dfs(u, p, INF);
//     // 关键：初始 mn = INF，不是 A[root]
//     return vec;
// };
// auto merger = [&](int root, vector<vector<Info>> &buckets) {
//     map<ll,ll> cnt;
//     for (auto &vec : buckets) {
//         for (auto &info : vec) {
//             ll a = info.val;
//             if (a >= A[root]) continue;
//             auto it = cnt.find(a);
//             if (it != cnt.end()) ans += it->second;
//         }
//         for (auto &info : vec) cnt[info.val]++;
//     }
//     ans += cnt[A[root]];
// };
// cd.run<Info>(1, collector, merger);
```

10.14 重链剖分（ACL `lazy_segtree`）

参考实现：CodeChef Beautiful Sum (XPS) 的提交。把树拆成若干条重链，每条链在线段树上变成一段连续区间；路径修改/查询 $O(\log^2 n)$，子树修改/查询 $O(\log n)$（一段连续区间 `[dfn[u], dfn[u] + siz[u] - 1]`）。下面这套含奇偶深度分开的异或 Tag、按位拆分的 Info、ACL `lazy_segtree`、HLD 的路径 / 子树双接口，以及对边权题”边挂儿子 + 子树边接口排除自身”的处理。线段树本体直接使用数据结构章的共享 ACL 模板。

```cpp
static constexpr int LOG = 21;
```

```cpp
// --- ACL lazy_segtree 的 S / F 五件套 ---
struct Tag {
    ll xor_val[2];      // [0]: 偶深, [1]: 奇深
    explicit Tag(ll e = 0, ll o = 0) {
        xor_val[0] = e;
        xor_val[1] = o;
    }
};

struct Info {
    int cnt[2];                     // 偶深 / 奇深节点数
    int bit_num[2][LOG];            // 对应奇偶性中每一位为 1 的个数

    Info() {
        memset(cnt, 0, sizeof(cnt));
        memset(bit_num, 0, sizeof(bit_num));
    }

    static Info make_structure(int depth) {
        Info res;
        res.cnt[depth & 1] = 1;
        return res;
    }
};

Info op(Info l, Info r) {
    Info res;
    FOR(p, 0, 1) {
        res.cnt[p] = l.cnt[p] + r.cnt[p];
        FOR(i, 0, LOG - 1)
            res.bit_num[p][i] =
                l.bit_num[p][i] +
                r.bit_num[p][i];
    }
    return res;
}

Info e() { return Info(); }

Info mapping(Tag f, Info x) {
    FOR(p, 0, 1) {
        FOR(i, 0, LOG - 1) {
            if ((f.xor_val[p] >> i) & 1)
                x.bit_num[p][i] =
                    x.cnt[p] - x.bit_num[p][i];
        }
    }
    return x;
}

Tag composition(Tag f, Tag g) {
    // 先 g 后 f; 异或可直接合并。
    return Tag(f.xor_val[0] ^ g.xor_val[0],
               f.xor_val[1] ^ g.xor_val[1]);
}

Tag id() { return Tag(); }

using HldSeg = lazy_segtree<Info, op, e, Tag,
        mapping, composition, id>;

// --- HLD ---
struct HLD {
    int n, cur;
    vector<vector<int>> g;
    vector<int> fa, dep, heavy, head, dfn, siz;

    HLD(int n_) : n(n_), cur(0), g(n_ + 1),
    ↪  fa(n_ + 1), dep(n_ + 1),
                heavy(n_ + 1), head(n_ + 1),
                ↪  dfn(n_ + 1), siz(n_ + 1)
                ↪  {}

    void addEdge(int u, int v) {
    ↪  g[u].push_back(v); g[v].push_back(u); }
    int dfs1(int u, int f) {
        fa[u] = f; dep[u] = dep[f] + 1; siz[u]
        ↪  = 1;
        int maxsz = 0;
        for (int v : g[u]) if (v != f) {
            int sub = dfs1(v, u);
            siz[u] += sub;
            if (sub > maxsz) { maxsz = sub;
            ↪  heavy[u] = v; }
        }
        return siz[u];
    }
    void dfs2(int u, int h) {
        head[u] = h; dfn[u] = ++cur;
        if (heavy[u]) dfs2(heavy[u], h);
        for (int v : g[u]) if (v != fa[u] && v
        ↪  != heavy[u]) dfs2(v, v);
    }
    void build(int root = 1) { cur = 0;
    ↪  dfs1(root, 0); dfs2(root, root); }

    int lca(int u, int v) {
        while (head[u] != head[v]) {
            if (dep[head[u]] > dep[head[v]]) u
            ↪  = fa[head[u]];
            else v = fa[head[v]];
        }
        return dep[u] < dep[v] ? u : v;
    }
    template<class F> void for_subtree(int u, F
    ↪  &&op) { op(dfn[u], dfn[u] + siz[u] -
    ↪  1); }
    template<class F> void for_path(int u, int
    ↪  v, F &&op) {
        while (head[u] != head[v]) {
            if (dep[head[u]] < dep[head[v]])
            ↪  swap(u, v);
            op(dfn[head[u]], dfn[u]);
            u = fa[head[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        op(dfn[u], dfn[v]);
    }
};

// --- HLD + ACL: 路径/子树修改、路径/子树查询、
↪  边接口 ---
struct HLDACL {
    HLD hld;
    HldSeg seg;
```

```
    HLDACL(int n) : hld(n), seg(n) {}

    void addEdge(int u, int v) { hld.addEdge(u,
    ↪ v); }

    void build() {
        hld.build(1);
        vector<Info> a(hld.n + 1, e());
        FOR(i, 1, hld.n)
            a[hld.dfn[i]] =
                Info::make_structure(hld.dep[i]」
                ↪ );
        seg = HldSeg(a);
    }

    void applySubtreeNode(int u, const Tag &t) {
        hld.for_subtree(u, [&](int l, int r) {
            seg.apply(l, r, t);
        });
    }

    void applyPathNode(int u, int v, const Tag
    ↪ &t) {
        hld.for_path(u, v, [&](int l, int r) {
            seg.apply(l, r, t);
        });
    }

    Info querySubtreeNode(int u) {
        Info res = e();
        hld.for_subtree(u, [&](int l, int r) {
            res = op(res, seg.prod(l, r));
        });
        return res;
    }

    Info queryPathNode(int u, int v) {
        Info res = e();
        hld.for_path(u, v, [&](int l, int r) {
            res = op(res, seg.prod(l, r));
        });
        return res;
    }

    // 边 (u, fa[u]) 挂在儿子 u 上;
    // 子树边集 = 子树点集去掉 u 自己。
    void applySubtreeEdge(int u, const Tag &t) {
        applySubtreeNode(u, t);
        int pos = hld.dfn[u];
        seg.apply(pos, pos, t); // 异或两次，撤销
        ↪ u
    }
};
```

10.15 欧拉回路与欧拉路径

欧拉回路存在的充要条件（无向连通图）：每个顶点的度数都是偶数。

欧拉路径存在的充要条件（无向连通图）：奇数度顶点数等于 0 或 2。0 时是回路，2 时起点和终点恰好是这两个奇度点。

构造时 DFS 在**弹栈时**（递归回来时）才把当前边塞进 `ans_ls`，最后整体 `reverse`。**为什么要反着塞**：因为最后要 `reverse(ans_ls)`，所以当前边必须等递归完后续点之后再 `push_back`，反过来才会得到正确顺序——和正常 DFS 顺序**反着写**。

```
if (odd_cnt != 0 && odd_cnt != 2) {
    cout << "NO\n";
    return;
}

vector<ll> ans_ls;
auto dfs = [&](auto &&self, ll u) -> void {
    while (!g[u].empty()) {
        auto it = g[u].begin();
        auto [to, id] = *it;
        g[u].erase(it);
        g[to].erase({u, id});
        self(self, to);
        ans_ls.push_back(id);              //
        ↪ 弹栈时才塞，最后 reverse
    }
};
dfs(dfs, max(start_node, g.begin()->first));

// 判断图是否连通：边数对不上就是不连通
if (SZ(ans_ls) != N) {
    cout << "NO\n";
    return;
}
reverse(all(ans_ls));
cout << "YES\n";
for (auto u : ans_ls) cout << u << " ";
cout << "\n";
```

10.16 Color Coding（随机染色 + 状压 DP）

Color Coding 适合处理这类题：图很大，但只需要找一个**点数不超过** `K` 的简单路径或小子图，并且 `K` 本身很小。它的核心想法是**把“判重节点”改写成“判重颜色”**。

每一轮先给所有点随机染上 `0` 到 `K - 1` 中的一种颜色。之后只允许路径走到**新颜色**的点上。于是状态里的 `sta` 不再表示“访问过哪些点”，而是表示“已经用过哪些颜色”。

这样做的好处是非常直接的：如果一条路径上颜色两两不同，那么这些点一定也两两不同，所以它天然就是简单路径。原本要维护一个 `n` 位的访问集合，现在只需要维护一个 `K` 位的颜色集合，复杂度就从“看起来完全做不了”变成了“`K` 小时可以接受”。

像 1009 走马观花 这题里，点权都在 `[0, K - 1]`，还要求路径点权和 $\bmod K = 0$。这时还有一个很关键的前置结论：**最优解至多只会经过 `K` 个点**。因为如果一条合法路径超过了 `K` 个点，那么看它的前缀和模 `K`，按鸽巢原理一定会有两个前缀余数相同，于是中间那一段的点权和就 $\equiv 0 \pmod K$。把这段删掉以后，合法性不变，但边权和更小，因此最优解不可能更长。

实现中状态是 `dp[u][sta]`，其中存的是一个二元组：

- `dp[u][sta][0]` 表示走到 `u`、颜色集合为 `sta` 时，当前最小的边权和。
- `dp[u][sta][1]` 表示这条最优方案对应的花韵总值模 `K` 的结果。

`timeId[u][sta]` 是一个用来时间戳优化的这个数组。

10.16.1 转移骨架

那么实际上，我们在转移的时候是**只对这个最短路进行优化**，我们**不会对这个花韵值有任何的这个选择，是最短路径，就直接接收其花韵总值**。

```
for (int i = 1; i <= N; ++i) {
  color[i] = uniform_int_distribution<int>(0, K
  ↪  - 1)(rng);
}
for (int i = 1; i <= N; ++i) {
  dp[i][1 << color[i]] = {0, A[i]};
  timeId[i][1 << color[i]] = curT;
  if (A[i] == 0) {
    ans = min(ans, 0ll);
    return;
  }
}
for (int use = 1; use <= K; ++use) {
  for (int sta = 1; sta < (1 << K); ++sta) {
    if (__builtin_popcount(sta) != use)
    ↪  continue;
    for (int u = 1; u <= N; ++u) {
      if (!check(u, sta)) continue;
      for (auto [to,w]: g[u]) {
        if (sta & (1 << color[to])) continue;
        ll tow = dp[u][sta][0] + w;
        if (tow >= ans) {
          break;
        }
        ll toSta = sta | (1 << color[to]);
        if (!check(to, toSta)) {
          dp[to][toSta] = {INF, -1};
          timeId[to][toSta] = curT;
        }
        if (tow < dp[to][toSta][0]) {
          dp[to][toSta][0] = tow;
          dp[to][toSta][1] = (A[to] +
          ↪  dp[u][sta][1]) % K;
          if (dp[to][toSta][1] == 0) {
            ans = min(dp[to][toSta][0], ans);
          }
        }
      }
    }
  }
}
```

上面这段代码里，真正的灵魂只有两句：

- `if (sta & (1 << color[to])) continue;`，表示这个颜色已经用过了，这一轮就不允许再走到这个点。
- `dp[to][toSta][1] = (A[to] + dp[u][sta][1]) % K;`，表示直接把“花韵和模 `K`”塞在状态二元组的第二项里维护，而不是额外拆出一维。

10.16.2 剪枝技巧

这份代码里还有两个很值得加进模板的剪枝和优化细节。

- 邻接表先按边权升序排序。这样在枚举 `u` 的出边时，一旦发现 `d + w >= ans`，后面的边只会更大，可以直接 `break`。（实测这个效果比较好）
- 不要每一轮都整块清空 DP。可以采用时间戳优化。

10.16.3 复杂度

如果一轮染色的状态是 `dp[u][sta]`，复杂度大致是 $O(2^K(N+M))$。像这题再加一维 `sum` 以后，可以记成 $O(K\cdot 2^K(N+M))$。单轮只会在目标路径刚好被染成“彩虹路径”时成功，所以要独立重复若干轮。最坏情况下，长度恰好为 `K` 的目标路径在一轮里全异色的概率是 $\frac{K!}{K^K}$，因此 `K = 6` 时通常要跑几百轮才稳。

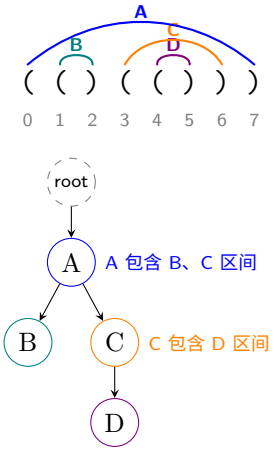
10.16.4 一句话总结

Color Coding 的本质，就是用**“`K` 个颜色的 bitmask”**去近似代替**“真实访问点集合”**。只要题目的目标结构规模小，且简单路径这个限制卡得很死，它往往就是那种平时用不上，一旦出现就很致命的技巧。

10.17 合法括号序列 $\leftrightarrow$ 有根有序树

结论：长度为 $2n$ 的合法括号序列与 $n+1$ 个结点的有根有序树一一对应；计数公式见前文“Catalan 数（合法括号串数量）”小节。扫描建树的过程就是一次 DFS 的进栈/出栈轨迹——遇到 (在当前结点下新建一个”最右儿子”并下去，遇到) 回到父结点，扫完恰好回到根。有一个”虚根”承担最外层括号，方便统一处理。

具体例子：序列 `(()(()))`（$n=4$）。上方是序列与配对弧（同色弧属同一对），下方是对应的 5 结点有根有序树（root 是虚根）。每个实心结点对应一对配对括号：



位置 ↔ 结点：A 对应位置 0–7，B 对应 1–2，C 对应 3–6，D 对应 4–5。注意 A 的两个儿子 B，C 就是位置上**相邻的两对最外层配对**——”兄弟 ↔ 并排配对”在这张图里一眼能看出。

对照表（括号序列位置从 0 编号）：

为什么有用：把”括号序列区间 $[l,r]$”翻译成”子树”之后，问题立刻具备**”子树内独立、跨子树只通过根交互”**的分治/树形 DP 结构。像 CF 2210D 这类”对合法括号序列做变换后再计数/极值”的题，先把序列想成树几乎是本能动作。

配对位置预处理（$O(n)$，栈一遍扫）：

```
// match[i]: 位置 i 的括号与哪个位置配对
vector<int> build_match(const string &s) {
  int n = s.size();
  vector<int> match(n);
  stack<int> st;
  for (int i = 0; i < n; ++i) {
    if (s[i] == '(') st.push(i);
    else {
      int j = st.top(); st.pop();
      match[i] = j;
      match[j] = i;
    }
  }
  return match;
}
```

顺手建真树：如果题目要的不止 match[]，而要父指针、儿子列表，只需在扫的时候每遇 (就 ++cnt 开一个新结点并挂到栈顶结点儿子末尾，同样 $O(n)$：

```
// 返回 {par, children}；结点从 0 开始，0 号为虚根
// 约定输入 s 合法（可外面先 assert）
struct ParenTree {
  vector<int> par;
  vector<vector<int>> ch;
};

ParenTree build_tree(const string &s) {
  ParenTree t;
  t.par.push_back(-1);
  t.ch.emplace_back();
  stack<int> st;
  st.push(0);
  for (char c : s) {
    if (c == '(') {
      int u = (int)t.par.size();
      t.par.push_back(st.top());
      t.ch.emplace_back();
      t.ch[st.top()].push_back(u);
      st.push(u);
    } else {
      st.pop();
    }
  }
  return t;
}
```

常见踩坑：很多题**不需要显式建树**，只要”配对数组 match[] + 深度数组”就足够做区间 → 子树的翻译；显式建树往往只在需要跨子树合并信息（例如儿子之间按下标有序 DP）时才值得做。

10.18 网络流（Dinic）：最大流 / 最小割容量及割边集合

Dinic 求最大流 $O(V^2E)$，二分图匹配 $O(E\sqrt{V})$。核心思路：BFS 在残量网络上分层，DFS 只沿层数 +1 的边增广；cur[u] 当前弧优化保证每条边在一轮里只被无效访问一次。

典型用法：

- **最大流 / 最小割容量**：maxFlow(s, t)。由最大流最小割定理，返回值同时是 s-t 最小割容量。
- **最小割顶点集**：跑完 maxFlow 后调 minCutSide(s)，返回 vis[]：vis[u]=1 表示 u 在 S 侧。原理是残量网络里从 s 沿剩余容量 >0 的边能到的就是 S 侧。
- **最小割边集**：minCutEdges(s) 返回所有”from 在 S 侧、to 在 T 侧”的原图边的输入编号。前提是 addEdge 时传过 id。
- **某条边实际流量**：getFlow(id) 返回 originCap - cap。

建图约定：点编号 1-based，Dinic dinic(n) 后 0 号点空着不用。无向容量边用 addUndirectedEdge(u, v, c)，等价于两个方向各加一条容量 c 的有向边。多测用 dinic.init(n) 重置。

```
class Dinic {
private:
    struct Edge {
        int from, to;
        ll cap;                                  //
        ↪ 当前剩余容量
        ll originCap;                            //
        ↪ 原始容量，用于求割边代价 / 查询流量
        int id;                                  //
        ↪ 输入边编号，不需要输出割边时可以不管
    };

    int n;
    vector<Edge> edges;
    vector<vector<int>> g;
    vector<int> level, cur;
    vector<int> edgeIdToIndex;

    bool bfs(int s, int t) {
        fill(all(level), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int edgeIndex : g[u]) {
                Edge &e = edges[edgeIndex];
                if (e.cap > 0 && level[e.to] ==
                ↪ -1) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
```

```
                }
            }
        }
        return level[t] != -1;
    }

    ll dfs(int u, int t, ll pushed) {
        if (u == t) return pushed;
        if (pushed == 0) return 0;
        for (int &i = cur[u]; i <
        ↪  (int)g[u].size(); i++) {
            int edgeIndex = g[u][i];
            Edge &e = edges[edgeIndex];
            if (e.cap == 0) continue;
            if (level[e.to] != level[u] + 1)
            ↪  continue;
            ll tr = dfs(e.to, t, min(pushed,
            ↪  e.cap));
            if (tr == 0) continue;
            e.cap -= tr;
            edges[edgeIndex ^ 1].cap += tr;
            return tr;
        }
        return 0;
    }

public:
    // 建立一个 1-base 流图，可用点编号 1..n，0
    ↪  号点空着不用。
    Dinic(int n = 0) { init(n); }

    // 多测时用 dinic.init(n) 清空旧图。
    void init(int n_) {
        n = n_;
        edges.clear();
        g.assign(n + 1, {});
        level.assign(n + 1, -1);
        cur.assign(n + 1, 0);
        edgeIdToIndex.clear();
    }

    // 加有向边 from -> to，容量 cap。
    // 只求最大流不需要传 id；
    ↪  要输出最小割边就把输入编号 i 传进来。
    void addEdge(int from, int to, ll cap, int
    ↪  id = -1) {
        if (id >= 0) {
            if ((int)edgeIdToIndex.size() <=
            ↪  id) {
                edgeIdToIndex.resize(id + 1,
                ↪  -1);
            }
            edgeIdToIndex[id] =
            ↪  (int)edges.size();
        }
        edges.push_back({from, to, cap, cap,
        ↪  id});
        g[from].push_back((int)edges.size() -
        ↪  1);
        edges.push_back({to, from, 0, 0, -1});
        g[to].push_back((int)edges.size() - 1);
    }

    // 加无向容量边：u -> v 和 v -> u 都加容量
    ↪  cap。
    // 普通无向图最小割通常这样建。
    void addUndirectedEdge(int u, int v, ll
    ↪  cap, int id = -1) {
        addEdge(u, v, cap, id);
        addEdge(v, u, cap, id);
    }

    // s 到 t 的最大流；由最大流最小割定理也是 s-t
    ↪  最小割容量。
    ll maxFlow(int s, int t) {
        ll flow = 0;
        while (bfs(s, t)) {
            fill(all(cur), 0);
            while (true) {
                ll pushed = dfs(s, t, INF);
                if (pushed == 0) break;
                flow += pushed;
            }
        }
        return flow;
    }

    // 跑完 maxFlow(s, t) 后调用。
    // vis[u] = 1 表示 u 在最小割 S 侧；vis[u] =
    ↪  0 表示在 T 侧。
    // 原理：残量网络里从 s 沿剩余容量 > 0
    ↪  的边能到的点。
    vector<int> minCutSide(int s) {
        vector<int> vis(n + 1, 0);
        queue<int> q;
        vis[s] = 1;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int edgeIndex : g[u]) {
                Edge &e = edges[edgeIndex];
                if (e.cap > 0 && !vis[e.to]) {
                    vis[e.to] = 1;
                    q.push(e.to);
                }
            }
        }
        return vis;
    }

    // 跑完 maxFlow(s, t) 后调用，
    ↪  返回一组最小割边的输入编号。
    // 一条原图边 u -> v 属于最小割 <=> u 在 S
    ↪  侧且 v 在 T 侧。
    // 使用前提：addEdge 时传过 id。
    vector<int> minCutEdges(int s) {
        vector<int> vis = minCutSide(s);
        vector<int> cutEdges;
        for (int edgeIndex = 0; edgeIndex <
        ↪  (int)edges.size(); edgeIndex += 2) {
            Edge &e = edges[edgeIndex];
            if (e.id != -1 && vis[e.from] &&
            ↪  !vis[e.to]) {
                cutEdges.push_back(e.id);
            }
        }
        return cutEdges;
```

```
    }

    // 跑完 maxFlow(s, t) 后调用，
    ↪ 返回当前这组最小割边的原始容量之和。
    // 正常情况下 dinic.minCutCost(s) ==
    ↪ dinic.maxFlow(s, t)。
    ll minCutCost(int s) {
        vector<int> vis = minCutSide(s);
        ll cost = 0;
        for (int edgeIndex = 0; edgeIndex <
        ↪ (int)edges.size(); edgeIndex += 2) {
            Edge &e = edges[edgeIndex];
            if (vis[e.from] && !vis[e.to]) {
                cost += e.originCap;
            }
        }
        return cost;
    }

    // 查询某条输入边实际流了多少；前提是 addEdge
    ↪ 时传过 id。
    ll getFlow(int id) {
        int edgeIndex = edgeIdToIndex[id];
        return edges[edgeIndex].originCap -
        ↪ edges[edgeIndex].cap;
    }
};
```

10.19 最小费用最大流（SPFA 势能 + Dijkstra 增广）

在最大流的基础上，每条边带单位流量费用，要求在所有最大流方案中费用最小的那个。核心思路：每次沿费用最短的增广路推流。直接 SPFA 找最短路也对，但残量网络可能出现负边权（反向边费用为 $-cost$），所以引入**势能**（Johnson 思想）做边权修正，让 Dijkstra 可用：把边 $u \to v$ 的权由 $cost$ 改成 $cost + h_u - h_v$，正确势能下这个修正后的权一定非负，每次 Dijkstra 跑完用 $h_v \mathrel{+}= dist_v$ 同步即可。

复杂度：$O(\mathrm{SPFA} + A \cdot m \log n)$，$A$ 为增广次数（每条增广路至少一条边饱和）。

典型用法：

- **原始边费用非负**：直接 `minCostMaxFlow(s, t)`，h 初始化为 0，第一次 Dijkstra 等价于在原图上跑费用最短路，势能自然生效。
- **原始边费用可能为负**：传 `useSPFA=true`，先用 SPFA 跑出从 s 到每点的初始势能再走 Dijkstra。注意是原始边出现负费用；反向边一开始 `cap=0` 不参与最短路。前提是原图**没有负费用环**——有负环时势能不存在，这套 SSP 会死循环，得改用消圈法。
- **某条边实际流量**：`getFlow(id)` 返回 `originCap - cap`，前提是 `addEdge` 时传过 `id`。费用流最常见的收尾就是还原方案——哪条边被选、谁配了谁。
- **最小割**：费用流跑到底同样是最大流，所以 `minCutSide(s)` / `minCutEdges(s)` / `minCutCost(s)` 与 Dinic 那节完全同义（割只看容量，跟费用无关）。

建图约定：点编号 1-based，反向边随 `addEdge` 自动加（容量 0、费用 $-cost$）。`addEdge` 的第五个参数 `id` 是输入边编号，可选，只在要 `getFlow` / `minCutEdges` 时才需要传。返回 {**最大流，最小费用**} 的二元组。多测用 `mcmf.init(n)` 重置。

```
class MinCostMaxFlow {
private:
    struct Edge {
        int from, to;                          //
        ↪ 边的起点和终点
        ll cap;                                //
        ↪ 当前残余容量
        ll originCap;                          //
        ↪ 原始容量，用于求割边代价 / 查询流量
        ll cost;                               //
        ↪ 单位流量费用
        int id;                                //
        ↪ 输入边编号，不需要查流量 /
        ↪ 输出割边可以不管
    };

    int n;
    vector<Edge> edges;
    vector<vector<int>> g;
    vector<int> edgeIdToIndex;

    // SPFA 初始化势能 h
    // 作用：当原始费用可能为负时，先算出从 s
    ↪ 到每个点的最短路势能
    void initPotentialBySPFA(int s, vector<ll>
    ↪ &h) {
        vector<int> inq(n + 1, 0);
        queue<int> q;
        fill(all(h), INF);
        h[s] = 0;
        q.push(s);
        inq[s] = 1;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            inq[u] = 0;
            for (int edgeIndex : g[u]) {
                Edge &e = edges[edgeIndex];
                if (e.cap <= 0) continue;
                int v = e.to;
                if (h[v] > h[u] + e.cost) {
                    h[v] = h[u] + e.cost;
                    if (!inq[v]) {
                        inq[v] = 1;
                        q.push(v);
                    }
                }
            }
        }
        // 不可达点的势能设为 0，避免后续计算 INF
        ↪ 参与运算
        for (int i = 1; i <= n; i++) {
            if (h[i] == INF) h[i] = 0;
        }
    }
```

```
public:
    // 建立一个 1-base 流图，可用点编号 1..n，0 号点空着不用。
    MinCostMaxFlow(int n = 0) { init(n); }

    // 多测时用 mcmf.init(n) 清空旧图。
    void init(int n_) {
        n = n_;
        edges.clear();
        g.assign(n + 1, {});
        edgeIdToIndex.clear();
    }

    // 加一条 u -> v 的边，容量 cap，单位费用 cost。
    // 同时加反向边 v -> u，容量 0，费用 -cost，用于撤销流量。
    // 只求费用流不需要传 id；要查单边流量 / 输出最小割边就把输入编号 i 传进来。
    void addEdge(int u, int v, ll cap, ll cost, int id = -1) {
        if (id >= 0) {
            if ((int)edgeIdToIndex.size() <= id) {
                edgeIdToIndex.resize(id + 1, -1);
            }
            edgeIdToIndex[id] = (int)edges.size();
        }
        int edgeIndex = (int)edges.size();
        edges.push_back({u, v, cap, cap, cost, id});
        edges.push_back({v, u, 0, 0, -cost, -1});
        g[u].push_back(edgeIndex);
        g[v].push_back(edgeIndex ^ 1);
    }

    // 返回 {最大流, 最小费用}
    // useSPFA 表示是否使用 SPFA 初始化势能
    pair<ll, ll> minCostMaxFlow(int s, int t, bool useSPFA = false) {
        ll maxFlow = 0;
        ll minCost = 0;
        vector<ll> h(n + 1, 0);                 // 势能，让 Dijkstra 可处理残量网络中的负边
        vector<ll> dist(n + 1);                 // 最短路距离
        vector<int> pre(n + 1);                 // pre[v] = 最短路上到达 v 的边下标

        if (useSPFA) {
            initPotentialBySPFA(s, h);
        }

        while (true) {
            fill(all(dist), INF);
            fill(all(pre), -1);
            priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
            dist[s] = 0;
            pq.push({0, s});

            // 在残量网络上跑 Dijkstra，找费用最短的增广路
            while (!pq.empty()) {
                auto [d, u] = pq.top();
                pq.pop();
                if (d != dist[u]) continue;
                for (int edgeIndex : g[u]) {
                    Edge &e = edges[edgeIndex];
                    if (e.cap <= 0) continue;
                    int v = e.to;
                    // 势能修正后的边权，正确势能下一定 >= 0
                    ll newCost = e.cost + h[u] - h[v];
                    if (dist[v] > dist[u] + newCost) {
                        dist[v] = dist[u] + newCost;
                        pre[v] = edgeIndex;
                        pq.push({dist[v], v});
                    }
                }
            }

            // t 不可达，说明已经没有增广路
            if (pre[t] == -1) break;

            // 更新势能
            for (int i = 1; i <= n; i++) {
                if (dist[i] < INF) {
                    h[i] += dist[i];
                }
            }

            // 找这条增广路上的瓶颈容量
            ll addFlow = INF;
            for (int cur = t; cur != s; cur = edges[pre[cur]].from) {
                addFlow = min(addFlow, edges[pre[cur]].cap);
            }

            // 沿着最短费用路增广
            for (int cur = t; cur != s; cur = edges[pre[cur]].from) {
                int edgeIndex = pre[cur];
                edges[edgeIndex].cap -= addFlow;
                edges[edgeIndex ^ 1].cap += addFlow;
                minCost += addFlow * edges[edgeIndex].cost;
            }

            maxFlow += addFlow;
        }

        return {maxFlow, minCost};
    }
```

```
    // 跑完 minCostMaxFlow(s, t) 后调用。费用流跑到底同样是最大流，
    // 所以下面三个割接口和 Dinic 完全同义：割只看容量，跟费用无关。
    // vis[u] = 1 表示 u 在最小割 S 侧；vis[u] = 0 表示在 T 侧。
    // 原理：残量网络里从 s 沿剩余容量 > 0 的边能到的点。
    vector<int> minCutSide(int s) {
        vector<int> vis(n + 1, 0);
        queue<int> q;
        vis[s] = 1;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int edgeIndex : g[u]) {
                Edge &e = edges[edgeIndex];
                if (e.cap > 0 && !vis[e.to]) {
                    vis[e.to] = 1;
                    q.push(e.to);
                }
            }
        }
        return vis;
    }

    // 跑完 minCostMaxFlow(s, t) 后调用，返回一组最小割边的输入编号。
    // 一条原图边 u -> v 属于最小割 <=> u 在 S 侧且 v 在 T 侧。
    // 使用前提：addEdge 时传过 id。
    vector<int> minCutEdges(int s) {
        vector<int> vis = minCutSide(s);
        vector<int> cutEdges;
        for (int edgeIndex = 0; edgeIndex < (int)edges.size(); edgeIndex += 2) {
            Edge &e = edges[edgeIndex];
            if (e.id != -1 && vis[e.from] && !vis[e.to]) {
                cutEdges.push_back(e.id);
            }
        }
        return cutEdges;
    }

    // 跑完 minCostMaxFlow(s, t) 后调用，返回这组最小割边的原始容量之和。
    // 正常情况下 mcmf.minCutCost(s) == mcmf.minCostMaxFlow(s, t).first。
    ll minCutCost(int s) {
        vector<int> vis = minCutSide(s);
        ll cost = 0;
        for (int edgeIndex = 0; edgeIndex < (int)edges.size(); edgeIndex += 2) {
            Edge &e = edges[edgeIndex];
            if (vis[e.from] && !vis[e.to]) {
                cost += e.originCap;
            }
        }
        return cost;
    }

    // 查询某条输入边实际流了多少；前提是 addEdge 时传过 id。
    // 费用流最常用的收尾动作：还原方案（哪条边被选、谁配了谁）。
    ll getFlow(int id) {
        int edgeIndex = edgeIdToIndex[id];
        return edges[edgeIndex].originCap - edges[edgeIndex].cap;
    }
};
```

10.20 图论常见结论

10.20.1 树直径端点的"远点"性质

对于任意点 a，树直径的两个端点中至少有一个是距离 a 最远的点。这就是为什么"两遍 BFS 求树直径"成立：第一遍从任意点 a 出发找最远点 u，u 一定是直径端点；第二遍从 u 出发找最远点 v，(u,v) 就是一条直径。可以用反证法证明。例题 ABC 428。

10.20.2 相邻点单源最短路差不超过 1

在无权图里，对任意一条边 (u,v)，恒有 $|dis_u - dis_v| \le 1$（dis_u 是某固定根到 u 的最短路长度，BFS 层数）。看起来像废话，但偶尔确实有用，如 CF 2179。

进一步地，若图是二分图，可以加强为 $|dis_u - dis_v| = 1$。原因：二分图 $\iff$ 所有环长度为偶数 $\iff$ 从固定根出发的最短路距离的奇偶性给出一个合法二分划分。所以对任意边 (u,v)，dis_u 与 dis_v 的奇偶性必相反，因此不可能相等。

10.20.3 广义凯莱公式（森林连通块版）

给定一个 n 个顶点 m 条边的森林，设有 k 个连通块、大小分别为 $s_1, s_2, \ldots, s_k$。把它们加 $k-1$ 条边合成一棵树的方案数为：

$$n^{k-2} \prod_{i=1}^{k} s_i.$$

直观理解：长度为 $k-2$ 的 prufer 序列贡献 n^{k-2}；乘以 $\prod_{i=1}^{k} s_i$ 是因为对所有非根连通块要确定它的"发送端"（每块只有一个发送端，否则会成环），还要确定根连通块的"接收端"——这个在 prufer 序列中没有直接体现，严格讲是**最后一个被删除的连通块**所连到的根连通块的接收点。

10.20.4 两条树上路径是否相交（LCA 判定）

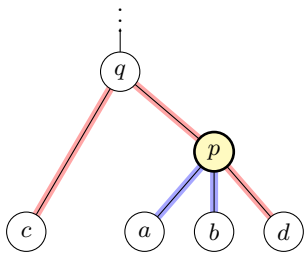
记两条路径的端点为 (a,b) 与 (c,d)，$p = \mathrm{LCA}(a,b)$、$q = \mathrm{LCA}(c,d)$，则

$$\text{两路径相交} \iff p \in \text{路径}(c,d) \ \text{ 或 } \ q \in \text{路径}(a,b).$$

证明：路径 a-b 是"a 上升到 p 再下降到 b"的 Λ 形，p 是其上深度最小的点，且路径上每个点都是 p 的后代。设 x 是两条路径的公共点，那么 p 与 q 都是 x 的祖先（含自身），故 p, q 必有祖先后代关系。不妨设 $\mathrm{dep}(p) \ge \mathrm{dep}(q)$，则 q 是 p 的祖先，而 p 又是 x 的祖先，所以 p 落在 x 上升到 q 的那一段上——这一整段都属于路径 c-d，故 p 在 c-d 上。反向显然。□

推论：只需检验 p, q 中**深度较大**的那个是否落在另一条路径上；实现时懒得比深度就两个方向各判一遍。

判定"x 是否在路径 u-v 上"：$\mathrm{LCA}(u,x)=x$ 且 $\mathrm{LCA}(x,v)=\mathrm{LCA}(u,v)$（或把 u,v 对调）；等价地 $\mathrm{dis}(u,x)+\mathrm{dis}(x,v)=\mathrm{dis}(u,v)$。模板与例题见本章开头「倍增法求 LCA」的 P3398 仓鼠找 sugar。



图中 $\mathrm{dep}(p)>\mathrm{dep}(q)$，两条路径的交集恰是 $\{p\}$：蓝路径 a-b 以 p 为顶，红路径 c-d 从 q 下降时正好经过 p。若把 p 从红路径上挪开（例如 d 改挂到 c 那一侧），两条路径就不相交了。

11 字符串算法

11.1 Manacher 算法

Manacher 算法在 $O(n)$ 时间内求出字符串每个位置为中心的最长回文半径。核心思想是利用已经求出的回文区间 $[L,R]$，把当前位置 cur 关于中心对称到 R + L - cur，借助对称位置已经计算好的半径作为初值，避免从 1 开始重新匹配。

下面分别给出奇回文和偶回文两个版本。

```
string s;
cin >> s;
const int n = s.size();
vector<int> manacher(n);
// 奇回文半径,manacher[cur] 表示以 cur
↪  为中心的最大回文半径
for (int cur = 0, L = 0, R = 0, k = 0; cur < n;
↪  cur++)
{
  if (cur <= R)
    k = min(manacher[R + L - cur], R - cur + 1);
  else
    k = 1;
  while (cur + k < n && cur - k >= 0 && s[cur +
  ↪  k] == s[cur - k])
    k++;
  if (cur + k - 1 > R) // 一定保证右端点最右
    R = cur + k - 1, L = cur - k + 1;
  manacher[cur] = k;
}
fill(all(manacher), 0);
// 偶回文半径，manacher[cur] 表示以 {cur - 1,
↪  cur} 为中心的最大回文半径
for (int cur = 0, L = 0, R = 0, k = 0; cur < n;
↪  cur++)
{
  if (cur <= R)
    k = min(manacher[R + L - cur + 1], R - cur
    ↪  + 1);
  else
    k = 0; // 注意这里要写成
    ↪  0,因为第一个位置初始不合法
  while (cur + k < n && cur - 1 - k >= 0 &&
  ↪  s[cur + k] == s[cur - 1 - k])
    k++;
  if (cur + k - 1 > R) // 一定保证右端点最右
    R = cur + k - 1, L = cur - 1 - k + 1;
  manacher[cur] = k;
}
```

11.2 Z 函数（扩展 KMP）

Z 函数 $z[i]$ 定义为字符串 s 与其后缀 $s[i..n-1]$ 的最长公共前缀长度，其中约定 $z[0]=n$。算法在 $O(n)$ 时间内维护一个最右匹配区间 $[l,r]$（即所有已计算的 i 中使 $i+z[i]-1$ 最大的那个 i 给出的区间）：当 cur 落在 $[l,r]$ 内时，可以用对称位置 cur $-\,l$ 处已经算出的 z 值作为初值，否则从 0 暴力匹配；任何一次扩展都不会让 r 后退，故总匹配次数线性。

```
// Z 函数：求字符串每一个后缀和原字符串的 LCP
auto Z = [&](string s) -> vector<int>
{
  int n = s.size();
  vector<int> Z(n);

  // 默认第一个位置是全匹配
  Z[0] = n;
  for (int cur = 1, l = 0, r = 0, k = 0; cur <
  ↪  n; cur++)
  {
    if (cur <= r)
      k = min(Z[cur - l], r - cur + 1); //
      ↪  复用维护的匹配区间信息
    else
      k = 0;
    // 暴力扩展
    while (cur + k < n && s[cur + k] == s[k])
      k++;

    // 当且仅当新匹配区间右端点更远的时候更新匹配」
    ↪  区间
    if (cur + k - 1 > r)
      l = cur, r = cur + k - 1;
    // 记录 Z 函数的值
    Z[cur] = k;
  }
  return Z;
};
```

函数版（与本章 kmp 同风格，直接复制即可用；此版把 $z[0]$ 留成 0 而不是 n，用到 $z[0]$ 时记得自己特判）：

```
vector<int> z_function(const string &s) {
  int n = SZ(s);
  vector<int> z(n, 0);
  for (int i = 1, l = 0, r = 0; i < n; ++i) {
    // 落在最右匹配区间内：用对称位置的答案做初值
    if (i <= r) z[i] = min(z[i - l], r - i + 1);
    while (i + z[i] < n && s[z[i]] == s[z[i] +
    ↪  i]) {
      z[i]++;
    }
    // 只有右端点更远才更新区间
    if (i + z[i] > r) {
      l = i, r = i + z[i] - 1;
    }
  }
  return z;
}
```

两个版本完全等价：min(z[i - l], r - i + 1) 里，z[i - l] 是对称位置已算好的答案，r - i + 1 是「不能超出已知区间」的截断，取小者当初值后再暴力外扩。

失配字符的用法：$z[i]$ 匹配到头之后，下标 $z[i]$（前缀侧）与 $i+z[i]$（后缀侧）这两个位置的字符必然**不同**（除非 $i+z[i]=n$ 已到串尾）。于是每个 i 免费给出一对有序关系 $(s[z[i]],\,s[i+z[i]])$，把「后缀 i 与原串的大小/字典序比较」压成一次单字符比较。牛客多

校 2026 第 6 场 Full Alphabet 就是用它对每个后缀取出这一对字符，攒成 26 个字母之间的前驱依赖，再交给状压 DP 数拓扑序。

```
auto z = z_function(s);
vector<int> pre(26);                    // pre[c]:
↪ 字母 c 的前驱依赖集合
for (int i = 0; i < n; ++i) {
  int u = z[i], v = i + z[i];
  if (v >= n) continue;                 //
  ↪ 后缀是原串前缀，无失配位
  u = s[u] - 'a', v = s[v] - 'a';
  if (u == v) continue;
  pre[v] |= 1 << u;                     //
  ↪ 这一对的先后由题意决定
}
```

11.3 KMP（前缀函数 pi）

KMP 的核心是 **前缀函数**：对串 A，$\pi[i]$ 表示 $A[0..i]$ 这一段中最长 真前缀 = 真后缀的长度（约定 $\pi[0]=0$）。求出 π 后做模式匹配的常用技巧是把模式串 s_2、文本串 s_1 拼成 s2 + '#' + s1 一起跑前缀函数，凡是 $\pi[i]=|s_2|$ 的位置就对应一次匹配。

```
/*
 * 采用 0-based 设计
 * 模式串(s2)+'#'+文本串(s1)
 ↪ 需要这样子传入,如果想要执行 kmp 的话
 * kmp 模板题 AC
 ↪ https://www.luogu.com.cn/record/269142391
 */
vector<ll> kmp(const string &A) {
  // 返回总前缀数组 pi
  // 其定义是 [0, i] 真前后缀相同的最长长度
  // 特别的, pi[0] = 0
  vector<ll> pi(SZ(A));
  for (int i = 1; i < SZ(A); ++i) {
    ll len = pi[i - 1];
    while (len > 0 && A[len] != A[i]) {
      len = pi[len - 1];
    }
    if (A[len] == A[i]) {
      pi[i] = len + 1;
    }
  }
  return pi;
}
```

典型调用方式：

```
void Solve() {
  string s1, s2;
  cin >> s1 >> s2;
  vector<ll> pi = kmp(s2 + "#" + s1);
  // 输出所有匹配的起始位置
  for (int i = SZ(s2); i < SZ(pi); ++i) {
    if (pi[i] == SZ(s2)) {
      ll ans = i - SZ(s2) - SZ(s2);
      ++ans;
      cout << ans << "\n";
    }
  }
  // 输出 s2 第 i 位的最长 border 长度
  for (int i = 0; i < SZ(s2); ++i) {
    cout << pi[i] << " ";
  }
  cout << "\n";
}
```

11.4 字符串哈希（mod $2^{61}-1$ 版本）

用 Mersenne 素数 $2^{61}-1$ 作模，乘法走 __uint128_t 中转，做一次「低 61 位 + 高位」折叠后至多减一次 M 就归约到 $[0,M)$；base 运行时随机，防离线构造碰撞。为了赛场手敲快，**不用自动取模类**，哈希值直接是 ull，只有一个 mul 函数。

接口：get(l, r) 取 $s[l..r]$（0-based 闭区间，不做越界检查）；push(c) / pop() 在末尾增删字符；整串哈希 = h.back()。约定越前面越高位（和写数字一样），所以两段拼接 $\text{hash}(XY)=\text{hash}(X)\cdot B^{|Y|}+\text{hash}(Y)$。子串计数这类需求在外面写一个循环即可（见代码末尾注释）。

```
// AC https://vjudge.net/solution/65151535
↪ string matching
// AC https://www.luogu.com.cn/record/245201000
↪ push / pop
const ull M = (1ULL << 61) - 1;
ull mul(ull a, ull b) {
  __uint128_t t = (__uint128_t) a * b;
  ull s = (t & M) + (t >> 61); // 一次折叠,
  ↪ 至多减一次 M
  return s >= M ? s - M : s;
}
// base 运行时随机, 防 anti-hash
const ull B = mt19937_64(chrono::steady_clock::」
↪ now().time_since_epoch().count())() % (M -
↪ 257) + 257;
// 越前面越高位; h[i] = s[0..i) 的哈希, p[i] =
↪ B^i
struct HS {
  vector<ull> h{0}, p{1};
  HS(const string &s) { for (char c : s)
  ↪ push(c); }
  void push(unsigned char c) {
    h.push_back((mul(h.back(), B) + c) % M);
    p.push_back(mul(p.back(), B));
  }
  void pop() { h.pop_back(); p.pop_back(); }
  ull get(int l, int r) { return (h[r + 1] + M
  ↪ - mul(h[l], p[r - l + 1])) % M; } //
  ↪ s[l..r], 0-based
};
// 子串计数: t 在 s 中出现几次
// HS hs(s), ht(t); int m = SZ(t), cnt = 0;
// for (int i = 0; i + m <= SZ(s); i++) cnt +=
↪ hs.get(i, i + m - 1) == ht.h.back();
```

11.5 子序列自动机

子序列自动机预处理 next[i][c]：从位置 i 起字符 c 下一次出现的下标（含 i）。建出来之后单串子序列匹

配可以做到 $O(|t|)$ 一次回答 t 是不是 s 的子序列。下面这一份适配小写字母 / 大写字母 / 数字，换字符集时只需调整 base 与第二维大小。

```
struct seqAuto {
  // 子序列自动机 subsequence automaton
  // AC
  ↪ https://www.luogu.com.cn/record/218545124
  vector<vector<int> > next;
  // 从 i 开始，字符下一次出现的位置（含 i）
  // 换字符集只需改 base（'a' / 'A' / '0'）
  ↪ 和第二维大小
  char base = 'a';

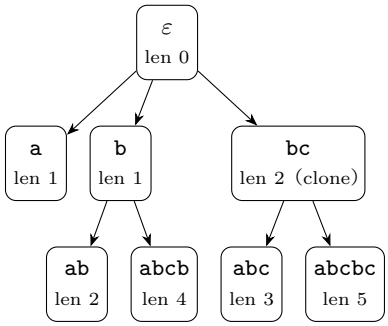
  seqAuto(const string &s) {
    next.assign(SZ(s) + 5, vector<int>(28, -1));
    for (int i = SZ(s) - 1; i >= 0; --i) {
      for (int j = 0; j < 26; ++j) {
        next[i][j] = next[i + 1][j];
      }
      next[i][s[i] - base] = i;
    }
  }

  inline bool isFind(const string &s) {
    ll idx = 0;
    for (int i = 0; i < SZ(s); ++i) {
      if (next[idx][s[i] - base] != -1) {
        idx = next[idx][s[i] - base] + 1;
      } else {
        return false;
      }
    }
    return true;
  }
};
```

11.6 后缀自动机（SAM）

后缀自动机是接受 s 全部子串的最小 DFA：从初始状态 0 出发沿转移边能走出的字符串，恰好是 s 的子串（走不动 = 不是子串）。状态数 $\le 2n-1$、转移数 $\le 3n-4$，在线增量构造，每次 sam_extend 均摊 $O(\Sigma)$（这里 $\Sigma=26$，转移用定长数组存）。

核心是把「结束位置集合 endpos 相同」的子串归成一个状态。同一状态里的串一定互为后缀且长度连续，占据区间 $[\texttt{len[link]}+1,\ \texttt{len}]$。后缀链接 link 指向「把首字母去掉后掉进另一个 endpos 等价类」的那个状态；所有 link 反过来看构成一棵以状态 0 为根的树（endpos 树 / parent 树），endpos(v) 就是 v 子树里所有「前缀状态」的集合。



上图是 $s=$ abcbc 的 link 树（结点标的是该状态的最长串）。读法：结点 bc 表示状态里装着 {c, bc}（长度区间 $[0+1,\ 2]$），它们的 endpos 都是 $\{3,5\}$；而 abc 的 endpos 只有 $\{3\}$，所以挂在 bc 下面。这个 bc 正是构造 abcbc 最后一个 c 时分裂出来的 clone。

sam_extend 里唯一麻烦的就是这个 clone：沿 link 链往上找到第一个已有 c 转移的 p，记 to = st[p].next[c]。若 st[p].len + 1 == st[to].len，说明 $\text{maxstr}(p)+c$ 就是 to 的最长串，to 整个都是新前缀的后缀，直接 link[cur] = to；否则 to 里只有长度 $\le$ len[p] $+1$ 的那一半多出了新的结束位置，必须复制出 clone（继承 to 的全部转移与 link，len 取 len[p] + 1），再把 link 链上仍指向 to 的那些 c 转移改挂到 clone。

常见套路：不同子串个数 $=\sum_{v\neq 0}(\texttt{len[v]}-\texttt{len[link[v]]})$；某子串的出现次数 = 它所在状态在 link 树上的子树里「前缀状态」个数（把状态按 len 桶排后倒序把 cnt 累加给父亲即可）；两串最长公共子串 = 拿另一串在自动机上走、走不动就沿 link 回退并把当前长度压到 len[link]。

两个坑：一是 st 要开 MAXLEN * 2 + 5（状态数上界 $2n-1$，MAXLEN 取单串最长长度）；二是多测每组都要重新 sam_init()，且 sam_extend 结尾的 last = cur 千万别漏——漏了整台自动机就是错的，而且小数据照样跑得出「看着对」的结果。

```
template<int MAXLEN>
struct SufAutomaton {
  struct State {
    int len; // 该状态包含的最长子串的长度
    int link; // 后缀链接，
    ↪ 指向包含当前状态最短子串的最长真后缀的状态
    // （把最短子串的第一个字母去掉，
    ↪ 也就是所谓的最长真后缀）
    int next[26]; // 字符转移边，
    ↪ 假设字符集为小写字母 a-z
  };

  State st[MAXLEN * 2 + 5]; // 状态数上界 2n-1，
  ↪ 多开几个保险
  int sz; // 当前自动机中的状态总数，合法编号 0 ..
  ↪ sz-1
  int last; //
  ↪ 指向当前整个字符串完整前缀所在的状态
  static constexpr char BASE = 'a';

  void sam_init() {
    st[0].len = 0;
    st[0].link = -1;
    for (int i = 0; i < 26; ++i) {
      st[0].next[i] = -1;
    }
    sz = 1;
    last = 0;
  }

  void sam_extend(char c) {
    int cur = sz++;
    st[cur].len = st[last].len + 1;
    st[cur].link = -1;
    for (int i = 0; i < 26; ++i) {
```

```
      st[cur].next[i] = -1;
    }
    int p = last;
    while (p != -1 && st[p].next[c - BASE] ==
    ↪  -1) {
      st[p].next[c - BASE] = cur;
      p = st[p].link;
    }
    if (p == -1) {
      st[cur].link = 0;
    } else {
      int to = st[p].next[c - BASE]; // 注意是
      ↪  st[p] 不是 st[cur]
      // to 可以作为我们的 link 啊
      // 但是我们需要其为最小串的这个去掉首字母以」
      ↪  后的串啊
      if (st[p].len + 1 == st[to].len) {
        // (p 是字符串) p + 'c' 就是 to
        ↪  中的最长子串，因此不需要对这个 to
        ↪  进行这个分裂
        // 根据 end pos 性质，to 中更短的，
        ↪  也是这个 cur 的这个后缀啊
        st[cur].link = to;
      } else {
        int clone = sz++;
        st[clone].link = st[to].link;
        st[clone].len = st[p].len + 1;
        for (int i = 0; i < 26; ++i) {
          st[clone].next[i] = st[to].next[i];
        }
        while (p != -1 && st[p].next[c - BASE]
        ↪  == to) {
          // 原先的，转移到 to 已经不合适了
          // maxstr(p) + c 的这个 endpos 包含 cur
          // 但是原来的 to 是不包含 cur 的，
          ↪  因此需要更新
          // 更新为这个这个 clone，因为 clone
          ↪  endpos 包含这个 cur
          st[p].next[c - BASE] = clone;
          p = st[p].link;
        }
        st[cur].link = clone;
        // 使用连续性定理
        ↪  (后缀自动机的性质里面提到的)，我们知道
        ↪  to 中包含 len(maxstr(p))+2
        ↪  的这个字符串啊
        // 因此也接到这个 clone 上面啊
        st[to].link = clone;
      }
    }
    last = cur; // 千万别漏：不推进 last，
    ↪  整台自动机就建错了
  }
};

// 全局开，别放栈上：MAXLEN=1e6 时 st 有 2e6+ 个
↪  State，约 200 MB 级别
// SufAutomaton<1000006> sam;
// sam.sam_init(); for (char c : s)
↪  sam.sam_extend(c);
```

11.7 Trie 树

支持小写字母、大写字母和数字共 62 种字符。cnt 表示经过节点的字符串数量，因此 query(s) 返回以 s 为前缀的字符串数量（重复插入会重复计数）。字符映射使用直接寻址数组，不需要 map；Reset() 只清空 Trie，保留固定字符映射和已经申请的容量，适合多测复用。

```
class Trie {
  static constexpr int SIGMA = 62;
  struct Node {
    array<int, SIGMA> ch{};
    int cnt = 0;
  };

  vector<Node> tr;
  array<int, 256> id;

public:
  explicit Trie(int max_nodes = 1) {
    id.fill(-1);
    int p = 0;
    for (int c = 'a'; c <= 'z'; ++c) id[c] =
    ↪  p++;
    for (int c = 'A'; c <= 'Z'; ++c) id[c] =
    ↪  p++;
    for (int c = '0'; c <= '9'; ++c) id[c] =
    ↪  p++;
    tr.reserve(max_nodes);
    Reset();
  }

  void Reset() {
    tr.clear();
    tr.emplace_back(); // 根节点编号为 0
  }

  void insert(const string &s) {
    int u = 0;
    ++tr[u].cnt; // 让 query("")
    ↪  也符合前缀计数语义
    for (unsigned char c : s) {
      int v = id[c];
      assert(v != -1); // 字符集仅为 [a-zA-Z0-9]
      if (!tr[u].ch[v]) {
        tr[u].ch[v] = (int) tr.size();
        tr.emplace_back();
      }
      u = tr[u].ch[v];
      ++tr[u].cnt;
    }
  }

  int query(const string &s) const {
    int u = 0;
    for (unsigned char c : s) {
      int v = id[c];
      assert(v != -1);
      if (!tr[u].ch[v]) return 0;
      u = tr[u].ch[v];
    }
    return tr[u].cnt;
  }
};
```

```
// 已知字符串总长度上界时传入，可避免 vector 扩容：
// Trie trie(MAXN); 每组数据开始前调用
↪ trie.Reset();
```

11.8 01 Trie（XOR Trie）

处理一族 unsigned long long 上的最大 / 最小异或问题。max_xor(x) / min_xor(x) 返回与 x 异或的极值；argmax_xor / argmin_xor 返回取得这个极值的原数本身。erase 走「沿路径减 cnt，遇到 cnt 归零的子树则懒摘除指针」的写法，避免正经回收带来的额外管理。

```
// AC https://judge.yosupo.jp/submission/308823
↪ min_xor / erase / insert
// AC https://codeforces.com/contest/706/submis
↪ sion/334623853 argmax / max
// AC https://www.codechef.com/viewsolution/118
↪ 3578991 综合
struct XORTrie {
  int B; // 最高位（含）
  struct Node {
    int nxt[2];
    int cnt; // 经过该节点的数的个数（叶子 =
    ↪ 出现次数）
    Node() { nxt[0] = nxt[1] = -1; cnt = 0; }
  };
  vector<Node> tr;
  explicit XORTrie(int max_bit = 63) :
  ↪ B(max_bit) {
    tr.reserve(1 << 20);
    tr.emplace_back(); // root
  }
  void clear() { tr.clear(); tr.emplace_back();
  ↪ }

  inline void insert(unsigned long long x) {
    int u = 0;
    for (int b = B; b >= 0; --b) {
      int bit = int((x >> b) & 1ull);
      if (tr[u].nxt[bit] == -1) {
        tr[u].nxt[bit] = (int) tr.size();
        tr.emplace_back();
      }
      u = tr[u].nxt[bit];
      ++tr[u].cnt;
    }
  }
  inline int count(unsigned long long x) const {
    int u = 0;
    for (int b = B; b >= 0; --b) {
      int bit = int((x >> b) & 1ull);
      int v = tr[u].nxt[bit];
      if (v == -1) return 0;
      u = v;
    }
    return tr[u].cnt;
  }
  inline bool erase(unsigned long long x) {
    if (tr.size() == 1) return false;
    static int pathBit[70];
    int u = 0, depth = 0;
    for (int b = B; b >= 0; --b) {
      int bit = int((x >> b) & 1ull);
      int v = tr[u].nxt[bit];
      if (v == -1 || tr[v].cnt == 0) return
      ↪ false;
      pathBit[depth++] = bit;
      u = v;
    }
    if (tr[u].cnt == 0) return false;
    u = 0;
    for (int i = 0; i < depth; ++i) {
      int bit = pathBit[i];
      int v = tr[u].nxt[bit];
      --tr[v].cnt;
      if (tr[v].cnt == 0) tr[u].nxt[bit] = -1;
      ↪ // 懒回收
      u = v;
    }
    return true;
  }
  inline unsigned long long max_xor(unsigned
  ↪ long long x) const {
    if (tr.size() == 1) return 0ull;
    int u = 0;
    unsigned long long ans = 0;
    for (int b = B; b >= 0; --b) {
      int bit    = int((x >> b) & 1ull);
      int prefer = tr[u].nxt[bit ^ 1];
      int same   = tr[u].nxt[bit];
      if (prefer != -1 && tr[prefer].cnt > 0) {
        u = prefer;
        ans |= (1ull << b);
      } else {
        u = same; // 至少有一条有效路
      }
    }
    return ans;
  }
  inline unsigned long long min_xor(unsigned
  ↪ long long x) const {
    if (tr.size() == 1) return 0ull;
    int u = 0;
    unsigned long long ans = 0;
    for (int b = B; b >= 0; --b) {
      int bit   = int((x >> b) & 1ull);
      int same  = tr[u].nxt[bit];
      int other = tr[u].nxt[bit ^ 1];
      if (same != -1 && tr[same].cnt > 0) {
        u = same;
      } else {
        u = other;
        ans |= (1ull << b);
      }
    }
    return ans;
  }
  inline unsigned long long argmax_xor(unsigned
  ↪ long long y) const {
    if (tr.size() == 1) return 0ull;
    int u = 0;
    unsigned long long chosen = 0;
    for (int b = B; b >= 0; --b) {
      int bit    = int((y >> b) & 1ull);
```

```
      int prefer = tr[u].nxt[bit ^ 1];
      int same   = tr[u].nxt[bit];
      if (prefer != -1 && tr[prefer].cnt > 0) {
        u = prefer;
        chosen |= (unsigned long long) (bit ^
        ↪  1) << b;
      } else {
        u = same;
        chosen |= (unsigned long long) bit << b;
      }
    }
    return chosen;
  }
  inline unsigned long long argmin_xor(unsigned
  ↪  long long x) const {
    if (tr.size() == 1) return 0ull;
    int u = 0;
    unsigned long long chosen = 0;
    for (int b = B; b >= 0; --b) {
      int bit   = int((x >> b) & 1ull);
      int same  = tr[u].nxt[bit];
      int other = tr[u].nxt[bit ^ 1];
      if (same != -1 && tr[same].cnt > 0) {
        u = same; // 让该位 XOR=0
        if (bit) chosen |= (1ull << b);
      } else {
        u = other; // 只能走相反位，该位 XOR=1
        if (bit ^ 1) chosen |= (1ull << b);
      }
    }
    return chosen;
  }
} tr;
// 处理无符号数
```

11.9 字符串分词与输出技术（大模拟读入）

getline + stringstream 是处理一行内含混合分隔符的标准搭配：先 getline 整行，再丢进 stringstream 按空白 / 自定义字符切分。下面两段一段是以 ':' / '+' 等多种字符做模板化分词的「跨天时间均值」例题，另一段是只用 '+' 切的最简加法器。

```
// 跨天时间区间均值：行内固定用 ':' / 空格分隔，
↪  末尾可选 (+day)
inline void solve() {
  init();
  FOR(i, 1, 2) {
    string line; getline(cin, line);
    stringstream sst(line); char fen;
    sst >> hh[1] >> fen >> mm[1] >> fen >> ss[1]
        >> hh[2] >> fen >> mm[2] >> fen >>
        ↪  ss[2];
    string dayst; ll day = 0;
    if (sst >> dayst) {
      stringstream st(dayst);
      st >> fen >> fen >> day;
    }
    sec[i] = day * 86400 + 3600 * (hh[2] -
    ↪  hh[1])
             + 60 * (mm[2] - mm[1]) + (ss[2] -
             ↪  ss[1]);
  }
  ll fsec = (sec[1] + sec[2]) / 2;
  ll h = fsec / 3600; fsec %= 3600;
  ll m = fsec / 60;   fsec %= 60;
  ll s = fsec;
  cout << setw(2) << setfill('0') << h << ":"
       << setw(2) << setfill('0') << m << ":"
       << setw(2) << setfill('0') << s << "\n";
}
```

```
// 用 '+' 作分词符的简单加法器
inline void __() {
  stringstream ss(line);
  string num;
  ll ans = 0;
  while (getline(ss, num, '+')) {
    ans += stoll(num);
  }
  cout << ans << "\n";
}
```

12 动态规划（dp）

12.1 二进制子集枚举技巧

如果要枚举 n 个元素的所有子集，最直接的写法是把每个集合看成一个二进制数，范围就是 $[0, 2^n - 1]$。

```
for (int s = 0; s < (1 << n); s++) {
  // s 的二进制表示对应一个子集
}
```

12.1.1 枚举某个集合的所有子集

如果已经有一个集合 mask，想要只枚举它的所有子集，常用下面这个写法。

```
for (int sub = mask; sub; sub = (sub - 1) &
↪  mask) {
  // sub 是 mask 的一个非空子集
}
```

它的原理是，sub - 1 会把 sub 最低位的一个 1 变成 0，并把它右边的若干位全部变成 1。随后再与 mask 做一次按位与，就会把不属于 mask 的位置全部清掉，于是得到一个更小但仍然合法的子集。

因此，这个过程会按照从大到小的顺序，不重不漏地枚举出 mask 的所有非空子集。

如果还需要处理空集，可以写成下面这样。注意 sub == 0 时必须及时跳出，不然会死循环。

```
for (int sub = mask;; sub = (sub - 1) & mask) {
  // sub 是 mask 的一个子集，包含空集
  if (sub == 0) {
    break;
  }
}
```

这个技巧在状压 DP 中非常常见。若外层再枚举所有 mask，内层枚举它的全部子集，总复杂度是 $O(3^n)$。

12.1.2 枚举固定大小的子集

如果只关心恰好选了 use 个元素的状态，就没必要把所有状态都扫一遍再用 popcount 过滤。可以直接枚举二进制中恰好有 use 个 1 的状态。

```
for (int use = 0; use <= K; ++use) {
  long long sta = (1LL << use) - 1;
  while (sta < (1LL << K)) {
    // sta 的二进制中恰好有 use 个 1

    if (sta == 0) break;  // use=0
    ↪  时只对应空集这一个状态，避免下面除零
    long long c = sta & -sta;
    long long r = sta + c;
    sta = (((r ^ sta) >> 2) / c) | r;
  }
}
```

这里先令 sta = (1LL << use) - 1，也就是二进制下最小的、恰好含有 use 个 1 的状态。后面的转移则负责直接跳到“下一个同样有 use 个 1 的状态”。

c = sta & -sta 会取出最低位的 1，r = sta + c 可以理解为把最靠右、还能向左移动的一位 1 往前推一格。这样一来，右侧结构会被打乱，再用最后那一行把剩余的若干个 1 重新挪到最右边，于是就得到了下一个更大的合法状态。

这个写法常被称为 Gosper's hack。它会按照从小到大的顺序，直接跳到下一个二进制中有相同个数 1 的状态。若 use = 0，则只对应空集这一个状态。

12.1.3 枚举补集元素做前向转移（附「交判子集」）

状压 DP 常写成**前向（push）**形式：站在状态 sta 上，枚举一个**还没被选进来**的元素 to，把 dp[sta] 推给 dp[sta | (1 << to)]。这时不要写成「扫 $0 \dots n-1$ 再用 if 过滤掉已在 sta 里的位」，而是直接把补集掏出来逐位剥：

```
int rem = full & (~sta);   // 还没选的元素集合
while (rem) {
  // 取出最低位的 1 所在下标
  int to = __builtin_ctz(rem);
  // 剥掉最低位的 1，进入下一轮
  rem &= rem - 1;
  // ... 用 to 做转移 ...
}
```

两点好处：循环次数等于**补集大小**而不是 n（sta 越满越省，整层下来常数明显更小）；并且 __builtin_ctz 直接给出**下标**，省掉「先枚举下标再判位」的那次 if。注意 full & (~sta) 的 full 掩码不能少，否则 ~sta 的高位全是 1，ctz 会跑到 n 以外的位上去。__builtin_ctz 只吃 unsigned int（32 位），64 位状态要用 __builtin_ctzll；另外 ctz(0) 是 UB，所以判空必须放在循环条件里。

用「交」判子集：转移往往还带一个前置条件——元素 to 有一个**前驱依赖集合** pre[to]，只有依赖全部就位才能选它，即 pre[to] $\subseteq$ sta。位掩码下的子集判定就是一次按位与：

$$\begin{aligned} A \subseteq B &\iff A \,\&\, B = A \\ &\iff A \,\&\, \sim B = 0 \\ &\iff A \mid B = B \end{aligned}$$

含义	位串	说明
sta	1 0 1 1	已选集合
pre[to]	0 0 1 1	依赖集合
pre[to]&sta	0 0 1 1	= pre[to]，可转移
pre[to]	0 1 1 0	换一个依赖
pre[to]&sta	0 0 1 0	≠ pre[to]，不可转移

合起来就是「有依赖约束的排列计数 / 拓扑序计数」的标准骨架：

```
// dp[sta] = 只用 sta 内元素、
↪  且满足依赖的合法前缀方案数
```

```cpp
vector<uint32_t> dp(1 << n);
dp[0] = 1;
const int full = (1 << n) - 1;
for (int sta = 0; sta < full; ++sta) {
  if (!dp[sta]) continue;          //
  ↪ 剪掉不可达状态
  int rem = full & (~sta);
  while (rem) {
    int to = __builtin_ctz(rem);
    rem &= rem - 1;
    // pre[to] 全部就位才允许放 to
    if ((pre[to] & sta) == pre[to]) {
      dp[sta | (1 << to)] += dp[sta];
    }
  }
}
cout << dp[full] << "\n";
```

复杂度 $O(2^n \cdot n)$。$n = 26$ 时 $2^{26} \cdot 26 \approx 1.7 \times 10^9$ 次极轻的位运算，配合 `if (!dp[sta]) continue` 的剪枝和 `uint32_t`（自然溢出当作 $\bmod 2^{32}$，省掉取模）仍然能过——牛客多校 2026 第 6 场 Full Alphabet 即此模型，依赖集合由 Z 函数导出。

12.2 SOS DP（Sum / Max over Subsets, 高维前缀和）

给定一个长度为 2^N 的数组 $f[\cdot]$（下标看成 N 位二进制掩码），希望对每个 $mask$ 求出

$$F[mask] = \bigoplus_{sub \subseteq mask} f[sub]$$

其中 $\bigoplus$ 可以是 $+$ / max / min / 异或等任何幺半群运算。直接枚举子集是 $O(3^N)$；SOS DP 通过**逐位扩展**把它压到 $O(N \cdot 2^N)$。

核心思想是把每一位看成一个维度，依次做「沿第 i 维的一维前缀」：当处理到第 i 位时，若 $mask$ 第 i 位是 1，就把 $mask$ 与 $mask \oplus (1 \ll i)$（去掉第 i 位）合并。等价于沿 N 个超立方体维度各做一遍一维前缀和。

对偶版：超集而非子集，即 $F[mask] = \bigoplus_{mask \subseteq sup} f[sup]$。把转移方向反过来，把「有第 i 位的从无第 i 位的吸收」改成「无第 i 位的从有第 i 位的吸收」即可。

12.2.1 典型应用：按位无交最大值

问题：给定 n 个非负整数 $a_1, \ldots, a_n$，对每个 i 找出最大的 a_j 使得 $a_i \,\&\, a_j = 0$（按位无交集，二进制位上互不相交）。

做法：

- 用 $dp[mask] = \max\{a_j : a_j \subseteq mask\}$（位掩码意义下的子集），初值 $dp[a_j] = a_j$。
- 跑一遍**子集 max** 的 SOS DP，得到「掩码 $mask$ 下所有子集中最大的 a_j」。
- 对询问 i，答案 $= dp[\overline{a_i}]$，因为 a_j 与 a_i 按位无交 $\iff a_j \subseteq \overline{a_i}$。

复杂度 $O((V + n) \cdot \log V)$，其中 $V = \max a_i$。

```cpp
void solve() {
  int n;
  cin >> n;
  vector<int> a(n);
  for (int &x : a) cin >> x;
  const int V = ranges::max(a);
  const int N = 32 - __builtin_clz(V); //
  ↪ 用到的位数
  vector<int> dp(1 << N);
  for (int x : a) dp[x] = x;           //
  ↪ 标记原始集合

  // SOS DP：dp[mask] = max{a_j : a_j 的位是 mask
  ↪ 的子集}
  for (int i = 0; i < N; i++) {
    for (int mask = (1 << N) - 1; mask >= 0;
    ↪ mask--) {
      if ((mask >> i) & 1) {
        dp[mask] = max(dp[mask],
                       dp[mask ^ (1 << i)]);
      }
    }
  }

  // 查询：与 a[i] 按位无交 等价于 是 ~a[i] 的子集
  for (int i = 0; i < n; i++) {
    int q = (~a[i]) & ((1 << N) - 1);
    cout << (dp[q] == 0 ? -1 : dp[q]) << "
    ↪ \n"[i == n - 1];
  }
}
```

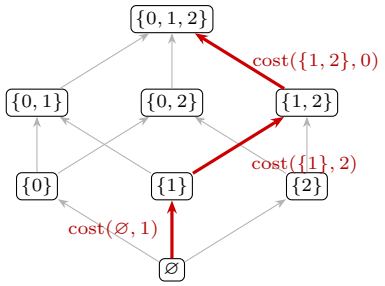
要点：

- 第二重循环**方向无关**（顺序、逆序都对），因为转移只涉及「同一 i 位下，$mask$ 与去掉该位的 $mask \oplus (1 \ll i)$」，去掉位的 $mask$ 在本轮不会被本轮的更新二次覆盖（它的第 i 位是 0，进不了 `if` 分支）。
- 用「0 表示未出现」的哨兵时，要确保 $a_j \neq 0$；否则要换 `-INF` 或者额外的 `exists` 标志位。
- 改成求**方案数**就把 `max` 换成 `+`，求**异或**就换成 `^`，模板完全一致。

12.3 子集格最短路（超集和 + 状压 DP 求最优排列）

通用模型：要给 m 个元素定一个处理顺序（一个排列），把元素 j 排在「已处理集合 S」之后的代价是 $\mathrm{cost}(S, j)$——注意它**只跟 S 和 j 有关**，跟 S 内部的先后顺序无关。求总代价最小的排列。

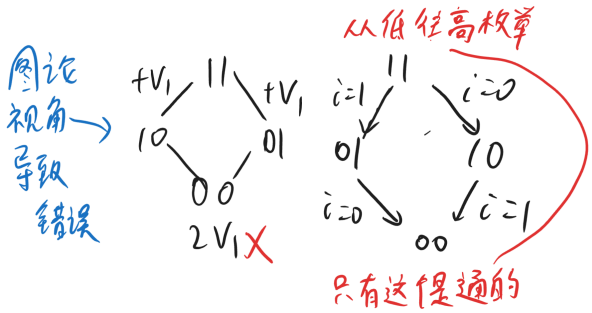
这类问题等价于**子集格上的最短路**：把 2^m 个子集当结点，从 S 向 $S \cup \{j\}$ 连一条权为 $\mathrm{cost}(S, j)$ 的边，答案就是 $\varnothing \to \{0, 1, \ldots, m-1\}$ 的最短路。因为边只从小集合指向大集合，图天然是 DAG，按 $mask$ 升序扫一遍就是拓扑序，不用 Dijkstra。



图中红色一条 $\varnothing \to \{1\} \to \{1,2\} \to \{0,1,2\}$ 对应排列 $(1,2,0)$，路径长度就是这个排列的总代价；$m!$ 种排列被压进 2^m 个结点里，这正是状压 DP 省下阶乘的地方。

代价怎么来：常见形态是「每条数据带一个能力掩码 $mask$ 与一组权 $w[\cdot]$，元素 j 排在 S 之后要付出所有 $mask \supseteq S$ 的那些数据的 $w[j]$」。先把每条数据按 $mask$ 装桶累加，再对每一维做**超集和**（上一节 SOS DP 的对偶版），桶就原地变成了 $\mathrm{cost}(S,j)$。

SOS DP 的精髓：外层先枚举维度，内层才枚举状态。高维前缀和 / 超集和的本质是「**对每一维各做一次**一维前缀和」，m 维就是 m 轮，每轮把这一维「压平」。两层循环**写反就是错的**：外层若是状态、内层是维度，同一个状态在一次内层里连着吃多个方向的超集，而那些超集已被部分更新，导致**重复计数**。$m=2$ 的反例（记初值 $f_{00}, f_{01}, f_{10}, f_{11}$，$S$ 倒序）：



左半（错）：按「图论最短路」的直觉去想，11 沿两条边各把 V_1 送给 10 与 01，两路再汇到 00，于是 00 吃进 $2V_1$——f_{11} 被经 10 与 01 各加了一次。**右半（对）**：维度在外时每一轮只沿一个维度走，$11 \to 00$ 的两条边必然分属 $i=1$ 与 $i=0$ 两个不同轮次，所以 11 只会被累加一次，「只有这条是通的」。

反过来，维度在外时**内层状态的枚举顺序无所谓**（正序倒序都对）：本轮只改第 i 位为 0 的状态，而被读的 $S \cup \{i\}$ 第 i 位是 1、本轮不会被改，层内没有依赖。口诀：**维度在外，状态在内**。

例题：2026 牛客多校 4 C. Retest Queue——$n \le 2\times10^4$ 个程序、$m \le 20$ 个测试点，测试队列可任意重排；每个程序从队首依次跑，**遇到第一个 rejected 的测试就停**（那个测试本身仍计时），程序 i 跑测试 j 花 $d_{i,j}$，求最小总耗时。令 $mask_i$ = 程序 i 能 accept 的测试集合，把测试 j 排在已排集合 S 之后时，此刻**还没停下来**的程序恰是那些 $mask_i \supseteq S$ 的，它们都要付 $d_{i,j}$，于是

$$\mathrm{cost}(S,j) = \sum_{mask_i \supseteq S} d_{i,j},$$

正是按 $mask_i$ 装桶后的超集和。样例最优顺序 $(2,1,3)$ 的三段耗时 10、$1+5$、$2+2+7$ 合计 27。

复杂度：装桶 $O(nm)$，超集和 $O(2^m m^2)$，最短路 $O(2^m m)$；空间 $2^m m$ 个 `ll`（$m=20$ 时约 160 MB，`vector<vector<ll>>` 还要多 2^m 个 vector 头约 24 MB——卡内存就换一维 `vector<ll>` 按 $S\cdot m+j$ 行主序拍平，顺带少一层间接寻址）。

```cpp
// N 个程序，M 个测试点
// D[i][j]  = 程序 i 跑测试 j 的耗时
// masks[i] = 程序 i 能 accept 的测试集合
void Solve() {
  cin >> N >> M;
  vector<vector<ll> > D(N + 2, vector<ll>(M));
  vector<ll> masks(N + 2);
  for (int i = 1; i <= N; ++i) {
    for (int j = 0; j < M; ++j) {
      cin >> D[i][j];
    }
    ll mask = 0;
    for (int j = 0; j < M; ++j) {
      char ch;
      cin >> ch;
      if (ch == 'A') {
        mask |= 1ll << j;
      }
    }
    masks[i] = mask;
  }

  // 装桶：把每个程序的耗时整行累加到
  // 「能力恰好等于 masks[i]」的那一行
  vector<vector<ll> > cost(1ll << M,
                           vector<ll>(M));
  for (int i = 1; i <= N; ++i) {
    for (int j = 0; j < M; ++j) {
      cost[masks[i]][j] += D[i][j];
    }
  }

  // ===== SOS DP（超集和）=====
  // 外层 i 是「维度」，不是状态!! 顺序写反就错
  for (int i = 0; i < M; ++i) {
    // 内层 sta 正序倒序都行：被加的 tosta
    // 第 i 位是 1，本层不会被改，无层内依赖
    for (int sta = (1 << M) - 1;
         sta >= 0; --sta) {
      if (!((sta >> i) & 1)) {
        ll tosta = sta | (1 << i);
        for (int j = 0; j < M; ++j) {
          cost[sta][j] += cost[tosta][j];
        }
      }
    }
  }

  // 子集格最短路：mask 升序天然是拓扑序，
  // 不需要 Dijkstra
  vector<ll> Dis(1 << M, INF);
  Dis[0] = 0;
  for (int sta = 0; sta < (1 << M); ++sta) {
    for (int j = 0; j < M; ++j) {
      if (!((sta >> j) & 1)) {
```

```
        ll tosta = sta | (1 << j);
        Dis[tosta] = min(Dis[tosta],
                Dis[sta] + cost[sta][j]);
      }
    }
  }
  cout << Dis[(1 << M) - 1] << "\n";
}
```

12.4 线段树维护 dp 求最大独立集

把「区间最大独立集」拍成可合并信息：每个区间维护 $dp[i][j]$, $i,j \in \{0,1\}$ 表示该区间左端点是否选、右端点是否选时的最大权和。合并时若左区间右端 $j=1$ 且右区间左端 $x=1$ 则非法（相邻同选）。单点 `Info(x)`：`dp[1][1] = x`, `dp[0][0] = 0`, 对角 $-\infty$。区间查询直接 `seg.query(l, r)` 得到该段最大独立集（四个状态取 max）。

参考提交：ABC 456 提交 75466400。

```
struct Info {
  array<array<ll, 2>, 2> dp;
  Info(ll x = 0) {
    dp[0][1] = dp[1][0] = -INF;
    dp[1][1] = x;
    dp[0][0] = 0;
  }
};

Info operator+(const Info &a, const Info &b) {
  Info fa;
  for (int i = 0; i <= 1; i++) {
    for (int j = 0; j <= 1; j++) {
      if (a.dp[i][j] == -INF) continue;
      for (int x = 0; x <= 1; x++) {
        for (int y = 0; y <= 1; y++) {
          if (b.dp[x][y] == -INF) continue;
          if (j == 1 && x == 1) continue;
          fa.dp[i][y] = max(a.dp[i][j] +
          ↪  b.dp[x][y], fa.dp[i][y]);
        }
      }
    }
  }
  return fa;
}

template <class InfoType = Info>
class LazySegmentTree {
  int n;
  vector<InfoType> tree;

  void pushUp(int id) {
    tree[id] = tree[id << 1] + tree[id << 1 |
    ↪  1];
  }

  void build(int id, int l, int r, const
  ↪  vector<InfoType> &a) {
    if (l == r) { tree[id] = a[l]; return; }
    int mid = (l + r) >> 1;
    build(id << 1, l, mid, a);
    build(id << 1 | 1, mid + 1, r, a);
    pushUp(id);
  }

  InfoType query(int id, int l, int r, int ql,
  ↪  int qr) {
    if (ql <= l && r <= qr) return tree[id];
    int mid = (l + r) >> 1;
    if (qr <= mid) return query(id << 1, l,
    ↪  mid, ql, qr);
    if (ql > mid)  return query(id << 1 | 1,
    ↪  mid + 1, r, ql, qr);
    return query(id << 1, l, mid, ql, qr) +
           query(id << 1 | 1, mid + 1, r, ql,
           ↪  qr);
  }

public:
  LazySegmentTree(int n = 0) { init(n); }
  LazySegmentTree(const vector<InfoType> &a) {
  ↪  build(a); }

  void init(int n_) {
    n = n_;
    tree.assign(4 * n + 5, InfoType());
  }

  void build(const vector<InfoType> &a) {
    init((int)a.size() - 1);
    if (n > 0) build(1, 1, n, a);
  }

  InfoType query(int l, int r) {
    if (l > r) return 0;
    return query(1, 1, n, l, r);
  }
};

void solve() {
  int n, k;
  cin >> n >> k;
  vector<ll> a(n + 1);
  for (int i = 1; i <= n; i++) cin >> a[i];
  vector<Info> b(n + 1);
  for (int i = 1; i <= n; i++) b[i] =
  ↪  Info(a[i]);
  for (int i = 1; i <= n; i++) a[i] = a[i] +
  ↪  a[i - 1];
  LazySegmentTree<Info> seg;
  seg.build(b);
  ll ans = INF;
  for (int len = k - 1; len <= k; len++) {
    for (int l = 1; l + len <= n; l++) {
      int r = l + len;
      ll candidate = a[r] - a[l - 1];
      auto info = seg.query(l + 1, r - 1);
      ll maxi = -INF;
      for (int i = 0; i <= 1; i++)
        for (int j = 0; j <= 1; j++)
          maxi = max(maxi, info.dp[i][j]);
      candidate -= maxi;
      ans = min(ans, candidate);
    }
```

```
  }
  cout << ans << "\n";
}
```

13 计算几何

13.1 点线基础模板

以下是计算几何最底层的几个基础类型：符号判定 sgn、点 / 向量 Point（点和向量同一类型，向量当成「以原点为起点」的点）、叉乘 / 点乘、两点式直线 Line。**后续所有子节的代码都建立在这些定义之上**，每节顶部会以注释形式列出本节用到的依赖（具体实现见此处）。

类型约定：坐标类型 T 只用 ll 或 DB。Wide<T> 把整数乘积（叉积、点积、模长平方）升为 i128，浮点保留原类型；坐标加减本身仍须落在 T 的范围内。Promote 让 Point<ll> 能隐式转成 Point<DB>，反向禁止。+ - cross dot 都只写同类型版本，混合类型表达式（如 Point<ll> - Point<DB>）靠这个隐式转换自动走 DB 版本；标量乘除 p * k、p / k 返回公共类型，所以 Point<ll> / DB 得到 Point<DB>，不会截断。sqrtL 对整数先精确开方再修正，避免 i128 转浮点丢精度。排序使用精确字典序，eps 只用于几何判定与 ==，不能放进排序比较器。拷贝与类型转换保留 id，算术派生点的 id=-1。

change log

```cpp
// C++17; ll / i128 / DB / eps / PI 由开赛模板定义。坐标类型 T 只用 ll 或 DB。
// Wide: 整数乘积升 i128 防溢出; Promote: 只允许 Point<ll> -> Point<DB> 这种无损方向的隐式转换。
template<class T> using Wide = conditional_t<is_floating_point_v<T>, T, i128>;
template<class T, class U> using Promote = enable_if_t<is_same_v<common_type_t<T, U>, T>>;
template<class T> int sgn(T x) { return (x > eps) - (x < -eps); }
template<class T> Wide<T> squared(T x) { return Wide<T>(x) * x; }
DB sqrtL(DB x) { return sqrtl(x); }
DB sqrtL(i128 n) { // 整数先精确开方再修正，避免 i128 转浮点丢精度
  if (n <= 0) return 0;
  i128 s = sqrtl((DB)n);
  while (s * s > n) --s;
  while ((s + 1) * (s + 1) <= n) ++s;
  return s + (DB)(n - s * s) / (sqrtl((DB)n) + s);
}

template<class T>
struct Point {
  T x, y;
  int id = -1; // 拷贝 / 类型转换保留下标，算术派生点为 -1
  explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
  template<class U, class = Promote<T, U>> Point(Point<U> p) : x(p.x), y(p.y), id(p.id) {}
  // 混合类型（ll 与 DB）运算靠隐式转换走 DB 版本；标量乘除返回公共类型，Point<ll> / DB 不会截断
  friend Point operator+(Point a, Point b) { return Point(a.x + b.x, a.y + b.y); }
  friend Point operator-(Point a, Point b) { return Point(a.x - b.x, a.y - b.y); }
  Point operator-() const { return Point(-x, -y); }
  template<class U> friend auto operator*(Point a, U k) { return Point<common_type_t<T, U>>(a.x *
  ↪  k, a.y * k); }
  template<class U> friend auto operator*(U k, Point a) { return a * k; }
  template<class U> friend auto operator/(Point a, U k) { return Point<common_type_t<T, U>>(a.x /
  ↪  k, a.y / k); }
  Point &operator+=(Point p) { return *this = *this + p; }
  Point &operator-=(Point p) { return *this = *this - p; }
  // 排序用精确字典序（eps 会破坏 strict weak ordering），== 才用 eps
  friend bool operator<(Point a, Point b) { return tie(a.x, a.y) < tie(b.x, b.y); }
  friend bool operator==(Point a, Point b) { return !sgn(a.x - b.x) && !sgn(a.y - b.y); }
  friend bool operator!=(Point a, Point b) { return !(a == b); }
  friend istream &operator>>(istream &s, Point &p) { return s >> p.x >> p.y; }
  friend ostream &operator<<(ostream &s, Point p) { return s << "(" << p.x << ", " << p.y << ")"; }
  friend Wide<T> cross(Point a, Point b) { return Wide<T>(a.x) * b.y - Wide<T>(a.y) * b.x; }
  friend Wide<T> dot(Point a, Point b) { return Wide<T>(a.x) * b.x + Wide<T>(a.y) * b.y; }
  friend Wide<T> cross(Point o, Point a, Point b) { return cross(a - o, b - o); } // 以 o 为原点
  friend Wide<T> dot(Point o, Point a, Point b) { return dot(a - o, b - o); }
  Wide<T> norm() const { return dot(*this, *this); } // 模长平方
  DB abs() const { return sqrtL(norm()); }
  Wide<T> normL1() const { return (Wide<T>)(x < 0 ? -x : x) + (y < 0 ? -y : y); }
  T normLinf() const { return max(x < 0 ? -x : x, y < 0 ? -y : y); }
  Point rot90() const { return Point(-y, x); }
```

```
  Point<DB> rot(DB a) const { return Point<DB>(x * cosl(a) - y * sinl(a), x * sinl(a) + y *
  ↪  cosl(a)); }
};

template<class T>
struct Line { // 直线 / 线段共用：p1 -> p2
  Point<T> p1, p2;
  Line(Point<T> a = Point<T>(), Point<T> b = Point<T>()) : p1(a), p2(b) {}
  Line(T k, T c) : p1(0, c), p2(1, k + c) {} // 斜截式 y = kx + c
  template<class U, class = Promote<T, U>> Line(Line<U> l) : p1(l.p1), p2(l.p2) {}
  friend bool operator<(Line a, Line b) { return tie(a.p1, a.p2) < tie(b.p1, b.p2); }
  friend bool operator==(Line a, Line b) { return a.p1 == b.p1 && a.p2 == b.p2; }
  friend istream &operator>>(istream &s, Line &l) { return s >> l.p1 >> l.p2; }
  friend ostream &operator<<(ostream &s, Line l) { return s << "<" << l.p1 << ", " << l.p2 << ">"; }
};
// 一般式 ax + by + c = 0（a、b 不同时为 0）
Line<DB> gen_line_from_general(DB a, DB b, DB c) {
  Point<DB> p = b != 0 ? Point<DB>(0, -c / b) : Point<DB>(-c / a, 0);
  return Line<DB>(p, p + Point<DB>(b, -a));
}
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
```

参考题单：计算几何模板题目总结（vjudge）以及 QOJ 模板赛 2646。

13.2 点在线上的投影点

vjudge 提交。

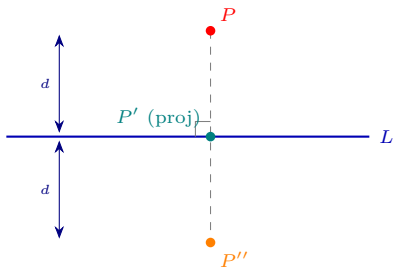
设 `vec = l.p2 - l.p1`，把向量 `p - l.p1` 在 `vec` 方向上的归一化投影长度记为 r，于是投影点等于 `l.p1 + r * vec`。**返回值固定为 `Point<DB>`**（除法走浮点，输入 `T / U` 是 `ll` 还是 `DB` 都向 `DB` 收敛），不再 `assert` 浮点。

```
// 依赖：点线基础；l 非退化。返回值固定为 Point<DB>
template<class T, class U>
Point<DB> project(Point<T> p, Line<U> l) {
  Point<DB> v = l.p2 - l.p1;
  return l.p1 + v * (dot(v, p - l.p1) / v.norm());
}
```

13.3 对称点

vjudge 提交。

设 `proj` 为投影点，则对称点 `= proj + proj - p`。直观上：**以投影点为中点**，原点和对称点处于直线两侧、距离相等。



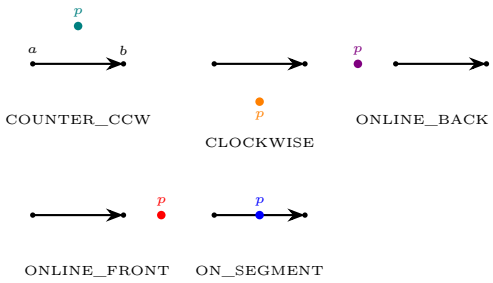
图注：P'（投影）是 P 到 L 的垂足；P''（对称点）满足 P' 是 P, P'' 中点，从而 $P'' = 2P' - P$。

```
// 依赖：project
template<class T, class U>
Point<DB> reflect(Point<T> p, Line<U> l) { return project(p, l) * 2 - p; }
```

13.4 点线位置判断（CCW）

vjudge 提交。

给一条有向直线 $\overrightarrow{a \to b}$，目标点 p 落在何处由 $\mathrm{cross}(\vec{ab}, \vec{ap})$ 与 $\mathrm{dot}(\vec{ab}, \vec{ap})$ 决定，共五种情况：



图注：先看 $\mathrm{cross}(\vec{ab}, \vec{ap})$ 的符号——正左 / 负右；为零再看 dot 与 $|\vec{ab}|, |\vec{ap}|$ 的比较，区分 p 落在线段反方向延长（BACK）、正方向延长（FRONT）还是线段内部（ON）。

```
// 依赖：点线基础
constexpr int ON_SEGMENT = 0, COUNTER_CLOCKWISE = 1, CLOCKWISE = -1, ONLINE_BACK = 2, ONLINE_FRONT
↪  = -2;
template<class T, class U>
int CCW(Point<T> p, Line<U> l) {
  auto a = l.p2 - l.p1; auto b = p - l.p1;
  if (int s = sgn(cross(a, b))) return s;
  if (sgn(dot(a, b)) < 0) return ONLINE_BACK;
  return a.norm() < b.norm() ? ONLINE_FRONT : ON_SEGMENT;
}
```

13.4.1 逆时针弧角 ccw_angle（标量 / 向量两版）

求「从 from 逆时针转到 to」的有向夹角，结果恒落在 $[0, 2\pi)$，而不是 $|\mathtt{to} - \mathtt{from}|$（后者在跨 $0/2\pi$ 处会算错）。两个重载：标量版输入两个极角，向量版输入两个方向向量、用单次 atan2(cross, dot)（叉积当 sin 分量、点积当 cos 分量）一步求出，比两次 atan2 相减更稳。

```
// 依赖：点线基础。从 from 逆时针转到 to 的角，落在 [0, 2pi)，不是 abs(to - from)
DB ccw_angle(DB from, DB to) {
  DB d = fmodl(to - from, 2 * PI);
  return d < 0 ? d + 2 * PI : d;
}
template<class T, class U> // 向量版：单次 atan2(cross, dot) 比两次 atan2 相减稳
DB ccw_angle(Point<T> from, Point<U> to) {
  DB d = atan2l(cross(from, to), dot(from, to));
  return d < 0 ? d + 2 * PI : d;
}
```

13.5 判断两直线是否垂直 / 平行

vjudge 提交。

分别取两条直线的方向向量，看 dot 是否为零（垂直）或 cross 是否为零（平行）。

```
// 依赖：点线基础
template<class T, class U> bool isOrthogonal(Line<T> a, Line<U> b) { return !sgn(dot(a.p2 - a.p1,
↪  b.p2 - b.p1)); }
template<class T, class U> bool isParallel(Line<T> a, Line<U> b) { return !sgn(cross(a.p2 - a.p1,
↪  b.p2 - b.p1)); }
```

13.6 判断两线段是否相交

vjudge 提交。

a 与 b 相交当且仅当 b 的两端点关于 a 异侧（含端点重合）**且** a 的两端点关于 b 异侧。利用 CCW 的符号相乘 ≤ 0 即可。

```cpp
// 依赖：CCW；含端点相交与退化为点的线段
template<class T, class U>
bool isSegIntersect(Line<T> a, Line<U> b) {
  if (a.p1 == a.p2) return CCW(a.p1, b) == ON_SEGMENT;
  if (b.p1 == b.p2) return CCW(b.p1, a) == ON_SEGMENT;
  return CCW(b.p1, a) * CCW(b.p2, a) <= 0 && CCW(a.p1, b) * CCW(a.p2, b) <= 0;
}
```

命名提示：函数名特意叫 `isSegIntersect`（不是 `isIntersect`）——`Line<T>` 在本模板里同时承担「直线」与「线段」两种语义，这里 `a`，`b` 是被当成线段判定的，函数名前加 `Seg` 防止与「两**直线**是否相交」（任意非平行直线都相交）混淆。

13.7 两条直线交点

vjudge 提交。

设两条直线分别用方向式 $p+t\vec{v}$、$q+s\vec{w}$。在交点处：

$$p+t\vec{v}=q+s\vec{w}$$

$$(p-q)\times\vec{w}+t\,\vec{v}\times\vec{w}=0 \quad (\text{两边叉乘}\vec{w},\ \text{自身叉乘消去}s\vec{w})$$

$$t=\frac{\vec{w}\times(p-q)}{\vec{v}\times\vec{w}}$$

交点即 $p+t\vec{v}$。**返回值固定为 `Point<DB>`**（除法走浮点，不论输入 `T` 是什么都向 `DB` 收敛）。函数体内先把所有 `Point<T>` 走 promotion 提到 `Point<DB>`，整数 `T` 也合法（实参为整数 `Line<ll>` 时不再 `assert`）。

```cpp
// 依赖：点线基础；两条非退化、不平行直线。返回值固定为 Point<DB>
template<class T, class U>
Point<DB> getintersect(Line<T> a, Line<U> b) {
  Point<DB> p = a.p1, v = a.p2 - a.p1, q = b.p1, w = b.p2 - b.p1;
  return p + v * (cross(w, p - q) / cross(v, w));
}
```

13.8 两线段间最小距离

vjudge 提交。

若两线段相交则距离为 0；否则取四个「端点 → 对方线段」距离的最小值。点到线段距离 `distancePS` 中要先判断投影是否落在线段两端之外，落到外面就退化为对应端点的距离。

```cpp
// 依赖：CCW, isSegIntersect。PPNorm 是距离平方（不开方，留到最后再 sqrt）；L1 / Linf 返回坐标公共类型
template<class T, class U> auto distancePPNorm(Point<T> a, Point<U> b) { return (a - b).norm(); }
template<class T, class U> DB distancePP(Point<T> a, Point<U> b) { return (a - b).abs(); }
template<class T, class U> auto distancePPL1(Point<T> a, Point<U> b) { return (a - b).normL1(); }
template<class T, class U> auto distancePPLinf(Point<T> a, Point<U> b) { return (a - b).normLinf();
↪  }
template<class T, class U>
DB distancePL(Point<T> p, Line<U> l) { // 点到直线
  auto v = cross(l.p1, l.p2, p);
  return (v < 0 ? -v : v) / (l.p2 - l.p1).abs();
}
template<class T, class U>
DB distancePS(Point<T> p, Line<U> l) { // 点到线段，AC https://qoj.ac/submission/2001668
  if (l.p1 == l.p2) return (p - l.p1).abs();
  if (sgn(dot(l.p2 - l.p1, p - l.p1)) < 0) return (p - l.p1).abs();
  if (sgn(dot(l.p1 - l.p2, p - l.p2)) < 0) return (p - l.p2).abs();
  return distancePL(p, l);
}
template<class T, class U>
DB distanceSS(Line<T> a, Line<U> b) { // 线段到线段，AC https://qoj.ac/submission/2001588
  if (isSegIntersect(a, b)) return 0;
  return min({distancePS(a.p1, b), distancePS(a.p2, b), distancePS(b.p1, a), distancePS(b.p2, a)});
}
```

13.9 多边形面积

vjudge 提交。

用「向量叉乘求三角形有向面积之和」（鞋带公式）：

$$A = \frac{1}{2}\left|\sum_{i=0}^{n-1} \mathrm{cross}(P_i, P_{(i+1) \bmod n})\right|$$

逆时针排列时 $\sum$ 本身已为正，顺时针为负，统一取绝对值。

拆出 `polygonSignedArea`：返回值 `Wide<T>` 跟着 `cross` 走（整数 `T` → `i128`，浮点 `T` → `T`）。**不要**像旧版那样写成 `DB res = 0; res += cross(...)`——在整数 `T` 下 `cross` 返回 `__int128_t`，赋给 `double` 累加器会在 $|val| > 2^{53}$ 时丢精度。除此之外，`reorder_polygon` 也复用 `polygonSignedArea` 取符号判方向，避免「前三点共线」时的局部退化。

```cpp
// 依赖：点线基础。2 倍有向面积（> 0 CCW，< 0 CW，0 退化）；整数返回 i128，不丢精度
template<class T>
Wide<T> polygonSignedArea(const vector<Point<T>> &poly) {
  Wide<T> res = 0;
  int n = poly.size();
  for (int i = 0; i < n; ++i) res += cross(poly[i], poly[(i + 1) % n]);
  return res;
}
template<class T>
DB polygonArea(const vector<Point<T>> &poly) {
  auto s = polygonSignedArea(poly);
  return (DB)(s < 0 ? -s : s) / 2;
}
```

```cpp
template<class T>
DB polygon_perimeter(const vector<Point<T>> &poly) {
  DB res = 0;
  int n = poly.size();
  for (int i = 0; i < n; ++i) res += (poly[i] - poly[(i + 1) % n]).abs();
  return res;
}
```

```cpp
// 正 n 边形面积 A = n l^2 / (4 tan(pi / n))，n >= 3
DB regular_polygon_area(ll n, DB l) { return n * l * l / (4 * tanl(PI / n)); }
```

13.10 圆 `Circle`：测度五件套

圆心 `c` + 半径 `r` 的最小描述，配套**弧长 / 弦长 / 弓形高 / 扇形面积 / 弓形面积**五件套测度。

角度约定：入参 `theta` 是弦所对的**圆心角**，单位**弧度**，约束 $\theta \in [0, 2\pi]$。$\theta < \pi$ 是 minor 弓形（小的那块），$\theta > \pi$ 是 major 弓形（大的那块），公式同一个，**不做内部 deg ↔ rad 转换**（调用方自己 `deg * PI / 180`）。`point_at` 接受任意 `theta`（含负值 / 超 2π），不抛 assert——绕圆走多圈合法。

`theta_at`：`point_at` 的逆：给定圆上一点 p，返回它相对圆心的极角。`atan2l` 出来的是 $(-\pi, \pi]$，加一次 2π 归一化到 $[0, 2\pi)$，于是 `point_at(theta_at(p)) = p`（p 在圆上时）。不检查 p 是否真在圆上，由调用方保证。典型用法：把圆上若干交点转成极角后排序，扫出相邻弧段。

`point_at` 直接用标准 `cosl/sinl`，不绑定编译器专有的 `sincos` 指针类型；`DB` 采用 `double` 或 `long double` 都能编译。

弓形面积公式：弓形 = 扇形 − 三角形，化简得

$$S_{\mathrm{seg}}(\theta) = \tfrac{1}{2} r^2 (\theta - \sin\theta).$$

AC 实战：Gym 105161B Area of the Devil（圆 − 五块 segment − 五块三角形）。

`ccw_angle` helper：从极角 `theta_from` CCW 转到 `theta_to` 的弧度差，落在 $[0, 2\pi)$。**不要**用 `abs(theta_to - theta_from)`——后者在 wrap 处（例如 from = 352°, to = 54°）会算成 298° 而非真实的 62°。

```cpp
// enum 值即两圆公切线条数；COINCIDENT = -1 表示重合（无穷多）
enum CircleRelation { COINCIDENT = -1, ONE_CONTAINS_OTHER, INTERNALLY_TANGENT, INTERSECTING,
↪  EXTERNALLY_TANGENT, EXTERNALLY_APART };
```

```cpp
// 依赖：点线基础，distancePPNorm。theta 均为弧度制圆心角
template<class T>
struct Circle {
  Point<T> c; T r;
  explicit Circle(Point<T> c = Point<T>(), T r = 0) : c(c), r(r) {}
  template<class U, class = Promote<T, U>> Circle(Circle<U> o) : c(o.c), r(o.r) {}
  Point<DB> point_at(DB t) const { return c + Point<DB>(cosl(t), sinl(t)) * (DB)r; } // 极角 t
  ↪  处的点，t 不限范围
  template<class U> DB theta_at(Point<U> p) const { // point_at 的逆：圆上点 p 的极角，[0, 2pi)
    Point<DB> d = p - c;
    DB t = atan2l(d.y, d.x);
    return t < 0 ? t + 2 * PI : t;
  }
  DB arc_length(DB t) const { return r * t; }
  DB chord_length(DB t) const { return 2 * r * sinl(t / 2); }
  DB sagitta(DB t) const { return r * (1 - cosl(t / 2)); } // 弓形高
  DB sector_area(DB t) const { return (DB)r * r * t / 2; }
  DB segment_area(DB t) const { return (DB)r * r * (t - sinl(t)) / 2; } // 弓形 = 扇形 - 三角形
  template<class U> bool is_point_in_circle(Point<U> p) const { return sgn(distancePPNorm(p, c) -
  ↪  squared(r)) <= 0; }
  friend istream &operator>>(istream &s, Circle &a) { return s >> a.c >> a.r; }
  friend ostream &operator<<(ostream &s, Circle a) { return s << "<" << a.c << "," << a.r << ">"; }
};
Circle<DB> circle_from_diameter(Point<DB> a, Point<DB> b) { return Circle<DB>((a + b) / 2,
↪  distancePP(a, b) / 2); }
```

13.11 两圆位置关系 `circle_relation`

vjudge 提交（Aizu CGL_7_A）。

按圆心距 d 与 $r_1 \pm r_2$ 的关系判 5 种位置：相离 / 外切 / 相交 / 内切 / 内含。**enum 取值即两圆公切线条数**（外离 4 / 外切 3 / 相交 2 / 内切 1 / 内含 0），方便直接拿来当答案输出。

重合圆 sentinel `COINCIDENT = -1`：同心同径时公切线无穷多，单独一档；调用方按需特判（如题面要求输出 `-1` / `Inf` / 任意大数）。

用 squared(d) 与 squared($r_1 \pm r_2$) 比较，不直接比 d：整数情形 $(r_1 + r_2)^2$ 在 `ll` 边界（输入 1e9 量级）会溢，走 `squared()` 自动升 `__int128_t` 防溢出；浮点情形也省一次 `sqrt`。

```cpp
// 依赖：Circle。全程平方比较，不开方
template<class T>
CircleRelation circle_relation(Circle<T> a, Circle<T> b) {
  auto d2 = distancePPNorm(a.c, b.c);
  if (!sgn(d2) && !sgn(a.r - b.r)) return COINCIDENT;
  int o = sgn(d2 - squared(a.r + b.r)), i = sgn(d2 - squared(a.r - b.r));
  if (o > 0) return EXTERNALLY_APART;
  if (o == 0) return EXTERNALLY_TANGENT;
  return i > 0 ? INTERSECTING : i == 0 ? INTERNALLY_TANGENT : ONE_CONTAINS_OTHER;
}
```

13.12 三角形内接圆 / 外接圆

内接圆 `incircle_triangle`：vjudge 提交（Aizu CGL_7_B）。内心 I 是三条**角平分线**的交点，用重心式直接写出：

$$I = \frac{|BC| \cdot A + |AC| \cdot B + |AB| \cdot C}{|AB| + |AC| + |BC|}.$$

内切圆半径 r = 内心到任意一条边的距离，取 `min` 三条边距离做防御性数值兜底（三个值理论上相等，浮点下取最小避免 r 越界出三角形）。

外接圆 `outcircle_triangle`：vjudge 提交（Aizu CGL_7_C）。外心 O 是三条**垂直平分线**的交点，取 AB 与 AC 两边的中垂线求交即可：

- AB 中点 $M_{AB} = (A + B)/2$，中垂线方向 $= (B - A)$`.rot90()`；

- AC 中点 $M_{AC}=(A+C)/2$，中垂线方向 $=(C-A)$`.rot90()`；
- $O=$`getintersect`(L_{AB},L_{AC})，$R=|O-A|$。

不要用「重心式」（$I=(a\cdot BC+b\cdot AC+c\cdot AB)/(AB+AC+BC)$）求外心——那是**内心**公式，等腰 / 等边三角形样例下两者重合，会侥幸过样例但 WA。

退化 sentinel：三点共线 / 任意两点重合（`cross(a, b, c) == 0`）时，内接圆 / 外接圆都不存在，分别返 `r = 0` 与 `r = -1` 作 sentinel；调用方用 `ci.r == 0` / `co.r < 0` 判退化即可，不会再吃到除零 NaN。

```
// 依赖：Circle, distancePP, getintersect。三点共线 / 重合时：内接圆返回 r = 0，外接圆返回 r = -1
Circle<DB> incircle_triangle(Point<DB> a, Point<DB> b, Point<DB> c) {
  DB s = cross(a, b, c);
  if (!sgn(s)) return Circle<DB>(a, 0);
  DB ab = distancePP(a, b), ac = distancePP(a, c), bc = distancePP(b, c), p = ab + ac + bc;
  return Circle<DB>((a * bc + b * ac + c * ab) / p, (s < 0 ? -s : s) / p);
}
Circle<DB> outcircle_triangle(Point<DB> a, Point<DB> b, Point<DB> c) { // 两条中垂线的交点
  if (!sgn(cross(a, b, c))) return Circle<DB>(a, -1);
  Point<DB> m1 = (a + b) / 2, m2 = (a + c) / 2;
  Point<DB> o = getintersect(Line(m1, m1 + (b - a).rot90()), Line(m2, m2 + (c - a).rot90()));
  return Circle<DB>(o, distancePP(o, a));
}
```

13.13 直线与圆的关系 / 交点 `circle_line_relation` / `getCrossPointsCL`

vjudge 提交（Aizu CGL_7_D）；弦长版 vjudge 提交（EOlymp-359）。

关系判定：圆心 C 到直线距离 $d=|\text{cross}(p_1,p_2,C)|/|p_2-p_1|$，与半径 r 比较。两边平方 + 乘 $|p_2-p_1|^2$ 走整数：cross^2 vs $r^2\cdot|p_2-p_1|^2$，`squared()` 自动升 `__int128_t` 防溢。

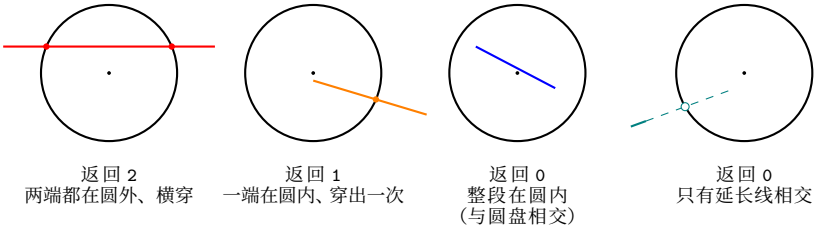
交点求法：先取垂足 $F=\text{project}(C,l)$，半弦长 $h=\sqrt{r^2-|CF|^2}$（斜边 r 与直角边 $|CF|$ 在直角三角形里推另一直角边），单位方向 $\hat{d}=(p_2-p_1)/|p_2-p_1|$，交点 $F\pm\hat{d}\cdot h$。

退化 sentinel：`l.p1 == l.p2` 时直接返 0 / 空集，不会跑到 `project` 的除零路径。

```
// 依赖：Circle, project。关系：0 相离 / 1 相切 / 2 相交；退化直线视为无交点
template<class T>
int circle_line_relation(Circle<T> cir, Line<T> l) {
  if (l.p1 == l.p2) return 0;
  return 1 - sgn(squared(cross(l.p1, l.p2, cir.c)) - squared(cir.r) * (l.p2 - l.p1).norm());
}
template<class T>
vector<Point<DB>> getCrossPointsCL(Circle<T> cir, Line<T> l) {
  int k = circle_line_relation(cir, l);
  if (!k) return {};
  Point<DB> f = project(cir.c, l), d = l.p2 - l.p1; // 垂足 f，沿 l 方向 +- 半弦长
  if (k == 1) return {f};
  d = d / d.abs() * (DB)sqrtl(max((DB)0, (DB)cir.r * cir.r - (cir.c - f).norm()));
  return {f - d, f + d};
}
```

13.14 线段与圆的关系 / 交点 `circle_segment_relation` / `getCrossPointsCS`

为什么要单独一份：`Line<T>` 在本模板里同时当直线与线段用，而上面的 `getCrossPointsCL` 是**直线**语义——交点可能落在 p_1p_2 之外的延长线上。线段版只需在直线版的结果上筛掉段外的交点：交点 P 已经在直线上，所以「P 落在线段内」等价于 $\text{dot}(A-P,B-P)\le 0$（P 看向两个端点的方向相反，$=0$ 即 P 就是某个端点）。



图注：四种情形下 circle_segment_relation 的返回值。右边两种 getCrossPointsCL 会分别给出 2 个交点（第四图中的空心点在延长线上）和 0 个交点，线段版都返回 0——这正是两者的差别。

返回值语义：circle_segment_relation 返回**线段与圆周的公共点个数** $0/1/2$，与 getCrossPointsCS 给出的交点个数恒等。它跟直线版的「0 相离 / 1 相切 / 2 相交」**不是一回事**：

- 返回 0 可能是整段在圆外，也可能是**整段在圆内**（与圆盘相交，只是不穿过圆周）；
- 返回 1 可能是真相切，也可能是一端在圆内、一端在圆外只穿出一次。

要判「线段与**圆盘**有没有公共点」，用 sgn(distancePS(cir.c, s) - cir.r) <= 0（点到线段距离 $\le r$），别拿本函数的返回值当它用。

退化：线段退化成一点（s.p1 == s.p2）时沿用直线版口径返回 0 / 空集，哪怕这个点正好落在圆周上。

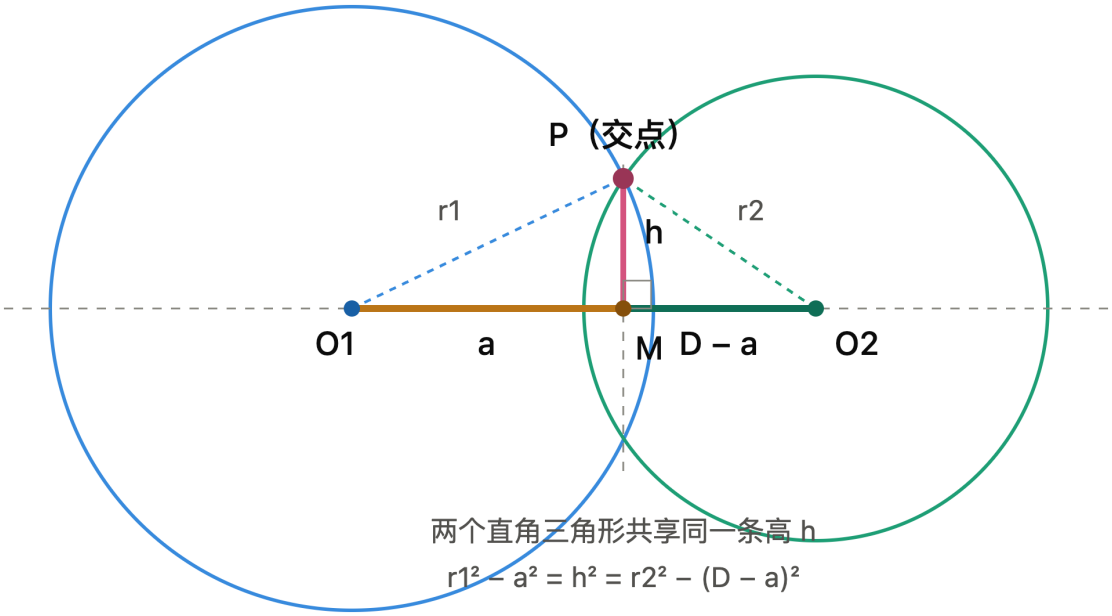
```cpp
// 依赖：getCrossPointsCL。线段语义：只保留落在 s 内部（含端点）的交点
template<class T>
vector<Point<DB>> getCrossPointsCS(Circle<T> cir, Line<T> s) {
  vector<Point<DB>> res;
  for (Point<DB> p : getCrossPointsCL(cir, s))
    if (sgn(dot(s.p1 - p, s.p2 - p)) <= 0) res.push_back(p);
  return res;
}
// 线段与圆周的公共点个数 0 / 1 / 2；整段在圆内也返回 0，判与圆盘相交用 distancePS <= r
template<class T>
int circle_segment_relation(Circle<T> cir, Line<T> s) { return getCrossPointsCS(cir, s).size(); }
```

13.15 两圆的关系 / 交点 circle_circle_relation / getCrossPointsCC

vjudge 提交（Aizu CGL_7_E）；含重合 sentinel 版 vjudge 提交（EOlymp-4779）。

关系判定：d^2、$(r_1+r_2)^2$、$(r_1-r_2)^2$ 全走整数 squared 比较，不开方。$d^2 > (r_1+r_2)^2$ 相离；$d^2 = (r_1+r_2)^2$ 外切；$(r_1-r_2)^2 < d^2 < (r_1+r_2)^2$ 相交；$d^2 = (r_1-r_2)^2$ 内切；$d^2 < (r_1-r_2)^2$ 内含。

交点求法：连心线方向 $\hat{d} = (c_2.c - c_1.c)/D$。弦中点 $M = c_1.c + \hat{d}\cdot a$，其中 $a = (r_1^2 - r_2^2 + D^2)/(2D)$（联立两圆方程相减消平方项得到）。沿连心线垂直方向 $\hat{d}.\mathrm{rot90}() \pm$ 半弦长 $h = \sqrt{r_1^2 - a^2}$ 即两交点。



图中 $PM \perp O_1O_2$ 把 $\triangle O_1PO_2$ 切成两个共享高 h 的直角三角形，两边各写一遍勾股定理就得到 a 的闭式：

$$r_1^2 - a^2 = h^2 = r_2^2 - (D-a)^2 \implies a = \frac{r_1^2 - r_2^2 + D^2}{2D}$$

注意 a **可以是负数**（M 落在 O_1 背离 O_2 的一侧，两圆一大一小且圆心很近时会出现），公式和代码都不需要特判——$\hat{d}\cdot a$ 自动往反方向走。

重合圆 sentinel：同心同径时 `circle_circle_relation` 返 -1（交点无穷多），`getCrossPointsCC` 据此返空，重合语义由 caller 自己处理。

类型提示：标量除法返回公共类型，`(c2.c - c1.c) / D` 在圆心是 `Point<ll>`、D 是 `DB` 时直接得到 `Point<DB>`，不会截断；代码里先写 `Point<DB> d = c2.c - c1.c;` 只是为了后面复用 `d`。

```cpp
// 依赖：Circle。关系：-1 重合 / 0 不交 / 1 相切 / 2 相交；重合与不交都返回空
template<class T>
int circle_circle_relation(Circle<T> a, Circle<T> b) {
  auto d2 = distancePPNorm(a.c, b.c);
  if (!sgn(d2) && !sgn(a.r - b.r)) return -1;
  int o = sgn(d2 - squared(a.r + b.r)), i = sgn(d2 - squared(a.r - b.r));
  return o > 0 || i < 0 ? 0 : o && i ? 2 : 1;
}
template<class T>
vector<Point<DB>> getCrossPointsCC(Circle<T> c1, Circle<T> c2) {
  int k = circle_circle_relation(c1, c2);
  if (k <= 0) return {};
  Point<DB> d = c2.c - c1.c;
  DB D = d.abs(), a = ((DB)c1.r * c1.r - (DB)c2.r * c2.r + D * D) / (2 * D); // 弦中点到 c1
  ↪ 的有向距离
  Point<DB> m = c1.c + d / D * a;
  if (k == 1) return {m};
  Point<DB> h = d.rot90() / D * (DB)sqrtl(max((DB)0, (DB)c1.r * c1.r - a * a)); // 半弦向量
  return {m + h, m - h};
}
```

13.16 过一点圆的切点 `getTangentPoints`

vjudge 提交（Aizu CGL_7_F）。

推导：过外部点 P 作圆 (C, r) 切线时，切点 T 同时满足 $PT \perp CT$ 与 $|CT| = r$，即 $\triangle PCT$ 在 T 处是直角三角形，$|PT| = \sqrt{|PC|^2 - r^2}$。切点 T 同时落在原圆 (C, r) 与辅圆 $(P, |PT|)$ 上，对两圆求交即得。

三种退化分类直接 early return：P 在圆内（含圆心）返空；P 在圆上唯一切点就是 P 自身；P 在圆外才走辅圆链。比" 全部喂给 `getCrossPointsCC` 让它兜底" 显式可读，也免依赖辅圆 $\mathrm{r} = 0$ 时的边界稳定性。

```cpp
// 依赖：getCrossPointsCC。圆内无切点，圆上返回自身，圆外与辅圆 (P, |PT|) 求交
vector<Point<DB>> getTangentPoints(Point<ll> P, Circle<ll> cir) {
  i128 d = (P - cir.c).norm() - squared(cir.r);
  if (d < 0) return {};
  if (!d) return {Point<DB>(P)};
  return getCrossPointsCC(Circle<DB>(cir), Circle<DB>(P, sqrtL(d)));
}
```

13.17 两圆公切线 `getCommonTangentPoints`

vjudge 提交（Aizu CGL_7_G）。函数返回的是公切线在 c_1 上的切点列表（每条公切线一个点；对应 c_2 上的切点由切线本身确定）。

推导：设 $T_1 = c_1.c + r_1 \cdot \vec{n}$，$\vec{n}$ 是单位法向量。令 $\vec{u} = (c_2.c - c_1.c)/D$，分外 / 内公切两类：

- 外公切：两边法向同向，$T_2 = c_2.c + r_2 \cdot \vec{n}$ 且 $(T_2 - T_1) \perp \vec{n}$，得 $(c_2.c - c_1.c) \cdot \vec{n} = r_1 - r_2$，故 $\cos\alpha = (r_1 - r_2)/D$；
- 内公切：两边法向反向，$T_2 = c_2.c - r_2 \cdot \vec{n}$，得 $\cos\alpha = (r_1 + r_2)/D$。

然后 $\vec{n} = \cos\alpha \cdot \vec{u} + \sin\alpha \cdot \vec{u}_\perp$，$\sin\alpha = \pm\sqrt{1 - \cos^2\alpha}$ 取两组。

退化：$|\cos\alpha| > 1$ 该类无切（内含 / 包含跳过）；$\sin\alpha = 0$ 该类只 1 条（内 / 外切临界）；$D = 0$（同心，含重合）统一返空。最后 `sort + unique` 去重，应对内 / 外切临界 $+ r = 0$ 退化下 4 个候选近重合的情形。

```cpp
// 依赖：Circle。返回各公切线在 c1 上的切点（外公切 + 内公切，已去重）；同心（含重合）返回空
vector<Point<DB>> getCommonTangentPoints(Circle<ll> c1, Circle<ll> c2) {
  vector<Point<DB>> ret;
  Point<DB> d = c2.c - c1.c;
  DB D = d.abs(), r1 = c1.r, r2 = c2.r;
  if (D < eps) return ret;
```

```
  for (DB cs : {(r1 - r2) / D, (r1 + r2) / D}) { // 法向与连心线夹角的 cos：外公切 / 内公切
    if (cs > 1 + eps || cs < -1 - eps) continue;
    DB sn = sqrtl(max((DB)0, 1 - cs * cs));
    Point<DB> p = c1.c + d / D * (cs * r1), v = d.rot90() / D * (sn * r1);
    ret.push_back(p + v);
    if (sn >= eps) ret.push_back(p - v);
  }
  sort(ret.begin(), ret.end());
  ret.erase(unique(ret.begin(), ret.end()), ret.end());
  return ret;
}
```

13.18 两圆交集面积 two_circle_intersect_area

vjudge 提交（Aizu CGL_7_I）。

推导：两圆相交时公共弦把每个圆切成两个弓形（segment / cap）。交集面积 = c_1 靠 c_2 那侧的弓形 + c_2 靠 c_1 那侧的弓形。圆心角由余弦定理给出：

$$\theta_1 = 2\arccos\frac{D^2 + r_1^2 - r_2^2}{2Dr_1}, \quad \theta_2 = 2\arccos\frac{D^2 + r_2^2 - r_1^2}{2Dr_2}.$$

单个弓形面积 = $\frac{1}{2}r^2(\theta - \sin\theta)$。

边界：$D \geq r_1 + r_2$ 不交 $\to 0$；$D \leq |r_1 - r_2|$ 一圆完全包含另一圆 $\to \pi \cdot r_{\min}^2$（重合圆自然落入此档，无需特判）。

```
// 依赖：Circle, distancePP。相交时 = 两个弓形之和
template<class T>
DB two_circle_intersect_area(Circle<T> c1, Circle<T> c2) {
  DB D = distancePP(c1.c, c2.c), r1 = c1.r, r2 = c2.r;
  if (sgn(D - r1 - r2) >= 0) return 0;
  if (sgn(D - fabsl(r1 - r2)) <= 0) return PI * min(r1, r2) * min(r1, r2);
  auto seg = [&](DB r, DB o) { // 圆 r 上、靠向圆 o 的弓形
    DB t = 2 * acosl(clamp((D * D + r * r - o * o) / (2 * D * r), (DB)-1, (DB)1));
    return r * r * (t - sinl(t)) / 2;
  };
  return seg(r1, r2) + seg(r2, r1);
}
```

13.19 简单多边形与圆的交集面积 polygon_circle_intersect_area

vjudge 提交（Aizu CGL_7_H）。

核心：把多边形按边累加「三角形 (O, A_i, A_{i+1}) 与 disk (O, r) 的有向面积」，O 是圆心。每条边的贡献由 tri_disk_signed_area 计算：先把 A, B 平移到以 O 为原点的坐标系；统一求边在圆内的参数区间，并截到 $[0, 1]$。区间两侧算扇形，中间算三角形；这同时覆盖以下四种情况：

- 两端都在圆内：整段就是真三角形，有向面积 = $\frac{1}{2}\text{cross}(A, B)$；
- 两端都在圆外但弦穿圆得入 / 出两点 P_1, P_2：扇形 OAP_1 + 真三角形 OP_1P_2 + 扇形 OP_2B；
- 两端都在圆外且不穿圆：整段对应一个扇形（从 $\vec{OA}$ 转到 $\vec{OB}$）；
- A 内 B 外（或反之）：在边上找穿圆点 P，OAP 是真三角形 + OPB 是扇形。

最后整圈累加取 abs，输入多边形 CW / CCW 都可。**扇形角度用 atan2(cross, dot) 算**：能拿到有向角度 $\in (-\pi, \pi]$，正负自动包含弧的方向（不像 acos 只给 $[0, \pi]$ 丢方向信息）。

退化：tri_disk_signed_area 在 $A = B$（多边形相邻顶点重合）时直接返 0，整体也吃得到。

```
// 依赖：Circle。三角形 (O, A, B) 与圆盘 (O, r) 的有向交面积，A、B 已平移到以圆心 O 为原点
DB tri_disk_signed_area(Point<DB> A, Point<DB> B, DB r) {
  if (A == B) return 0;
  auto sector = [&](Point<DB> p, Point<DB> q) { return r * r * atan2l(cross(p, q), dot(p, q)) / 2;
  ↪  };
  Point<DB> d = B - A;
```

```
  DB dd = d.norm(), ad = dot(A, d), disc = ad * ad - dd * (A.norm() - r * r);
  if (disc <= 0) return sector(A, B);
  DB s = sqrtl(disc), l = max((DB)0, (-ad - s) / dd), h = min((DB)1, (-ad + s) / dd); //
  ↪  边在圆内的参数区间
  if (l >= h) return sector(A, B);
  Point<DB> P = A + d * l, Q = A + d * h;
  return sector(A, P) + cross(P, Q) / 2 + sector(Q, B);
}
// 简单多边形（CW / CCW 均可）与圆的交集面积
template<class T>
DB polygon_circle_intersect_area(Circle<DB> cir, const vector<Point<T>> &poly) {
  DB res = 0;
  int n = poly.size();
  for (int i = 0; i < n; ++i) res += tri_disk_signed_area(poly[i] - cir.c, poly[(i + 1) % n] -
  ↪  cir.c, cir.r);
  return res < 0 ? -res : res;
}
```

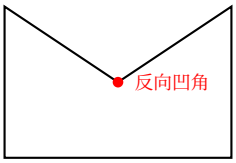
13.20 多边形凸性检验

允许的输入：**不允许自交**的多边形，顺序可顺可逆，函数会顺手把 poly 旋转 / 反向到「最左下点位 [0] + CCW 顺序」。

下面是三类**不被认为是凸多边形**的反例。

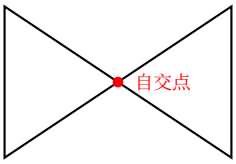
13.20.1 反例 1：内角为反向凹角（外角为锐角）

外角是锐角时 $\sin > 0$，叉乘也 > 0，看似 CCW；但其实形成的是凹陷。



图注：标红顶点的内角 $> 180°$（反射角），isConvex 中相对 poly[0] 的扇形检查 cross(poly[0], poly[i], poly[i+1]) < strict 触发返回 false。

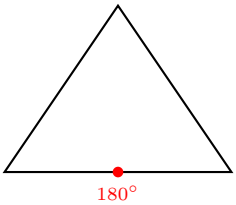
13.20.2 反例 2：自交多边形（蝴蝶结）



图注：四个顶点排成蝴蝶结，相对 poly[0] 不再是单调 CCW 排列，扇形检查即被排除。

13.20.3 反例 3：内角恰为 180°

三点共线（叉乘 = 0）也不允许，否则会让边数虚假增加。



图注：底边上多了一个 180° 顶点，相邻两边叉乘 s < strict 在 s = 0 时触发返回 false（严格模式要求大于零）。

实现：reorder_polygon 把多边形旋转 / 反向到「最左下点位 [0] + CCW 顺序」，再在同一个循环里检查两件事：(a) 每个内角（相邻两边的叉乘）符号合规；(b) 所有顶点相对于 poly[0] 按 CCW 排列。

`reorder_polygon` **用整体** `polygonSignedArea` **判方向，不要**用「前三点 `cross`」局部判向：含三点共线的多边形（如长方形 + 边密集共线）前三点 `cross = 0`，会被旧版 `<= 0` 误判为 CW 而强 reverse，下游 `isConvex` / 旋转卡壳全部失效（QOJ 784 实测）。

`isConvex` **加** `strict` **参数**：

- `strict = true`（默认）：严格凸，拒绝三点共线 / 内角 $= 180°$；
- `strict = false`：弱凸，允许内角 $= 180°$（题面用「凸」时常允许共线，要的就是这个口径）。

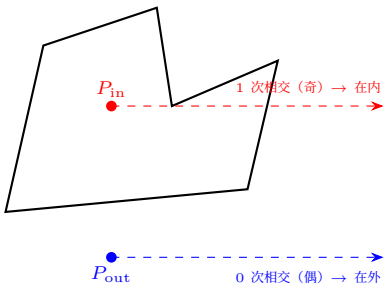
加宽接受范围有时是发现底层 bug 的好工具：本次就是因为加 `strict = false` 后 lax 模式 fail，才暴露出 `reorder_polygon` 的隐藏 bug——strict 模式因为 `<= 0` 提前 short-circuit 把它掩盖了。

```cpp
// 依赖：polygonSignedArea。整理成 CCW 且 [0] 为 y 最小（再 x 最小）的点；整体面积判向，不受前三点共线影响
template<class T>
void reorder_polygon(vector<Point<T>> &poly) {
  if (sgn(polygonSignedArea(poly)) < 0) reverse(poly.begin(), poly.end());
  auto byY = [](Point<T> a, Point<T> b) { return tie(a.y, a.x) < tie(b.y, b.x); };
  rotate(poly.begin(), min_element(poly.begin(), poly.end(), byY), poly.end());
}
```

```cpp
// 依赖：reorder_polygon。不允许自交，顺序任意（会整理成 CCW）。strict：拒绝内角 180°（三点共线）
// AC https://qoj.ac/submission/2002121
template<class T>
bool isConvex(vector<Point<T>> poly, bool strict = true) {
  int n = poly.size();
  if (n <= 2) return false;
  reorder_polygon(poly);
  bool turn = false; // 至少一处真左转，排除全共线
  for (int i = 0; i < n; ++i) {
    int s = sgn(cross(poly[i], poly[(i + 1) % n], poly[(i + 2) % n]));
    if (s < strict) return false;
    turn |= s > 0;
    // 所有顶点相对 poly[0] 也要按 CCW 排列，否则是星形 / 乱序
    if (i && i < n - 1 && sgn(cross(poly[0], poly[i], poly[i + 1])) < strict) return false;
  }
  return turn;
}
```

13.21 点与多边形关系

多边形未必凸——「有向面积一定为正」的方法在凹多边形上失效，所以采用经典的**射线奇偶法**。



图注：从待判定点向任意方向打一条射线，统计与多边形边的相交次数。**奇 = 在内，偶 = 在外**。代码使用水平射线，边的纵坐标采用半开区间 $[y_{\min}, y_{\max})$，穿过顶点时只计一次。

返回值约定：2 = 严格在内、1 = 点落在某条边上、0 = 严格在外。

```cpp
// 依赖：CCW。水平射线 + 半开区间计数：0 外部 / 1 边界 / 2 内部；凹多边形也可
template<class T>
int isInPolygon(Point<T> p, const vector<Point<T>> &poly) {
  bool inside = false;
  int n = poly.size();
  for (int i = 0; i < n; ++i) {
```

```
    auto a = poly[i], b = poly[(i + 1) % n];
    if (CCW(p, Line<T>(a, b)) == ON_SEGMENT) return 1;
    if (a.y > b.y) swap(a, b);
    if (a.y <= p.y && p.y < b.y && cross(a, b, p) > 0) inside = !inside;
  }
  return inside ? 2 : 0;
}
```

13.22 极角排序

按 atan2(y, x) 排序时，注意 atan2 浮点本身的精度问题。**规避调用 atan2** 的做法：先按「下半平面 / x 轴正向 / 上半平面」三档分桶，桶内再用叉乘符号判定相对方位。这样所有比较都用整数叉乘完成，没有浮点误差。

起点是 x 轴正半轴（含原点），逆时针环绕一周到 x 轴负半轴（不含）。

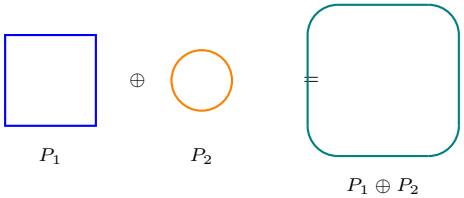
```
// 依赖：点线基础。极角序：x 正半轴（含原点）起逆时针一圈；全整数叉积，不用 atan2
template<class T>
int get_part_of_point(Point<T> p) { return p.y < 0 ? 0 : p.y == 0 && p.x >= 0 ? 1 : 2; }
template<class T> // AC https://qoj.ac/submission/2001485
void polar_angle_sort(vector<Point<T>> &a) {
  sort(a.begin(), a.end(), [](Point<T> p, Point<T> q) {
    int u = get_part_of_point(p), v = get_part_of_point(q);
    return u != v ? u < v : cross(p, q) > 0;
  });
}
// 有向直线按方向向量极角排序；同向的线「左半平面更紧」的排前（半平面交用）
template<class T>
void polar_angle_sort(vector<Line<T>> &a) {
  sort(a.begin(), a.end(), [](Line<T> l, Line<T> m) {
    Point<T> p = l.p2 - l.p1, q = m.p2 - m.p1;
    int u = get_part_of_point(p), v = get_part_of_point(q);
    if (u != v) return u < v;
    if (auto c = cross(p, q)) return c > 0;
    return cross(p, m.p1 - l.p1) < 0;
  });
}
// 排序后同向线只留每组最紧的一条（即每组第一条）
template<class T>
void unique_line_polar_angle_sort(vector<Line<T>> &a) {
  polar_angle_sort(a);
  a.erase(unique(a.begin(), a.end(), [](Line<T> l, Line<T> m) {
    Point<T> p = l.p2 - l.p1, q = m.p2 - m.p1;
    return get_part_of_point(p) == get_part_of_point(q) && cross(p, q) == 0;
  }), a.end());
}
```

13.23 Minkowski 闵可夫斯基和

给定两个凸包 P_1、P_2，定义其闵可夫斯基和：

$$P_1 \oplus P_2 = \{a + b \mid a \in P_1,\ b \in P_2\}$$

结果仍是凸包。**减法**就等价于「加上对方关于原点的中心对称图形」，这是判断两凸包是否相交、求最近距离的常用技巧。



图注：正方形 + 圆 = 圆角矩形——四条直边长度等于原正方形的边长，四个角是半径等于圆半径的四分之一圆弧（这是几何上严格的 Minkowski 和）。

算法核心：先把两个凸包都 `reorder_polygon`（最左下点起、CCW 顺序），然后**把它们的边向量按极角合并**（类似归并排序的归并步）：每次比较两条候选边向量的叉乘符号，挑极角更小的那条接到答案末尾。起点设为 $P_1[0]+P_2[0]$，沿合并后的边向量逐条累加即得新凸包。

```
// 依赖：reorder_polygon。输入凸多边形（可退化为点 / 线段），原地整理；返回闭合边界（末点重复首点），
↪  保留共线边
template<class T>
vector<Point<T>> minkowski(vector<Point<T>> &P1, vector<Point<T>> &P2) {
  if (P1.empty() || P2.empty()) return {};
  reorder_polygon(P1); reorder_polygon(P2);
  int n = P1.size(), m = P2.size(), i = 0, j = 0;
  vector<Point<T>> ans{P1[0] + P2[0]};
  while (i < n || j < m) { // 按极角归并两组边向量
    auto a = P1[(i + 1) % n] - P1[i % n], b = P2[(j + 1) % m] - P2[j % m];
    bool takeA = j == m || (i < n && cross(a, b) > 0);
    ans.push_back(ans.back() + (takeA ? a : b));
    takeA ? ++i : ++j;
  }
  return ans;
}
```

13.24 半平面交（half_plane_cut）

把每条有向直线 `Line(p1, p2)` 看作半平面——「沿 `p1 → p2` 方向走，左手边」是合法区域（如果想要这个东西按照你所想的方式运行，应该传入方向是 CCW）。所有半平面的交集若非空则为一个凸多边形（可能退化为无界、零面积等情形，本模板返回 `vector<Point<DB>>`，空表示交集为空 / 退化）。

算法核心：双端队列扫描法。先 `unique_line_polar_angle_sort` 把直线按极角排序 + 同向去重，然后逐条加入 deque `q`：每次新直线 `l` 进来，先用 `l` 反向清理队尾（队尾两条线的交点若在 `l` 右侧，说明该交点被 `l` 切掉，对应的队尾线已经无贡献），再清理队首（对称地）；扫完一轮后还要做一次 **wrap-around**：用队首 / 队尾互相再清一遍（处理环形闭合时残留的失效线段）。

复杂度：$O(N\log N)$（瓶颈是极角排序，扫描本身均摊 $O(N)$）。

常见用法：

- **多个半平面求公共凸区域**：每个不等式 $a_ix+b_iy\le c_i$ 表示一个半平面，加入对应的有向直线即可。
- **多边形并的总面积（旋转切割型）**：经典题 *Uyuw's Concert*（多次直线切凸多边形求剩余面积）/ 洛谷 P4196 [CQOI2006] 凸多边形（多个凸多边形求交）。
- **有界化**：当输入只有半平面、没有边界 box 时，可手动加 4 条「大正方形边界」直线（如 `[-1e9, 1e9]` 见方），避免交集无界。

```
// 依赖：unique_line_polar_angle_sort, getintersect。p1 -> p2 左侧闭半平面；求有界、正面积交集，
// 空 / 无界 / 退化返回空。无界输入需自行加题目允许的边界
template<class T>
vector<Point<DB>> half_plane_cut(vector<Line<T>> lines) {
  unique_line_polar_angle_sort(lines);
  deque<Line<T>> q;
  auto out = [&](Line<T> a, Line<T> b, Line<T> c) { return sgn(cross(c.p1, c.p2, getintersect(a,
  ↪  b))) < 0; };
  auto para = [&](Line<T> a, Line<T> b) { return cross(a.p2 - a.p1, b.p2 - b.p1) == 0; };
  for (auto l : lines) {
    while (q.size() > 1 && out(q[q.size() - 2], q.back(), l)) q.pop_back();
    while (q.size() > 1 && out(q[0], q[1], l)) q.pop_front();
    if (!q.empty() && para(q.back(), l)) return {}; // 反向平行：交集为空或退化
    q.push_back(l);
  }
  while (q.size() > 2 && out(q[q.size() - 2], q.back(), q.front())) q.pop_back();
  while (q.size() > 2 && out(q[0], q[1], q.back())) q.pop_front();
  if (q.size() < 3) return {};
  vector<Point<DB>> poly;
  for (int i = 0; i < (int)q.size(); ++i) {
    if (para(q[i], q[(i + 1) % q.size()])) return {};
```

```
    poly.push_back(getintersect(q[i], q[(i + 1) % q.size()]));
  }
  return poly;
}
```

使用范例（Uyuw's Concert 框架：给定 N 条有向直线 + 题目自带的 $[0,10000]^2$ 边界，求公共半平面交的面积）：

```
ll N; cin >> N;
vector<Line<DB>> lines(N);
for (auto &l : lines) cin >> l;
// 手动补 4 条边界直线（CCW 顺序：左下 → 右下 → 右上 → 左上）
auto a1 = Point<DB>(0, 0), a2 = Point<DB>(10000, 0);
auto a3 = Point<DB>(10000, 10000), a4 = Point<DB>(0, 10000);
lines.push_back(Line<DB>(a1, a2));
lines.push_back(Line<DB>(a2, a3));
lines.push_back(Line<DB>(a3, a4));
lines.push_back(Line<DB>(a4, a1));
auto poly = half_plane_cut(lines);
cout << fsp(1) << polygonArea(poly) << "\n";
```

13.25 凸包（Andrew 算法）

按 x 升序、x 相同按 y 升序排序后，从左到右扫一遍维护下凸包，再从右到左扫一遍维护上凸包，把两段拼起来即可。

`strict` **参数**决定共线点处理：`strict = true` 时 `cross == 0`（共线）也弹栈，**不保留共线点**；`strict = false` 时只在 `cross < 0`（凹陷）才弹，**保留共线点**（题目要求「凸包上所有点」时用）。

`HullMode` **参数**：三种返回都是 CCW 方向。`HULL_FULL` 完整凸包（**按 Y 排序**，从**最低点**（y 最小、再 x 最小）起 CCW 绕一圈；这样剥层第一步就拿到最低点起步，不必再调 `reorder_polygon` 找最低点）；`HULL_LOWER` 下凸壳（最左 → 最右）；`HULL_UPPER` 上凸壳（最右 → 最左，复用单调栈函数反向扫描）。**易踩**：`HULL_UPPER` 等价「直线上半包络」时调用前需 dedupe 同斜率（strict 处理不了「同 x 不同 y 竖边」退化），见 STL 节 `sort + unique` 模板。

```
// 依赖：polygonSignedArea。strict：不保留共线点。FULL 从最低点起 CCW；LOWER 左到右；UPPER 右到左
enum HullMode { HULL_FULL, HULL_LOWER, HULL_UPPER };
template<class T>
vector<Point<T>> make_convex_hull(vector<Point<T>> A, bool strict, HullMode mode) {
  if (mode == HULL_FULL) sort(A.begin(), A.end(), [](Point<T> a, Point<T> b) { return tie(a.y, a.x)
  ↪  < tie(b.y, b.x); });
  else sort(A.begin(), A.end());
  A.erase(unique(A.begin(), A.end()), A.end());
  int n = A.size();
  if (n <= 2) {
    if (mode == HULL_UPPER) reverse(A.begin(), A.end());
    return A;
  }
  auto chain = [&](int start, int step) { // 单调栈扫一侧
    vector<Point<T>> h;
    for (int i = start; i >= 0 && i < n; i += step) {
      while (h.size() > 1 && sgn(cross(h[h.size() - 2], h.back(), A[i])) < strict) h.pop_back();
      h.push_back(A[i]);
    }
    return h;
  };
  if (mode == HULL_LOWER) return chain(0, 1);
  if (mode == HULL_UPPER) return chain(n - 1, -1);
  auto h = chain(0, 1), u = chain(n - 1, -1);
  h.pop_back(); u.pop_back();
  h.insert(h.end(), u.begin(), u.end());
  return !strict && !sgn(polygonSignedArea(h)) ? A : h; // 全共线且保留共线点：直接返回排好序的点
}
```

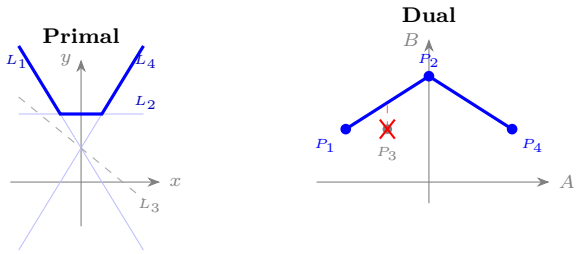
13.25.1 对偶点技巧：直线上半包络 ⇔ 对偶点上凸壳

洛谷 P3194 [HNOI2008] 水平可见直线：给 N 条直线 $y = A_i x + B_i$，从 $y = +\infty$ 俯视，问哪些「有某段 x 露头」。

直线-点对偶：把 L_i 映射到对偶平面的点 $P_i = (A_i, B_i)$，则

$$L_i \text{ 可见} \iff P_i \text{ 在所有对偶点的「严格上凸壳」上.}$$

为什么：3 条 $a_1 < a_2 < a_3$，L_2 被 L_1, L_3 在某 x 处压下 $\iff \text{cross}(P_2 - P_1, P_3 - P_1) \geq 0$（$P_2$ 在 P_1P_3 下方或共线）。共线时三线共点、L_2 只在一点露头不是 segment，`strict` 模式正好弹掉。



图注：左 Primal 4 条线，上半包络 $= L_1 \to L_2 \to L_4$（粗蓝折线），L_3 全程在下被压（灰虚线）。右 Dual 4 个对偶点，$P_3 = (-1, 1)$ 严格落在弦 P_1P_2 下方（红虚示意垂直差 0.5），上凸壳 $P_1 \to P_2 \to P_4$ 不收 P_3。**这就是「L_i 可见 ⇔ P_i 在上凸壳上」的视觉对照。**

落地：(1) 把 (A_i, B_i) 当点（套 `Point<ll>` 的 `id` 字段绑原始下标）；(2) 同 A 多条只留 B 最大那条（`sort + unique` 模板见 STL 节）；(3) `make_convex_hull(pts, true, HULL_UPPER)` 直接返回上凸壳点列，按 `id` 升序输出。$|A|, |B| \leq 5 \times 10^5$ 时全程整数，`cross` 走 `__int128`。

13.26 凸多边形直径（旋转卡壳）

QOJ 784（凸多边形直径模板题）。

给一个 **逆时针顺序**的 n 凸多边形（允许三点共线），求最远点对的距离。**旋转卡壳**的标准技巧：把每条边 $A[r] \to A[r+1]$ 当成一条「卡尺」，对踵点 $A[l]$ 是离这条边最远的顶点。r 沿 CCW 扫一圈时 l 单调（不回退），整体 $O(n)$。

关键判据：l 该不该往前推一步？看「以边 $(r, r+1)$ 为底、$l+1$ vs l 为顶」的两个三角形面积——后者 $\geq$ 前者就推。**比直接比距离平方更鲁棒**：曲率非均匀的凸包（如 spike 形 $r(\theta) = 1 + 0.1\sin(7\theta)$）下 $|A[l] - A[r]|^2$ 沿 r 不是单峰会卡死，但「以边为底的三角形面积」仍是单峰。

4 个不要踩的坑（QOJ 784 session 实录，每个都有真实反例）：

1. **不要做点对踵**：以 $|A[l] - A[r]|^2$ 沿 r 找单峰只在「曲率均匀」凸包成立，cyaron 默认凸包形是盲区——必死于 spike 形；
2. l, r **角色不能反**：必须「外层 r 是边、内层 l 是点」，反过来 |edge| 乘子会破坏单峰；
3. **ans 候选取两个**：$(A[l], A[r])$ 和 $(A[l], A[r+1])$ 都要更新，只取一个会漏掉「边端点本身就是最远对」的情形；
4. **推进用 $\leq$ 不是 $<$**：plateau（两个候选三角形面积相等）起点要继续推到 plateau 末端，严格 $<$ 会卡在起点。

对拍提醒：几何题对拍**必须用曲率非均匀的形状**（spike / heart / lemon / 椭圆），cyaron `Polygon.convex_hull` 默认是近圆形，stress 5000+ case 全过会让你错误推断「算法对」。详见 `cyaron-stress` skill。

```
// 依赖：点线基础。CCW 凸边界（允许共线边、重复顶点）。返回最远两点下标，空输入 {-1, -1}
template<class T>
pair<int, int> convex_diameter_indices(const vector<Point<T>> &A) {
  int n = A.size();
  if (!n) return {-1, -1};
  int lo = min_element(A.begin(), A.end()) - A.begin(), hi = max_element(A.begin(), A.end()) -
  ↪  A.begin();
  bool collinear = true;
  for (auto p : A) if (sgn(cross(A[lo], A[hi], p))) collinear = false;
  if (collinear) return {lo, hi};
  pair<int, int> best{0, 0};
  Wide<T> ans = 0;
```

```
  auto update = [&](int i, int j) {
    auto d = (A[i] - A[j]).norm();
    if (d > ans) ans = d, best = {i, j};
  };
  for (int i = 0, j = 0; i < n; ++i) { // 边 (i, k) 为卡尺，j 是以它为底面积最大的顶点（单调推进，用 >=
  ↪ 越过 plateau）
    int k = (i + 1) % n;
    if (A[i] == A[k]) continue;
    auto area = [&](int p) { return cross(A[i], A[k], A[p]); };
    while (area((j + 1) % n) >= area(j)) j = (j + 1) % n;
    update(i, j); update(k, j);
  }
  return best;
}
template<class T>
DB convex_diameter(const vector<Point<T>> &A) {
  auto [i, j] = convex_diameter_indices(A);
  return i < 0 ? 0 : (A[i] - A[j]).abs();
}
```

13.27 最远点对（旋转卡壳，返回原始下标）

Luogu P16223。

convex_diameter 只返回距离；要把最远点对的原始输入下标也输出回来，靠 Point::id 字段：读入时填 A[i].id = i，make_convex_hull 不做派生运算（仅 sort + 拷贝原点入栈），凸包顶点继承原始 id。

```
// 依赖：make_convex_hull, convex_diameter_indices。返回原始下标（读入时需设置 id），空输入 {-1, -1}
template<class T>
pair<int, int> furthest_pair_of_point(vector<Point<T>> A) {
  A = make_convex_hull(A, true, HULL_FULL);
  auto [i, j] = convex_diameter_indices(A);
  return i < 0 ? pair(-1, -1) : pair(A[i].id, A[j].id);
}
```

调用样板（读入时给 id）：

```
ll N; cin >> N;
vector<Point<ll>> A(N);
for (int i = 0; i < N; ++i) {
  cin >> A[i];
  A[i].id = i;
}
auto [l, r] = furthest_pair_of_point(A);
cout << l << " " << r << '\n';
```

13.28 最小矩形覆盖（旋转卡壳，4 指针）

Luogu P3187 [HNOI2007] 最小矩形覆盖。

给一个点集，求面积最小、把所有点框住的矩形（不必轴对齐）。**关键定理**：最小覆盖矩形必有一条边与凸包某条边重合。所以枚举每条凸包边 e 当矩形底边，三指针单调推进维护：

- up：以 e 为底时离 e 最远的顶点（决定矩形高 h）；
- r / l：在 e 方向上投影最大 / 最小的顶点（决定宽 w 的右 / 左端）。

外层 edge 沿 CCW 扫一圈，内层 up, r, l 各自单调（不回退），整体 $O(n)$。

共用投影还原矩形：令 $v = e^{\perp}$、$d = |e|^2$。代码中的 x, w, h 是沿 e, v 的系数（不是实际长度）：

$$x = \frac{e \cdot (A[l] - A[i])}{d}, \qquad w = \frac{e \cdot (A[r] - A[l])}{d}, \qquad h = \frac{v \cdot (A[up] - A[i])}{d}.$$

左下角 $p = A[i] + xe$，其余三角为 $p + we$、$p + we + hv$、$p + hv$，面积为 whd。这样宽高计算和顶点还原共用同一组投影，不再构造四条直线求交。接口仍接收 vector<Point<DB>>，返回浮点面积和四个 CCW 顶点。

```cpp
// 依赖：make_convex_hull。返回最小面积 + BL/BR/TR/TL 四角（CCW）；空输入全零，点 / 线段面积 0
pair<DB, array<Point<DB>, 4>> minimum_rectangle_cover(vector<Point<DB>> A) {
  A = make_convex_hull(A, true, HULL_FULL);
  int n = A.size();
  if (!n) return {0, array<Point<DB>, 4>()};
  if (n <= 2) return {0, {A.front(), A.back(), A.back(), A.front()}};
  int up = 0, l = 0, r = 0;
  DB best = INFINITY;
  array<Point<DB>, 4> corners;
  for (int i = 0; i < n; ++i) { // 枚举底边 e = A[i] -> A[i+1]，up / r / l 分别沿法向、e、-e 投影最大，单调推进
    auto e = A[(i + 1) % n] - A[i], v = e.rot90();
    auto advance = [&](int &j, Point<DB> dir) {
      if (!i) for (int k = 0; k < n; ++k) if (dot(dir, A[k]) > dot(dir, A[j])) j = k;
      while (dot(dir, A[(j + 1) % n] - A[j]) > 0) j = (j + 1) % n;
    };
    advance(up, v); advance(r, e); advance(l, -e);
    DB d = e.norm(), x = dot(e, A[l] - A[i]) / d, w = dot(e, A[r] - A[l]) / d, h = dot(v, A[up] - A[i]) / d;
    if (w * h * d < best) {
      best = w * h * d;
      auto p = A[i] + e * x; // 左下角
      corners = {p, p + e * w, p + e * w + v * h, p + v * h};
    }
  }
  return {best, corners};
}
```

13.29 平面最近点对

经典分治：把所有点按 x 排序后，递归求左右两半的最小距离 d，再处理跨越中线、与中线 x 距离 $< \sqrt{d}$ 的「窄带」候选点。

难点在于把窄带内的候选点收集起来后再按 y 暴力扫一遍——技巧是从中线向两侧分别推进，一旦 $|\Delta x| \geq \sqrt{d}$ 立刻 break，避免把窄带退化成 $O(n)$。

```cpp
// 依赖：点线基础，MAXN。调用 dfs(0, n - 1) 前把 A 按 x 排序。O(n log^2 n)，返回距离平方与两点
Point<ll> A[MAXN];
struct Best {
  i128 d;
  Point<ll> a, b;
  bool operator<(const Best &o) const { return d < o.d; }
};
Best mindis(vector<Point<ll>> &a, Best res) { // 窄带内按 y 扫
  sort(a.begin(), a.end(), [](Point<ll> p, Point<ll> q) { return p.y < q.y; });
  for (int i = 0; i < (int)a.size(); ++i)
    for (int j = i - 1; j >= 0 && squared(a[i].y - a[j].y) <= res.d; --j)
      res = min(res, Best{(a[i] - a[j]).norm(), a[i], a[j]});
  return res;
}
Best dfs(int l, int r) {
  if (l == r) return {numeric_limits<i128>::max(), A[l], A[l]};
  int m = (l + r) / 2;
  Best res = min(dfs(l, m), dfs(m + 1, r));
  if (!res.d) return res;
  vector<Point<ll>> strip;
  for (int i = l; i <= r; ++i) if (squared(A[i].x - A[m].x) < res.d) strip.push_back(A[i]);
  return mindis(strip, res);
}
```

13.30 两凸多边形最近距离（旋转卡壳）

OpenJ_Bailian 3851 Bridge Across Islands。给两个不相交的 CCW 凸多边形 A, B，求两边界最小距离。

初始 `eA` 取 A 最低点、`eB` 取 B 最高点（一对反向支撑线起步）；按 $\texttt{cross}(\vec{e_A}, \vec{e_B}) > 0$ 单调推进 `eB`、否则推进 `eA`，合计 $\le n+m$ 步。每步取段-段距离 `distanceSS`（A 边长且 B 边整段在 A 边投影内时单纯点-边距离会漏，必须段-段）。

必须双向各调一次：单次 `cal_min_dis_two_convex(A, B)` 的对偶覆盖不完整，要 `(A, B)` 和 `(B, A)` 各跑一次取 min 才覆盖全。漏一次会在「A 顶点 $\to$ B 边」和「B 顶点 $\to$ A 边」非对称分布下系统性偏大（实测反例存在）。

```cpp
// 依赖：reorder_polygon, distanceSS。两个不相交凸多边形的边界最小距离；须 (A, B) 与 (B, A) 各调一次取 min
// AC https://vjudge.net/solution/69793877
template<class T>
DB cal_min_dis_two_convex(vector<Point<T>> A, vector<Point<T>> B) {
  if (A.empty() || B.empty()) return INFINITY;
  reorder_polygon(A); reorder_polygon(B);
  int n = A.size(), m = B.size(), i = 0; // A 从最低点、B 从最高点起步（一对反向支撑线）
  int j = max_element(B.begin(), B.end(), [](Point<T> a, Point<T> b) { return a.y < b.y; }) - B.begin();
  DB ans = INFINITY;
  for (int _ = 0; _ < n + m; ++_) {
    while (cross(A[(i + 1) % n] - A[i], B[(j + 1) % m] - B[j]) > 0) j = (j + 1) % m;
    ans = min(ans, distanceSS(Line<T>(A[i], A[(i + 1) % n]), Line<T>(B[j], B[(j + 1) % m])));
    i = (i + 1) % n;
  }
  return ans;
}
```

13.31 随机凸多边形生成

返回指定数量的顶点，支持整数或浮点、允许或禁止共线；依赖基础模板的随机数引擎。整数坐标范围必须足够容纳所需凸多边形，否则拒绝采样可能无法结束；此工具不替代非均匀曲率对拍数据。

```
// 依赖：点线基础，rng。T = ll 时坐标取整（范围 [-R, R]）。拒绝采样：R 太小装不下 N 个整点时不会结束
template<class T>
vector<Point<T>> gen_convex_polygon(int N, bool allow_collinear, ll R) {
  DB phi = uniform_real_distribution<DB>(0, 2 * PI)(rng), asp = uniform_real_distribution<DB>(1,
  ↪  3)(rng);
  auto fix = [](DB v) { return T(is_floating_point_v<T> ? v : (DB)llround(v)); };
  while (true) {
    vector<DB> th(N);
    for (auto &t : th) t = uniform_real_distribution<DB>(0, 2 * PI)(rng);
    sort(th.begin(), th.end());
    vector<Point<T>> A(N);
    for (int i = 0; i < N; ++i) { // 椭圆 (R cos, R / asp sin) 再旋转 phi
      DB x = R * cosl(th[i]), y = R / asp * sinl(th[i]);
      A[i] = Point<T>(fix(x * cosl(phi) - y * sinl(phi)), fix(x * sinl(phi) + y * cosl(phi)));
    }
    bool ok = N < 2 || A[0] != A[1];
    for (int i = 0; i < N && N > 2 && ok; ++i) {
      auto c = cross(A[i], A[(i + 1) % N], A[(i + 2) % N]);
      ok = A[i] != A[(i + 1) % N] && (allow_collinear ? c >= 0 : c > 0);
    }
    if (ok) return A;
  }
}
```

13.32 最小圆覆盖

随机增量维护覆盖圆；依赖点线基础、圆类及三角形外接圆。空集返回原点处半径为零的圆，单点或重复点也可直接调用。

```
// 依赖：circle_from_diameter, outcircle_triangle。随机增量，期望 O(n)；空集返回原点半径 0
// AC https://vjudge.net/solution/69974356
template<class T>
Circle<DB> min_circle_cover(vector<Point<T>> A) {
  shuffle(A.begin(), A.end(), rng);
  Circle<DB> res;
  for (int i = 0; i < (int)A.size(); ++i) {
    if (i && res.is_point_in_circle(A[i])) continue;
    res = Circle<DB>(A[i], 0);
    for (int j = 0; j < i; ++j) {
      if (res.is_point_in_circle(A[j])) continue;
      res = circle_from_diameter(A[i], A[j]);
      for (int k = 0; k < j; ++k) {
        if (res.is_point_in_circle(A[k])) continue;
        res = outcircle_triangle(A[i], A[j], A[k]);
        if (res.r < 0) // 浮点近共线：三点无外接圆，退化为最远两点的直径圆
          res = circle_from_diameter(A[k], distancePPNorm(A[i], A[k]) > distancePPNorm(A[j], A[k])
          ↪  ? A[i] : A[j]);
      }
    }
  }
  return res;
}
```

13.33 常用公式

13.33.1 正多边形面积公式

设边数为 n、边长为 l:

$$A = \frac{n \cdot l^2}{4 \cdot \tan\left(\frac{\pi}{n}\right)}$$

13.33.2 三角恒等式

和差角:

$$\begin{aligned}
\sin(\alpha \pm \beta) &= \sin\alpha\cos\beta \pm \cos\alpha\sin\beta \\
\cos(\alpha \pm \beta) &= \cos\alpha\cos\beta \mp \sin\alpha\sin\beta \\
\tan(\alpha \pm \beta) &= \frac{\tan\alpha \pm \tan\beta}{1 \mp \tan\alpha\tan\beta}
\end{aligned}$$

倍角:

$$\sin 2\theta = 2\sin\theta\cos\theta, \qquad \cos 2\theta = \cos^2\theta - \sin^2\theta = 1 - 2\sin^2\theta = 2\cos^2\theta - 1$$

$$\tan 2\theta = \frac{2\tan\theta}{1 - \tan^2\theta}$$

降幂（把 θ 整体升到 2θ）:

$$\sin^2\theta = \frac{1 - \cos 2\theta}{2}, \qquad \cos^2\theta = \frac{1 + \cos 2\theta}{2}, \qquad \sin\theta\cos\theta = \frac{\sin 2\theta}{2}$$

辅助角（线性叠加 $\rightarrow$ 单一正弦）:

$$a\sin\theta + b\cos\theta = \sqrt{a^2 + b^2}\,\sin(\theta + \varphi), \qquad \tan\varphi = \frac{b}{a}$$

极值即 $\pm\sqrt{a^2 + b^2}$，把单变量旋转优化变成闭式。

积化和差:

$$\begin{aligned}
\sin\alpha\cos\beta &= \tfrac{1}{2}\big[\sin(\alpha + \beta) + \sin(\alpha - \beta)\big] \\
\cos\alpha\cos\beta &= \tfrac{1}{2}\big[\cos(\alpha + \beta) + \cos(\alpha - \beta)\big] \\
\sin\alpha\sin\beta &= \tfrac{1}{2}\big[\cos(\alpha - \beta) - \cos(\alpha + \beta)\big]
\end{aligned}$$

和差化积:

$$\begin{aligned}
\sin\alpha + \sin\beta &= 2\sin\tfrac{\alpha+\beta}{2}\cos\tfrac{\alpha-\beta}{2} \\
\sin\alpha - \sin\beta &= 2\cos\tfrac{\alpha+\beta}{2}\sin\tfrac{\alpha-\beta}{2} \\
\cos\alpha + \cos\beta &= 2\cos\tfrac{\alpha+\beta}{2}\cos\tfrac{\alpha-\beta}{2} \\
\cos\alpha - \cos\beta &= -2\sin\tfrac{\alpha+\beta}{2}\sin\tfrac{\alpha-\beta}{2}
\end{aligned}$$

半角:

$$\sin\frac{\theta}{2} = \pm\sqrt{\frac{1 - \cos\theta}{2}}, \qquad \cos\frac{\theta}{2} = \pm\sqrt{\frac{1 + \cos\theta}{2}}, \qquad \tan\frac{\theta}{2} = \frac{\sin\theta}{1 + \cos\theta} = \frac{1 - \cos\theta}{\sin\theta}$$

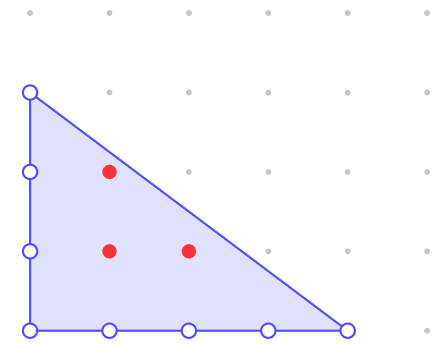
13.34 结论

本节收录不需要写模板、直接用结论出答案的几何定理。

13.34.1 Pick 定理

设简单多边形的**顶点全部是整点**，A 为面积、i 为严格位于内部的整点数、b 为落在边界上的整点数（含顶点），则

$$A = i + \frac{b}{2} - 1$$



○ 边界格点 $b = 8$　● 内部格点 $i = 3$

$$A = 6 = 3 + \tfrac{8}{2} - 1$$

竞赛里几乎只用变形式：面积和边界点数都好算，内部点数难数，所以反解

$$i = A - \frac{b}{2} + 1 = \frac{2A - b + 2}{2}$$

右式全是整数（$2A$ 由叉积直接得到），**整个过程不碰浮点**。

b 的算法：一条从 P 到 Q 的线段上，除去一端后的整点数恰是 $\gcd(|\Delta x|, |\Delta y|)$，绕多边形一圈求和即得

$$b = \sum_{i=0}^{n-1} \gcd\big(|x_{i+1} - x_i|,\ |y_{i+1} - y_i|\big)$$

```cpp
// 依赖：polygonSignedArea。整点简单多边形：b = 边界整点数（含顶点），i = 内部整点数 = (2A - b + 2) / 2，全程整数
template<class T>
ll pick_boundary(const vector<Point<T>> &poly) {
  ll b = 0;
  int n = poly.size();
  for (int i = 0; i < n; ++i) { // 别写 abs(d.x)：只 include <cstdlib> 时 ::abs 是 int 版
    Point<T> d = poly[(i + 1) % n] - poly[i];
    b += gcd((ll)(d.x < 0 ? -d.x : d.x), (ll)(d.y < 0 ? -d.y : d.y));
  }
  return b;
}
template<class T>
ll pick_interior(const vector<Point<T>> &poly) {
  auto s = polygonSignedArea(poly);
  return (ll)(((s < 0 ? -s : s) - pick_boundary(poly) + 2) / 2);
}
```

适用前提（踩了就是 WA）：

- 顶点**必须**是整点。有一个顶点不是整点，定理直接失效。
- 多边形**必须简单**（边不自交），凹凸不限。自交的要先拆成若干简单多边形。

- 带 h 个洞的多连通区域用推广式 $A = i + \dfrac{b}{2} + h - 1$，其中 b 把所有洞的边界也算进去。

两条顺手的推论：整点多边形的面积一定是 $\frac{1}{2}$ 的整数倍（由 $A = i + \frac{b}{2} - 1$ 立得）；顶点为整点的三角形若内部和边上除顶点外都没有整点，则面积恰为 $\frac{1}{2}$。

14 STL 及其使用

14.1 gp_hash_table（pb_ds 哈希表，替 unordered_map）

开放寻址哈希表，比 `std::unordered_map` 快 2–5 倍（本机实测 $N = 5 \times 10^6$ 下 gp + resize 比 unordered_map 快约 4.85×，见 gist: gp_cc_hash 速度测试）。默认 `hash<int>` 是恒等映射会被卡成 $O(n)$ 单次（参考 Codeforces blog/60737 "Anti-hash test"），**哈希函数 H 自己写**（不给默认值，强制显式传；现成写法见本章 **unordered_map 自定义 hash** 一节的 `custom_hash`）。默认 resize policy 锁死了 resize，用下面的 HashMap 别名把第 7 个模板参数换成 `external_size_access = true` 才能预留桶（`mp.resize(1 << 20)`，插入提速约 3.8×）。

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

// 可 resize 版别名（默认 resize_policy 锁死 resize，
// 这里把第 7 个模板参数换成 external_size_access=true）
template<class K, class V, class H>
using HashMap = gp_hash_table<K, V, H,
    equal_to<K>,
    direct_mask_range_hashing<>,
    linear_probe_fn<>,
    hash_standard_resize_policy<
        hash_exponential_size_policy<>,
        hash_load_check_resize_trigger<true>,
        true>>;
```

14.2 pb_ds 红黑树（tree）

`__gnu_pbds::tree` 是 GCC 自带的策略式 BST（默认红黑树），加上 `tree_order_statistics_node_update` 之后可在 $O(\log n)$ 内做**第 k 小**（`find_by_order`）和**排名**（`order_of_key`），等于 `std::set` + 内置 order statistics。

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
using ordered_set = tree<int, null_type, less<>,
    rb_tree_tag,
    tree_order_statistics_node_update>;
```

`find_by_order(k)`（迭代器，0-based）：

- `find_by_order(0)`：最小元素。
- `find_by_order(n - 1)`：最大元素（共 n 个时）。
- 越界（$k \geq n$）返回 `s.end()`。

`order_of_key(x)`：返回**严格小于** `x` 的元素个数，即 `x` 插入后排序序列中的 0-based 索引。例：集合 $\{2, 5, 8, 10, 15\}$，`order_of_key(8) = 2`（小于 8 的有 2 个），`order_of_key(12) = 4`，`order_of_key(20) = 5`。

自带去重 → 想要可重时改 pair：`tree` 同 `set` 一样去重。要存可重值时把 key 包成 `pair<value, id>`，id 取插入序号即可：

```
using ordered_multiset =
    tree<pair<int, int>, null_type,
         less<pair<int, int>>,
         rb_tree_tag,
         tree_order_statistics_node_update>;
```

查询**值落在区间 $[l, r]$ 内有几个**（多重集）：

```
// 用 (l - 1, INF) 排除 = l - 1 的等价类
// 用 (r + 1, 0)   排除 > r 的全部
ll cnt = s.order_of_key({r + 1, 0})
       - s.order_of_key({l - 1, INF});
```

应用 1：树上 dfs + 启发式合并 + 区间第 k 小（LeetCode kthSmallest）：每个节点维护一个 ordered_set，子树合并时把小集合并进大集合，在 dfs 离开节点前用 `find_by_order(k - 1)` 取答案。

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
using ordered_set = tree<int, null_type, less<>,
    rb_tree_tag,
    tree_order_statistics_node_update>;

class Solution {
public:
    vector<int> kthSmallest(vector<int>& par,
        vector<int>& vals,
        vector<vector<int>>& queries) {
        vector<vector<int> > g(par.size() + 5);
        vector<vector<array<int, 2> > >
            que(par.size() + 5);
        for (int i = 1; i < par.size(); ++i) {
            g[par[i]].push_back(i);
        }
        for (int i = 0; i < queries.size();
        ↪  ++i) {
            que[queries[i][0]].push_back(
                {queries[i][1], i});
        }
        vector<int> ans(queries.size());
        auto dfs = [&](auto&& self, int u, int
        ↪  val)
            -> ordered_set* {
            val ^= vals[u];
            ordered_set* st = new ordered_set();
            st->insert(val);
            for (auto v : g[u]) {
                ordered_set* t = self(self, v,
                ↪  val);
                // 启发式合并：小集合并入大集合
                if (t->size() > st->size())
                    swap(st, t);
                for (int k : *t) st->insert(k);
                delete t;
            }
```

```
            for (auto& t : que[u]) {
                int k = t[0], idx = t[1];
                if (k > st->size()) {
                    ans[idx] = -1; continue;
                }
                ans[idx] = *st->find_by_order(k
                ↪  - 1);
            }
            return st;
        };
        dfs(dfs, 0, 0);
        return ans;
    }
};
```

应用 2：按某个 key 维护时间序列、查 $[l,r]$ 计数（上师校赛 Problem F. 统计数据）：用 `priority_queue` 拿全局最早的事件，`map<ll, ordered_set<time_id>>` 按 type 分桶；查询 type=t 在时间窗 $[l,r]$ 内的件数 = `order_of_key({r+1, 0}) - order_of_key({l-1, INF})`。

```
struct type_time {
    ll type, time, id;
    bool operator<(const type_time& o) const {
        if (time != o.time) return time >
        ↪  o.time;
        return id > o.id;
    }
};
struct time_id {
    ll time, id;
    bool operator<(const time_id& o) const {
        if (time != o.time) return time <
        ↪  o.time;
        return id < o.id;
    }
};
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
using ordered_set = tree<time_id, null_type,
↪  less<>,
    rb_tree_tag,
    tree_order_statistics_node_update>;

void Solve() {
    ll Q; cin >> Q;
    priority_queue<type_time> pq;
    map<ll, ordered_set> type_setOfTime;
    auto pop = [&]() {
        if (pq.empty()) return;
        auto [type, time, id] = pq.top();
        pq.pop();
        type_setOfTime[type].erase({time, id});
    };
    FOR(i, 1, Q) {
        ll opt; cin >> opt;
        if (opt == 1) {
            ll type, time; cin >> type >> time;
            pq.push({type, time, i});
            type_setOfTime[type].insert({time,
            ↪  i});
        } else if (opt == 2) {
            pop();
        } else {
            ll type, l, r;
            cin >> type >> l >> r;
            ll ans
                = type_setOfTime[type].order_of
                ↪  _key(
                    {r + 1, 0})
                - type_setOfTime[type].order_of
                ↪  _key(
                    {l - 1, INF});
            cout << ans << "\n";
        }
    }
}
```

14.3 string::find 的起始位置参数与 string_view

find 可以指定「从哪个下标开始找」，但不能直接指定「查找到哪个下标结束」。

```
string s = "abacaba";
string t = "aba";

size_t pos = s.find(t, 2); // 从下标 2 开始找
if (pos == string::npos) { // string::npos 就是
↪  -1
  // 没找到
}
```

如果要限制在区间 [l, r] 内查找，可以先从 l 开始找，再判断匹配是否完整落在区间内（std::string_view 是在 C++ 17 引入的）。

```
string_view sv = s;
auto sub = sv.substr(l, r - l + 1); // 只看 [l,
↪  r]
size_t p = sub.find(t);
if (p != string::npos) {
  size_t real_pos = l + p; // 映射回原串下标
}
```

14.4 mt19937_64 套 splitmix64 抗 BM 攻击

```
std::mt19937_64 rng(std::chrono::steady_clock::
↪  now().time_since_epoch().count());

// 把 rng() 输出再过一层非线性混合，挡住 F2 上的
↪  Berlekamp-Massey 攻击
uint64_t splitmix64(uint64_t x) {
  x ^= x >> 30; x *= 0xbf58476d1ce4e5b9ULL;
  x ^= x >> 27; x *= 0x94d049bb133111ebULL;
  return x ^ (x >> 31);
}
uint64_t rnd() { return splitmix64(rng()); } //
↪  XOR hashing 等场景用这个，别用裸 rng()
```

攻击模型出处：*Hacking Incorrect XOR Hashing with mt19937_64*（Sugar_fan，CF blog）。mt19937_64 的 XOR、移位、稀疏 $\mathbb{F}_2$ 矩阵乘全部在 $\mathbb{F}_2$ 上线性，Berlekamp–Massey 拿前 2×19937 个输出位即可复盘最小多项式、预测后续输出，构造 XOR 多重集 hash 碰撞。换种子、加 seed_seq、混 random_device 全无效——线性结构和种子无关。

splitmix64 包装够不够：本地 Apple M5 + GCC 15.2 上的 benchmark（gist）实证——裸 mt19937_64 上 BM 在 5 个 bit lane 一致复盘出 degree = 19937、命中率 100%；套上 splitmix64 后 BM 度数饱和到训练长度的一半、命中率锁在 50%（bit 要么 0，要么 1，基本就是闭眼瞎猜），攻击失效。

14.5 unordered_map 自定义 hash & 结构体做 key

为何要自定义：GCC 整数默认 hash 是恒等（hash<ll>(x) == x）、桶数又走内部写死的质数序列，二者都可离线预测。攻击者构造大量落同一桶的 key，让查询退化 $O(n)$、整体 $O(n^2)$ TLE。防御 = 每次运行播一个 static 随机盐、混进 key 再过 splitmix64 打散（复用上一节的 splitmix64）：

```
struct custom_hash {
  size_t operator()(const ull &x) const {
    static ull R = rnd();       // 随机盐，rnd() 见上一节 mt19937
    return splitmix64(x ^ R);   // 盐 + avalanche
  }
};
// 用 alias 一键注入，声明处跟原生一样
template <class K, class V>
using hash_map = unordered_map<K, V, custom_hash>;
hash_map<ll, int> mp;
```

Me (zzy)

如果要存的东西里面有负数，会造成问题吗?

Mentor (Claude)

不会，负数 key 完全安全。关键在于 custom_hash 的形参是 const ull &（ull 即 64 位无符号整数）：往 hash_map<ll,int> 里塞一个负的 ll（如 -5）时，会先做一次 **signed → unsigned 隐式转换**再进哈希。C++ 标准规定这个转换是**良定义**的——结果 = 原值对 2^{64} 取模，即 $-1 \to 2^{64}-1$、$-5 \to 2^{64}-5$。

对 64 位的 ll，这个转换是一个**双射**（2^{64} 个有符号值一一对应 2^{64} 个位模式，无信息丢失），于是：

- **相等的 key** 转出**相等**的 ull ⇒ 哈希一致，一定查得到；
- **不同的 key** 转出**不同**的 ull ⇒ 负数不会跟某个正数「假撞」，不会平白增加冲突。

后续 x ^ R 只是拿随机盐做**按位异或**（连溢出都不涉及），再交给 splitmix64 打散——它的设计就是吃满 64 位输入。再兜一层底：即便两个 key 真落进同一个桶，unordered_map 还会用 std::equal_to<ll> 去比**真实的 ll 值**，-5 和 5 照样分得清。

唯一真会出事的不是「负数」，而是「key 宽于 64 位」：如果拿 __int128 当 key，形参 ull 会**截断**高 64 位，两个只差在高位的 key 才可能假撞。ll / int 的负数不涉及截断，安全。

自定义结构体做 key 需要两样东西：(1) 一个 hash 把**每个字段逐个混入**（禁止直接异或，(a,b) 与 (b,a) 会撞成同值）；(2) operator== 供哈希冲突时比较。两个 operator **都要加** const，否则 unordered_map 对 const key 调用时编译不过：

```
struct Point {
  ll x, y;
  bool operator==(const Point &o) const { // const
    return x == o.x && y == o.y;
  }
};
struct PointHash {
  size_t operator()(const Point &p) const { // const
    static const uint64_t R =
      chrono::steady_clock::now().time_since_epoch().count();
    uint64_t h = splitmix64(p.x + R);
    return splitmix64(p.y + h); // 逐字段混，顺序敏感
  }
};
unordered_map<Point, int, PointHash> mp;
```

变长序列（vector<ll>）做 key：思路一样，只是字段数不定——用一个初值为随机盐 R 的 h，把每个元素**依次 fold** 进去（h ^ x 再过 splitmix64）。这样天然**顺序敏感 + 长度敏感**，不会像裸异或那样让 {a,b} 和 {b,a} 撞成同值：

```
struct VecHash {
  size_t operator()(const vector<ll> &v) const {
    static const uint64_t R = rnd(); // 随机盐，rnd() 见上一节 mt19937
    uint64_t h = R;
    for (auto x : v) {
      h = splitmix64(h ^ (uint64_t)x); // 逐元素 fold，顺序/长度敏感
    }
    return h;
  }
};
unordered_set<vector<ll>, VecHash> st; // vector 直接做 key
```

坑：mp.reserve(n) / max_load_factor(0.25) 只是把桶数顶大 / 保持稀疏，桶数**仍然可预测**——是弱防御 + 常数优化，**不能单独防 hack**，真正的防线永远是随

机盐。追求速度可换 __gnu_pbds::gp_hash_table<K, V, custom_hash>（开放寻址，常数更小），custom_hash 用法相同。

14.6 views::keys / views::values 遍历 map 的键 / 值（C++20 ranges）

C++20 的 <ranges> 提供 views::keys / views::values，把 map（或任何 pair 序列）**投影**成「只有 key」/「只有 value」的惰性视图——省掉 for (auto &[k, v] : mp) 里那个用不到、却还得写的另一半。视图元素是**原容器的引用**，auto & 拿到就能**原地改**（如对每个 value 排序）：

```
#include <ranges>                  // 需要
namespace views = std::views; // 或 using
↪  namespace std::views;

unordered_map<ll, vector<array<ll, 2> > > mp;
// 只遍历 value：免写 auto &[_, vec]，直接拿
↪  value 的引用，能原地改
for (auto &vec : mp | views::values)
↪  sort(all(vec));
// 只遍历 key：关联容器 key 是 const，这里拿到
↪  const 引用
for (ll k : mp | views::keys) cout << k << " ";
```

同族常用视图（都能用 | 串接）：views::filter(pred) 过滤、views::transform(f) 逐元素映射、views::reverse 反序、views::take(n) / views::drop(n) 取前 / 弃前 n 个。视图**惰性求值**、不拷贝底层数据，遍历到才逐个算。

14.7 sort + unique 按 key 去重并保留指定值

std::unique 把相邻相等元素合并为一个，**保留每段的第一个**（不是最后）。配合 sort 可以做「按某个 key 分组、每组只留一个特定属性最大 / 最小的元素」。

标准套路：(1) 主 key 升序排（保证同 key 元素挤在一起）；(2) 次 key **反向**排，让"想留的"挤到每组最前面；(3) unique 用主 key 比较器去重，留下每组第一个。

例（洛谷 P3194 上半包络题 dedupe 步骤）：直线 $y = A_i x + B_i$，同斜率 A 多条只留 B 最大那条（其余被同斜率平行压死，对偶点全部出现在上凸壳「同 x 不同 y 竖边」上 strict 也救不了）。

```
sort(poly.begin(), poly.end(), [](const
↪  Point<ll> &a, const Point<ll> &b) {
  if (a.x != b.x) return a.x < b.x; // 主 key
  ↪  升序
  return a.y > b.y;                  // 次 key
  ↪  降序：想留的（B 大）在前
});
poly.erase(unique(poly.begin(), poly.end(),
↪  [](const Point<ll> &a, const Point<ll> &b) {
  return a.x == b.x;                 //
  ↪  比较器只看主 key，不看次 key
}), poly.end());
```

坑：次 key 写反成 a.y < b.y（升序），unique 留下的是每组 B **最小**那条——和题意完全相反，是高频低级错误。**记忆口诀**：「unique 留前 → 想留谁就 sort 把谁放最前」。

14.8 lower_bound / upper_bound 自定义比较器参数顺序相反

先厘清概念：lower_bound / upper_bound 的第三个参数是**待查找的值**（拿来和元素的排序键比较的目标值），**不是下标 / 位置**；返回的迭代器才是「位置」。即**输入一个值，输出一个位置**。下例数组 es 按 node::val 升序排好，target 是要找的目标值，e.val 是元素的排序键字段（也是个值，不是下标）。

带自定义比较器时，lower_bound 与 upper_bound 的比较器两个参数**谁前谁后是相反的**——标准强制，记反轻则二分边界静默错、重则编译不过：

- lower_bound(first, last, target, comp) 内部按 comp(*it, target) 调用——**数组元素在前，目标值在后**，返回首个满足 e.val ≥ target 的位置。
- upper_bound(first, last, target, comp) 内部按 comp(target, *it) 调用——**目标值在前，数组元素在后**，返回首个满足 e.val > target 的位置。

场景 A ——找第一个 e.val ≥ target 的元素，用 lower_bound：

```
auto it = lower_bound(es.begin(), es.end(),
↪  target,
  [](const node &e, int t) {   // 元素 node
  ↪  在前，目标值 int 在后
    return e.val < t;
  });
```

场景 B ——找第一个 e.val > target 的元素，用 upper_bound：

```
auto it = upper_bound(es.begin(), es.end(),
↪  target,
  [](int t, const node &e) {   // 目标值 int
  ↪  在前，元素 node 在后
    return t < e.val;
  });
```

坑（编译错 vs 静默逻辑错）：比较器喂错函数的后果取决于**元素类型与目标值类型是否同构**。本例元素是 node、目标值是 int，二者**异构且不可互转**：把场景 B 的 (int, node) 比较器误传给 lower_bound，它要按 comp(node, int) 调，参数对不上又无 node/int 转换，**直接编译失败**。但若元素与目标值**同类型**（如拿 node 二分 node），顺序写反**能编译通过，却是静默错的二分边界**——更难查。

记忆口诀：「lower 找 ≥，元素在前；upper 找 >，目标在前」。

14.9 __int128 读入输出（重载运算符版）

iostream 原生不支持 __int128。重载 << / >> 后，cin / cout 就能像内建整型一样直接读写（也自动覆盖 Point<i128> 等模板类里转发的 >> / <<）：

```
ostream &operator<<(ostream &os, __int128 x) {
  if (x < 0) os << '-', x = -x;
  if (x > 9) os << x / 10;
  return os << (char) ('0' + x % 10);
}

istream &operator>>(istream &is, __int128 &x) {
  string s;
  if (!(is >> s)) return is;
  x = 0;
  size_t i = 0;
  bool neg = false;
  if (s[0] == '-') {
    neg = true;
    i = 1;
  } else if (s[0] == '+') { i = 1; }
  for (; i < s.size(); ++i) x = x * 10 + (s[i]
  ↪  - '0');
  if (neg) x = -x;
  return is;
}
```

输出走递归除 10（深度 ≤ 39 层，无爆栈风险）；读入按字符串整体读进来再逐位累加，支持前导 - / +。

读入开头 if (!(is >> s)) return is; 的用意：让 while (cin >> v) 「多组输入读到 EOF 自动停」的写法对 __int128 同样成立。is >> s 返回流本身，流可以转 bool（语义是「上一次读取成功与否」）：EOF 时读不到内容、流被置上失败位、转出 false，这行就直接带着失败位原样返回且**不动** x——与内建整型 cin >> n 读失败的行为对齐，外层循环条件拿到 false 正确退出。

坑：输出对负数先 x = -x，恰为 -2^{127}（__int128 最小值）时取负溢出是 UB——竞赛值域顶到 $\pm 1.7 \times 10^{38}$ 边界才会踩，正常使用不受影响。

14.10 next_permutation 枚举全排列

next_permutation(first, last) 把区间 [first, last) 原地重排成**字典序的下一个排列**并返回 true；当前已是最大排列（完全降序）时返回 false，并把区间重排回最小排列（完全升序）。标准套路——**先 sort 到最小排列，再 do-while**：

```
vector<int> p = {3, 1, 2};
sort(p.begin(), p.end());  // 必须从最小排列出发，
↪  否则枚举不全
do {
  // 处理当前排列 p（do-while
  ↪  保证初始排列本身也被处理）
} while (next_permutation(p.begin(), p.end()));
```

- **不 sort 就 do-while 是高频错误**：只会从当前排列往字典序后面枚举，前面的排列全部漏掉。
- **复杂度**：单次调用均摊 $O(1)$（最坏 $O(n)$），枚举全部共 $n!$ 个——$n = 10$ 约 3.6×10^6 随便跑，$n = 11$ 约 4×10^7 是边界，$n \ge 12$ 基本不可行。
- **有重复元素时自动去重**：按「不同的排列」恰好各枚举一次（如 {1, 1, 2} 只出 3 种），这是它比手写 DFS 全排列省心的地方。
- **枚举组合的技巧**：开一个 $n-k$ 个 0 + k 个 1 的 01 数组，sort 后用同一套 do-while 枚举它的全部排列，就是 $\binom{n}{k}$ 个组合的指示向量。
- 对称版本 prev_permutation：字典序上一个，完全升序时返回 false；string / 数组 / vector 都能用（任何双向迭代器区间）。

14.11 FMT_STRUCT：让结构体能被 format / println 打印

配合开头模板里的 debug(x)（println(stderr, ...) 版）使用：给自定义结构体特化 std::formatter<T>，在 format 里用结构化绑定把成员拆出来，std::tie 成 tuple 交给 C++23 自带的 tuple formatter 打印。注册之后，结构体本身、装着结构体的 vector / map、嵌在别的结构体里的成员都能直接 debug。需要 -std=c++23。

```
#define FMT_STRUCT(T, ...)
↪  \
template<> struct std::formatter<T> {
↪  \
    constexpr auto parse(auto& ctx_) { return
    ↪  ctx_.begin(); } \
    auto format(const T& obj_, auto& ctx_) const
    ↪  {            \
        auto& [__VA_ARGS__] = obj_;
        ↪  \
        return std::format_to(ctx_.out(), "{}",
        ↪  \
                              std::tie(__VA_ARG
                              ↪  S__));
                              ↪  \
    }
    ↪  \
};
```

调用方式：第一个参数是类型名，后面按成员声明顺序写**绑定名**，个数必须等于成员个数（名字随便取，跟成员名无关）：

```
struct P { int x, y; };
FMT_STRUCT(P, x, y)          // 全局作用域，
↪  结尾不用分号
struct E { string name; ll w; P p; };
FMT_STRUCT(E, a, b, c)       // 绑定名随意，个数 =
↪  成员数

P p{1, 2};
vector<P> v{{1, 2}, {3, 4}};
E e{"ab", 5, {1, 2}};
debug(p); debug(v); debug(e);
```

打印效果（stderr，本机 clang++ / g++-15 + `-std=c++23` 实测）：

```
p:(1, 2)
v:[(1, 2), (3, 4)]
e:("ab", 5, (1, 2))
```

结构体打成圆括号 tuple，容器是方括号，`string` 带双引号、`char` 带单引号（`pair<int,char>{3,'c'}` 打成 `(3, 'c')`），`map` 打成 `{1: "one"}`。

- **必须写在全局作用域**（在 `namespace std` 外特化 `std::formatter`），不能写在 `main` 或别的函数里。
- **嵌套结构体**：内层类型要先 `FMT_STRUCT`，外层才能打印（如上面先 `P` 后 `E`）。
- **结构化绑定的限制**：成员全部 public、不能有带数据成员的基类；绑定名个数不等于成员数直接编译错误。
- 宏内形参刻意叫 `obj_` / `ctx_`：若叫 `x`，`FMT_STRUCT(P, x, y)` 的绑定名会和形参撞名报 `redefinition of 'x'`。

15 常见问题与错误

15.1 -O2 下的 FMA 乘加融合踩崩浮点 ceil

2026 HDU 春季联赛 6「绩点查询」一题，坑的是一个很不起眼的浮点细节：**同一份代码在 -O0（CLion Debug 默认）下跑样例全对，提交到判题机却 WA**。最小复现如下：

```
ll s1 = 36, s2 = 96;
ll s = ceil(0.6L * s1 + 0.4L * s2);
// 数学真值：3*36 + 2*96 = 300，除 5 恰好 60，ceil
↪  应为 60
// 但 -O2 下，s = 61
```

罪魁祸首是编译器在 -O2 默认启用的 FMA（fused multiply-add）乘加融合。ARM64 上对应指令 fmadd，x86 上对应 vfmadd。把表达式 a * b + c * d 合成一条带单次舍入的融合指令后，原本两次乘法各自产生的舍入误差无法互相抵消。

具体数值：0.6 * 36 真值 21.6，double 误差约 -2.13e-15；0.4 * 96 真值 38.4，double 误差约 +5.68e-15。分别计算再相加，两个反号误差近似抵消，落在精确的 60.0。一旦用 fmadd 融合，0.6 * 36 不再做中间舍入，加法也只做一次舍入，结果变成 60.0000000000000071，ceil 直接顶到 61。

实测不同优化等级下的行为：

编译参数	$0.6L \cdot 36 + 0.4L \cdot 96$	ceil
-O0 / -O1	60.0 精确	60（对）
-O2 / -O3	60.0000000000000071	61（错）
-O2 -ffp-contract=off	60.0 精确	60（对）

教训：

1. **ceil(浮点表达式) 永远不要信**。只要表达式里有 a * x + b * y 这种结构，-O2 的 FMA 合并就可能让结果比真值多出约 eps 的误差，刚好顶过整数边界。
2. **本地过、判题机挂**的经典场景：CLion / VSCode 的 Debug 构建是 -O0，不会触发 FMA；判题机普遍用 -O2。本地样例永远测不出来。
3. **long double 不是救命稻草**。Apple 平台（macOS ARM64 / Intel）的 long double ABI 就是 64-bit double；MinGW-w64 的 long double 宽度由发行版配置决定，不能指望一定是 80-bit。更要命的是 FMA 合并跟宽度无关，80-bit long double 也一样会被合并。

三种应对手段，优先级从高到低：

1. **首选——化成整数运算**。系数是有理数时，先通分成精确整数：

```
// 有理系数 0.6, 0.4 -> 3, 2, 分母 5
ll s = (3 * s1 + 2 * s2 + 4) / 5;  // 整数
↪  ceil
```

一行、零误差、对编译选项免疫。

2. **不得不用浮点时——关掉 FMA 合并**。用 #pragma 或 volatile 阻断合并：

```
// 函数级关 FMA（GCC）
__attribute__((optimize("fp-contract=off")))
ll calc(ll s1, ll s2) {
    return ceil(0.6L * s1 + 0.4L * s2);
}

// 或者用 volatile 分步，强制内存来回，
↪  编译器无法融合
volatile double a = 0.6 * s1;
volatile double b = 0.4 * s2;
ll s = (ll)ceil(a + b);
```

3. **真的需要更高精度时——上 __float128**（GCC / Clang 扩展，128-bit IEEE 754 quad precision，精度 $\sim 10^{-34}$）：

```
// 注意：表达式里不要出现 double 字面量 0.6、
↪  0.4，
// 那会先以 double 存储（带误差），升到
↪  float128 也救不回来。
// 所有系数都要用整数 / 有理构造：
__float128 v = (__float128)3 * s1 +
↪  (__float128)2 * s2;
v /= 5;
ll s = (ll)v;
if ((__float128)s < v) ++s;  // 手写 ceil
```

额外限制：0.6Q 这种 quad 字面量后缀在 -std=c++20 下默认被禁用，要加 -fext-numeric-literals 才认，而判题机场景下你无法指定编译参数。__float128 也没有 std::ostream 重载，不能直接 cout <<，只能把结果转回整数或 long double 再输出。

所以 __float128 能用的”安全姿势”几乎等价于整数法，反而更啰嗦。除非中间必须保留非整数有理值（比如小数系数来自输入），才值得上。

15.2 __float128 使用要点

- **类型本身**：GCC / Clang 扩展，x86_64 与 ARM64 均支持。sizeof(__float128) = 16，尾数 112-bit，eps $\approx 2^{-112} \approx 1.9 \times 10^{-34}$。
- **字面量后缀 Q**：需要 -fext-numeric-literals 编译参数，在严格 -std=c++20 / c++17 下被禁用。
- **头文件 <quadmath.h> 与 -lquadmath**：ceilq、sqrtq、quadmath_snprintf 等函数依赖这个库，判题机不一定链接。若只做基础算术（+ - * / 与比较），不需要任何额外库，就能直接用。

- **没有流 I/O**：`cin >> __float128` 和 `cout << __float128` 都不存在，必须手动转成 `double` / `long double` / 整数再输出。
- **陷阱**：`(__float128)0.6` 这种写法**不是**在 `__float128` 里存 `0.6`——`0.6` 先以 `double` 字面量存储(带 `5.5e-17` 的误差),再升到 `__float128`,误差直接继承。想保证精度要么用 `0.6Q`（需要上面那个 flag），要么干脆把系数换成整数比例构造。

15.3 DP 不可达状态必须用 INF/-INF 显式标识

线性 DP 形如 $f[r+1] = \max\{f[l]+1 : (l, r)$ 合法转移$\}$ 时，**不可达的前缀**（即 $a[0..l-1]$ 没有合法划分的状态）**必须用 -INF 标记**，不能图省事用 `vector<ll> dp(N + 1)`（默认初值 0）。

反例：CF 1094C *Median Partition* 写成

```
vector<ll> dp(N + 1);   // 全 0，BUG!
dp[0] = 0;
for (int r = 0; r < N; ++r)
    for (int l = 0; l <= r; ++l)
        if (checkLr(l, r))
            dp[r + 1] = max(dp[r + 1], 1 +
            ↪  dp[l]);
```

会让" 不可达的 $a[0..l-1]$" 被误读成"0 段已成功"，从而 $\texttt{dp}[r+1] = 1 + 0 = 1$ ——这等价于**凭空跳过**前面无法划分的部分、把 $a[l..r]$ 当作整个数组的合法第一段。最大段数被高估。

正解：

```
vector<ll> dp(N + 1, -INF);   // 不可达 = -INF
dp[0] = 0;                    // 空前缀是唯一
↪  base
// 转移时 dp[l] = -INF 自动让 1 + dp[l] 落败
```

记忆口诀：**base 之外一律 -INF**（求 max）/ INF（求 min）。0 只能用于" 求计数 / 默认无贡献" 语义，不能用于" 段数 / 长度" 语义——后者 0 是合法答案值，与" 不可达" 撞车。

15.4 大网格 DFS 遇到障碍要跳过，不要继续标记递归

2026 陕西省赛 M *Group Effect* 一题，$n \cdot m = 4 \times 10^6$。如果为了实现方便，遇到障碍不跳过而是继续标记已访问递归，会爆栈。**这里 RE 不是代码写错了，是 C++ 爆栈了**。

16 debug 技巧

16.1 除了打印之外，善用 assert

我们在头文件中已经定义了 dbg 宏，用来在本地把变量、容器打印到 cerr。但打印并不是调试的唯一手段，**配合 assert 把关键的「不变量」写成断言**，往往能更快定位错误。

assert 的好处是：一旦条件不成立，程序会立刻崩溃在出错的那一行，而不是让错误悄悄蔓延到很久之后再体现为 WA 或段错误。这对于 DP、图论、状压之类有很多前置约束的算法尤其有用，能帮我们把「这里应该满足某个条件」这种口头约定，变成程序能自检的硬约束。

下面是一段 Color Coding 的代码（随机染色 + 状压 DP，求包含 K 个不同颜色的最短路），我们同时使用了 dbg 打印和 assert 断言。

```
void execColorCoding() {
  for (int i = 1; i <= N; ++i) {
    dp[i][1 << color[i]] = {0, A[i]};
    timeId[i][1 << color[i]] = curT;
  }
  for (int sta = 1; sta < (1 << K); ++sta) {
    for (int u = 1; u <= N; ++u) {
      if (!check(u, sta)) continue;
      for (auto [to, w] : g[u]) {
        if (sta & (1 << color[to])) continue;
        ll toSta = sta | (1 << color[to]);
        if (!check(to, toSta)) {
          dp[to][toSta] = {INF, -1};
          timeId[to][toSta] = curT;
        }
#ifdef LOCAL
        assert(check(u, sta));
        assert(check(to, toSta));
#endif
        // 省略与 debug 无关的正常转移
      }
    }
  }
  dbg(dp);
  dbg(timeId);
  cEnd;
}
```

这道题目是一个**随机化算法**，每次跑出来的结果大概率是错的，**常规的这个 dbg 打印其实效果不是很好**。因为我们不知道预期输出是什么。在这种情况下，我们就不得不采用这个 **assert 来进行 debug**。

这道题目当中，我们使用这个 assert 来判断来源状态，即**即将进行转移的这个状态是否已经被初始化过了**。

16.1.1 增量 debug

如果一份代码原本是对的，而你后来**只新加了两样东西**，随后程序就挂了，那么一个很自然的想法就是做**增量 debug**。

具体来说，不要一上来就在新代码里到处打印，而是先把其中**一处改动暂时注释掉**，只保留另一处，然后重新运行：

- 如果问题消失了，说明刚刚注释掉的那部分很可疑。
- 如果问题还在，说明更可能是剩下那部分导致的。

这样就能把「两个新增改动共同成锅」的局面，快速缩小成「大概率是哪一个改动出了问题」。如果还有更多新增内容，也可以继续按这个思路一段一段地回退，逐步缩小可疑范围。

这个方法本质上是在做**控制变量**。很多时候，真正高效的 debug 不是打印更多信息，而是**一次只改一个东西**，让错误来源自己暴露出来。

16.2 std::string 条件断点的陷阱

在调试器（GDB、LLDB、CLion 等）里设置**条件断点**时，如果条件写成：

```
ops == "?00?00?00"
```

你会发现断点**根本不按预期停**——它可能在条件完全不成立时就触发，退化成了无条件断点。

原因：调试器的表达式求值器在求值条件时，需要在运行时调用 std::string::operator==。但这个运算符是一个重载函数，调试器往往**无法可靠地解析并调用**它，导致比较结果始终为真（或行为未定义），断点每次都停。这一问题在 JetBrains、VS Code、GDB 等主流工具的 issue tracker 上均有记录（另见 gist 整理）。

正确写法：改用成员函数 compare，调试器能正确调用它：

```
ops.compare("?00?00?00") == 0
```

或者用 C 风格的 strcmp（部分调试器更容易解析 C 函数符号）：

```
strcmp(ops.c_str(), "?00?00?00") == 0
```

17 环境测试

17.1 试机赛与纪律

- 检查身份证件：护照、学生证、胸牌以及现场所需通行证。
- 确认什么东西能带进场。特别注意：智能手表、金属（钥匙）等等。
- 测试鼠标、键盘、显示器和座椅。如果有问题，立刻联系工作人员。
- 测试比赛提交方式。如果有 submit 命令，确认如何使用。
- 设置自动保存、自动备份。

17.2 编译器版本与扩展头

测试编译器版本。C++20 cin >> (s + 1); C++17 auto [x, y] : a; C++14 [](auto x, auto y); C++11 auto; bits/stdc++.h; pb_ds。

```
#include <ext/rope>
using namespace __gnu_cxx;
rope<int> R; R.insert(y, x); R[x]; R.erase(x,
↪  1);

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
tree<int, null_type, less<int>, rb_tree_tag,
    tree_order_statistics_node_update> s;
s.insert(1); s.find_by_order(0);
↪  s.order_of_key(5);
```

17.3 __int128, __float128, long double 实测

long double 不是跨平台稳定类型：在 **x86-64 Linux + GCC**（CF / AtCoder / QOJ 评测机）上是 80-bit x87 扩展精度（尾数 64 bits、约 18–19 位有效、LDBL_EPSILON $\approx 1.08 \times 10^{-19}$），但 **Apple arm64 / MSVC** 上 long double == double（尾数 53 bits、约 15 位、$\epsilon \approx 2.22 \times 10^{-16}$）。本机过 $\not\Rightarrow$ 评测机过。

```
#include <bits/stdc++.h>
// pb_ds / rope 是否可用：能编过即 OK
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/rope>

// long double 是不是真扩展精度（CE = 退化成
↪  double 或编译器异常）
static_assert(sizeof(long double) >
↪  sizeof(double), "ld == double");
static_assert(LDBL_MANT_DIG >= 64, "ld mantissa
↪  < 64 bits");

int main() {
    std::cout << "sizeof(long double)=" <<
    ↪  sizeof(long double) << '\n';
    std::cout << "LDBL_MANT_DIG=" <<
    ↪  LDBL_MANT_DIG << '\n';
    // QOJ x86-64 Linux 实测 sizeof=16 (10B
    ↪  80-bit 数据 + 6B 对齐 padding), mant=64
    // 相对精度 LDBL_EPSILON ~ 2^-63 ~ 1.08e-19
    ↪  (vs double 2.22e-16, 多 ~3 位有效数字)
    // __int128 是否可用（不可用直接 CE）。assert
    ↪  必须在函数体内，namespace scope 不合法
    __int128 x = (__int128)1 << 100; assert(x >
    ↪  0);
    // __float128 是否可用（GCC 扩展，MSVC 无；
    ↪  不可用 CE）
    // 1.0Q 字面量需 -fext-numeric-literals 才认；
    ↪  用 cast 跨编译选项更稳
    __float128 y = (__float128)1; assert(y ==
    ↪  1);
}
```

注意 __int128 的 cin/cout 不支持，需手写 read/print；__float128 是软件模拟，慢一档。重载 operator<< / operator>> 后 cin >> x / cout << x 即可直接吃 __int128：

```
ostream &operator<<(ostream &os, __int128 x) {
    if (x < 0) os << '-', x = -x;
    if (x > 9) os << x / 10;
    return os << (char) ('0' + x % 10);
}

istream &operator>>(istream &is, __int128 &x) {
    string s;
    if (!(is >> s)) return is;
    x = 0;
    size_t i = 0;
    bool neg = false;
    if (s[0] == '-') {
        neg = true;
        i = 1;
    } else if (s[0] == '+') { i = 1; }
    for (; i < s.size(); ++i) x = x * 10 +
    ↪  (s[i] - '0');
    if (neg) x = -x;
    return is;
}
```

17.4 评测机限制与编译选项

- 测试代码长度限制，尝试触发 NO-OUTPUT、OUTPUT-LIMIT、RUN-ERROR。
- 测试 #pragma GCC optimize / #pragma GCC target 是否会被判为 CE（本机自带一半答案，详见下面「#pragma 开优化 / SIMD 实测」一节）。
- 测试 clarification：如果问不同类型的愚蠢问题，得到的回复是否不一样？
- 测试 -fsanitize=address、-fsanitize=undefined、-D_GLIBCXX_DEBUG 是否可用。

- 测试 `/usr/bin/time -v ./a.out` 是否能显示内存占用。

17.5 #pragma 开优化 / SIMD 实测

开 O3 三条路径，可叠加；**源码 pragma 优先级高于命令行**（同 TU 内覆盖 `-O` flag）。

```
// 命令行：本机训练默认；-march=native 评测机别带
// g++ -O3 -std=c++20 a.cpp

// optimize 放 include 之前；target 必须放
↪ include 之后（见下方踩坑 d）
#pragma GCC optimize("O3,unroll-loops")
#include <bits/stdc++.h>
#ifdef __x86_64__
#pragma GCC target("avx2,bmi,bmi2")
#endif
int main() { std::vector<int> v(5); v[0] = 1; }
↪ // 自检要触发 STL 实例化
```

实测（$N = 2^{20}$ 浮点 mul-add 200 轮，5 次中位数）：

编译方式	时间
`-O2`（评测机默认）	120 ms
`-O3`	117 ms
`-O3 -funroll-loops`	113 ms
`#pragma optimize("O3,unroll-loops")`	114 ms

评测机默认 `-O2`，所以 `#pragma optimize("O3")` 的实际收益是 **O3 比 O2 快约 3%、加 `unroll-loops` 共约 6%**——不是”几倍加速”。其更主要价值是兜底：评测机若被设成 `-O0`/`-O1`（少见但确实有），pragma 自动把模板抬回 O3。

踩坑：(a) `target("avx2")` 在 arm64 硬 CE，模板里此类代码段默认假设 x86 评测机。(b) pragma 路径下 `__OPTIMIZE__` 不定义（后端才 apply），不能拿来自检。(c) Mac 系统 `g++` 是 clang shim、不认这套 pragma 也找不到 `bits/stdc++.h`，本地必须显式调 `g++-15`。(d) **`#pragma GCC target` 必须放在 `#include <bits/stdc++.h>` 之后**：放之前会让 libstdc++ 头里 `always_inline` 模板（如 `~allocator` / `_Vector_impl`）带上 SIMD target，CF 用的 winlibs MinGW GCC 上 user 代码用 `vector<>` 等触发 inline 时报 `target specific option mismatch` CE（Linux GCC 不触发）。

评测机自检：把上段提交到任意评测机——CE 就是 pragma 被拒，AC 就是 pragma 全收。**自检源码必须实例化 STL**（如 `vector<int> v(5)`），空 `main` 不触发模板 inline、是假绿灯。

17.6 性能基准（map 跑分）

跑分覆盖 **STL 容器 + 随机访存 + 比较器内联**这套 CP 真实热点。`std::map` 的插入查找把红黑树指针追逐 + L2/L3 miss + `operator<` 内联效果都能压出来。计时用 `chrono::steady_clock`（wall-clock，纳秒分辨率，跨平台语义一致）而不是 `clock()`（Windows MSVC 上是 wall-clock、Linux glibc 上是 CPU 时间，跨评测机数字没法直接比）。标定 `1s` 能跑多少次：

```
#include <bits/stdc++.h>
using namespace std;

// 文件作用域：高精度 steady_clock 种子 +
↪ splitmix64 非线性混合（同 §10.4）
mt19937_64 rng(chrono::steady_clock::now().time
↪ _since_epoch().count());
uint64_t splitmix64(uint64_t x) {
    x ^= x >> 30; x *= 0xbf58476d1ce4e5b9ULL;
    x ^= x >> 27; x *= 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

int main() {
    const int N = 1 << 20;
    vector<uint64_t> keys(N);
    for (int i = 0; i < N; i++) keys[i] =
    ↪ splitmix64(rng());
    auto t0 = chrono::steady_clock::now();
    map<uint64_t, int> mp;
    for (int i = 0; i < N; i++) mp[keys[i]] =
    ↪ i;          // 插入
    long long s = 0;
    for (int i = 0; i < N; i++) s +=
    ↪ mp[keys[i]];          // 查找
    assert(s != 0);
    ↪ // 防 DCE + 弱正确性
    auto dt = chrono::steady_clock::now() - t0;
    cout << chrono::duration_cast<chrono::milli
    ↪ seconds>(dt).count() << "ms\n";
}
// 参考（N=2^20 一轮）：本机 440ms; CF ~2434ms
↪ (judge Used ~1988ms); QOJ 1024-1558ms
↪ (连发越跑越快，冷启动 +50%)
```

18 常用 Python 命令

18.1 启动交互解释器

启动命令因环境而异，依次试：`python3` / `python` / `py`。

- **Linux 赛场机**：一般只有 `python3`（裸 `python` 可能不存在或指向 Python 2）。
- **Windows**：官方安装器装的是 py launcher，敲 `py`；从 Microsoft Store 装的或勾了 PATH 的则 `python` 直接可用。
- 退出：`exit()`（或 Unix `Ctrl-D` / Windows `Ctrl-Z` + `Enter`）。

18.2 当计算器：科学计数法速估

赛场想快速估算（阶乘 / 组合数 / log / 大整数幂会不会爆 `ll`）时，进解释器先敲这两行：

```
>>> from math import *
>>> def p(x): print(f"{x:.2e}")
...
>>> p(factorial(20))
2.43e+18
>>> p(comb(30, 15))
1.55e+08
>>> p(2**64)
1.84e+19
>>> log2(1e9)
29.897352853986263
```

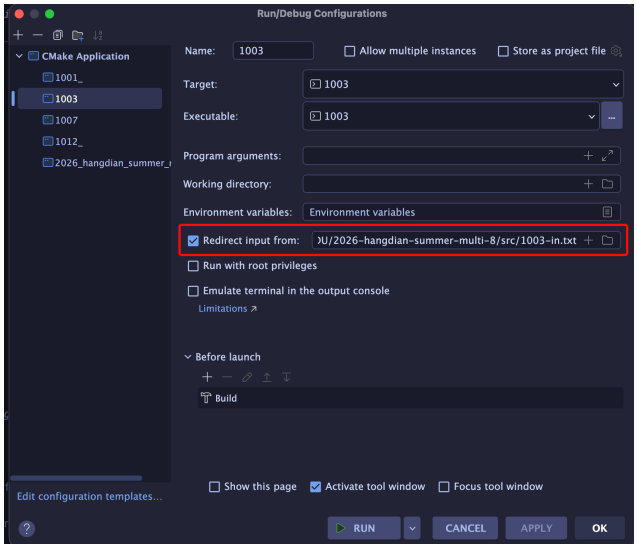
`p(x)` 按科学计数法两位小数打印，配合 Python 天然无界的大整数，判「爆不爆 `ll`（上限 ≈ `9.22e18`）/ 复杂度量级多大」一眼出结果。

19 常用技巧（卡常，对拍，打表，调试等）

19.1 快读快写

下面这个模板来自 wida 的 GitHub 仓库并补充了一些功能，适合需要高性能读写的场景。

一般使用快读以后，就选择这个重定向输入从这个输入文件啊。Clion 即便 SendEOF 了，有可能也会无法输出这个这个 stdout。




```cpp
// 来自wida github仓库
↪  https://github.com/hh2048/XCPC/tree/main
↪  加了一些新功能
//46 ms https://acm.hdu.edu.cn/contest/view-cod
↪  e?cid=11798&rid=14912 读入字符getc(a) 的AC记录
// AC https://vjudge.net/solution/63197697
↪  一道测试读写的codechef题目
namespace QuickRead {
  // 快读
  char buf[1 << 21], *p1 = buf, *p2 = buf;

  inline int getc() {
    return p1 == p2 && (p2 = (p1 = buf) +
    ↪  fread(buf, 1, 1 << 21, stdin), p1 ==
    ↪  p2) ? EOF : *p1++;
  }

  template<typename T>
  void Cin(T &a) {
    T ans = 0;
    bool f = 0;
    char c = getc();
    for (; c < '0' || c > '9'; c = getc()) {
      if (c == '-') f = 1;
    }
    for (; c >= '0' && c <= '9'; c = getc()) {
      ans = ans * 10 + c - '0';
    }
    a = f ? -ans : ans;
  }

  void getc(char &a) {
    char c = getc();
    while (isspace(c)) {
      c = getc();
    }
    a = c;
  }

  template<typename T, typename... Args>
  void Cin(T &a, Args &... args) {
    Cin(a), Cin(args...);
  }

  // ===== 输出 =====
  static constexpr size_t OUT_CAP = 1 << 21; //
  ↪  大概是2MB
  char obuf[OUT_CAP];
  size_t op = 0;

  inline void flush() {
    if (op) {
      fwrite(obuf, 1, op, stdout);
      op = 0;
    }
  }

  inline void putc(char c) {
    if (op == OUT_CAP) flush();
    obuf[op++] = c;
  }

  template<typename T>
  void write(T x) {
    // 注意，这里输出不带换行
    if (x < 0) putc('-'), x = -x;
    if (x > 9) write(x / 10);
    putc(x % 10 + '0');
  }

  struct Flusher {
    // 析构时自动刷新
    ~Flusher() { flush(); }
  } flusher;
} // namespace QuickRead
using namespace QuickRead;
```

19.1.1 读入，输出科学计数法小数

正常来说是用不到这个东西的。

```cpp
// https://www.luogu.com.cn/record/230330447
↪  科学计数法小数
template<typename T>
void Cinf(T &a) {
  // 新增：读取浮点数，支持科学计数法
  T ans = 0;
  bool f = 0; // 负号
  char c = getc();
  // 跳过非数字和小数点前的空白/符号
  while ((c < '0' || c > '9') && c != '.') {
    if (c == '-') f = 1;
    c = getc();
  }
  // 读取整数部分
```

```
  while (c >= '0' && c <= '9') {
    ans = ans * 10 + (c - '0');
    c = getc();
  }
  // 如果有小数点，读取小数部分
  if (c == '.') {
    c = getc();
    T frac = static_cast<T>(1);
    while (c >= '0' && c <= '9') {
      frac /= 10;
      ans += (c - '0') * frac;
      c = getc();
    }
  }
  a = f ? -ans : ans;
  // 处理科学计数法部分
  if (c == 'E' || c == 'e') {
    c = getc();
    bool exp_neg = false;
    if (c == '+') {
      c = getc();
    } else if (c == '-') {
      exp_neg = true;
      c = getc();
    }
    int exp = 0;
    while (c >= '0' && c <= '9') {
      exp = exp * 10 + (c - '0');
      c = getc();
    }
    T multiplier = pow(10.0, exp_neg ? -exp :
    ↪  exp);
    a *= multiplier;
  }
}

template<typename T>
void writef(T x) {
  // 新增：输出浮点数，默认输出6位小数，
  ↪  支持四舍五入
  if (x < 0) putchar('-'), x = -x;
  using ll = long long;
  int prec = 6; // 默认精度 6，可根据需要调整
  // 添加圆整因子
  T round_add = static_cast<T>(5);
  for (int i = 0; i < prec + 1; ++i) {
    round_add /= 10;
  }
  x += round_add;
  ll int_part = static_cast<ll>(x);
  write(int_part); // 输出整数部分
  putchar('.');
  T frac_part = x - int_part;
  for (int i = 0; i < prec; ++i) {
    frac_part *= 10;
    int digit = static_cast<int>(frac_part);
    putchar(digit + '0');
    frac_part -= digit;
  }
}
```

19.2 开大栈空间

如何开大栈空间

正常应该是不需要选手去干这件事情的。

杭电 OJ 暂不支持调整栈空间大小。如果使用 G++ 提交代码，且代码中递归层数较深，可能会返回 Runtime Error (STACK_OVERFLOW)。可以在 main() 开头加入内联汇编手动切换到更大的栈空间。注意必须用 exit(0) 退出，不能 return，否则会访问已失效的旧栈帧。

x86_64（Codeforces、洛谷、AtCoder 等绝大多数 OJ）：

```
int main() {
    int size(256 << 20);  // 256M
    __asm__ volatile("movq %0, %%rsp" ::
    ↪  "r"((char *)malloc(size) + size));

    // 关同步流代码
    // 主程序代码

    exit(0);
}
```

AArch64 / ARM64（Apple Silicon 本地调试等）：

```
int main() {
    int size(256 << 20);  // 256M
    asm volatile("mov sp, %0" :: "r"((char
    ↪  *)malloc(size) + size));

    // 关同步流代码
    // 主程序代码

    exit(0);
}
```

19.3 C++ 中 struct 自定义比较器与 std::array 性能对比

在作为 std::set 和 std::priority_queue 的元素时，使用自定义的 struct（配合自定义比较器）通常比使用 std::array （无论是自定义比较器还是默认比较器）具有更好的性能。根据基准测试，在 std::set 中，struct 的操作耗时比 std::array 显著更少（快约 30% 到 50%）。在 std::priority_queue 中两者性能相近，但整体上 priority_queue 远快于 set。因此，在对常数要求较高的题目中，建议使用 struct 替代 std::array 或 std::tuple，并优先考虑使用 priority_queue。

详细的基准测试代码与输出结果记录 (gist 链接)。

19.4 计时

```
#include <chrono>
#include <iostream>

int main() {
    auto start = std::chrono::high_resolution_c
    ↪  lock::now();

    // ... 你的代码 ...
```

```
    auto end = std::chrono::high_resolution_clo
    ↪  ck::now();

    // 转换为毫秒
    auto duration = std::chrono::duration_cast<
    ↪  std::chrono::milliseconds>(end - start);
    std::cout << "耗时: " << duration.count()
    ↪  << " ms\n";

    // 转换为微秒
    auto us = std::chrono::duration_cast<std::c
    ↪  hrono::microseconds>(end - start);
    std::cout << "耗时: " << us.count() << "
    ↪  us\n";

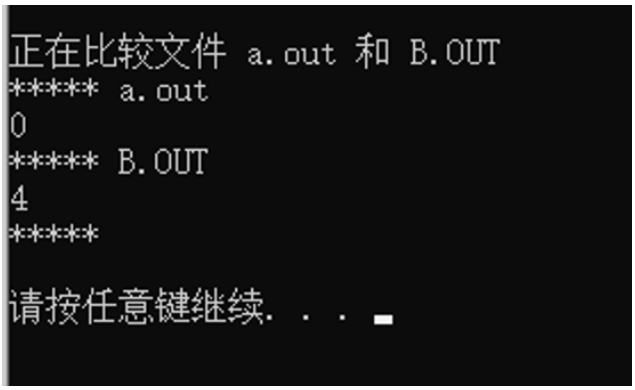
    // 或者直接用浮点数秒
    std::chrono::duration<double> sec = end -
    ↪  start;
    std::cout << "耗时: " << sec.count() << "
    ↪  s\n";
}
```

19.5 对拍

对拍程序大致框架（windows）（不写 .exe 也是可以的）

```
#include <bits/stdc++.h>

int main() {
  // For Windows
  // 对拍时不开文件输入输出
  // 当然，这段程序也可以改写成批处理的形式
  while (true) {
    system("gen > test.in");  //
    ↪  数据生成器将生成数据写入输入文件
    system("mySol.exe < test.in > a.out");  //
    ↪  获取程序1输出
    system("std.exe < test.in > b.out");  //
    ↪  获取程序2输出
    if (system("fc a.out b.out")) {
      // 该行语句比对输入输出
      // fc返回0时表示输出一致，否则表示有不同处
      system("pause");  // 方便查看不同处
      return 0;
      // 该输入数据已经存放在test.in文件中，
      ↪  可以直接利用进行调试
    }
  }
}
```



fc 函数的输出结果如上。

Linux 可以像下面这么写。（其实就是把 fc 改成了 diff，其他其实差不太多）

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    // 对拍时不开文件输入输出
    // 当然，这段程序也可以改写成批处理的形式
    int idx=0;
    while (true) {
        cerr<<"running on "<<++idx<<"
        ↪  test"<<"\n";
        // 下面的这些都是可执行程序
        // （相对）路径也是以这个可执行程序位置为准
        system("./D2_Tree_Orientation_Gen >
        ↪  test.in");  //
        ↪  数据生成器将生成数据写入输入文件
        system("./D2_Tree_Orientation_Hard_Vers
        ↪  ion < test.in > a.out");  //
        ↪  获取希望检验输出
        system("./D1_Tree_Orientation_Easy_Vers
        ↪  ion < test.in > b.out");  //
        ↪  获取暴力程序输出
        if (system("diff a.out b.out")) {
            // 该行语句比对输入输出
            return 0;
            // 该输入数据已经存放在test.in文件中，
            ↪  可以直接利用进行调试
        }
    }
}
```

```
zzy@zzysever:~/myCppProgram$ clang++ bruteCMP.cpp  -o bruteCMP
zzy@zzysever:~/myCppProgram$ ./bruteCMP
1c1
< 16
---
> 8
sh: 1: pause: not found
zzy@zzysever:~/myCppProgram$
```

19.5.1 Special Judge（答案不唯一）题目怎么对拍

答案不唯一时（「输出任意一组方案」/ Yes 后跟构造 / 浮点误差），diff 逐字节比对会把「两个都对但长得不一样」误报成不一致。做法：把 diff a.out b.out 换成自己写的 **checker**，通过返回 0、发现错返回非 0——仍然沿用 system() 返回值判断，框架其余部分完全不变：

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
  // 对拍时不开文件输入输出
  // 当然，这段程序也可以改写成批处理的形式
  int idx=0;
  while (true) {
    cerr<<"running on "<<++idx<<" test"<<"\n";
    // 下面的这些都是可执行程序
    // （相对）路径也是以这个可执行程序位置为准
    system("./H_Modulo_Triples_gen > test.in");
    ↪ // 数据生成器将生成数据写入输入文件
    system("./H_Modulo_Triples < test.in >
    ↪ a.out"); // 获取希望检验输出
    if (system("./H_Modulo_Triples_check test.in
    ↪ a.out")) {
      system("cat a.out");
      // 该行语句比对输入输出
      return 0;
      // 该输入数据已经存放在test.in文件中，
      ↪ 可以直接利用进行调试
    }
  }
}
```

抓到反例时顺手把 test.in / a.out cat 到终端，肉眼直接看现场（**Windows 下没有 cat，换成 type**，与打表一节相同）。

checker 骨架（argv 传文件名即可，本地对拍不必上 OJ 官方 checker 用的 testlib.h）：

```cpp
// 可以不使用 namespace std，防止使用成 cin，
↪ 导致错误啊
// checker.cpp: 任一步失败 => cerr 打出原因，
↪ return 1
signed main(int argc, char **argv) {
  std::ifstream in(argv[1]); // 输入数据
  std::ifstream out(argv[2]); // smart 输出
  ↪ （待验）
  // ifstream ref(argv[3]); // （可选）brute 输出
  ↪ -- 仅比对最优值时才传
  in >> N;
  std::set<ll> st;
  for (int i = 0; i < 3 * N; ++i) {
    st.insert(i);
  }
  for (int i = 0; i < 3 * N; ++i) {
    ll p;
    out >> p;
    st.erase(p);
  }
  if (st.empty()) {
    return 0;
  }
  // 直接使用 cerr 输出错误原因
  for (auto x: st) {
    std::cerr << x << " ";
  }
  std::cerr << "\n";
  return -1;
}
```

19.6 打表

用 system() 调用编译好的生成器和暴力程序，批量跑数据并输出结果，方便观察规律。

```cpp
struct Solve {
  // ll N;

  Solve() {
    // cin >> N;
    for (int i = 1; i <= 100; ++i) {
      system("./Good_Array_1_gen > in.txt");
      system("cat in.txt");
      system("./Good_Array_1__brute < in.txt >
      ↪ out.txt");
      cout << "\n"; // 这里需要加一个换行，
      ↪ 才能更好地看清 out.txt
      system("cat out.txt");
      cout << "\n---------------\n";
    }
  }
};
```

Windows 注意：Windows 下没有 cat 命令，需要换成 type。

重定向 cerr 到文件：如果程序的输出走的是 cerr（比如调试信息），普通的 > 只能捕获 stdout。需要用 2> 或 2>&1 进行重定向：

```cpp
# 只捕获 cerr (stderr) 到 out.txt
./brute < in.txt 2> out.txt

# 同时捕获 cout (stdout) 和 cerr (stderr) 到
↪ out.txt
./brute < in.txt > out.txt 2>&1
```

不过一般我们只需要在 brute.cpp 中在 cerr 输出调试信息，所以不需要这个。

重定向后，**被重定向的流就不会再实时显示在终端里，而是写入文件**。比如用了 2> out.txt 之后，cerr 会消失在终端；用了 > out.txt 2>&1 之后，cout 和 cerr 都会消失在终端。

这里为什么不直接用 brute.cpp 程序的标准输出流，而是写入文件？因为如果直接用标准输出流，那么 brute.cpp 的输出和这个 in.txt 的 cat 输出会混在一起，不方便观察。

19.6.1 打表中需要输出的东西（答案）

Binary Construction 这个是一道典型的打表题目，只需要使用暴力输出答案，即可一眼看出规律。

```
1
01
011
1001
10001
100001
1000001
.......
```

```
10000000000001
100000000000001
1000000000000001
10000000000000001
```

但是，由于我们采用了放弃大脑思考，以对拍代替大脑思考的策略，显然，我们不止在打表题目进行打表，而是在很多题目上都打表，那么如何打表才能够更好地观察规律，找到解法，帮助我们解决问题呢?

19.6.2 打表中需要输出的东西（前缀答案）

Good Array 1 输出前缀答案，可以帮助我们看出来这道题目是不是前缀 dp。

```
1
8
2 7 1 4 6 6 4 4
[dp] = [0, 0, 0, 0, 0, 1, 1, 2]

************************
2

************************
1
7
4 3 3 1 4 5 6
[dp] = [0, 0, 1, 1, 1, 1, 1]

************************
1
```

不过其实不是每道题目都是适合这个打表的，对于一些题目，头脑风暴或者其他方法可能更加适合。

19.7 凸包数据生成

对拍几何题（凸包、旋转卡壳、闵可夫斯基和等）需要快速生成 CCW 序凸多边形。每次随机选椭圆长短轴比 $\in [1,3]$ + 随机旋转角，覆盖近圆 / 椭圆 / 倾斜椭圆等曲率非均匀形状（破除「近圆形 monotonicity 盲区」），然后随机 N 个角度排序映射上去。`T` 模板参数：`T = ll` 走 `llround` 钉整数（题面整数坐标）、`T = DB` 保留 cos/sin 原浮点（精度/边界更细）；坐标范围 $x, y \in [-R, R]$（椭圆 $R\cos\theta$、$\frac{R}{\text{asp}}\sin\theta$ 内嵌半径 R 圆盘，旋转 φ 不变半径，`llround` 边界 ± 1 不溢出）。`allow_collinear` 切换严格凸（cross > 0）和允许三点共线（cross ≥ 0，但仍要求至少一处真左转保证多边形面积非零）。可行域实测：$R = 10^9$ 时 $N \leq 2000$ 安全（$N = 5000$ 严格模式 75 % 失败）；$N/R \gtrsim 0.3$ 严格模式 retry 显著上升、$\gtrsim 0.5$ 不可行（如 $R = 100$、$N = 50$ 100 % 卡死）。要求 $N \geq 3$，超出可行域时自旋等死（外面套 retry 上限自己加）。

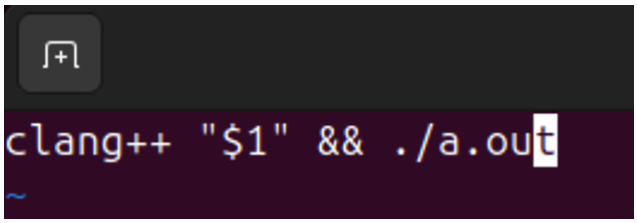
```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

// T = ll : 整数坐标（llround）   /   T = DB : 浮点坐标（保留原值）
// 坐标范围 |x|, |y| <= R
template<class T>
vector<Point<T>> gen_convex_polygon(int N, bool allow_collinear = false,
                                    ll R = 1'000'000'000) {
    // 退化 case (N <= 2): 主循环对 N <= 2 必死循环
    // (N=1: j=(0+1)%1=0=i, A[i]==A[j] 永真;
    //  N=2: k=(0+2)%2=0=i, cross 三点退化恒 0).
    // 直接撒 N 个不重点返回，跳过 CCW 校验.
    if (N <= 2) {
        set<pair<T, T>> S;
        while ((int) S.size() < N) {
            if constexpr (is_floating_point_v<T>) {
                S.insert({uniform_real_distribution<T>(-(T) R, (T) R)(rng),
                          uniform_real_distribution<T>(-(T) R, (T) R)(rng)});
            } else {
                S.insert({uniform_int_distribution<T>(-(T) R, (T) R)(rng),
                          uniform_int_distribution<T>(-(T) R, (T) R)(rng)});
            }
        }
        vector<Point<T>> A;
        for (auto [x, y] : S) A.push_back(Point<T>(x, y));
        return A;
    }
    DB phi = uniform_real_distribution<DB>(0, 2 * M_PI)(rng);
    DB asp = uniform_real_distribution<DB>(1.0, 3.0)(rng); // 1=圆, >1=椭圆
    DB cp = cos(phi), sp = sin(phi);
    while (true) {
        vector<DB> th(N);
        for (auto& t : th) t = uniform_real_distribution<DB>(0, 2 * M_PI)(rng);
        sort(th.begin(), th.end());
        vector<Point<T>> A(N);
        for (int i = 0; i < N; ++i) {
            DB x = R * cos(th[i]), y = R / asp * sin(th[i]);  // 椭圆
            DB rx = cp * x - sp * y, ry = sp * x + cp * y;    // 旋转 phi
            if constexpr (is_floating_point_v<T>) {
                A[i] = Point<T>(rx, ry);
            } else {
                A[i] = Point<T>((T) llround(rx), (T) llround(ry));
            }
        }
```

```
        bool ok = true, has_turn = false; //
        ↪ has_turn: 至少一处真左转
        for (int i = 0; i < N && ok; ++i) {
            int j = (i + 1) % N, k = (i + 2) %
            ↪ N;
            if (A[i] == A[j]) { ok = false;
            ↪ break; }
            auto c = cross(A[i], A[j], A[k]);
            if (allow_collinear ? c < 0 : c <=
            ↪ 0) { ok = false; break; }
            if (c > 0) has_turn = true;
        }
        // allow_collinear 下若全程 cross == 0
        ↪ 则点全共线、面积 = 0，得重抽
        if (ok && !has_turn) ok = false;
        if (ok) return A;
    }
}
```

19.8 编译 shell

```
clang++ "$1" && ./a.out
```




```
zzy@zzysever:~/myCppProgram$ ./cr gen.cpp
1
5 4
2 1
3 1
4 1
5 1
```